



应用与微服务
使用手册

版本 V1.00

前言

产品版本

与本手册相对应的产品版本如下所示。

产品名称	产品版本	手册版本
应用与微服务	V3.7.0.0	V1.00

内容介绍

章节	主要内容
第 1 章	产品概述
第 2 章	工作原理
第 3 章	功能描述
第 4 章	产品架构
第 5 章	安全和可靠性

修改记录

修订记录累积了每次手册更新的说明。最新版本的文档包含以前所有版本的更新内容。

版本 V1.00 （2023-08-12）

随着悟空平台 V3.7.0.0 版本的发布，本手册首次发布。

目录

- 1 产品概述.....7
- 2 工作原理.....9
 - 2.1 JAVA 探针原理.....9
 - 2.2 TINGYUNAGENT 原理.....9
 - 2.3 KUBERNETES TINGYUNAGENT 原理.....10
 - 2.4 产品监测原理.....11
- 3 功能描述.....12
 - 3.1 公共操作.....12
 - 3.1.1 统计周期.....12
 - 3.1.2 添加图表.....13
 - 3.1.3 框选图表.....13
 - 3.2 全局拓扑.....13
 - 3.2.1 应用场景.....14
 - 3.2.2 横向拓扑.....15
 - 3.2.3 纵向拓扑.....17
 - 3.2.4 拓扑操作说明.....17
 - 3.2.4.1 详情分析.....17
 - 3.2.4.2 过滤节点.....21
 - 3.2.4.3 图例说明.....22
 - 3.2.4.4 节点设置.....23
 - 3.2.4.5 其他说明.....24
 - 3.3 网络.....24
 - 3.3.1 网络指标监控.....26
 - 3.3.1.1 Workload 详情.....26

3.3.1.2	Pod 详情.....	27
3.3.1.3	连线详情.....	29
3.3.2	过滤节点.....	31
3.3.3	图例说明.....	31
3.3.4	操作说明.....	31
3.4	业务系统.....	32
3.4.1	新建业务系统.....	32
3.4.1.1	控制台创建.....	32
3.4.1.2	配置文件创建.....	33
3.4.2	新建服务组.....	33
3.4.3	业务系统列表.....	34
3.4.4	业务系统全局拓扑.....	34
3.4.5	业务系统详情.....	37
3.4.6	服务组拓扑.....	40
3.5	应用与实例.....	41
3.5.1	应用/实例概览.....	42
3.5.1.1	应用概览.....	42
3.5.1.2	实例概览.....	46
3.5.2	应用详情.....	47
3.5.2.1	服务视角分析.....	47
3.5.2.2	调用者视角分析.....	48
3.5.2.3	实例分析.....	48
3.5.2.4	请求分析.....	49
3.5.3	响应时间性能分解.....	50
3.5.3.1	响应时间分解.....	53
3.5.3.2	响应时间分解趋势.....	54
3.5.3.3	Top 响应时间分解.....	54
3.5.3.4	响应时间分解表格.....	55
3.5.4	响应时间分布.....	56
3.5.4.1	响应时间分布.....	56
3.5.4.2	Top 请求.....	57
3.5.5	调用者分析.....	57

3.5.5.1	请求来源	58
3.5.5.2	请求分析	59
3.5.6	热点方法	60
3.5.6.1	饼图	60
3.5.6.2	堆叠图	61
3.5.6.3	火焰图	61
3.5.7	错误分析	62
3.5.7.1	错误列表	62
3.5.7.2	影响分析	63
3.5.7.3	根因分析	63
3.5.7.4	追踪列表	64
3.5.8	异常分析	65
3.5.8.1	异常列表	65
3.5.8.2	影响分析	65
3.5.8.3	根因分析	66
3.5.8.4	追踪列表	67
3.5.9	OneTraces	67
3.5.10	拓扑图	67
3.5.11	外部服务	69
3.5.11.1	性能	69
3.5.11.2	异常	70
3.5.12	自定义指标	70
3.5.13	线程剖析	71
3.5.14	JVM	72
3.5.15	环境信息	74
3.6	事务	91
3.6.1	事务列表	91
3.6.2	概览	92
3.6.3	事务拓扑图	95
3.6.4	响应时间	95
3.6.5	错误	98
3.6.6	线程剖析	99
3.7	服务接口	101

3.8	后台任务	103
3.8.1	概览	104
3.8.2	错误	104
3.8.3	后台任务追踪	105
3.9	DATABASE 服务组件	105
3.9.1	概览	106
3.9.2	响应时间	107
3.9.3	错误	107
3.9.4	SQL 分析	108
3.9.4.1	SQL 语句列表	109
3.9.4.2	SQL 语句详情	109
3.9.5	操作分析	111
3.9.5.1	概览	112
3.9.5.2	响应时间	113
3.9.5.3	错误	113
3.9.6	连接池分析	113
3.10	NoSQL 服务组件	114
3.11	MQ 服务组件	115
3.12	错误分析	115
3.12.1	错误分析	116
3.12.1.1	错误统计	116
3.12.1.2	异常统计	118
3.12.1.3	公共操作	119
3.12.2	错误详情	120
3.12.2.1	错误影响分析	120
3.12.2.2	错误趋势图	120
3.12.2.3	Exception Messages	120
3.12.2.4	Stacktrace	121
3.12.2.5	Root cause	121
3.12.2.6	异常追踪	122
3.12.2.7	Referring pages	122
3.12.2.8	Client IPs	122
3.12.2.9	Caller Applications	123

3.13	诊断工具	123
3.13.1	对象统计	124
3.13.1.1	启用对象统计	124
3.13.1.2	创建对象统计	125
3.13.1.3	对象统计诊断	126
3.13.2	内存 Dump 诊断	129
3.13.2.1	启用内存 Dump	129
3.13.2.2	创建内存 Dump	130
3.13.2.3	内存 Dump 诊断	131
3.14	请求追踪	132
3.14.1	请求追踪列表	132
3.14.2	请求追踪详情	134
3.14.2.1	追踪概览	134
3.14.2.2	性能分解图	135
3.14.2.3	疑似问题	135
3.14.2.4	拓扑	135
3.14.2.5	Call Table	135
3.14.2.6	Call Tree	136
3.14.2.7	全栈快照	142
3.14.2.8	异常	145
3.14.2.9	参数信息	146
3.14.2.10	Code 统计	146
3.14.2.11	SQL 统计	146
3.14.2.12	NoSQL 统计	146
3.14.2.13	Servers	146
3.14.2.14	日志	147
3.15	连接池	147
3.16	动态基线和健康度	148
3.16.1	动态基线	149
3.16.1.1	新建动态基线	149
3.16.1.2	管理动态基线	150
3.16.2	健康度	150
3.16.2.1	新建健康规则	151

3.16.2.2	管理健康规则.....	154
3.16.2.3	健康度分析.....	156
3.17	部署管理.....	156
3.17.1	TingyunAgents 管理.....	156
3.17.1.1	列表项说明.....	157
3.17.1.2	操作说明.....	157
3.17.1.3	进程信息.....	159
3.17.2	Collector 管理.....	162
3.17.2.1	Collector 列表.....	162
3.17.2.2	Collector 主机详情.....	165
3.17.3	下载中心.....	165
3.17.3.1	TingyunAgent 下载.....	165
3.17.3.2	Agent Collector 下载.....	166
3.17.3.3	Agent 下载.....	166
3.17.4	第三方探针接入.....	167
3.17.4.1	通过 OpenTelemetry 接入 Java 应用数据.....	167
3.17.4.2	通过 SkyWalking 接入 Java 应用数据.....	170
3.18	配置.....	175
3.18.1	全局配置.....	176
3.18.1.1	Apdex T.....	176
3.18.1.2	日志溯源.....	176
3.18.1.3	采样.....	180
3.18.1.4	采集调用成功率和响应率.....	181
3.18.1.5	获取源代码.....	182
3.18.1.6	组件钻取.....	183
3.18.1.7	追踪选项.....	184
3.18.1.8	错误及异常.....	185
3.18.1.9	探针熔断.....	186
3.18.2	业务系统设置.....	187
3.18.2.1	常规选项.....	187
3.18.2.2	用户溯源.....	187
3.18.2.3	自定义嵌码.....	189

3.18.2.4	数据项.....	193
3.18.2.5	自定义指标.....	195
3.18.2.6	性能诊断.....	198
3.18.3	应用设置.....	198
3.18.3.1	常规选项.....	199
3.18.3.2	采样.....	200
3.18.3.3	探针熔断.....	200
3.18.3.4	错误及异常.....	200
3.18.3.5	事务.....	200
3.18.3.6	热点方法.....	208
3.18.3.7	数据项.....	209
3.18.4	实例设置.....	210
3.19	权限说明.....	211
4	产品架构.....	212
5	安全和可靠性.....	213
5.1	安全.....	213
5.2	可靠性.....	213

1 产品概述

基调听云应用与微服务是一套全新的应用性能管理平台，提供包括拓扑自动发现、调用链追踪、性能问题诊断以及故障告警和定位在内的多种应用性能监测手段和管理服务，同时在性能监控的基础上基于全新设计的探针架构，系统提供了全量数据采集能力。

应用与微服务具有以下特点：

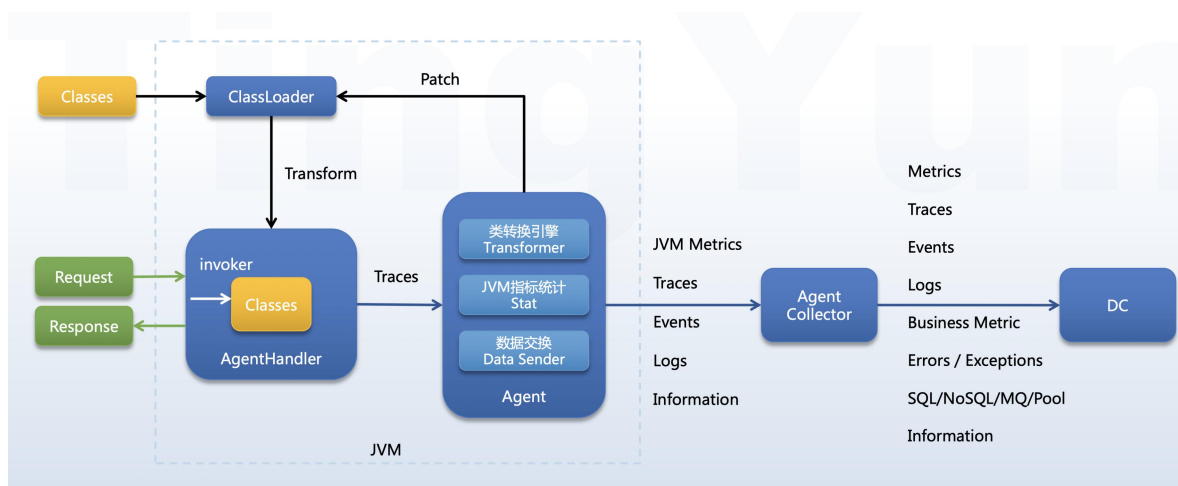
- 采用全新的应用探针架构，可以在几乎不影响被监控应用性能的情况下提供全量的业务和性能数据采集，同时极大降低了对 JVM 虚拟机资源的消耗。同时引入了全新的探针管理系统，可以在线对探针进行批量的升级部署。
- 引入了业务系统和全局事务的定义，以便更好地对业务系统中的应用进行归类整理和可视化，同时对业务系统提供更完整监控、可视化和追踪，更方便用户对业务系统进行梳理和监控，对业务性能问题进行快速发现、隔离和定位，降低 MTTR（平均故障恢复时间）。

- 引入了动态基线和健康度概念。用户可以根据不同的业务周期性变化需求选择不同趋势的动态基线作为健康度计算的基准。对包含业务系统、应用、实例、事务、组件等在内的多种被监控对象，用户可分别选择多种指标和规则来进行健康度计算并以直观的形式展示在报表中。
- 通过可视化的业务埋点技术可以与业务分析、用户体验分析等平台相结合，提供从业务表现分析、用户体验分析到应用性能分析的数据关联和追踪分析，提升 IT 部门对业务和用户体验的把控。

2 工作原理

2.1 Java 探针原理

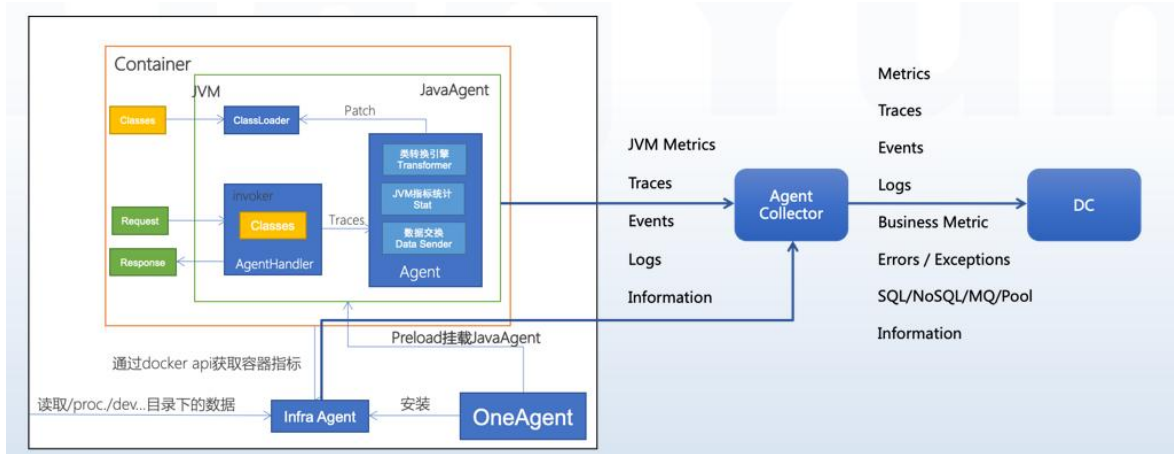
Java Agent 基于 JDK 提供的 Instrumentation 机制，在 class 文件被加载的时候，通过字节码技术，动态对 Framework、数据库、NoSQL、Web Service、组件等特定方法实施监控，从而获得方法执行时间、数据库调用时间、NoSQL 响应时间以及外部服务响应时间；并在这些时间超过一定阈值时，抓取调用堆栈。



2.2 TingyunAgent 原理

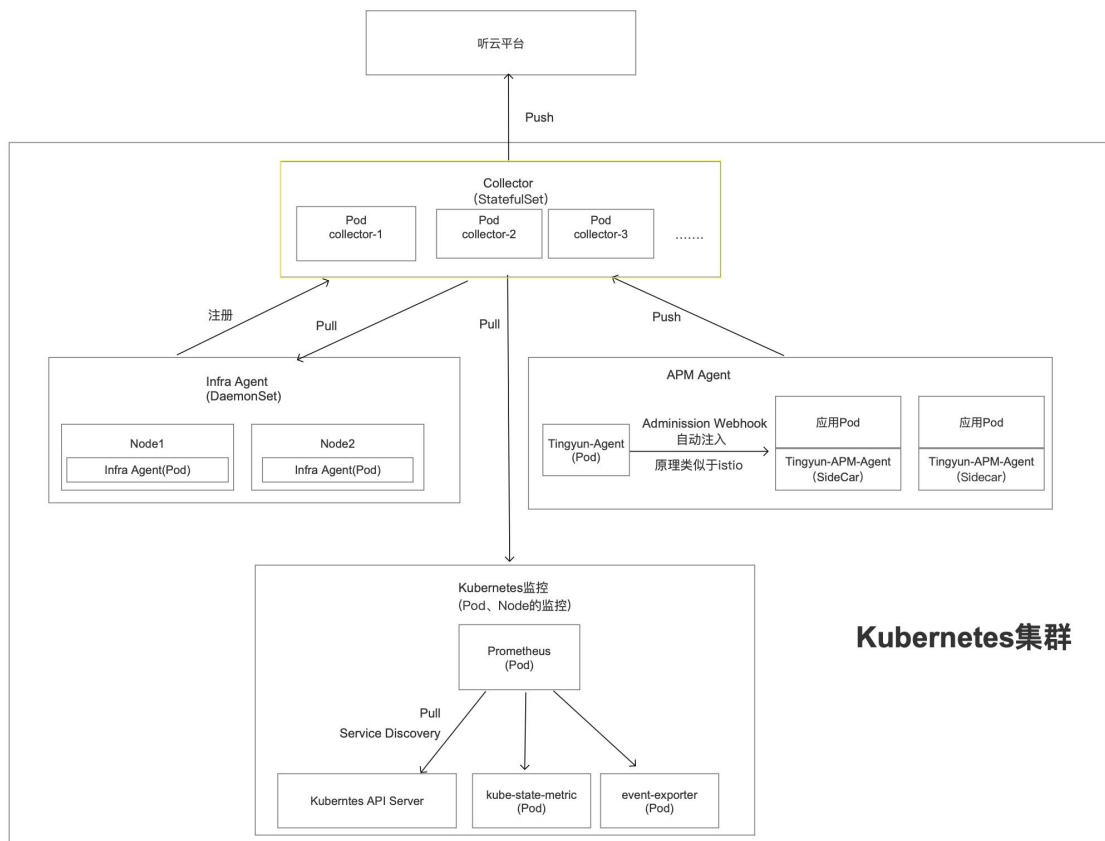
TingyunAgent 基于 Preload 技术实现主机、应用程序、组件、服务、数据库等监控对象的性能指标的采集。Preload 技术是 Linux 系统自身支持的模块预加载技术，进程加载器在加载进程时，会在模块表中首先插入指定的预加载模块，然后再插入进程所依赖的其他模块，预加载模块在模块表的位置总是位于加载器之后。

Java Agent 基于 JDK 提供的 Instrumentation 机制，在 class 文件被加载的时候，通过字节码技术，动态对 Framework、数据库、NoSQL、Web Service、组件等特定方法实施监控，从而获得请求调用次数、方法执行时间、数据库调用时间、NoSQL 响应时间、外部服务响应时间、异常、堆栈以及方法参数。



2.3 Kubernetes TingyunAgent 原理

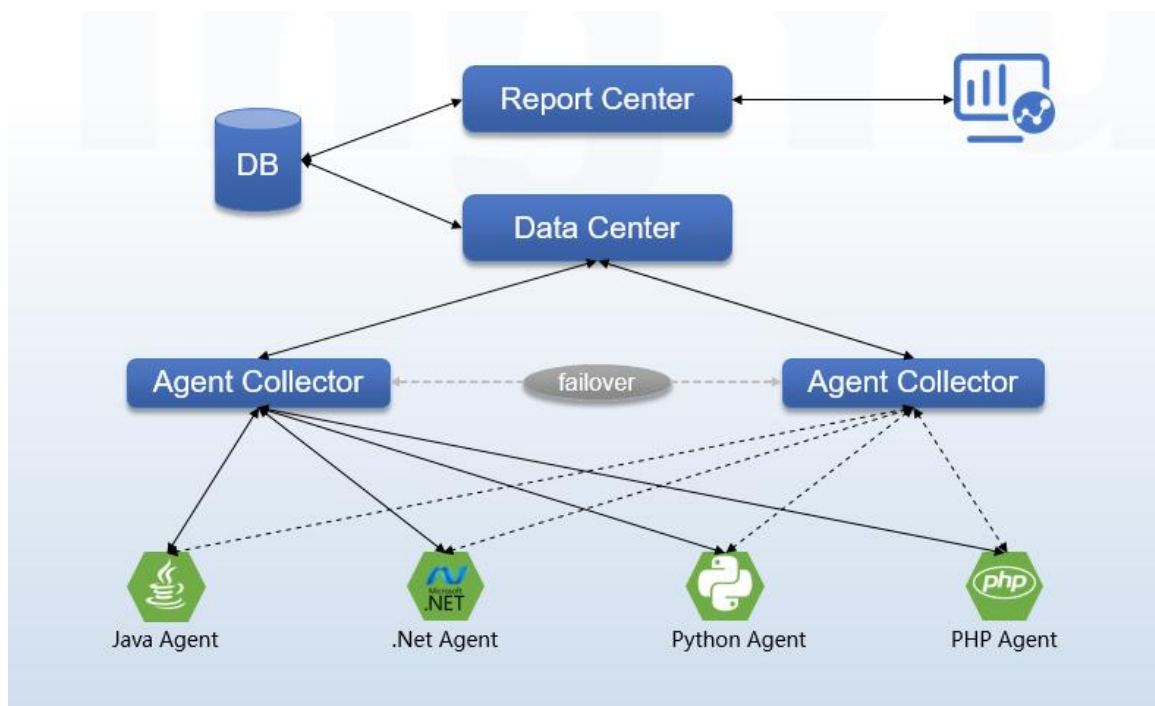
- APM 探针：采用 Adminissson Webhook+SideCar 自动注入。
- Infra 探针：采用 daemonset 自动扩展 Node 节点。
- Collector：采用 Statefulset 自动扩展。
- Kubernetes 集群监控：对接 Prometheus。



2.4 产品监测原理

探针采用两级架构，包括运行在被监控应用上的 Agent 部分和运行在独立服务器上的 Collector 部分。其中 Agent 部分负责对应用进行嵌码和原始的数据采集，采集后的信息全部通过局域网直接传输到 Collector 上，由 Collector 进行数据的统计和关联，并最终上报到平台的数据中心上。

由于 Agent 会采集全量的业务和性能数据并实时传输到 Collector 上，因此 Agent 与 Collector 之间必须保证通畅的本地千兆网络连接。Collector 支持 Failover 的高可用，对一组 Agent 可以部署多台 Collector 服务器，当其中个别服务器出现故障无法正常工作，探针可通过 Failover 机制将数据实时传输到其他 Collector 上，以实现数据采集的高可用。Collector 的高可用机制如下图所示：

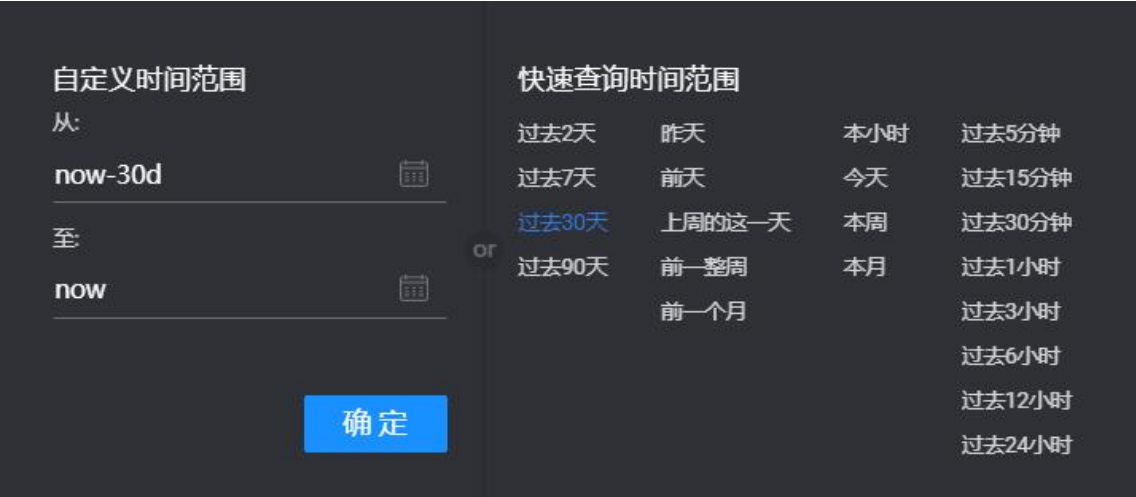


3 功能描述

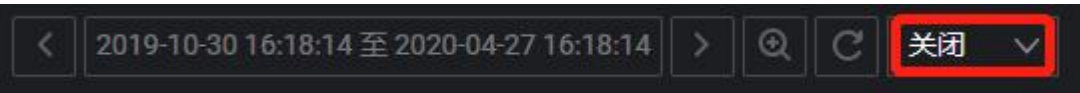
3.1 公共操作

3.1.1 统计周期

应用与微服务能够存储最近 90 天的性能监控数据，在此区间内，您可以指定任意的统计周期对监控的数据进行查询。应用与微服务提供两种统计周期选择方式：自定义时间范围和快速查询时间范围。



- **自定义时间范围**：单击  图标后，在日历中选择月份、年份和日。完成后，日期输入框中默认展示所选择日期的 00:00:00，可手动修改**时分秒**部分。需分别设置开始日期和结束日期。
- **快速查询时间范围**：系统预置了部分常用查询范围，可单击后快速完成时间范围设置。
- 单击时间范围放大图标 ，可根据您之前选择的时间范围，动态的扩大统计区间。
- 单击时间控件最右侧的下拉菜单，可开启或关闭图表数据自动刷新功能。默认为关闭状态。开启只需选择刷新的时间间隔即可。



- 单击重置刷新计时图标 ，可刷新各个图表中的数据。如果同时开启了自动刷新功能，则数据刷新要开始重新计时。

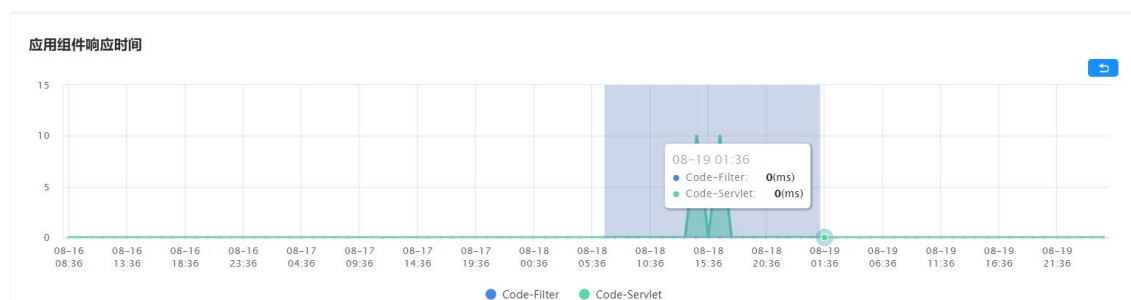
3.1.2 添加图表

将鼠标悬浮在图表上，单击图表右下角的 图标，可将当前图表添加到基调听云大屏的组件列表中。单击图表右下角的 图标，可将当前图表添加到基调听云智能报告的图表库中。不同的图表支持情况不同，请根据实际情况添加。

3.1.3 框选图表

当用户需要对图表中某一时间段的数据进行细粒度查看时，无需手动调整右上角的统计周期，按住鼠标左键即可在图表中框选时间范围，缩小图表的展示周期，框选时间范围需要大于一分钟。松开鼠标后，图表最左侧对应的时间为开始框选的时间点，图表最右侧对应的时间为结束框选的时间点，页面右上角的统计时间会随之调整，页面中所有图表的展示周期都会变化。单击图表右上角的返回按钮可恢复之前的统计区间。

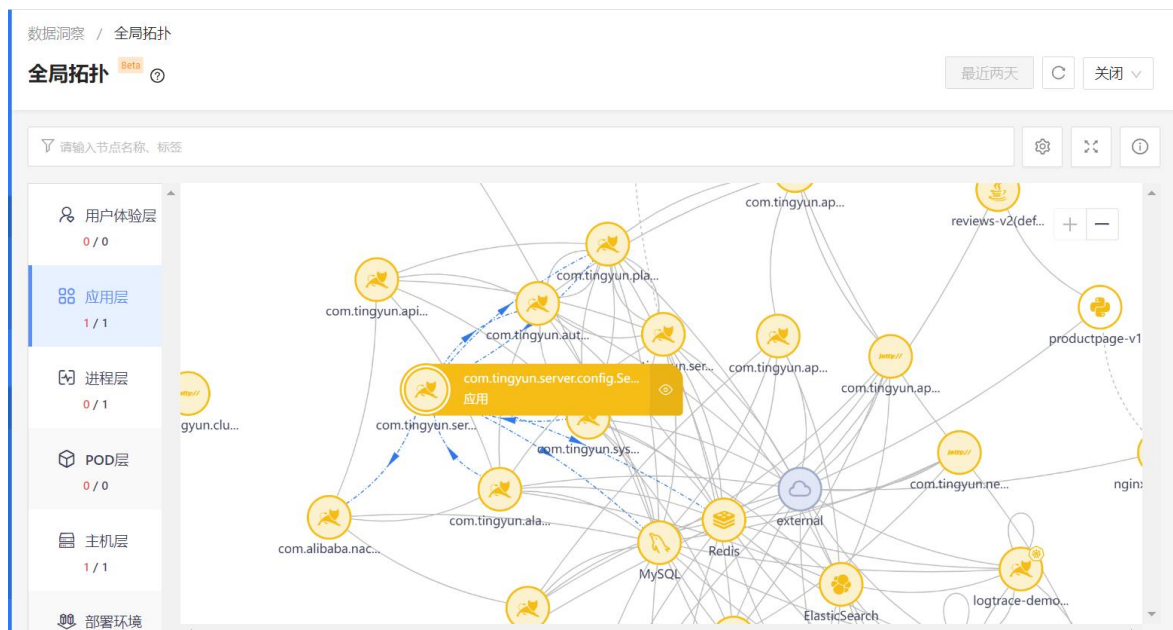
支持框选功能的图表不包括事务/服务接口/后台任务追踪详情中的图表、连接池图表、健康度图、业务系统/应用详情中的 Apdex 图、弹出的对话框中的图表。



3.2 全局拓扑

全局拓扑是一种全景式的应用程序拓扑图，它可以自动发现和绘制整个应用程序环境的拓扑结构，包括用户体验、应用、进程、Pod、主机、部署环境间的关系。功能说明如下：

- 全局拓扑可以帮助用户快速了解应用程序的整体架构，并提供实时的性能指标和事件数据，指标、告警、日志等数据彼此关联打通，以便用户能够快速诊断和解决性能问题。
- 全局拓扑通过横向调用关系拓扑和纵向依赖关系拓扑，帮助您解决企业复杂多变的服务治理难题、业务流程治理难题以及变更影响分析难题，实现数字业务的全景可观测性。
- 利用基调听云的智能一体化探针 TingyunAgent，全局拓扑能够自动发现、监控和拓扑监控对象。
- 借助无监督的知识图谱和决策树等算法，它可以主动为您标记疑似问题节点及根因节点，为故障提供答案。
- 全局拓扑融合了指标体系和标签系统，用于指标探索和深度分析。



在左侧导航栏中依次单击**数据洞察>全局拓扑**，进入全局拓扑图页面。页面左侧展示层级名称，名称下方左侧为受疑似问题影响的对象数量，右侧为当前层级的对象总数量。页面右侧展示全局拓扑图。

3.2.1 应用场景

- 分级拓扑

面向大规模的分布式的微服务业务系统，全新设计的分级和分层的拓扑图可以更好地展示复杂的服务依赖和业务调用关系，方便用户更快、更方便地把握全局。

- 数据纵向关联

除了横向的应用、服务组件间的数据关联外，系统还能纵向的将 RUM 访问、应用、进程、容器、主机、数据中心等多维度的数据关联在一起，帮助帮用户快速定位问题。

- 依赖关系映射

全局拓扑直观展示组件访问的层级关系，通过警告规则设置，系统计算出应用、实例、进程的访问健康状态，进而判断出业务流程的健康状态。

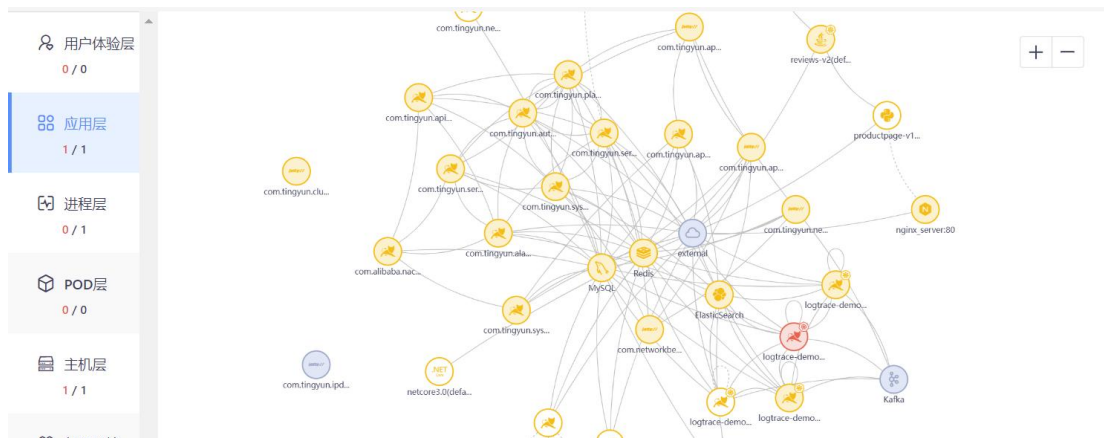
- 端到端用户体验追踪

在企业日益复杂的 IT 环境中，全局拓扑能够自动为 IT 团队建立从用户端，后端服务器、API、消息队列、存储等全方位的实时动态监控，实现跨系统、多业务部门的完整用户体验追踪。

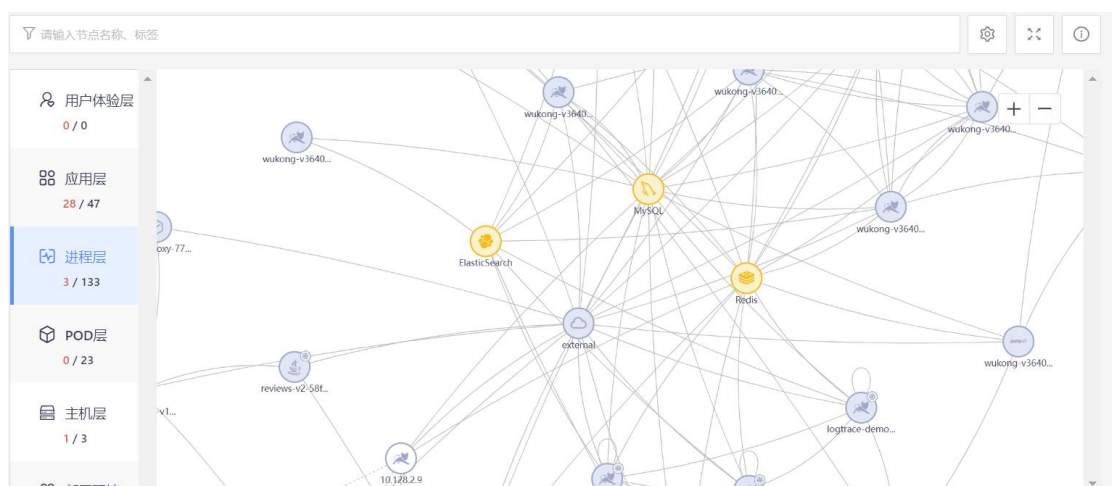
3.2.2 横向拓扑

横向拓扑可分别展示用户体验层、应用层、进程层、POD层、主机层、部署环境层的全局拓扑图。

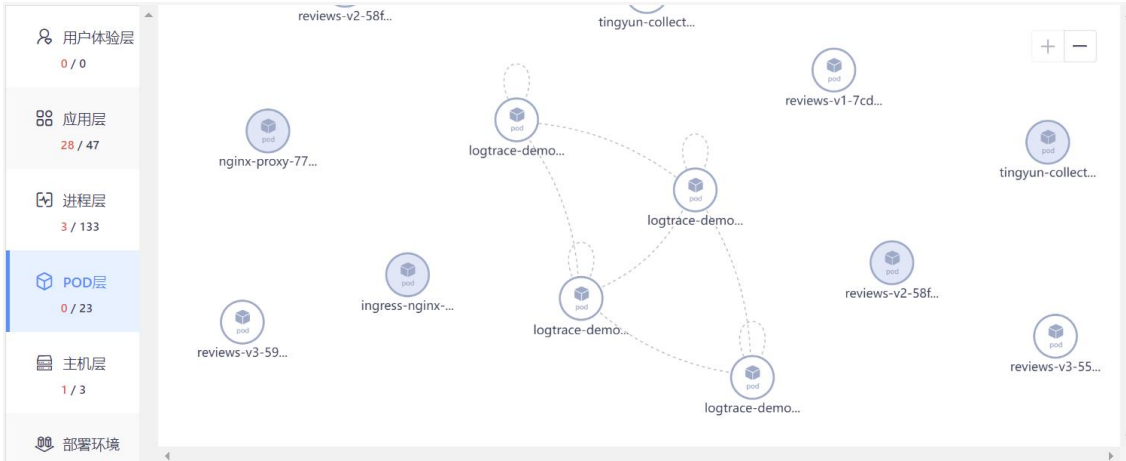
- **用户体验层**展示在悟空平台中创建的 Web 应用、App 应用和小程序应用。
- **应用层**展示当前账户下所有的应用、数据库组件、NoSQL 组件、MQ 组件、外部服务组件信息，以及应用和组件之间的调用关系。



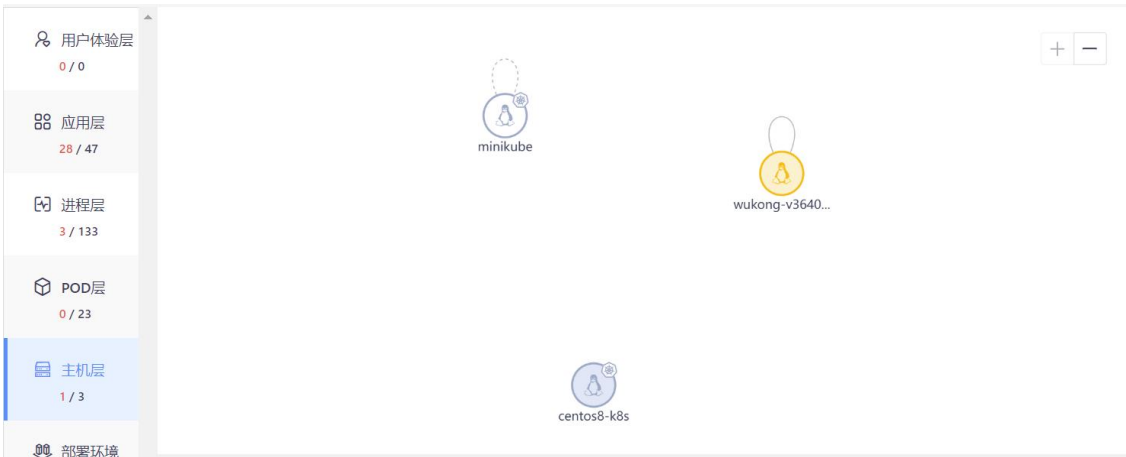
- **进程层**展示当前环境中正在运行的所有实例和进程，以及它们之间的调用关系，包括应用实例、数据库进程、NoSQL 进程、MQ 进程、外部服务进程、其他基础设施进程等。



- **POD 层**展示 Kubernetes 环境中正在运行的 POD，仅监控 Kubernetes 环境时会展示数据。



- 主机层展示环境中正在运行的所有主机。



- 部署环境层展示环境中正在运行的所有主机所在的机房。



说明：应用访问组件的拓扑需要满足以下条件：

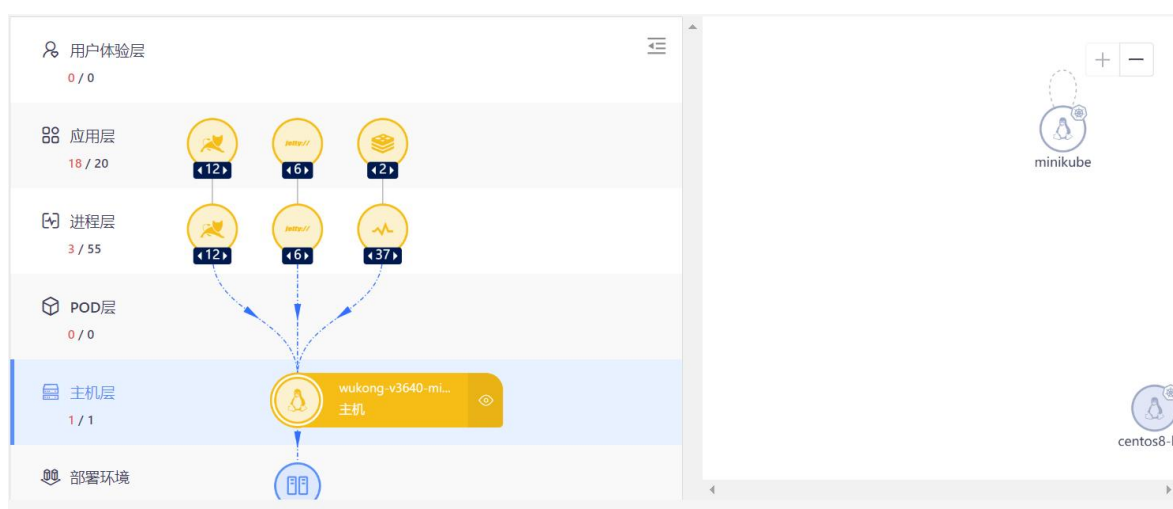
- Collector：3.6.6.0 及以上版本
- TingyunAgent：2.3.3.4 及以上版本

- Kubernetes TingyunAgent: 2.4.0.3 及以上版本

3.2.3纵向拓扑

您在横向拓扑图中，可以单击任意节点，在左侧查看其纵向的拓扑关系。例如，当前节点为应用节点，则用户体验层展示的是访问该应用的全部节点（集合 A），进程层为该应用下的全部实例和相关进程（集合 B），主机层为部署集合 B 的全部主机（集合 C），数据中心层为部署集合 C 的所有数据中心。

每一层的相关节点默认都是分类展示，图标下显示节点数量，单击图标或者节点数量会展开节点，再次单击节点数量会收起。将鼠标悬浮在图标上，可查看节点的名称、类型以及上下层节点。

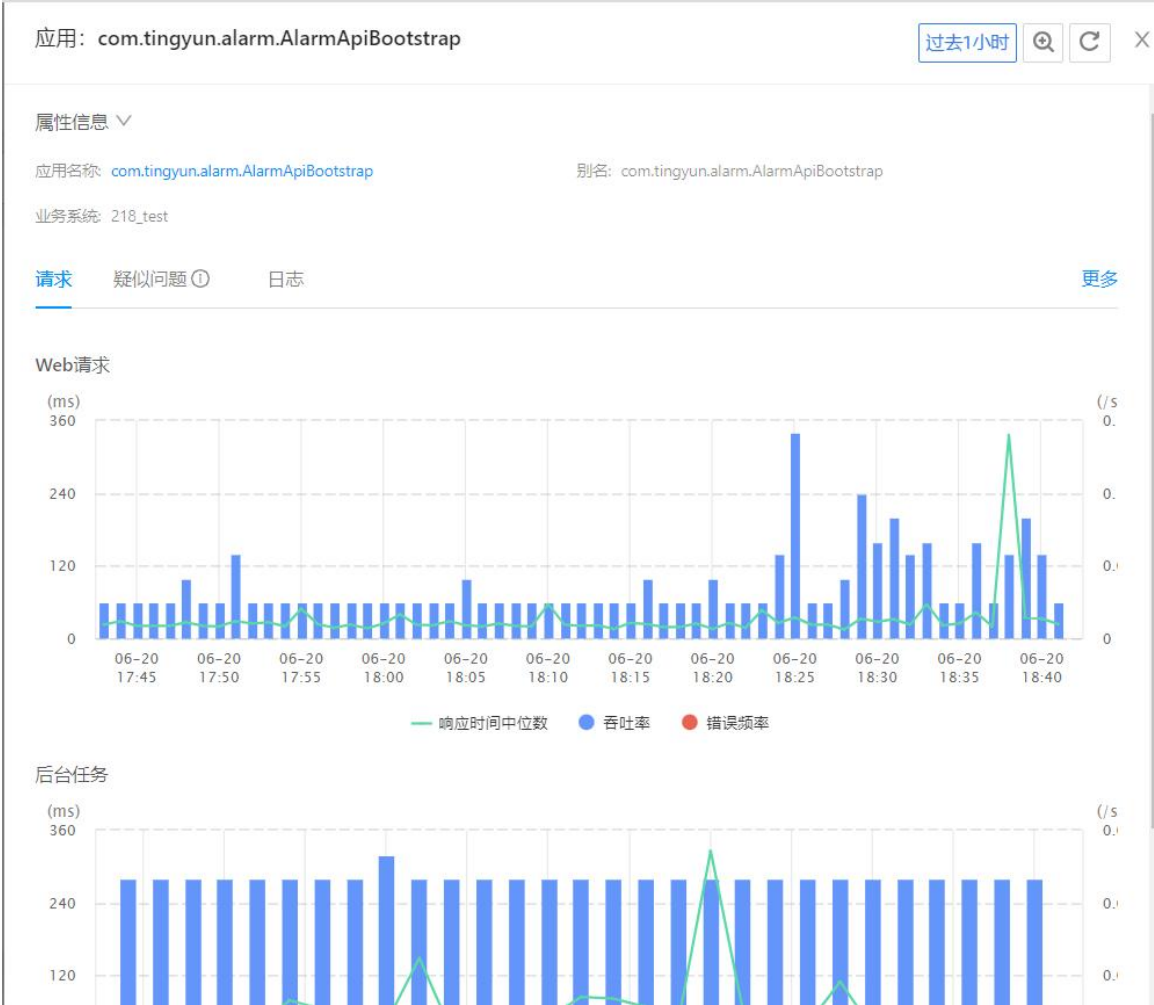


3.2.4拓扑操作说明

3.2.4.1 详情分析

3.2.4.1.1 应用分析

单击应用节点，名称右侧会出现  图标，单击该图标可展开应用详情面板，可查看应用别名、所属业务系统，Web 请求和后台任务的响应时间中位数、吞吐率、错误频率图表，疑似问题和应用日志。单击疑似问题的描述可进入疑似问题分析详情页面进行进一步分析。



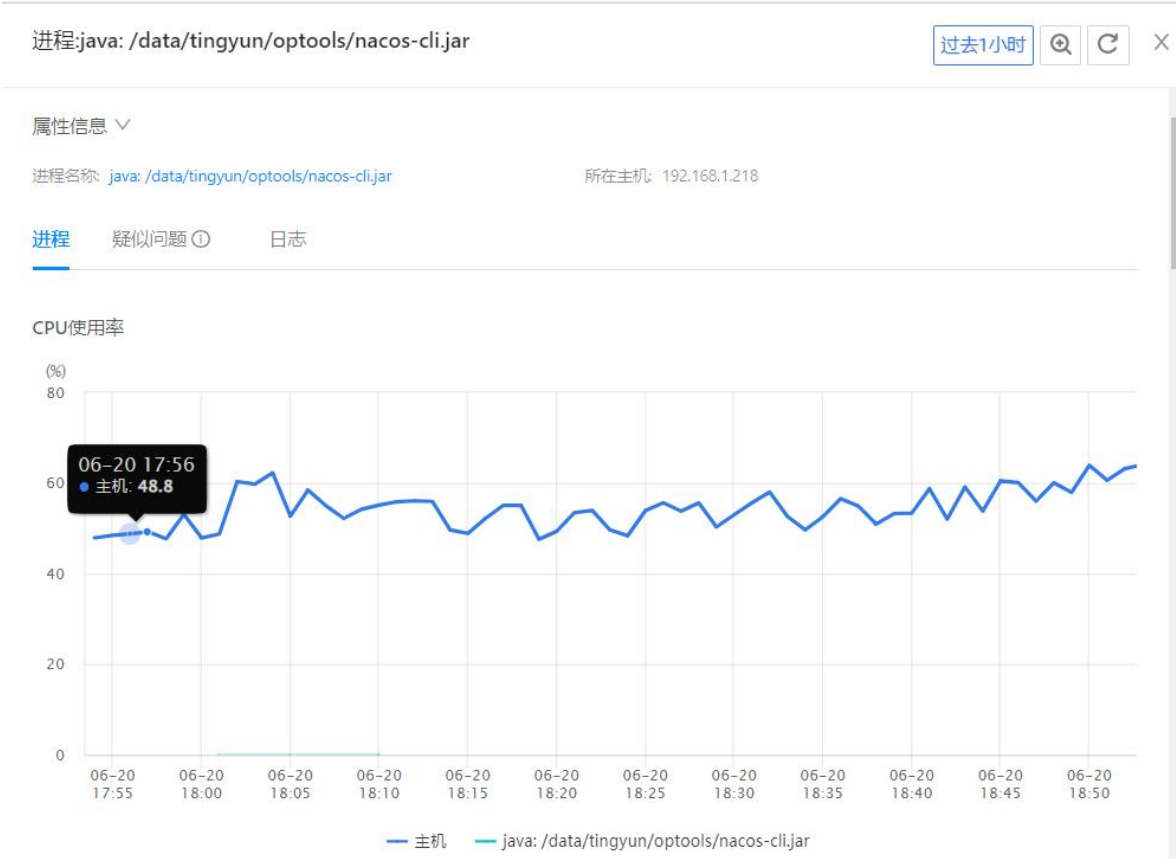
3.2.4.1.2 实例分析

单击应用实例节点，名称右侧会出现 图标，单击该图标可展开实例详情面板，可查看所属应用名称、IP 地址、技术栈、探针名称、进程名称、所属主机，Web 请求和后台任务的响应时间中位数、吞吐量、错误频率图表，疑似问题、进程分析和实例日志。单击疑似问题的描述可进入疑似问题分析详情页面进行进一步分析。



3.2.4.1.3 进程分析

单击进程节点，名称右侧会出现图标，单击该图标可展开进程详情面板，可查看进程名称、所在主机、运行时长、进程分析、疑似问题和进程日志。单击疑似问题的描述可进入疑似问题分析详情页面进行进一步分析。



3.2.4.1.4 主机分析

单击主机节点，名称右侧会出现图标，单击该图标可展开主机详情面板，可查看主机名称、IP 地址、CPU 核数、CPU 架构、内存总量、运行环境、指标分析、疑似问题、主机日志和进程分析。单击**更多**进入主机页面查看更多主机指标。单击疑似问题的描述可进入**疑似问题分析详情**页面进行进一步分析。

主机: wukong-v3640-micro.syh.com

过去1小时

X

属性信息

主机名称: wukong-v3640-micro.syh.comIP地址: 192.168.1.218

CPU核数: 16CPU架构: x86_64

内存总量: 62.76GB运行环境: test1

指标疑似问题 ①日志进程更多

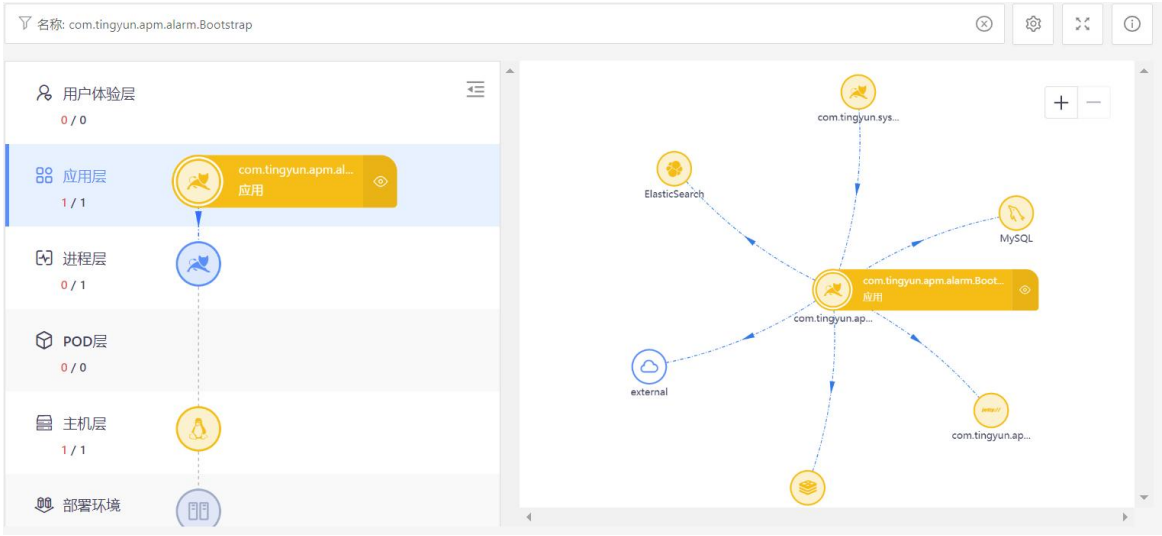
进程名称	运行时长	CPU(%)	内存	磁盘吞吐率 (读)	磁盘吞吐率 (写)	Swap	进程数	打开文件数	Major page fau
apache-exporter	3天3小时1分钟36秒	0	9.98MB	700.00	0	0	1	10	0
elasticsearch-exporter	3天3小时1分钟34秒	0	9.80MB	474.77	0	0	1	10	0
elasticsearch: /data/tingyun/base/base-elasticsearch	4天4小时48分钟22秒	4.44	3.45GB	13054.37	2188670.04	79961565.86666666	2	9853	0.04
java: /data/tingyun/alarm/alarm-	4小时57分25秒	0.18	1.03GB	1507.42	21397.13	0	1	635	0

3.2.4.2 过滤节点

单击页面顶部的过滤框，可根据名称和标签来迅速定位想找的节点。设置完成后目标节点将在拓扑图中放大显示，且展示该节点关联的上下游节点以及级联节点，同时在左侧可查看该节点的上下层级关系。

- 名称包括应用名称、应用实例名称、主机名称、Pod 名称、进程名称、Database 名称、部署环境、NoSQL 名称、MQ 名称。
- 标签包括应用名称、业务系统、部署环境、主机 IP、主机名称、实例名称、kubernetes 集群 ID、kubernetes 集群名称。

除上述过滤方式外，您还可以直接双击目标节点进行过滤。



3.2.4.3 图例说明

用户可以通过节点颜色来识别节点状态，通过连线来查看节点调用关系。单击页面右上角的  图标，可查看拓扑图的图例。



节点颜色：

- 蓝色 ：代表普通节点，状态正常。
- 黄色 ：代表普通故障节点。
- 红色 ：代表故障根因节点。

节点图标线条

- 实线：代表已安装探针。
- 虚线：代表未安装探针。

节点图标填充

- 填充：代表 1 小时内发生了调用。
- 不填充：代表 1 小时内无调用。

连线形式：

- ：1 小时内有拓扑关联。
- ：1 小时内无拓扑关联，2 天内有拓扑关联。
- ：service 下的 Workload 或 Pod，service 不直接与 Pod 产生网络连接。

连线颜色：

- 灰色：调用正常。
- 红色：应用层或者进程层的调用关系出现错误产生告警。

3.2.4.4 节点设置

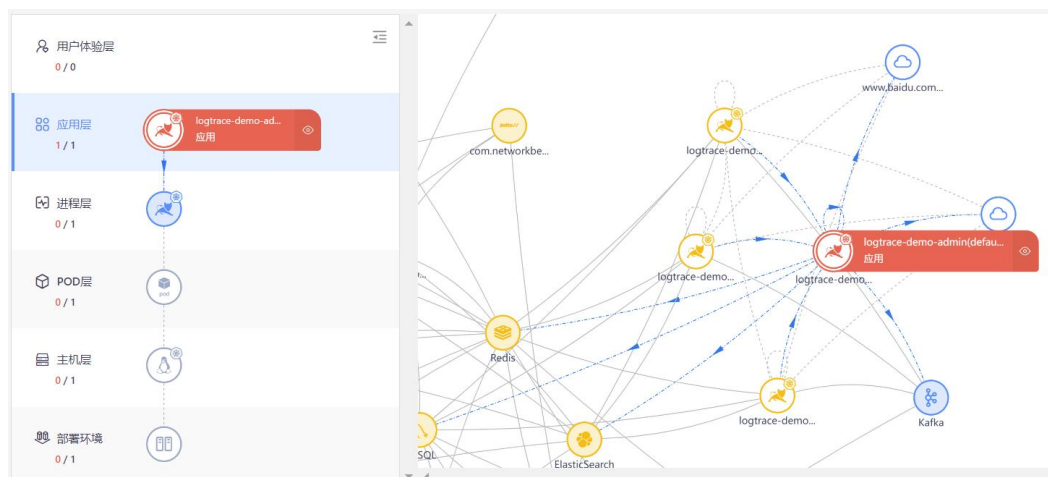
单击页面右上角的  图标，可对拓扑图节点进行以下操作。

- 隐藏孤立节点：默认为关闭状态，即不隐藏没有数据关联的节点。开启后，拓扑图中将不再展示孤立节点。
- 合并 Database：当合并显示时，同一类的数据库在拓扑图中只合并显示为一个。默认为合并显示。
- 合并 NoSQL：当合并显示时，同一类的 NoSQL 在拓扑图中只合并显示为一个。默认为合并显示。
- 合并 MQ：当合并显示时，同一类的 MQ 在拓扑图中只合并显示为一个。默认为合并显示。
- 合并外部服务：当合并显示时，外部服务在拓扑图中合并显示为 **External**，关闭后则按域名来分别显示。默认为合并显示。

- 修改展示节点数：用户可调整拓扑展示的节点数量，范围为 1~500。根因节点和影响节点不受范围限制，必定会展示。默认展示 100 个节点，包括所有故障节点、根因节点和与故障节点产生数据交互的整条链路所有结点。

3.2.4.5 其他说明

- 在横向拓扑图中，单击任意节点，拓扑图会放大到最大状态，该节点、相关联的上下游节点、连线都会被高亮展示，同时连线上也会展示调用方向。在左侧查看其纵向的拓扑关系。



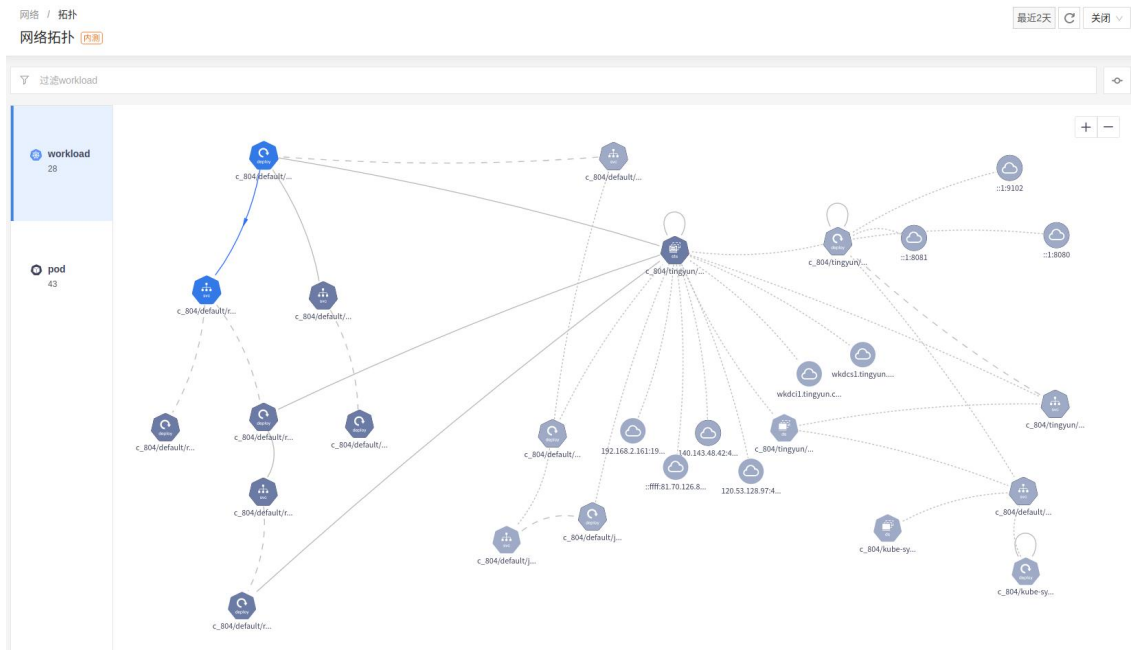
- 在纵向拓扑图中，单击任意节点，右侧横向拓扑图中该节点、相关联的上下游节点、连线都会被高亮展示，同时连线上也会展示调用方向。
- 将鼠标悬浮在节点上，可查看节点的类型和名称。
- 缩放：单击拓扑图右上角的 按钮可放大拓扑图，单击 按钮可缩小拓扑图。
- 全屏：单击拓扑图右上角的 按钮可全屏显示拓扑图。按 ESC 键可退出全屏模式。
- 拖动：在横向拓扑图上按住鼠标左键，可以移动整个拓扑图；在节点上按住鼠标左键，可上下左右移动节点位置。

3.3 网络

基调听云 eBPF 探针可获取网络指标，其集成在 Kubernetes TingyunAgent 之中。Kubernetes TingyunAgent 启用 eBPF 功能后，用户即可在网络分析页面中查看 Pod 与 Pod 的拓扑关系以及之间的网络性能，也可以查看 Workload 与 Workload 的网络拓扑关系以及之间的网络性能。

网络分析页面分别展示 **Workload 层**、**Pod 层** 的拓扑关系。

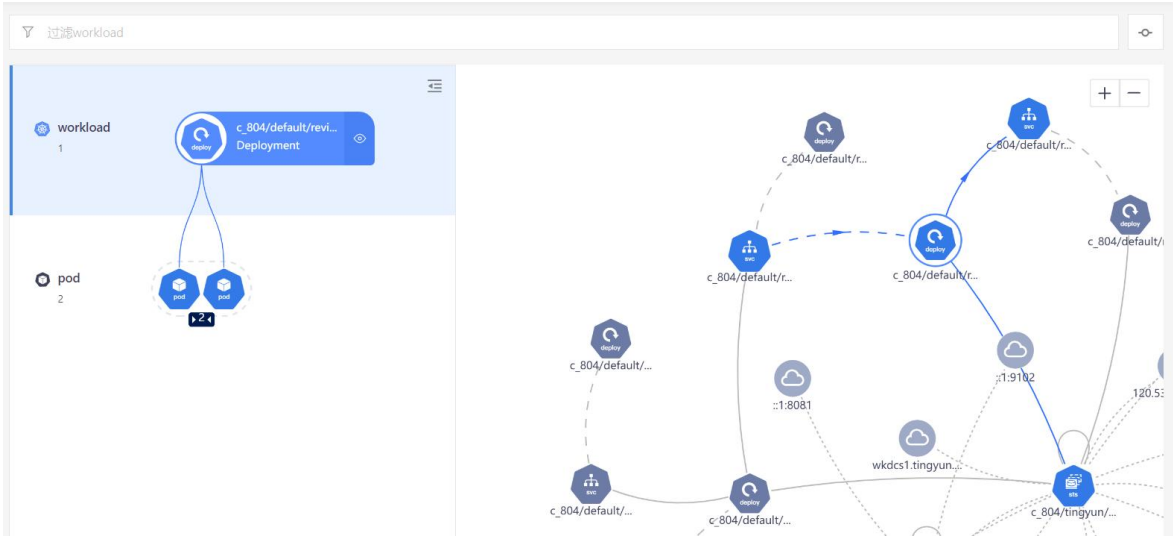
- **Workload 层**展示 Kubernetes 集群中所有 Workload 之间的网络连接关系，包括 Deployment、StatefulSet、DaemonSet 类型的 Workload，以及 Workload 调用外部服务的网络拓扑关系。



- **Pod 层**展示 Kubernetes 集群中所有 Pod 之间的网络连接关系，以及 Pod 调用外部服务的网络拓扑关系。



您在拓扑图中单击任意 Workload 节点，可在左侧查看该 Workload 和其中的 Pod 的拓扑关系，同时右侧会高亮显示与该 Workload 有直接调用和被调用关系的节点。左侧节点图标下显示节点数量，单击图标或者节点数量会展开节点，再次单击节点数量会收起。单击一个 Pod，右侧拓扑图中会高亮显示与该 Pod 有直接调用和被调用关系的节点。



3.3.1网络指标监控

3.3.1.1 Workload 详情

单击 Workload 节点，基本信息右侧会出现图标，单击该图标可展开详情面板，可查看该 Workload 的网络和性能指标。

网络页签上方展示当前 Workload 所调用的 Pod 和服务的入方向吞吐量、出方向吞吐量、TCP 建连时间、TCP 重传次数和 Establish 连接数。如果调用的是 Pod，目的列会展示 Pod 名称，否则展示域名+端口，没有域名则展示 IP+端口。下方展示当前 Workload 被哪些 Pod 和服务所调用，以及入方向吞吐量、出方向吞吐量、TCP 建连时间、TCP 重传次数和 Establish 连接数。

Deployment:c_804/default/reviews-v2 过去1小时 ×

网络 性能 请求

c_804/default/reviews-v2网络调用关系

目的	吞吐量 (In)	吞吐量 (Out)	TCP建连时间	TCP重传次数	Establish连接数
tingyun-collector	11.83 Bytes/s	171.91 Byte...	--	0	0
ratings.default.svc.cluster.local:9080	25.41 Bytes/s	30.27 Bytes/s	78.258 us	0	240

< 1 >

c_804/default/reviews-v2网络被调用关系

源	吞吐量 (In)	吞吐量 (Out)	TCP重传次数	Establish连接数
productpage-v1-5f887878db	24.40 Bytes/s	40.20 Bytes/s	0	240

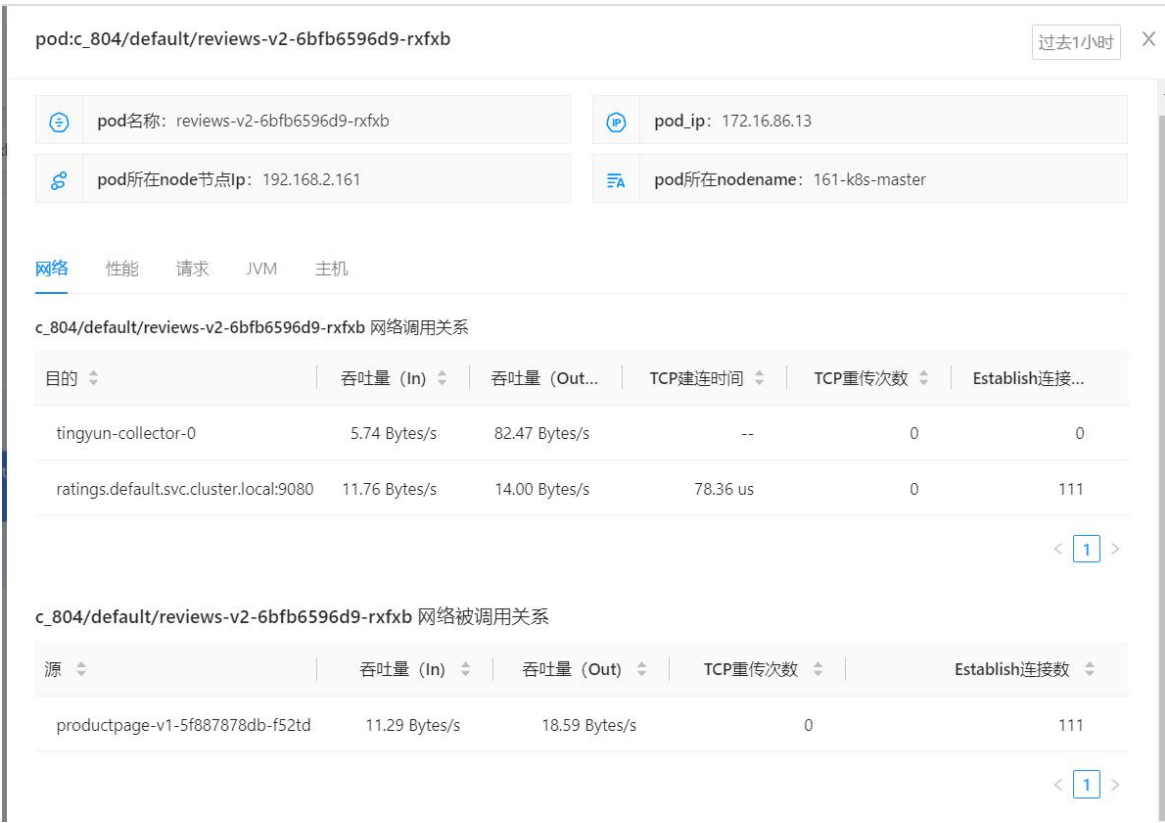
< 1 >

性能页签展示当前 Workload 的 CPU、内存、网络吞吐、磁盘吞吐的变化趋势图，单击右上角的**更多**可到基础设施监控模块查看 Kubernetes 的 Workload 监控详情。

3.3.1.2 Pod 详情

单击 Pod 节点，基本信息右侧会出现图标，单击该图标可展开详情面板，可查看该 Pod 的名称、IP 地址、Pod 所在 Node 的 IP 地址、Pod 所在 Node 的名称、网络指标、性能指标、请求指标、JVM 和主机监控信息。

网络页签上方展示当前 Pod 所调用的 Pod 和服务的入方向吞吐量、出方向吞吐量、TCP 建连时间、TCP 重传次数和 Establish 连接数。如果调用的是 Pod，目的列会展示 Pod 名称，否则展示域名+端口，没有域名则展示 IP+端口。下方展示当前 Pod 被哪些 Pod 和服务所调用，以及入方向吞吐量、出方向吞吐量、TCP 建连时间、TCP 重传次数和 Establish 连接数。

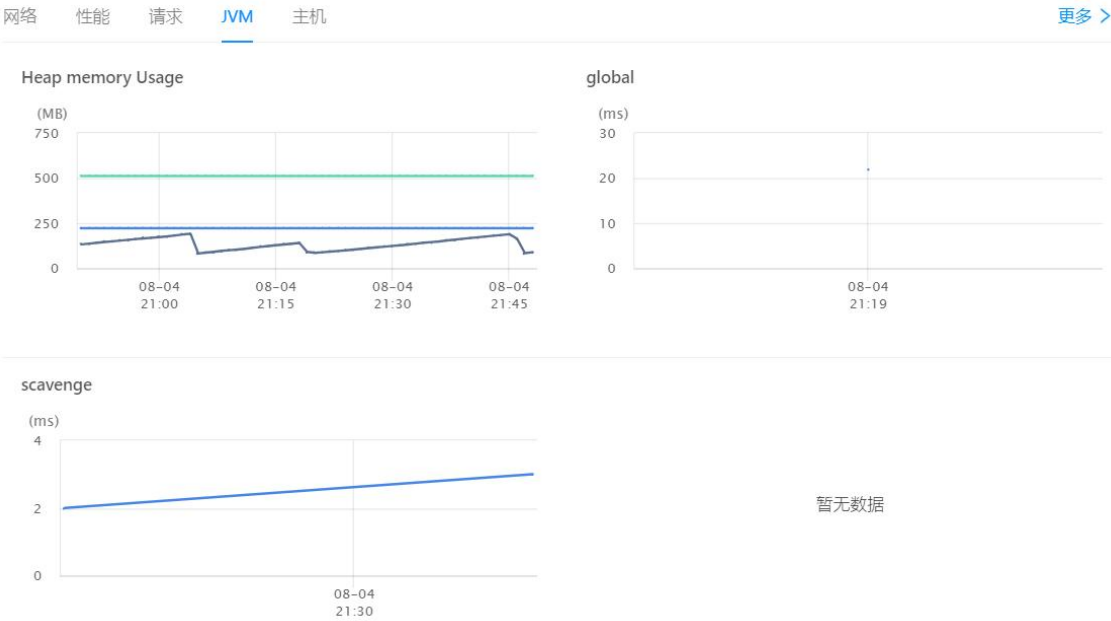


性能页签展示当前 Pod 的 CPU、内存、网络吞吐、磁盘吞吐的变化趋势图，单击右上角的**更多**可到基础设施监控模块查看 Kubernetes 的 Pod 监控详情。

请求页签展示当前 Pod 的所有 Web 请求响应时间中位数、吞吐率和错误频率的变化趋势。单击右下角的**Web 请求分析**按钮，可进入应用与微服务模块查看当前实例的请求分析详情。



JVM 页签展示当前 Pod 所运行在的 JVM 的堆内存使用量、global、Non-Heap/Metaspace、Copy、scavenge 等指标的变化趋势。单击 global 趋势图可进入应用与微服务模块中查看详情。单击右上角的**更多**，可进入应用与微服务模块查看当前实例的 JVM 详情。



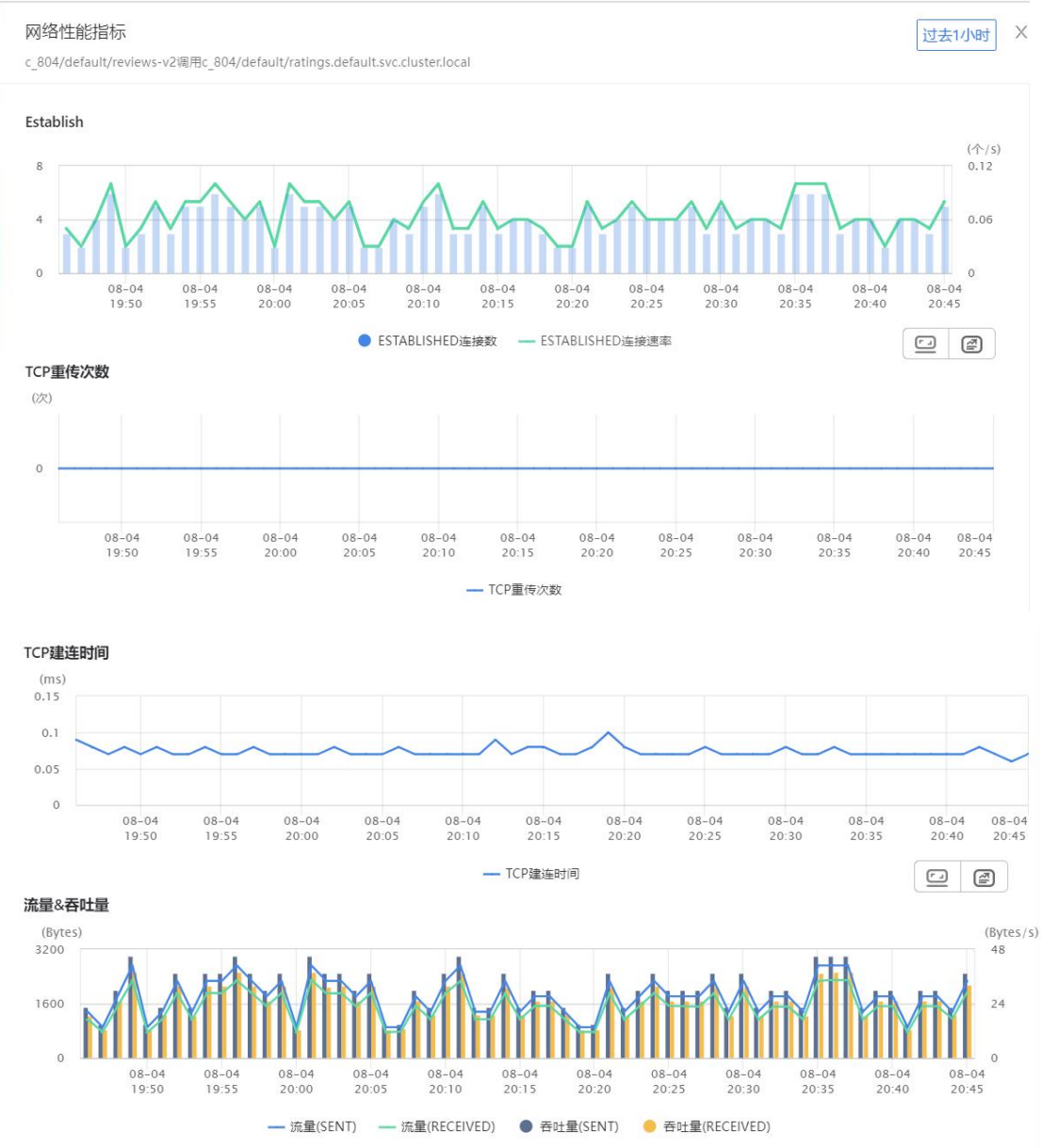
主机页签展示当前 Pod 所在主机的 CPU 使用率、1 分钟平均每核负载、内存使用、Major Page Faul、网络吞吐、磁盘吞吐、IOPS、延迟、IOutil 和分区使用率的变化趋势。具体指标的说明，可参见基础设施产品的[主机指标说明](#)。



3.3.1.3 连线详情

单击 1 小时内有网络数据的对象连线（实线）和 1 小时外 2 天内有网络数据的对象连线（虚线），可查看两个对象之间的网络性能指标，包括已建立的连接数、TCP 重传次数、TCP 建连时间、流量和吞吐量的变化趋势。如果单击 1 小时内有网络数据的对象连线，默认展示最近 1 小时内的统计数据；如果单击 1 小时外 2 天内有网络数据的对象连线，默认展示最近 2 天内的统计数据。

- **Establish** 图表展示统计周期内 Established 连接数和 Established 连接率的变化趋势。
Established 连接率是每秒建立的连接数。
- **TCP 重传次数**图表展示统计周期内 TCP 重传次数的变化趋势。
- **TCP 建连时间**图表展示统计周期内 TCP 建连时间的变化趋势。
- **流量&吞吐量**图表展示统计周期内发送流量、接收流量、发送吞吐量和接收吞吐量的变化趋势。流量的单位为 Byte，吞吐量的单位为 Byte/s。



3.3.2过滤节点

单击页面顶部的过滤框，您可根据 Workload 名称或 Pod 名称来迅速定位想找的节点，支持模糊搜索。设置完成后目标节点将在拓扑图中放大显示，且展示该节点关联的上下游节点以及级联节点，同时在左侧可查看该节点的上下层级关系。

除上述过滤方式外，您还可以直接双击目标节点进行过滤。

3.3.3图例说明

单击页面右上角的  图标，可查看拓扑图的图例。

节点类型：

-  代表 Deployment，即无状态的工作负载。
-  代表 StatefulSet，即有状态的工作负载。
-  代表 DaemonSet，即守护进程工作负载。
-  代表 Pod，Kubernetes 调度最小单元。
-  代表 Service，Kubernetes 中的服务。

节点图标颜色

-  代表 1 小时内有网络数据。
-  代表 1 小时外 2 天内有网络数据。

连线形式：

-  代表对象与对象之间发生实际网络连接（1 小时内）。
-  代表对象与对象之间发生实际网络连接（1 小时外 2 天内）。
-  代表 Service 下的 Workload 或 Pod，Service 不直接与 Pod 产生网络连接。

3.3.4操作说明

- 在拓扑图中，单击任意节点，该节点、相关联的上下游节点、连线都会被高亮展示，同时连线上也会展示调用方向。



- 将鼠标悬浮在节点上，可查看节点的类型和名称。
- 缩放：单击拓扑图右上角的  按钮可放大拓扑图，单击  按钮可缩小拓扑图。
- 拖动：在拓扑图上按住鼠标左键，可以移动整个拓扑图；在节点上按住鼠标左键，可上下左右移动节点位置。

3.4 业务系统

应用与微服务引入了业务系统和服务组的概念，以便更好地组织和管理被监控的对象，并实现业务监控。

- 业务系统是指共同完成某一类特定业务（例如电商系统、OA 系统、电子邮件系统等等）的一组应用的集合。业务系统中可以包含一到多个应用，每个应用只能归属于一个业务系统。
- 服务组指一组应用的集合，是完成某一类特定业务的某个功能模块。例如电商系统下的账户模块，OA 系统下的权限管理模块。服务组中可以包含一到多个应用，每个应用只能归属于一个服务组。

3.4.1 新建业务系统

用户可以将属于共同完成同一个业务的相关应用加入新建的业务系统中。目前支持两种方式创建业务系统，分别是通过控制台创建和通过配置文件创建。

3.4.1.1 控制台创建

请按照以下步骤新建业务系统：

1. 在左侧导航栏中单击 **APM>业务系统**，进入业务系统全局拓扑图页面。
2. 单击左上角的**新建业务系统**按钮。
3. 在弹出的**创建新业务系统**对话框中进行配置。

- 业务系统名称：输入新业务系统的名称，支持任意字符。
- 从左侧待选应用列表中，选择隶属于新业务系统的应用，单击图标加入右侧列表。
反之，单击图标可取消选择。两侧列表均支持全选。

4. 单击**确定**完成创建。

新创建的业务系统将会根据字典排序规则显示在**业务系统区**，并展示在拓扑图和业务系统列表中。单击名称右侧的编辑图标，可修改该业务系统的名称、更改所包含的应用和删除业务系统。用户也可以选择在不同的业务系统之间互相迁移应用。

3.4.1.2 配置文件创建

用户可通过 TingyunAgent 的 oneagent.conf 配置文件，在控制台中自动创建业务系统，该 TingyunAgent 默认将上传数据到此业务系统。

创建方式：在 TingyunAgent 第一次启动前，修改文件/opt/tingyun-oneagent/conf/oneagent.conf 中 default_business_system 配置项。默认值为 default。如果填写的是一个新的业务系统，将新创建并展示在业务系统列表或拓扑中，以及 **TingyunAgents 管理>新增**页面的**业务系统**下拉菜单中；如果填写的是一个已存在的业务系统，可指定 TingyunAgent 默认上传数据的业务系统。

```
#----- 容器配置项 -----  
#  
# 是否启用alpine容器版本配置  
# x86_64 环境缺省值 true  
# aarch64 环境缺省值 false  
# enabled_alpine=true  
  
# 业务系统名称，不能包含空格，中文名称必须为UTF-8编码  
# 类型:字符串  
# 默认值:default  
default_business_system=default
```

TingyunAgent 启动后，如果您想修改探针所属的业务系统，可在控制台中直接修改即可。

3.4.2 新建服务组

用户可以将完成某一类特定业务的某个功能模块的一组应用加入到一个服务组中。服务组必须隶属于业务系统。

请按照以下步骤新建服务组：

1. 在左侧导航栏中单击 **APM>业务系统**，进入业务系统全局拓扑图页面。
2. 在左侧操作面板区，单击新服务组所要隶属于的业务系统。
此时**新建业务系统**按钮的右侧将出现**新建服务组**按钮。
3. 单击**新建服务组**按钮。
4. 在弹出的**服务组**对话框中进行配置。

名称：支持输入任意字符，最多可输入 128 个字符。

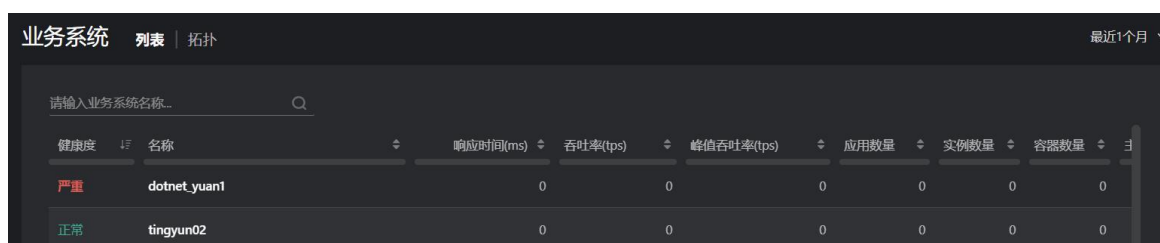
5. 单击**确定**完成创建。

新创建的服务组将会根据字典排序规则显示在所属的业务系统层级下。单击名称右侧的编辑图标，可修改该服务组的名称和删除服务组。用户也可以选择在不同的服务组之间互相迁移应用。

3.4.3 业务系统列表

业务系统页面分别以列表和拓扑图的形式展示当前监控的所有业务系统及其相关的性能指标数据，在页面的左上角单击**列表**或**拓扑**页签可进行切换。默认情况下，系统以拓扑图的形式展示。

业务系统列表支持名称关键字查询和按不同指标项进行排序。单击**修改**可对业务系统中的应用进行添加或移除。如果业务系统连续 7 天无数据，那么该列中会显示**删除**链接，您可以单击来删除该业务系统。



健康度	名称	响应时间(ms)	吞吐量(tps)	峰值吞吐量(tps)	应用数量	实例数量	容器数量
严重	dotnet_yuan1	0	0	0	0	0	0
正常	tingyun02	0	0	0	0	0	0

当健康度显示为“严重”或者“警戒”时，可单击健康度进入**健康度分析**页面，查看该业务系统的健康详情。具体请参见**健康度分析**。

3.4.4 业务系统全局拓扑

在**业务系统**页面的左上角单击**拓扑**页签，可以切换到业务系统全局拓扑图页面。业务系统全局拓扑以图标和连线的形式展示各个业务系统和未分组的应用之间复杂的调用关系，并展示相关性能指标，当出现性能问题时能帮助用户迅速定位故障所在。

业务系统和服务组由用户定义，具体请参见**新建业务系统**和**新建服务组**。缺省状态下部署了探针的应用将显示在操作面板的**应用区**，不属于任何业务系统和服务组。

业务系统拓扑界面包括三大区域：左侧的操作面板区，中间的拓扑图展示区和右下角的按钮区。如下图所示：



- 操作面板区：上方业务系统区展示创建的业务系统和服务组，下方应用区展示未被加入到业务系统或服务组的应用。
 - 用户可以将应用区的应用拖拽到业务系统区的业务系统或服务组中。
 - 单击目录中的业务系统，系统会显示**业务系统拓扑图**。单击目录中的服务组，系统会显示**服务组拓扑**。
 - 在业务系统区的搜索框中输入关键字可搜索业务系统、服务组和已分组的应用。
 - 在应用区的搜索框中输入关键字可搜索未分组的应用。
 - 底部的**隐藏 7 天无数据的应用**复选框默认为勾选状态，拓扑图中将不再展示 7 天内无数据的应用，取消勾选后将恢复显示，如果此时刷新页面，会恢复为勾选状态。
 - 单击该区右侧的箭头可使操作面板区隐藏，再次单击可展开。



- 拓扑展示区：默认展示所有业务系统、未分配应用之间的相互调用关系。

- 业务系统由七个六边形图标表示。应用由一个六边形图标表示。
- 图标的颜色代表健康度，状态为正常时显示绿色，状态为警戒时显示黄色，状态为严重时显示红色，状态为无数据时显示灰色，业务系统中无服务组或应用时显示蓝色。
- 当业务系统触发了警报时，图标右上角会显示铃铛，单击铃铛可进入告警列表查看警报详情。
- 业务系统和业务系统、业务系统和未分配应用之间连线上展示三个数据。第一个是统计周期内源应用调用目的应用的平均响应时间（s），第二个是统计周期内源应用调用目的应用的吞吐率（即平均每分钟目的应用被调用的次数），第三个是统计周期内源应用调用目的应用的错误率。
- 将鼠标悬浮于业务系统、应用图标上，会出现指标悬浮窗。当业务系统的健康度条形图显示为红色或黄色时，即健康度为严重或警戒时，可单击条形图进入健康度分析页面，查看该业务系统的健康详情。具体请参见健康度分析。
- 单击业务系统的名称进入业务系统内拓扑页面，可查看本业务系统内部的服务组、应用、组件之间的相互调用关系，系统还会自动展示和本业务系统内部应用有直接调用关系的业务系统。
- 单击服务组的名称进入服务组拓扑页面，可查看本服务组内部的应用、组件之间的相互调用关系，系统还会自动展示和本服务组内部应用有直接调用关系的业务系统。
- 单击连线会以弹出对话框窗口的形式展示业务系统与业务系统、业务系统与服务组、业务系统与应用之间的上下游服务调用信息和事务追踪信息。

业务系统-业务系统

上游服务 | 下游服务 | 追踪

事务/服务接口	类型	应用	响应时间	平均响应时间(ms)	平均吞吐率(tps)	峰值吞吐率(tps)
#0		sun1.8-192.168.2.207-8071-ku...		3002	0	
#0		apache-tomcat-8.0.36		3000	0	

- 按钮区：通过各个按钮对拓扑图进行操作。

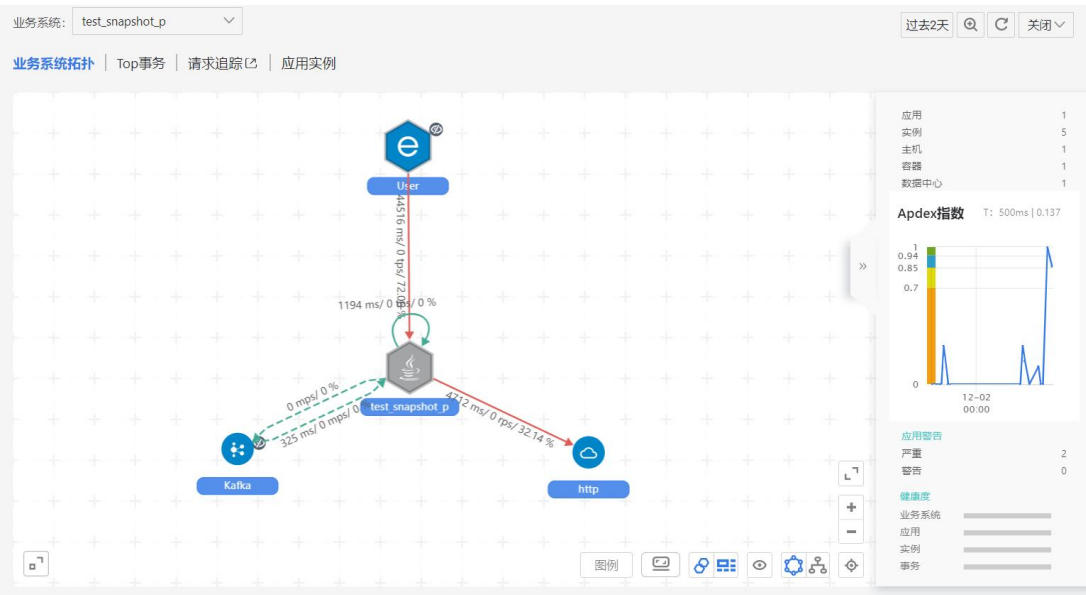
-  图例：展示图例说明。
- ：以圆型的形式展示拓扑。
- ：以树型的形式展示拓扑。
- ：以全屏模式展示拓扑。按 ESC 键可退出全屏模式。

- : 整个拓扑图最大可放大到标准大小的 1.6 倍。拓扑图默认展示的大小为标准大小的 0.85 倍。
- : 整个拓扑图最小可缩小到标准大小的 0.6 倍。当缩小到标准大小的 0.6 倍时，将不再显示文字。
- : 单击该按钮后，拓扑图将恢复到默认的大小，即标准大小的 0.85 倍。
- 缩略图：通过缩略图可以观察当前所显示的拓扑图部分在整个拓扑图中的位置和大小。

3.4.5 业务系统详情

在业务系统全局拓扑图页面，用户可以单击拓扑展示区中的业务系统名称或图标进入业务系统的详情页面。

业务系统详情页面展示当前业务系统的拓扑图、业务系统基本指标、健康度和指标趋势图。还可以通过上方的页签切换快速访问当前业务系统的 Top 事务、请求追踪和应用实例列表。



● 业务系统拓扑图

业务系统拓扑图展示当前业务系统中活跃应用、服务组件以及其他相关的业务系统之间的逻辑调用关系，以及前端浏览器（显示为 User，即用户发起的请求）、Web 产品和 App 产品对应用的访问情况。每个服务组、应用、服务组件和客户端都以图标形式展示，并以带有箭头的连线来展示应用之间、应用与服务组件以及应用与业务系统等等之间的调用关系。

- 服务组由三个六边形图标表示.

- 图标的颜色表示所代表的监控对象的健康度，状态为正常时显示绿色，状态为警戒时显示黄色，状态为严重时显示红色，状态为无数据时显示灰色，服务组中无应用时显示蓝色。
- 当服务组触发了警报时，图标右上角会显示铃铛，单击铃铛可进入告警列表查看警报详情。
- 将鼠标悬浮于业务系统、服务组、应用、外部接口、服务组件和前端图标上，会出现指标悬浮窗。
- 单击应用、服务组件和前端图标右上角的隐藏图标，可以隐藏当前服务组件或者同类服务组件。当组件数量超过 50 个时，系统会根据响应时间从高到低排序，仅展示前 50 个组件。
- 连线的粗细根据调用的对象的吞吐率大小而不同，以拓扑图上同类型连线（如前端到应用、应用到应用等等）的最大吞吐率和最小吞吐率为上下边界划分为 5 档粗细，吞吐率越大线越粗。单击连线会以弹出对话框窗口的形式展示应用与各种组件、应用与应用、应用与业务系统之间的详细调用事务、服务接口、SQL 查询和相关的错误、追踪信息。



- 连线颜色表示的意义如下：

颜色	说明
绿色	正常
黄色	下游应用相关事务错误率>=3%
红色	下游应用相关事务错误率>=5%

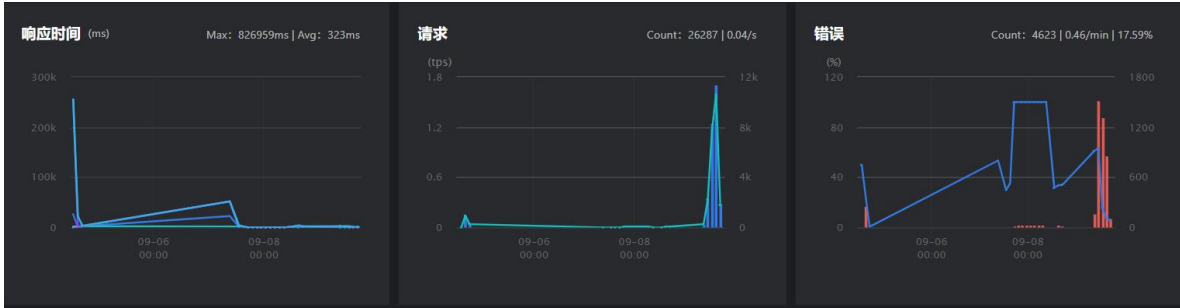
- 服务组和应用之间、应用和应用之间、服务组件和应用之间、外部接口和应用之间连线上展示三个数据。第一个是统计周期内源应用调用目的应用的平均响应时间（s），第二个是统计周期内源应用调用目的应用的平均吞吐率（即平均每分钟目的应用被调用的次数），第三个是统计周期内源应用调用目的应用的平均错误率。
- 前端和应用之间的连线上展示两个数据。第一个是统计周期内该类客户端访问目标对象时应用服务器的平均响应时间（s），第二个是统计周期内该类客户端访问目标对象的吞吐率（即平均每分钟目标对象被访问的次数）。
- 拓扑图可使用鼠标滚轮进行缩放，在缩放的同时，可在左下角的全局缩略图观察当前所显示的拓扑图部分在整个拓扑图中的位置和大小。通过拓扑图右下角的操作按钮还可对拓扑图进行以下操作：

- 缩放：缩放整个拓扑图，当前可见区域与整个拓扑图的相对关系和大小在左下角的缩略图上展示。
- 复位：以缺省的缩放比例显示拓扑图。
- 全屏：以全屏显示拓扑图。按 ESC 键可退出全屏模式。
- 布局：可选圆形和树型结构两种拓扑图布局。
- 隐藏的应用和组件：显示被隐藏的应用、业务系统、服务组件、前端和外部接口图标列表，并可以重新在拓扑图上显示出来。
- 展示级联业务系统：切换显示或隐藏与当前业务系统相关（由当前业务系统的事务所访问）的其他业务系统图标。
- 合并同类组件：切换合并或单独显示同一类型的服务组件图标，当单独显示时，按每个服务组件的实例显示当前业务系统中调用的每个服务组件。当合并显示时，同一类的服务组件在拓扑图中只合并显示为一个，且相应的数据也会合并显示。默认为合并显示。
- 图例：显示当前业务系统拓扑图的图例说明。

- 业务系统基本信息

在业务系统拓扑图右侧展示当前业务系统的基本信息，包括：

- 应用：当前业务系统中的活跃应用数量。
 - 实例：当前业务系统中活跃应用的实例数量。
 - 主机：当前业务系统中活跃应用所在的主机数量。
 - 容器：当前业务系统中活跃应用所在的容器数量。
 - 数据中心：当前业务系统中活跃应用所在的数据中心的数量。
 - Apdex 趋势图：当前业务系统中所有应用的平均 Apdex 指数的变化趋势。右上角显示用于计算 Apdex 指数的 Apdex T 和统计时间周期内的平均 Apdex 指数值。
 - 应用警告：当前业务系统中所有严重级别和警告级别的警报数量。
 - 健康度：以条形图的形式展示当前业务系统中业务系统自身、应用、实例和事务的健康度，以及各种健康状态的对象数量和百分比。当业务系统或事务的健康状态存在严重或警告时，可单击查看健康度分析详情。具体请参见[健康度分析](#)。
- 基本指标趋势图：提供当前业务系统的响应时间、请求和错误三个重要指标的历史趋势图。



- 响应时间：显示当前业务系统的平均响应时间和 4 个自定义分位值响应时间的历史曲线，分位值的计算方法请参见[分位值](#)。鼠标悬停在曲线上时显示当前时间点的对应响应时间指标数值并在其他两个趋势图中联动显示其他指标。右上角展示统计时间段内业务系统的平均响应时间。
- 请求：显示吞吐率的历史趋势图和请求数的条形图，并在其他两个趋势图中联动显示其他指标。
- 错误：显示当前业务系统事务错误率的趋势图和错误数的条形图，并在其他两个趋势图中联动显示其他指标。右上角的 3 个数值分别代表统计时间段内发生的错误总次数、每分钟的错误数和平均错误率，不含异常的统计。

在业务系统详情页面，还可以通过上方的页签切换快速访问当前业务系统的 Top 事务、事务追踪和应用实例列表。

3.4.6 服务组拓扑

用户可以通过以下三种方式进入服务组拓扑图页面：

- 在业务系统拓扑图页面中单击左侧的服务组名称
- 业务系统拓扑图页面单击服务组名称
- 业务系统拓扑图页面单击服务组图标

服务组拓扑图展示当前服务组内的应用之间、应用和服务组件之间、应用和其相关的其他业务系统之间的相互调用关系，以及浏览器（显示为 User，即用户发起的请求）、Web 产品和 App 产品对应用的访问情况。

- 图标的颜色代表[健康度](#)，状态为正常时显示绿色，状态为警戒时显示黄色，状态为严重时显示红色，状态为无数据时显示灰色。
- 连线颜色表示的意义如下：

颜色	说明
绿色	正常
黄色	下游应用相关事务错误率>=3%

- 单击页面右上角的**返回旧版**，可进入旧版应用页面。如需返回新版应用页面，单击**体验新版**即可。

 新版应用分析（Beta），需要Collector 3.6.6.0+，[返回旧版](#)

- 单击列表右上角的自定义表头图标，可以勾选想要展示的列名，部分字段必选。
- 单击列表右上角的**导出**按钮，可将应用列表导出为 CSV 格式。当设置了过滤条件时，导出的是过滤后的应用列表。
- 在列表右上角的下拉菜单中，可调整列表中应用数据的统计范围。
 - 当前值：仅在选择最近时间（快速查询方式）时生效，表示过去 5 分钟内最后一个时间点的值。
 - 整个时间范围内的值：所选时间范围内，有数据时间段的值。
- 单击右侧**操作**列的“...”，可以对目标应用进行多维观测和分析，可查看**拓扑图**、**调用者分析**、**错误分析**、**响应时间分布**、**OneTraces** 和**异常分析**。

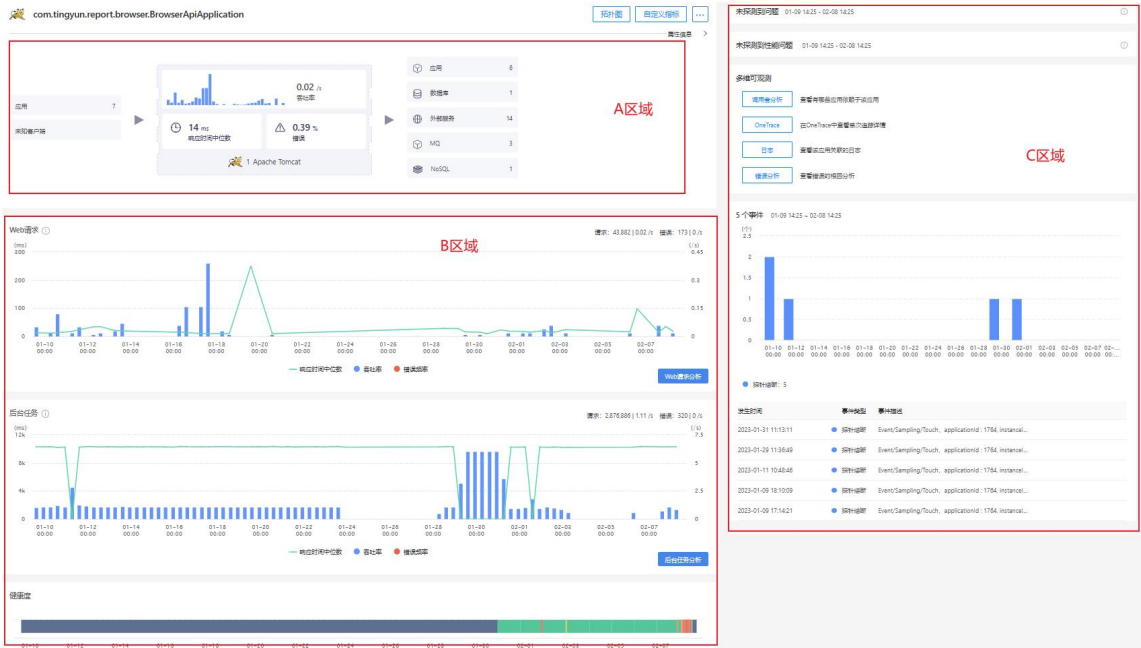
错误率	错误次数	操作
0.03 %	1	...
3.03	02-10 13:51 ~ 02-10 14:21	...
	多维可观测	...
	拓扑图	...
	调用者分析	...
	错误分析	...
	响应时间分布	...
	OneTraces	...
	异常分析	...
55.56		...

- 当健康度列显示为“严重”或者“警戒”时，可单击健康度进入**健康度分析**页面，查看该应用的健康详情。

3.5.1 应用/实例概览

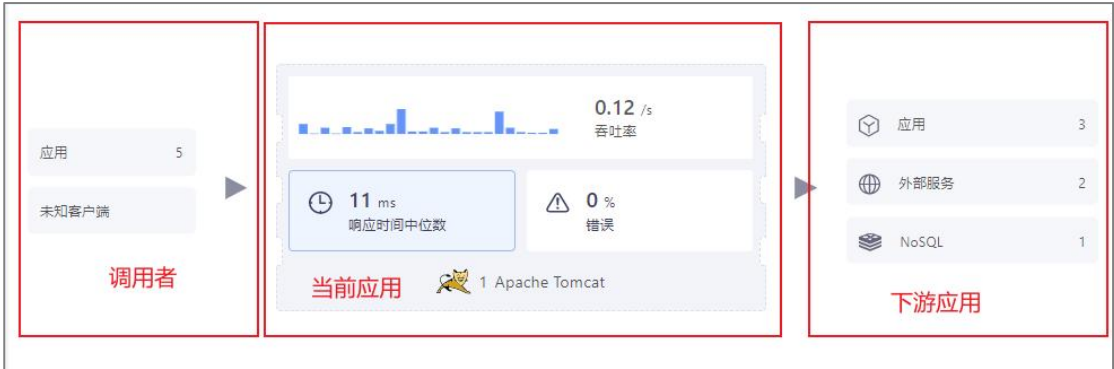
3.5.1.1 应用概览

在应用列表页面单击应用名称，可以进入目标应用的概览页面。概览页面包括 A、B、C 三个区域。



3.5.1.1.1 A 区域

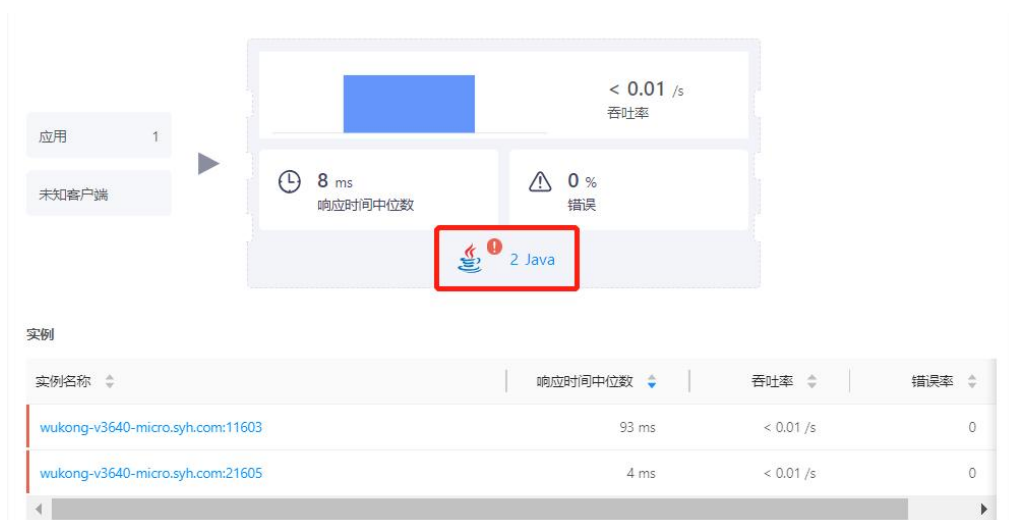
A 区域展示当前应用的调用关系，从左依次是调用者、当前应用、下游应用或服务组件。



在该区域中，您可以进行以下操作：

- 应用名称默认展示应用的别名，如果没有设置应用别名则展示应用名称。
- 单击应用名称右侧的**拓扑图**按钮，可查看应用拓扑；单击**自定义指标**按钮，可查看该应用下配置的自定义指标性能数据；单击“...”可进行应用配置、线程剖析和异常检测配置。
- 单击“...”下方的**属性信息**，可查看应用名称、应用别名、业务系统、进程技术栈、服务组、Namespace、Workload 信息。只有在 Kubernetes 平台部署的探针，应用才会显示 Namespace、Workload 两个属性信息。
 - 单击业务系统名称进入指定业务系统分析页面。
 - 单击**添加标签**可以添加一些属性信息。

- 调用者区域展示调用当前应用的应用及个数，应用包括 App、Web、小程序、应用、未知客户端五种。
 - 单击调用者应用个数，下方会显示调用者应用列表。单击列表中的应用名称可进入该应用的概览页面。单击**操作**列的“...”，选择**请求分析**进入应用详情页面。
 - 当调用者为未知客户端时，不支持点击。
- 当前应用区域展示当前应用的吞吐率、吞吐率柱状图、响应时间中位数、错误率、实例数量及技术栈。
 - 单击吞吐率、响应时间中位数、错误率任意一项都会进入应用详情页面。
 - 单击实例数及技术栈，下方显示对应的实例列表信息。当实例存在持续中的疑似问题时，技术栈图标处会显示红色叹号，实例列表左侧存在疑似问题的实例会显示红色竖线。单击列表中实例名称进入实例概览页面，具体描述可参考[实例概览](#)。单击**操作**列的“...”，可进行多维观测，包括请求分析、拓扑图分析、JVM 分析、查看环境信息、线程剖析。



- 当有多个技术栈时显示最后一个上传实例的技术栈。
- 下游应用区域展示当前应用所调用的下游应用或服务组件信息，包括应用、外部服务、数据库、NoSQL 和 MQ。单击后，下方显示对应的列表信息，单击列表中的名称可进入其概览页面。单击列表**操作**列的“...”，选择**请求分析**进入应用详情页面。

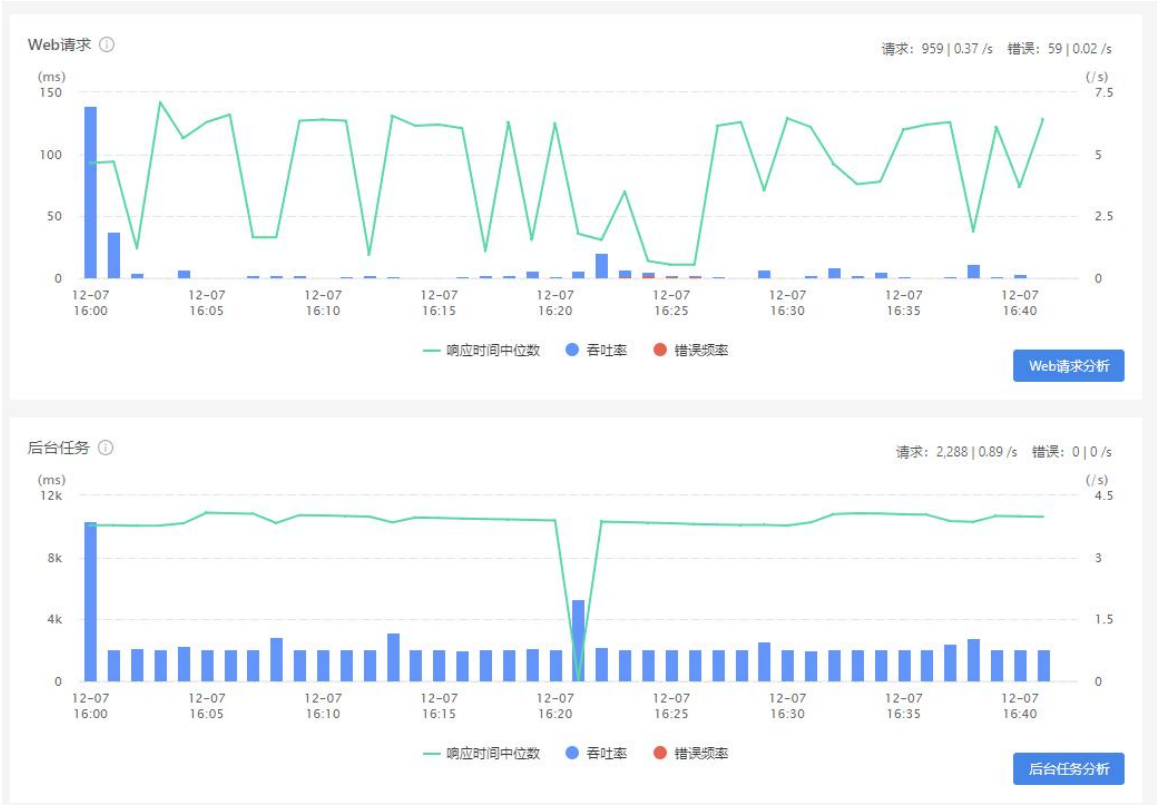
3.5.1.1.2 B 区域

B 区域展示当前应用的 Web 请求分析图表、后台任务分析图表以及健康度条形图。此处的 Web 请求是指由程序外部发起、使用 TCP/IP 协议传输的网络请求，包括当前应用所属事务和服务接口。指标数据来自服务端。

Web 请求分析和后台任务趋势图展示当前应用的请求响应时间中位数、吞吐率以及错误频率三个重要指标的历史趋势图。鼠标悬停在曲线上时，可查看当前粒度时间的响应时间中位数、吞

吐率和错误频率。图的右上角从左到右依次展示该应用在统计周期内的总请求次数、吞吐率、错误次数和错误频率。

单击图表右下角的 **Web 请求分析** 按钮，进入应用详情页面，仅展示 Web 请求的统计数据；同理，单击**后台任务分析**按钮，仅展示后台任务的统计数据。



系统以条形图的形式展示当前应用的健康度变化。当健康状态存在严重或者警告时，可以单击条形图的严重或者警告部分查看[健康度分析详情](#)。



3.5.1.1.3 C 区域

- 疑似问题：展示所选时间范围内应用的疑似问题，包括持续中和已关闭的，持续中的问题显示为红色，已关闭的显示为灰色。您可以查看问题的问题 id 、问题描述、影响对象、开始结束时间、持续时间。疑似问题可能是数据库响应时间恶化、错误率增加、应用存活探

针数过低等。默认展开最新的 6 个持续中的疑似问题，折叠已关闭的问题。单击问题可查看疑似问题根因和影响范围。

- 多维可观测：从不同维度查看统计时间范围内当前应用的分析数据，包括调用者分析、OneTrace 分析、日志分析和错误分析。
 - 调用者分析：单击后可查看有哪些应用依赖于该应用。
 - OneTrace：单击后可查看单次追踪详情。
 - 日志：单击后可查看该应用关联的日志。
 - 错误分析：单击后可查看错误的根因分析。
- 事件：所选时间范围内的所有事件，事件的类型包括：进程崩溃、进程重启、探针熔断和应用发布。
 - 柱状图展示事件的个数，支持添加到大屏和智能报告。
 - 列表展示事件类型、对象和事件发生的时间。

3.5.1.2 实例概览

在应用概览页面，单击实例列表中的实例名称进入实例概览页面。该页面是针对实例进行的统计，描述跟应用概览基本一致。下面只介绍不同之处。

B 区域除了展示 Web 请求分析图表、后台任务分析图表以及健康度条形图外，还会展示 JVM、WebServer（仅 Nginx 应用展示）、主机、进程、Container 和 Pod 信息，单击页签进行切换。

- JVM：单击页签右上角的**更多**进入实例 JVM 页面，详情请参见 [JVM](#)。
- 主机：单击页签右上角的**更多**进入主机概览页面，详情请参见《基调听云_基础设施_使用手册》中的**主机监控**小节。
- 进程：单击页签右上角的**更多**进入进程页面，详情请参见《基调听云_基础设施_使用手册》中的**进程监控**小节。
- Container：单击页签右上角的**更多**进入容器页面，详情请参见《基调听云_基础设施_使用手册》中的**Docker 监控**小节。
- Pod：单击页签右上角的**更多**进入 Pod 页面，详情请参见《基调听云_基础设施_使用手册》中的**Kubernetes 监控**小节。

 **说明：**主机、进程、Container 和 Pod 页签需要安装 TingyunAgent 才会展示。

3.5.2 应用详情

应用详情页面包括响应时间分析、错误分析、吞吐率分析、实例分析和请求分析。对于应用的响应时间、错误和吞吐率，系统又可以从服务视角和调用者视角对数据进行分析，默认展示服务视角对应趋势图。

页面数据支持按照请求名称、请求类型、HTTP Method 、HTTP Status Code、实例和数据项过滤。

 **说明：**调用者视角显示的数据暂不支持 HTTP Method 、HTTP Status Code、数据项的过滤。

3.5.2.1 服务视角分析

系统展示响应时间、错误和吞吐率趋势图，单击上方页签可进行切换。

- **响应时间趋势图：**展示指定时间范围内当前应用的响应时间中位数、吞吐率、平均响应时间变化趋势，默认展示 50 分位值，即中位数，在图表右上角可切换分位值。将鼠标悬停在图表上，可查看当前粒度时间的响应时间中位数、平均响应时间以及吞吐率。
- **错误趋势图：**展示指定时间范围内当前应用的错误率、吞吐率、错误频率变化趋势。将鼠标悬停在图表上，可查看当前粒度时间的错误率、吞吐率以及错误频率。
- **吞吐率趋势图：**展示指定时间范围内当前应用的吞吐率、请求、慢请求变化趋势。将鼠标悬停在图表上，可查看当前粒度时间的吞吐率、请求以及慢请求数。

在左下角的下拉菜单中可以选择分析时间范围，分析最近指定时间的数据，并展示总请求次数。可选分析范围会根据您选择的统计时间而变化。当您需要对图表中某一时间段的数据进行细粒度查看时，无需手动调整右上角的统计周期，按住鼠标左键即可在图表中框选时间范围，缩小图表的展示周期，框选时间范围需要大于一分钟。单击图表右上角的返回按钮可恢复之前的统计区间。

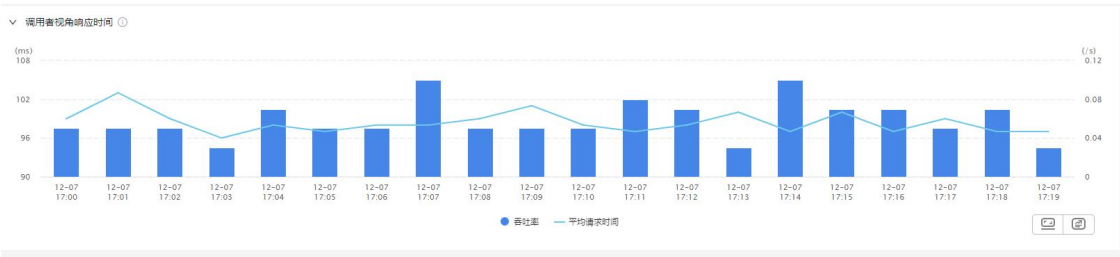
单击下方的[响应时间性能分解](#)、[响应时间分布](#)、[错误分析](#)、[异常分析](#)、[热点方法](#)、[调用者分析](#)和 [OneTraces](#) 按钮可进一步对应用进行分析，查看详情。响应时间我们推荐查看[响应时间性能分解](#)和[响应时间分布](#)；错误推荐查看[错误分析](#)和[异常分析](#)；吞吐率推荐查看[调用者分析](#)。



3.5.2.2 调用者视角分析

默认折叠不展示，单击调用者视角表头展开对应的趋势图。单击服务视角上方的**响应时间**、**错误**和**吞吐率**页签可进行切换。

- 响应时间趋势图：从调用者视角展示当前时间范围平均请求时间和吞吐率的变化趋势。
- 错误趋势图：从调用者视角展示当前时间范围错误率、吞吐率和错误频率的变化趋势。
- 吞吐率趋势图：从调用者视角展示当前时间范围吞吐率和请求数的变化趋势。



3.5.2.3 实例分析

默认折叠不展示，单击实例表头展开列表，可查看实例名称、主机、总耗时、响应时间中位数。

实例				
名称过滤 显示 1 / 1				
实例名称	主机	总耗时	响应时间中位数	操作
VM_2_34_centos8091	VM_2_34_centos	6.82 s	96.00 ms	...

您还可以对列表进行以下操作：

- 鼠标悬停在列表条目上时，可查看当前实例的吞吐率、请求次数、响应时间中位数、响应时间 95 分位值、响应时间 99 分位值和错误率。
- 在搜索框中输入实例的名称，然后单击搜索图标，可查看指定实例的信息。支持模糊搜索，不区分大小写。
- 在**操作**列单击 可将该实例的名称作为过滤条件添加到过滤框中，同时可立即看到过滤后的统计数据。
- 在**操作**列单击“...”，然后单击菜单选项可进行多维观测，并支持单击跳转至主机或进程。

3.5.2.4 请求分析

3.5.2.4.1 Top 请求

Top 请求列表展示 Top 1000 的慢请求相关信息。切换服务视角上方的响应时间、错误和吞吐率页签后，慢请求列表的排序会发生变化，变化如下。

- 响应时间：展示响应时间中位数 Top 1000 的请求。
- 错误：展示错误率 Top 1000 的请求。
- 吞吐率：展示吞吐率 Top 1000 的请求。

列表默认按照响应时间中位数倒排序：

14 个请求 请求命名规则

Top 请求 数据项

名称过滤 显示 14 / 14 导出

名称	错误率	错误频率	错误次数	请求次数	操作
URI/auth-api/routePerms	100.00 %	< 0.01 /s	2	2	...
URI/auth-api/menus	42.86 %	< 0.01 /s	3	7	...
SpringController/getUserPermissionByToken (POST)	17.65 %	< 0.01 /s	3	17	...
SpringController/goLogin	0	0	0	2	...

您可以对 Top 请求列表进行以下操作：

- 鼠标放在请求条目上，可查看当前请求的吞吐率、请求次数、响应时间中位数、99 分位值和错误率。
- 在搜索框中输入 Top 请求的名称，然后单击搜索图标，可查看指定 Top 请求的信息。支持模糊搜索，不区分大小写。
- 单击列表右上角的请求命名规则按钮进入应用设置页面，可配置事务命名，具体描述可参考应用设置。
- 单击列表右上角的导出按钮，可将 Top 请求列表导出为 CSV 格式。

说明：

- 当过滤设置中有过滤条件时，导出的列表为过滤后的结果。
- 当前如果是响应时间 Top 请求，导出列表 CSV 的命名就是“响应时间-Top 请求-报表”，以此类推。
- 单击操作列的 可将该 Top 请求名称作为过滤条件添加到过滤框中，同时可立即看到过滤后的统计数据。
- 在操作列单击“...”，然后单击菜单选项可进行多维观测。

3.5.2.4.2 数据项

数据项列表展示 Top 50 的慢请求相关的数据项信息，每个数据项只显示前 50 个对应值。切换服务视角上方的**响应时间**、**错误**和**吞吐率**页签后，数据项列表的排序会发生变化，变化如下。

- 响应时间：展示响应时间中位数 Top 50 的数据项。
- 错误：展示错误率 Top 50 的数据项。
- 吞吐率：展示吞吐率 Top 50 的数据项。

16 个请求

请求命名规则

Top请求 数据项

名称过滤

显示 3 / 3

名称	吞吐率	请求次数	慢请求	操作
URL	0.47 /s	853	655	
/Unknown	0.39 /s	706	628	
http://10.128.2.95/sense-api/vi/dataSource/component/getData/1/3224	0.02 /s	40	16	
http://10.128.2.95/sense-api/vi/dataSource/component/getData/1/3225	0.01 /s	14	5	
http://10.128.2.95/sense-api/vi/admin/hasPermission	0.01 /s	9	0	
http://10.128.2.95/sense-api/vi/template/list	0.01 /s	9	0	
http://10.128.2.95/sense-api/v2/license/check	0 /s	8	0	

您可以对数据项列表进行以下操作：

- 在搜索框中输入的数据项名称，然后单击搜索图标，可查看指定数据项的信息。支持模糊搜索，不区分大小写。
- 单击操作列的 可将当前页按数据项=value 过滤条件添加到过滤框中，同时可立即看到过滤后的统计数据。
- 单击列表右上角的请求命名规则按钮进入应用设置页面，可配置事务命名，具体描述可参考[应用设置](#)。

3.5.3 Service Flow

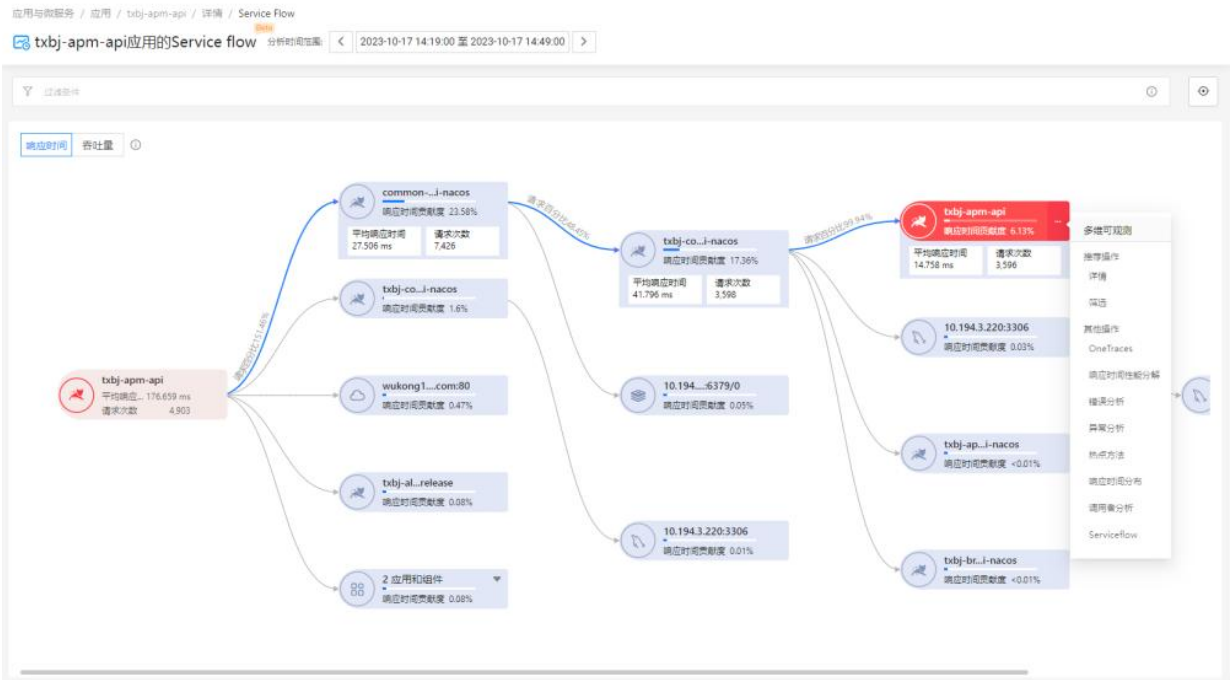
Service Flow 展示了应用程序中应用、组件之间的依赖关系和性能指标，它能帮您：

- 了解应用、组件调用的顺序和依赖关系
- 定位性能瓶颈，查看哪些应用影响了整体的响应时间
- 识别繁忙的应用、组件

下图显示了应用 apm-api 的调用链路以及每个节点对整体响应时间的影响。单机节点显示节点的响应时间贡献条形图、平均响应时间、请求次数。

- 分析时间范围默认是最近 30 分钟，用户可在页面顶部分析时间范围处修改时间范围。时间范围不能超过 24 小时。
- 响应时间贡献度=当前节点的总响应时间/根节点的总响应时间。

- 请求百分比=当前节点的请求次数/父节点的请求次数。
- 单击右侧操作列的“...”，可以对目标应用进行多维观测和分析，包括详情、OneTraces、响应时间性能分解、错误分析、异常分析、响应时间分布、调用者分析。

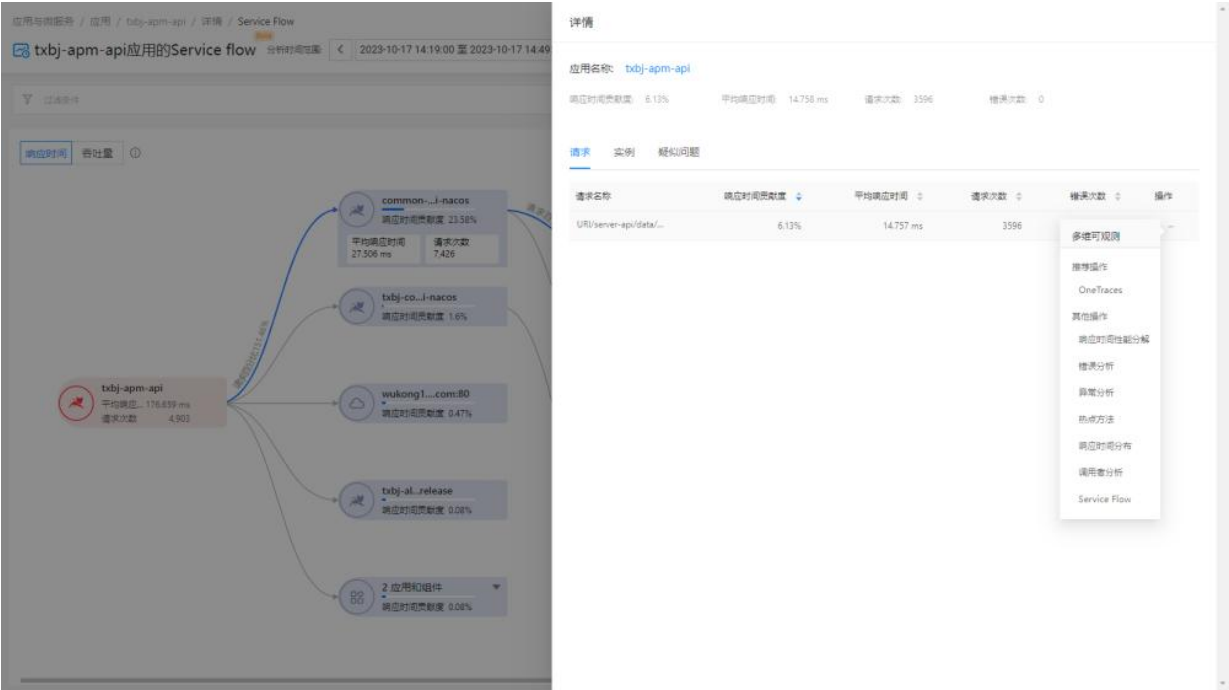


3.5.3.1 详情

单击详情后，右侧会展示调用了该应用的详情，包括调用该应用的请求、实例、疑似问题。

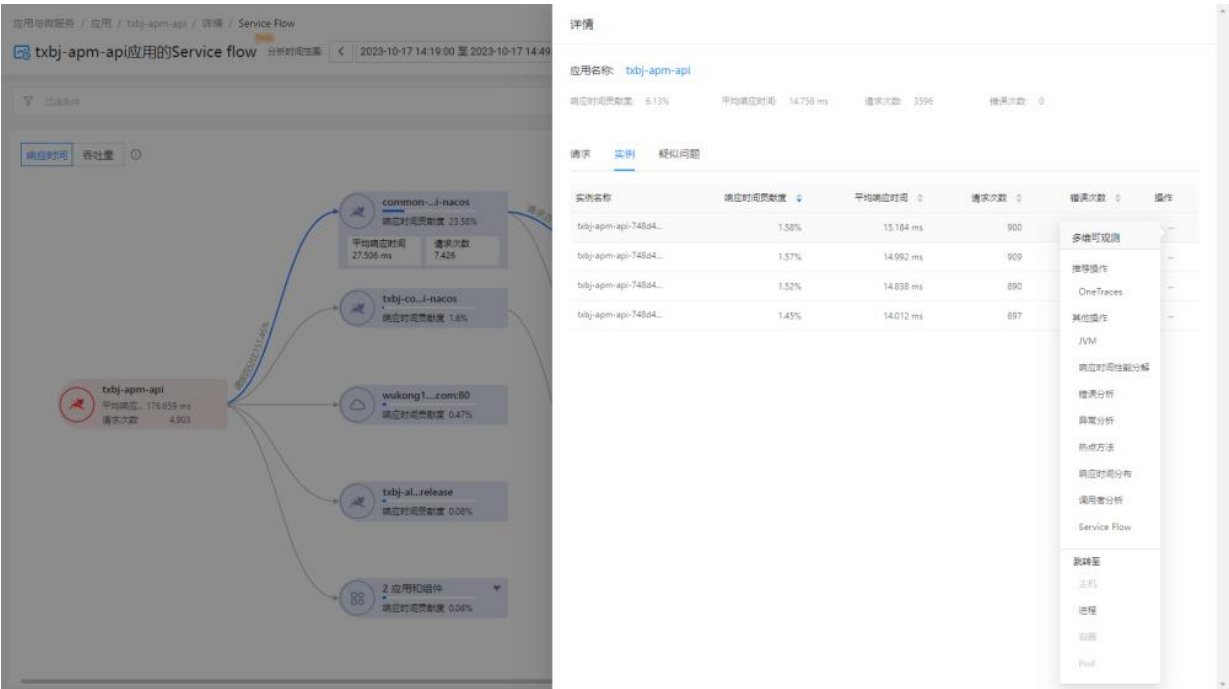
3.5.3.1.1 请求

在请求页签中，展示调用所选应用的请求。



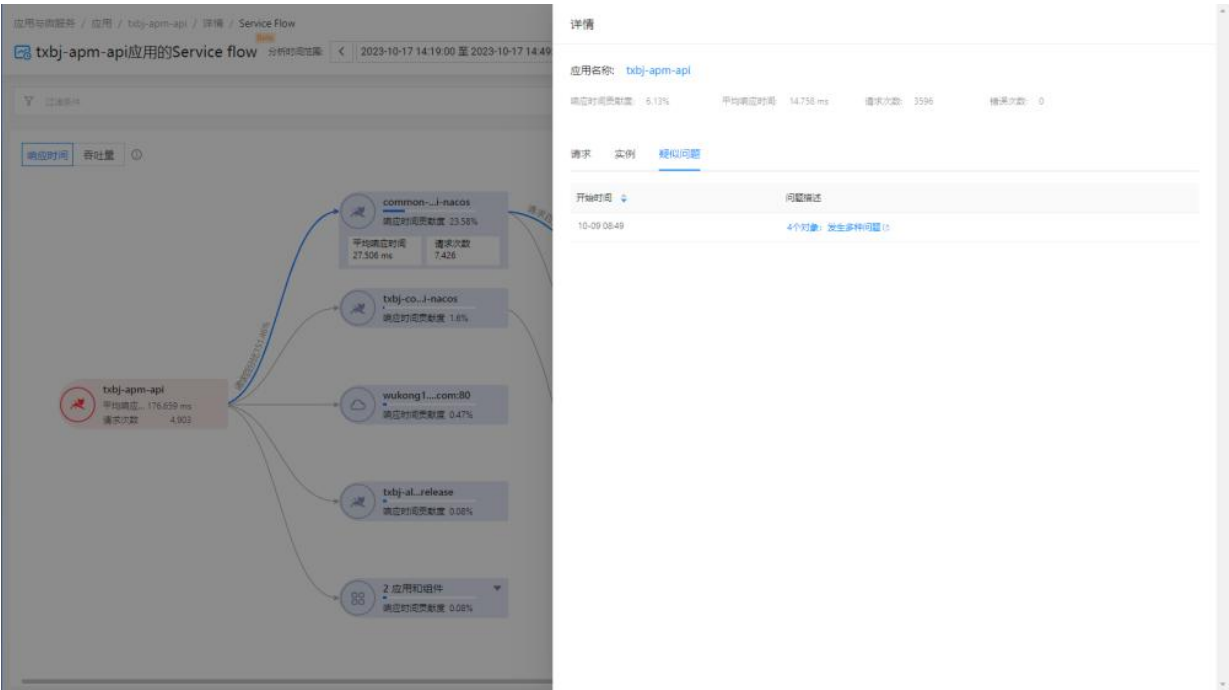
3.5.3.1.2 实例

实例页签展示所选应用的实例信息，包括实例名称、响应时间贡献度、平均响应时间、请求次数、错误次数。



3.5.3.1.3 疑似问题

疑似问题页签展示所选应用的持续中的疑似问题，单击问题描述可查看疑似问题详情。



3.5.4 响应时间性能分解

在应用详情页面单击详情部分的响应时间性能分解进入该页面。

- 修改分析时间范围：该页面中，所有图表的分析时间范围是应用详情页面服务视角下选择的分析范围，可在页面顶部分析时间范围处修改时间范围。

说明：如果时间跨度超过 1 天，开始时间修改为结束时间减 1 天。

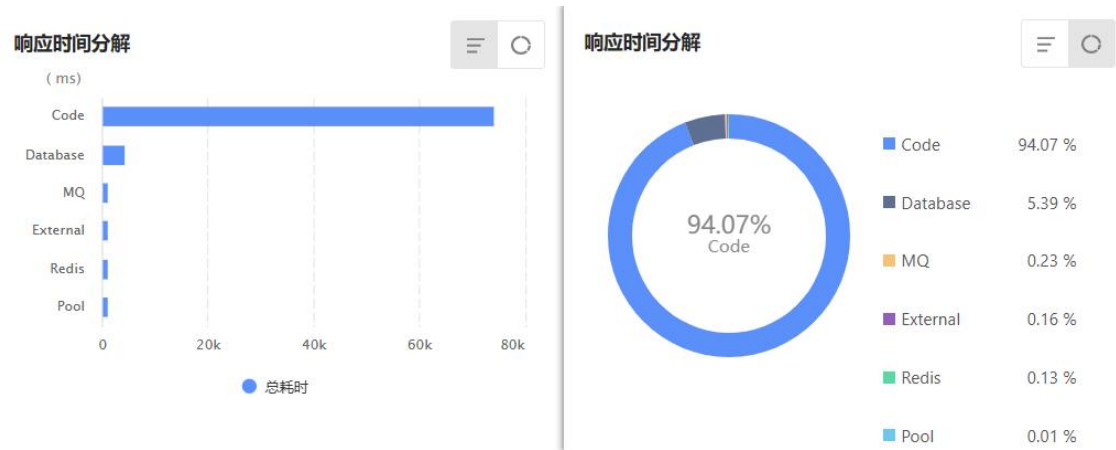
- 过滤：单击页面顶部的过滤框，弹出过滤条件，选择想要查询的条件，立即可看到过滤后的统计数据。过滤条件包括：实例名称、请求名称和请求类型。

3.5.4.1 响应时间分解

以 Bar 图（条形图）和环形图的方式展示应用响应时间的构成部分分别所占用的时间。图形默认展示 Bar 图（条形图）。

- Database：当前应用依赖的数据库组件。
- Code execution：当前应用代码执行。
- Redis：当前应用的缓存数据库组。
- External：当前应用依赖外部组件。

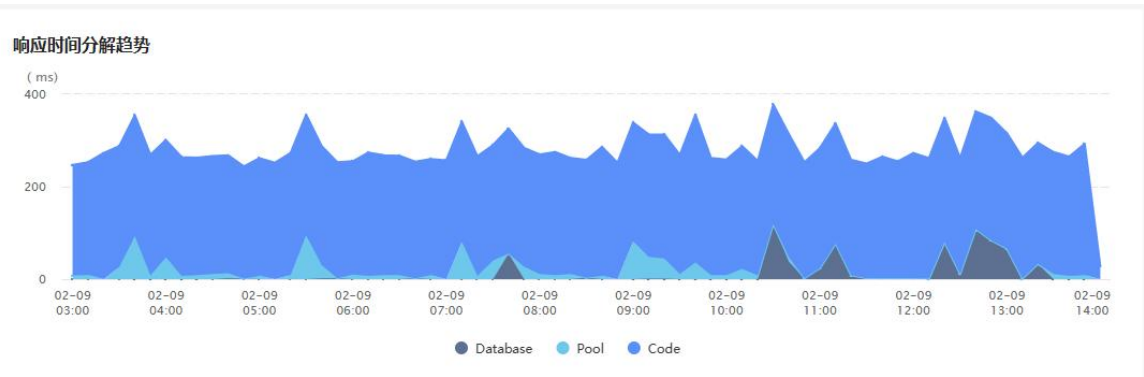
- MQ：当前应用的消息中间件。
- Memcached：当前应用的分布式高速缓存组件。
- Pool：当前应用连接池组件。



3.5.4.2 响应时间分解趋势

以堆叠图的形式展示应用响应时间构成的变化趋势。响应时间包含 Database(数据库)、Code execution(代码执行)、Redis、External(外部组件)、MQ、Memcached（分布式高速缓存组件）和连接池的获取连接耗时 7 种应用组件。

将鼠标悬浮在堆叠图曲线上，可查看粒度时间内当前应用组件的独占时间、调用次数、组件执行时间占比。



3.5.4.3 Top 响应时间分解

展示当前应用的各个重要方法（即代码段）的耗时趋势图。图表展示前 5 个最耗时的代码段以及其他（耗时小于 2%的代码段会汇总到“其他”类别中）。每个点的响应时间数值为粒度时间内代码段的平均执行时间。

将鼠标悬浮在堆叠图曲线上，可查看粒度时间内当前代码段的响应时间、总耗时以及总次数。



3.5.4.4 响应时间分解表格

展示当前应用（对应着多个请求）涉及的所有代码段性能分类、代码段、所属应用和服务接口、总耗时、耗时百分比、调用次数和平均执行时间。代码段按总耗时降序排序。

响应时间分解表格

所有分类

代码段过滤

显示 13/13

导出

性能分类	代码段	总耗时	总耗时占比	调用次数	平均执行时间
Java	com.mq.QueueReceiver/run	50.87 s	64.36 %	50	1.02 s
Java	oracle.*	20.03 s	25.34 %	100	200.30 ms
Oracle	10.128.1.29:1521	4.26 s	5.39 %	50	85.24 ms
Java	com.Exceptions.SQLExceptions/OracleErrorTest1	2.16 s	2.74 %	50	43.24 ms
Java	com.Exceptions.MySimpleJob/execute	1.03 s	1.31 %	50	20.64 ms
Java	com.Exceptions.SQLExceptions/OracleTest	240.00 ms	0.3 %	50	4.80 ms
ActiveMQ	192.168.2.213:61616	182.00 ms	0.23 %	250	0.73 ms
thrift	/ErrorTestDemo/	130.00 ms	0.16 %	50	2.60 ms
Redis	192.168.2.213:6379/0	100.00 ms	0.13 %	100	1 ms
Java	org.springframework.*	12.00 ms	0.02 %	50	0.24 ms

< 1 2 >

10条/页

您可以对表格进行以下操作：

- 表格默认展示所有分类的代码块，单击所有分类下拉菜单，可以根据已有应用组件进行分类选择。
- 在搜索框中输入代码段的名称，然后单击搜索图标，可查看指定代码段的信息。支持模糊搜索，不区分大小写。
- 单击 Database 代码段可进入 **SQL 分析** 页面，具体描述可参见 [SQL 分析](#)。
- 单击 NoSQL 代码段可查看详情。
- 表格默认一页显示 10 条数据，单击右下角的下拉菜单可以选择 20 条/页、30 条/页、40 条/页、50 条/页。
- 单击列表右上角的自定义表头图标，可以勾选想要展示的列名。默认展示全部。
- 单击列表右上角的**导出**按钮，可将全部代码段列表导出为 CSV 格式。

3.5.5 响应时间分布

要进入指定应用的响应时间分布页面，有以下两种方式：

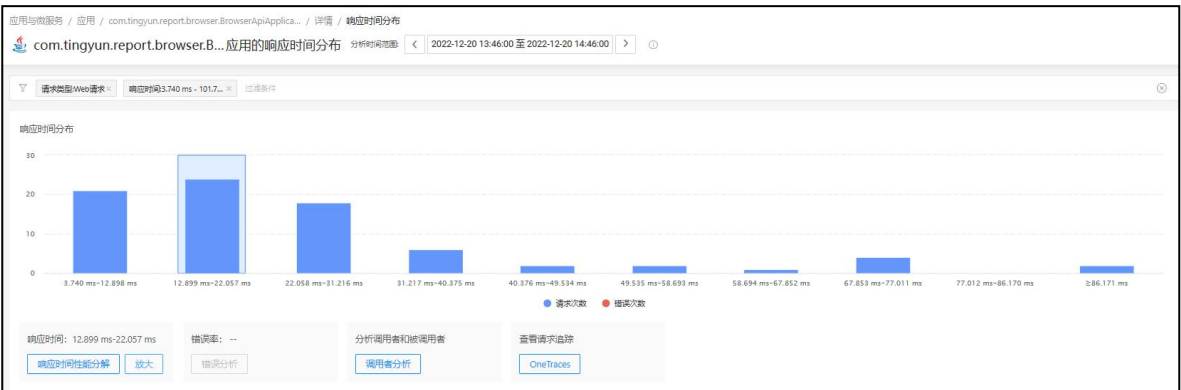
- 单击应用列表**操作**列的“...”，在菜单中选择**响应时间分布**。
- 在**应用详情**页面单击**详情**部分的**响应时间分布**按钮。

在该页面中，您可以修改分析时间范围和过滤数据。

- **修改分析时间范围**：该页面中，所有图表的分析时间范围是**应用详情**页面服务视角下选择的分析范围，可在页面顶部**分析时间范围**处修改时间范围。
- **过滤**：单击页面顶部的过滤框，弹出过滤条件，选择想要查询的条件，立即可看到过滤后的统计数据。过滤条件包括实例名称、请求名称、请求类型、调用者应用、调用者实例和响应时间。当按响应时间过滤时，左侧填写最小响应时间，右侧填写最大响应时间，响应时间支持输入小数，最多为三位小数，单击**确定**即可。

3.5.5.1 响应时间分布

响应时间分布图是将响应时间最小值和最大值的区间范围分成 10 个区间作为横坐标，响应时间的请求次数和错误次数作为纵坐标。默认选中请求次数最多的响应时间区间，如果请求次数相同，则选中响应时间最大的区间。



- 分布图底部展示已选择的响应时间区间和错误率。
- 单击**响应时间性能分解**按钮，可查看所选区间的响应时间性能分解详情。
- 单击**放大**，可以将当前响应时间范围添加到上方的过滤条件，并将所选时间范围继续分成 10 个区间展示在图表中。

说明：当所选时间范围的请求次数 ≤ 1 ，或响应时间区间 $< 1\text{ms}$ 时，**放大**按钮会置灰。

- 单击**错误分析**按钮，可进入错误的分析页面。
- 单击**调用者分析**，可进入应用的**调用者分析**页面，查看调用者和被调用者。
- 单击 **OneTraces**，可进入 OneTraces 请求追踪页面，查看所选响应时间区间内的请求记录。

3.5.5.2 Top 请求

Top 请求列表展示所选响应时间区间内前 50 的请求信息，包括名称、总耗时、响应时间中位数、错误率和请求次数。默认按总耗时倒序展示。

11个Top请求 响应时间: 0.601 ms-144.439 ms 导出

名称过滤 显示 11/11

名称	总耗时	平均响应时间	错误率	请求次数	操作
SpringController/getAccountBaseInfo (POST)	618.923 ms	5.526 ms	0 %	112	
SpringController/verify (POST)	526.927 ms	35.128 ms	0 %	15	
SpringController/menus (POST)	201.299 ms	67.099 ms	0 %	3	
URI/auth-api/actuator/prometheus	168.634 ms	5.621 ms	0 %	30	
SpringController/getCommonPermissionByModule (POST)	65.977 ms	65.977 ms	0 %	1	
SpringController/routePerms (POST)	24.788 ms	24.788 ms	0 %	1	
SpringController/login (POST)	20.262 ms	20.262 ms	0 %	1	
SpringController/preLogin (POST)	14.965 ms	14.965 ms	0 %	1	
SpringController/goLogin	9.936 ms	9.936 ms	0 %	1	

您可以对列表进行以下操作：

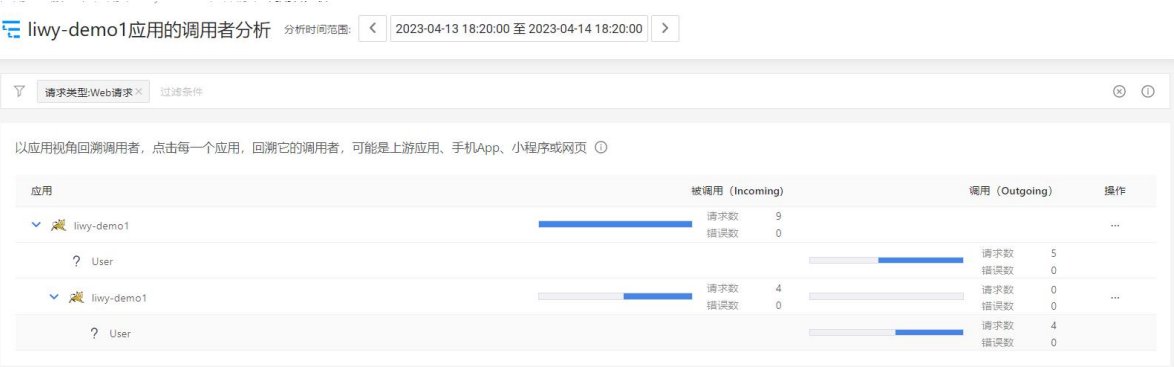
- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定 Top 请求的信息。支持模糊搜索，不区分大小写。
- 单击列表右上角的导出按钮，可将全部 Top 请求列表导出为 CSV 格式。导出的数据会受选中的响应时间范围影响。
- 在操作列中单击 可将该 Top 请求名称作为过滤条件添加到顶部的过滤框中，同时可立即看到过滤后的统计数据。
- 在操作列中单击“...”，然后单击菜单选项进行多维观测，包括 OneTraces、错误分析、异常分析、热点方法、响应时间性能分解和调用者分析。

3.5.6 调用者分析

调用者分析功能是以应用视角回溯调用者。

- 单击每一个应用前的 图标，可在下方查看它的调用者，可能是上游应用、手机 App、小程序或网页。手机 App、小程序或网页会统一显示为“User”。如果是未知的客户端，会显示为“? User”。应用前会展示应用种类图标。
- 页面默认展开所分析的应用的调用者。
- 被调用（Incoming）列展示被调用请求数条形图、应用被调用的请求总数、应用被调用的错误请求总数。条形图中，整个条形代表被调用请求总数，红色部分代表错误请求总数。
- 调用（Outgoing）列展示调用的请求数条形图、调用的请求总数、调用的错误请求总数。条形图中，整个条形代表调用的请求总数，红色部分代表错误请求总数。

- 将鼠标悬浮在**被调用（Incoming）**列或**调用（Outgoing）**列区域，可查看调用的应用情况和被调用的情况。
- 分析时间范围默认是最近 24 小时，用户可在页面顶部**分析时间范围**处修改时间范围。时间范围不能超过 24 小时。
- 单击右侧**操作**列的“...”，可以对目标应用进行多维观测和分析，包括**错误分析**、**异常分析**、**OneTraces 分析**、**热点方法分析**、**响应时间性能分解**、**响应时间分布分析**和**调用者分析**。



3.5.6.1 请求来源

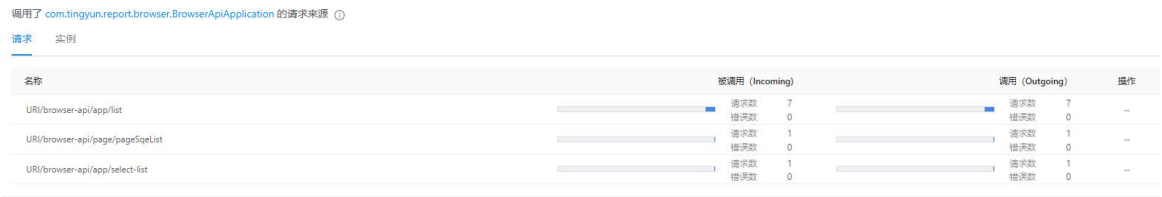
在上方调用链中单击一个应用名称后，下方会展示调用了该应用的请求来源，包括调用该应用的请求和该应用的实例信息。

说明：如果单击手机 App、小程序、网页或未知客户端，系统不展示请求来源。

3.5.6.1.1 请求

在请求页签中，展示调用所选应用请求数前 50 的请求。

- **被调用（Incoming）**列展示被调用请求数条形图、当前请求被调用的总次数、当前请求被调用的错误总次数。条形图中，整个条形代表当前请求被调用的总次数，红色部分代表当前请求被调用的错误总次数。
- **调用（Outgoing）**列展示调用的请求数条形图、当前请求调用所选应用的次数、当前请求调用所选应用的错误次数。条形图中，整个条形代表当前请求调用所选应用的次数，红色部分代表当前请求调用所选应用的错误次数。
- 单击右侧**操作**列的“...”，可以对目标请求进行多维观测和分析，包括**错误分析**、**异常分析**、**OneTraces 分析**、**热点方法分析**、**响应时间性能分解**和**响应时间分布分析**。



3.5.6.1.2 实例

实例页签展示所选应用的实例信息，包括实例名称、实例被调用的请求总数、实例被调用的错误请求总数、实例调用的请求总数、实例调用的错误请求总数。

单击右侧操作列的“...”，可以对目标实例进行多维观测和分析，包括错误分析、异常分析、OneTraces 分析、热点方法分析、响应时间性能分解和响应时间分布分析。



3.5.6.2 请求分析

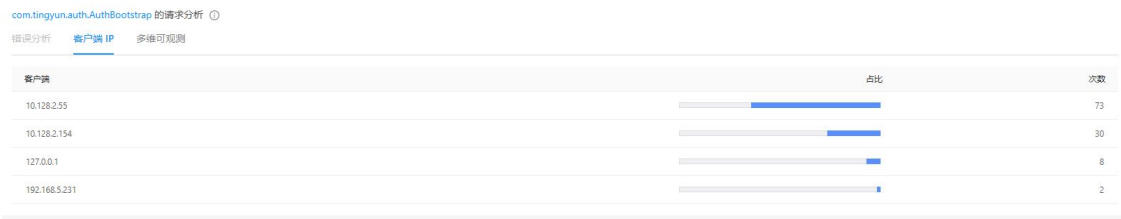
在上方调用链中单击一个应用名称后，页面底部展示该应用请求分析，包括错误分析、客户端 IP 和多维可观测。

- 错误分析：**展示应用在调用中出现的错误，默认展示前 50 个错误，如果没有出现错误，页签将不可用。

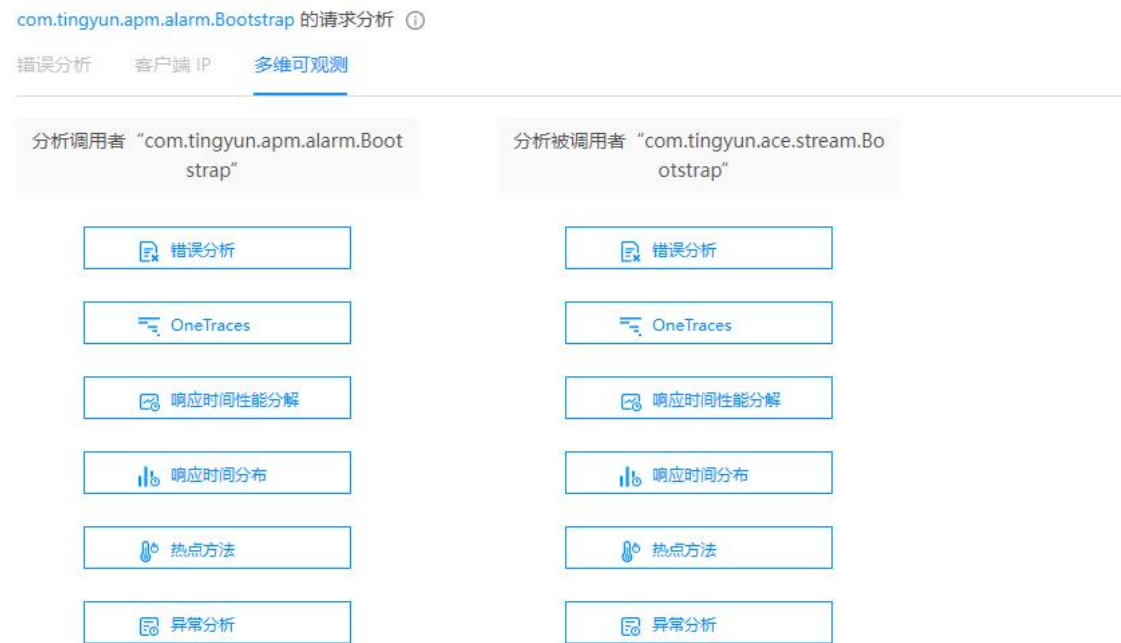
说明：此处只列出错误名称和次数，具体错误根因分析请单击多维可观测页签里面的错误分析。



- 客户端 IP：**展示客户端 IP、占比和次数。占比为当前 IP 地址的调用次数/总调用次数。默认按次数倒排序，没有数据时客户端 IP 页签不可用。



- **多维可观测**：对所选的应用和被调用者进行多维度分析，包括**错误分析**、**异常分析**、**OneTraces** 分析、**热点方法分析**、**响应时间性能分解**和**响应时间分布分析**。当选择客户端（包括 App、小程序、网页和未知客户端）时，**多维可观测**页签不可用。



3.5.7 热点方法

热点方法页面展示当前应用发起请求时调用的线程方法信息。系统通过全栈快照备份对方法栈进行统计分析。热点方法按照线程状态可分为五类，分别是：代码执行、线程等待、线程锁、文件读写和网络读写。

默认分析时间范围是当前应用详情页面的分析时间范围。用户可在页面顶部**分析时间范围**处修改时间范围。

3.5.7.1 饼图

- 不同的颜色代表不同类线程状态。
- 图形中间默认显示占比最大的线程状态。
- 右侧展示各种线程状态和所占百分比。单击一种线程状态后，可在下方的火焰图查看该状态的方法调用栈，符合条件的方法显示为黄色。



3.5.7.2 堆叠图

- 横坐标表示当前选择的时间段，纵坐标表示线程状态下方法调用次数。
- 将鼠标悬浮在堆叠图上，可查看当前粒度时间内各种线程状态下方法的调用次数。

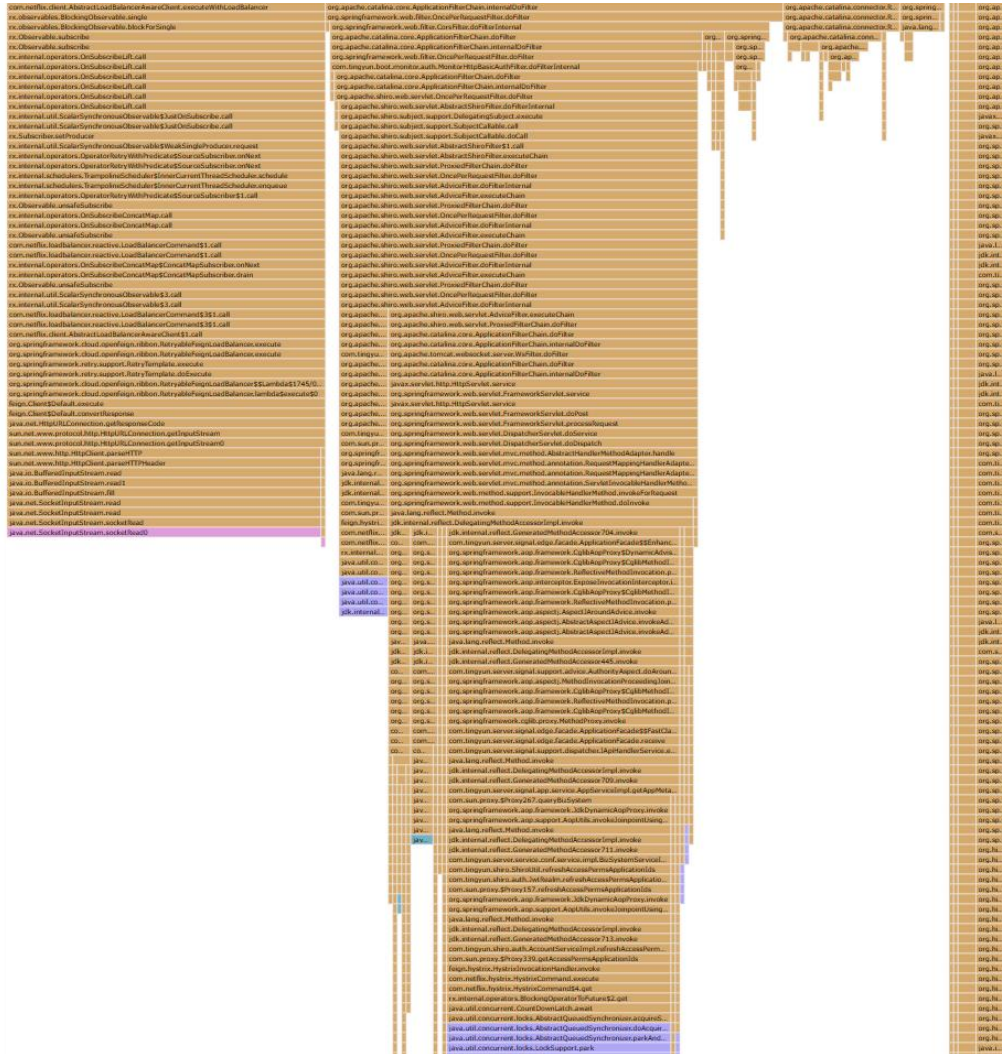


3.5.7.3 火焰图

- 纵轴表示调用栈深度，按照调用关系从上到下排列，火焰越高说明该方法被调用的层级越多。
- 横轴表示调用频次，一个格子的宽度越大，说明该方法调用次数越多。

📖说明：横轴并不代表时间，而是所有方法栈合并后，按字母顺序的排序。
- 每一个格子代表一个方法调用栈，不同的颜色代表不同类型的方法栈。方法调用栈格式为：包名.类名.方法名.线程状态。

- 单击任意一个格子后，详细的方法调用栈会被展开。
- 将鼠标悬浮在格子上，可查看当前方法栈的线程状态、堆栈样本占比和方法栈调用次数。其中堆栈样本占比是指根方法的调用次数/总的方法调用次数。



3.5.8 错误分析

在应用分析页面中单击**错误分析**进入**错误分析**页面，您可以查看当前应用发生的错误请求（包括 Web 请求和后台任务），并定位错误的影响范围和错误的根因。

您可以通过选择过滤条件筛选数据，过滤条件包括：实例、请求类型、请求名称和错误名称。

3.5.8.1 错误列表

错误分析列表展示当前应用出现的错误信息。列表默认选中错误次数最多的数据，如果没有选择任何数据时，页面仅显示错误分析列表和影响分析，不显示根因分析和追踪列表。

- 名称：请求错误名称。

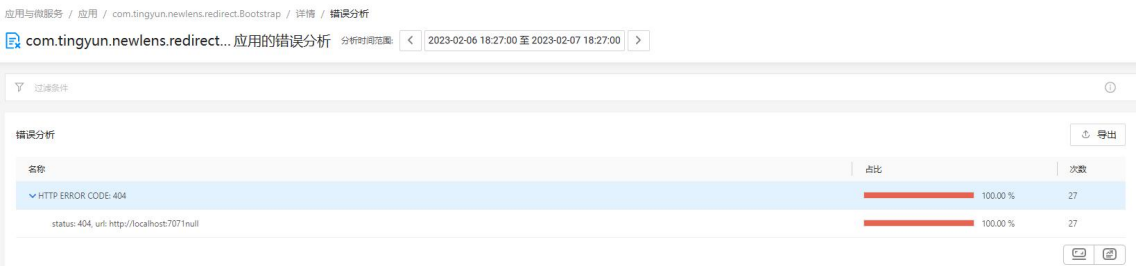
- 占比：错误次数/应用的错误总数。

说明：错误总数为 Web 请求和后台任务的错误总数。如果请求类型选择了 Web 请求，那么错误总数为 Web 请求的错误总数，反之就是后台任务的错误总数。

- 次数：请求错误次数，列表默认按照错误次数倒排序。

您可以对列表进行以下操作：

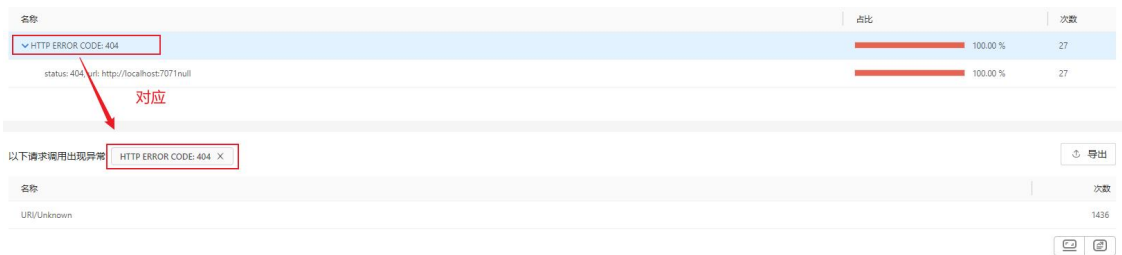
- 单击列表右上角的**导出**按钮，可以将整个错误分析列表导出为 CSV 格式。
- 单击一个错误，下方的影响分析、根因分析和追踪列表也相应的展示该错误的分析结果。



3.5.8.2 影响分析

影响分析列表默认显示错误次数最多的错误的请求信息。当单击错误分析列表中任意一条错误信息，请求列表会对应展示所选的这条错误的请求信息。

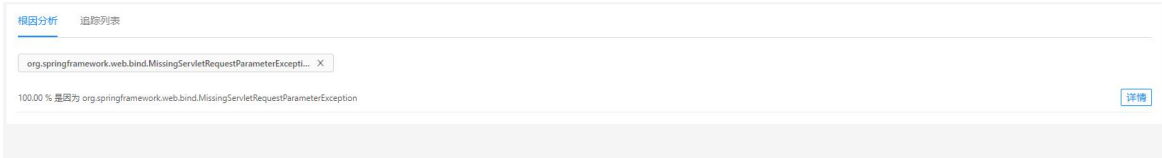
- 单击标题右侧的**错误名称**后的叉号，可以删除所选错误信息。删除后会显示所有错误的请求信息。
- 单击列表右上角的**导出**按钮，可以将整个请求列表导出为 CSV 格式。
- 单击一个请求，下方的追踪列表展示相应的追踪结果。



3.5.8.3 根因分析

根因分析展示某个错误/异常的根因。单击 **StackTrace 分析** 进入新页面查看 root cause 的 Messages 和 StackTrace。

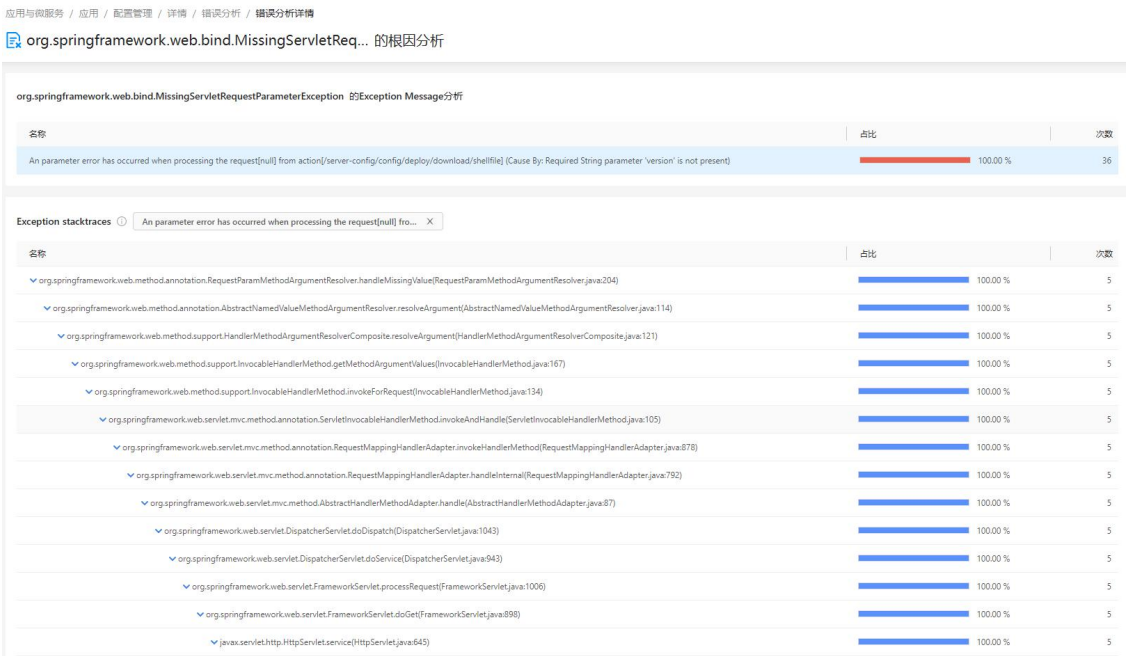
说明：当前错误存在异常堆栈时才显示根因分析，如果没有则默认置灰。



Exception Message 分析：展示当前错误的根因、根因占比和次数。列表默认按照次数倒序排列。

Exception StackTrace 分析：展示上方对应错误的 StackTrace（堆栈追踪）信息。

- +：合并单子集且包名相同的方法。
- 单击  将多个同级节点展开。
- 列表默认展示到第一次分支的地方，子方法默认收起。



3.5.8.4 追踪列表

追踪列表展示发生错误的请求追踪信息。

- 发生时间：请求追踪的发生时间，默认按发生时间倒序显示。
- Referer：请求来源地址信息。
- 单击请求名称进入请求追踪详情，具体描述可参考[请求追踪](#)。
- 单击详情会弹出错误根因详情，包括错误根因的 Messages 和 StackTrace。

 **说明：** 错误存在堆栈时才展示 StackTrace 信息，否者只显示 Messages。

- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定请求信息。支持模糊搜索，不区分大小写。

根因分析

追踪列表

名称过滤

Q

显示 3 / 3

发生时间	请求名称	实例	Referer	详情
2023-02-07 18:26:19	URI/Unknown	VM-2-55-centos7071	--	Exception details
2023-02-07 18:25:19	URI/Unknown	VM-2-55-centos7071	--	Exception details
2023-02-07 18:24:19	URI/Unknown	VM-2-55-centos7071	--	Exception details

3.5.9 异常分析

在应用分析页面中单击**异常分析**进入**异常分析**页面，您可以查看当前应用发生的异常，并定位异常的影响范围和异常的根因。

您可以通过选择过滤条件筛选数据，过滤条件包括：实例、请求类型、请求名称和异常名称。

3.5.9.1 异常列表

异常列表展示当前应用出现的异常请求信息。列表默认选中异常次数最多的数据，如果没有选择任何数据时，页面仅显示异常列表和影响分析，不显示根因分析和追踪列表。

- 名称：请求异常名称。
- 占比：异常次数/应用的异常总数。

说明：

异常总数为 Web 请求和后台任务的异常总数。如果请求类型选择了 Web 请求，那么异常总数为 Web 请求的异常总数，反之是后台任务的异常总数。

- 次数：请求异常次数，列表默认按照异常次数倒排序。

您可以对列表进行以下操作：

- 单击列表右上角的**导出**按钮，可以将整个异常分析列表导出为 CSV 格式。
- 单击一个异常，下方的影响分析、根因分析和追踪列表也相应的展示该异常的分析结果。

应用与服务 / 应用 / com.tingyun.maka.MakaApplication / 详情 / 异常分析

com.tingyun.maka.MakaAppli...应用的异常分析

分析时间范围

<

2023-02-06 18:12:00 至 2023-02-07 18:12:00

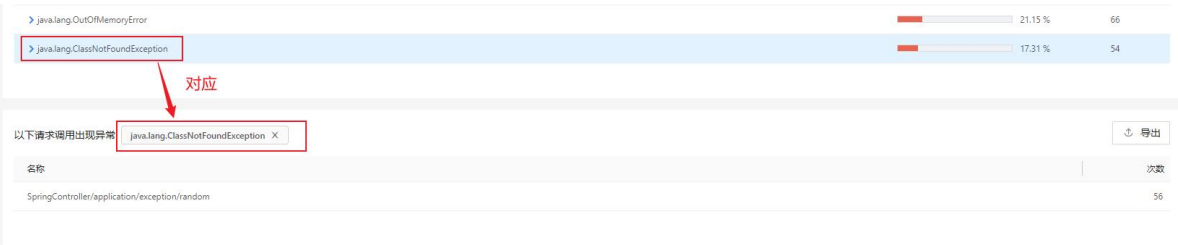
>

名称	占比	次数
org.springframework.web.util.NestedServletException	40.38 %	126
java.lang.IndexOutOfBoundsException	21.15 %	66
java.lang.OutOfMemoryError	21.15 %	66
java.lang.ClassNotFoundException	17.31 %	54

3.5.9.2 影响分析

请求列表默认显示异常次数最多的请求信息。当单击异常分析列表中任意一条异常信息，请求列表会对应展示所选的这条异常的请求信息。

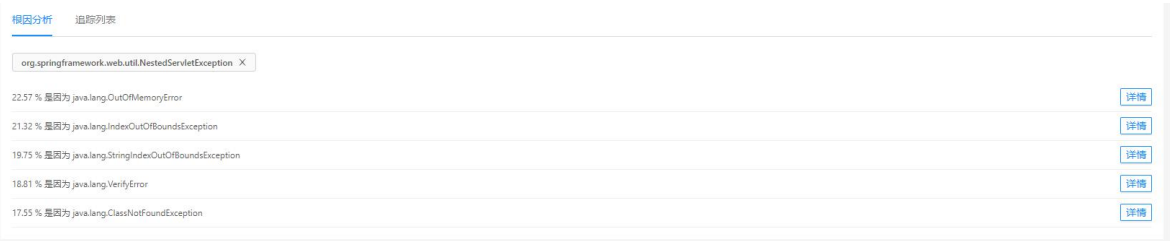
- 单击标题右侧的**异常名称**后的叉号，可以删除所选异常信息。删除后会显示所有异常的请求信息。
- 单击列表右上角的**导出**按钮，可以将整个请求列表导出为 CSV 格式。
- 单击一个请求，下方的追踪列表展示相应的追踪结果。



3.5.9.3 根因分析

根因分析展示某个错误/异常的根因。单击 **StackTrace 分析** 进入新页面查看 root cause 的 Messages 和 StackTrace。

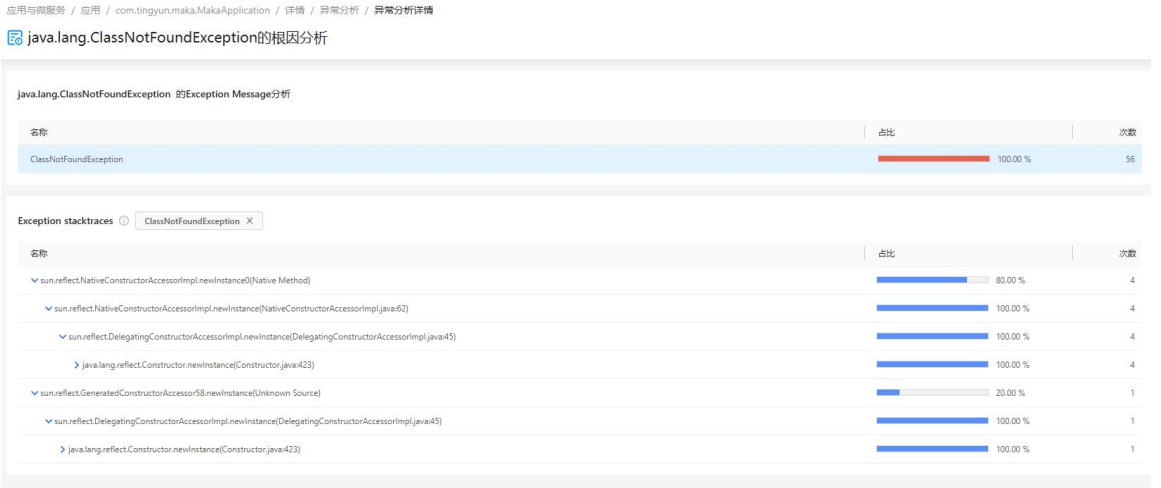
说明：当前异常存在异常堆栈时才展示根因分析，如果没有则默认置灰。



Exception Message 分析：展示当前异常的根因、根因占比和次数。列表默认按照次数倒序排列。

Exception StackTrace 分析：展示上方对应根因的 StackTrace（堆栈追踪）信息。

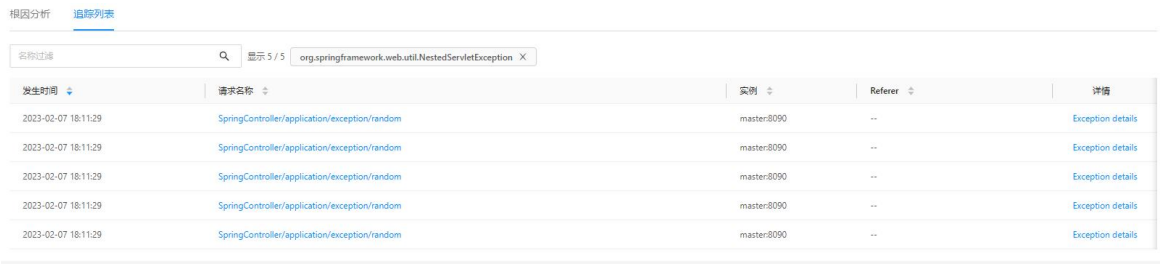
- **+**：合并单子集且包名相同的方法。
- 单击  将多个同级节点展开。
- 列表默认展示到第一次分支的地方，子方法默认收起。



3.5.9.4 追踪列表

追踪列表显示发生异常的请求追踪信息。

- 发生时间：请求追踪的发生时间，默认按发生时间倒序展示。
- Referer：请求来源地址信息。
- 单击请求名称进入请求追踪详情，具体描述可参考[请求追踪](#)。
- 单击详情会弹出异常根因详情，包括异常根因的 Messages 和 StackTrace。
- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定请求信息。支持模糊搜索，不区分大小写。



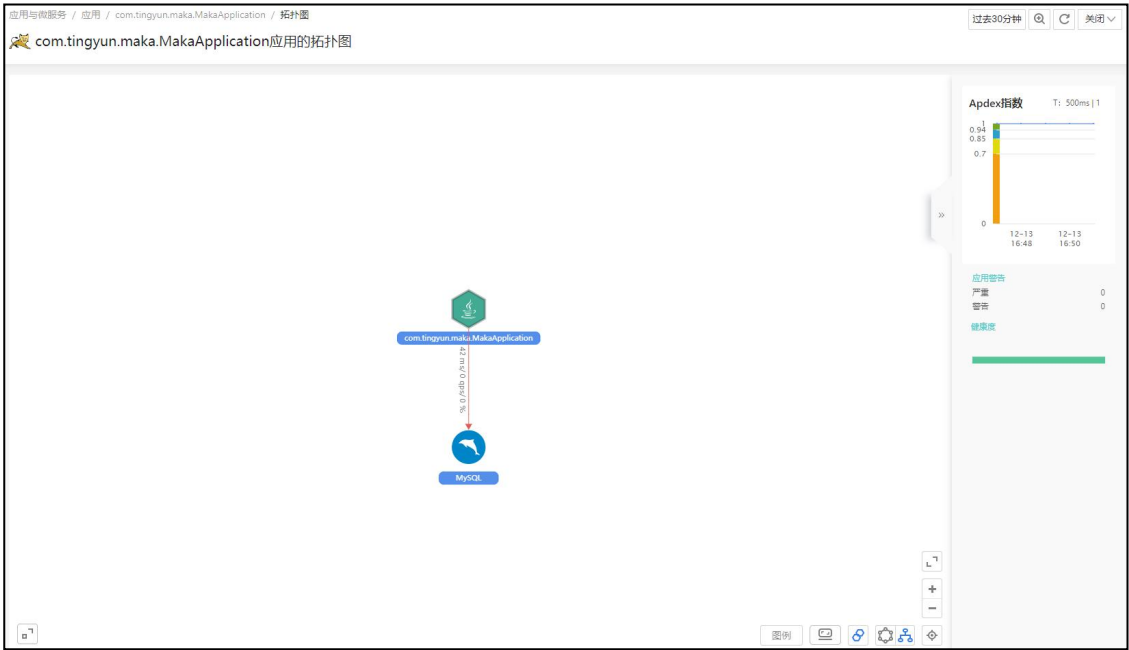
3.5.10 OneTraces

该页面展示当前应用或实例所属的事的务、服务接口和后台任务。详细介绍请参见[请求追踪](#)。

3.5.11 拓扑图

应用/实例拓扑

拓扑页面展示当前应用或实例与服务组件、外部服务、其他相关的应用、其他相关的业务系统之间的逻辑调用关系。每个业务系统、应用和服务组件都以图标形式展示，并以带有箭头的连线来展示应用之间、应用与服务组件以及应用与业务系统之间的调用关系。连线上显示的是平均响应时间和吞吐率。选择具体的实例后，描述同样适用。



- 图标的颜色表示所代表的监控对象的健康度。
- 当组件数量超过 50 个时，系统会根据响应时间从高到低排序，仅展示前 50 个组件。
- 拓扑图可使用鼠标滚轮进行缩放，在缩放的同时，可在左下角的全局缩略图观察当前所显示的拓扑图部分在整个拓扑图中的位置和大小。通过拓扑图右下角的操作按钮还可对拓扑图进行以下操作：
 - 缩放：缩放整个拓扑图，当前可见区域与整个拓扑图的相对关系和大小在左下角的缩略图上展示。
 - 复位：以缺省的缩放比例显示拓扑图。
 - 全屏：以全屏显示拓扑图。
 - 布局：可选圆形和树型结构两种拓扑图布局。
 - 图例：显示当前应用拓扑图的图例说明。
- 单击添加到大屏：将拓扑图添加到基调听云大屏的组件列表中。

应用基本信息

在拓扑图右侧展示当前应用的基本信息，包括：

- Apdex 趋势图：当前应用的 Apdex 指数的变化趋势。右上角显示用于计算 Apdex 指数的 Apdex T 和统计时间周期内的平均 Apdex 指数值。
- 应用警告：显示当前应用中出现的告警程度的警报和严重程度的警报的数量。
- 健康度：以条形图的形式展示当前应用的健康度。当应用的健康状态存在严重或警告时，可单击条形图的严重或警告部分查看[健康度分析详情](#)。

实例基本信息

实例基本信息除了展示 Apdex 趋势图、应用告警和健康度以外，还展示了实例主机名称以及 IP 地址。

3.5.12 外部服务

在应用概览页面 A 区域的下游应用部分，单击**外部服务**，查看当前应用下统计周期内调用的所有外部服务。目前，系统支持监控 HTTP、HTTPS、Thrift、Dubbo、Web Service、EJB、gRPC 和 MINA 外部服务协议。

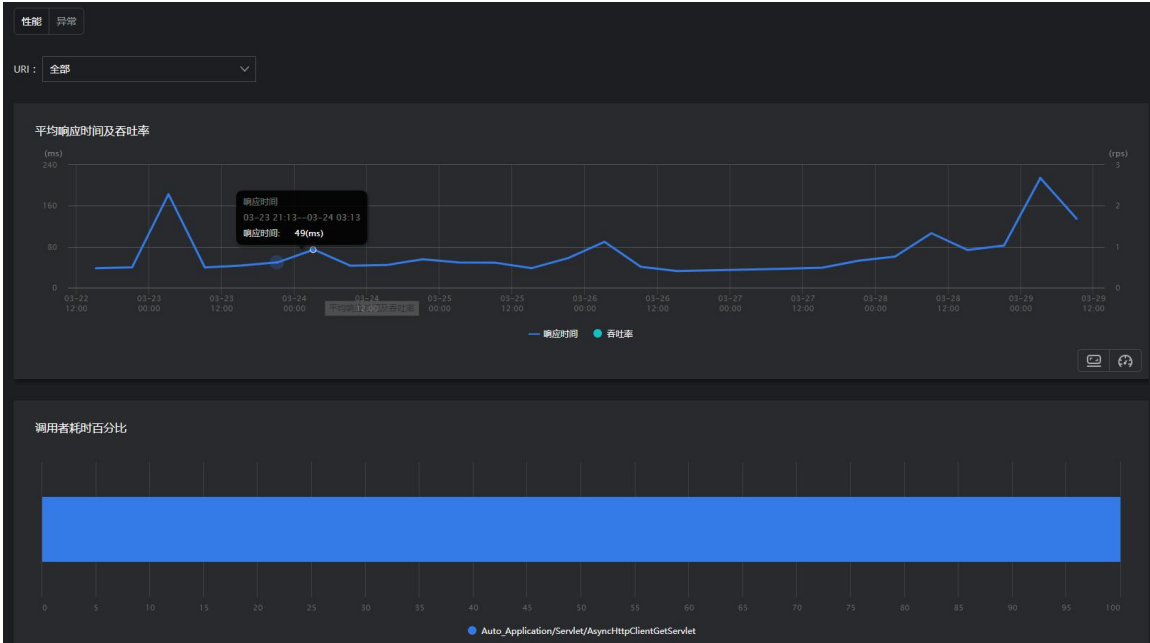
列表项说明如下：

- 类型：外部服务的协议类型。
- 名称：外部服务名称，格式为主机名加端口号，或者为服务名。
- 平均响应时间：当前外部服务在统计周期内的平均响应时间。支持正反排序。
- 吞吐率：当前外部服务在统计周期内的平均调用次数，单位 rps。支持正反排序。
- 错误率：当前外部服务在统计周期内发生异常（包括错误和未影响事务的异常）的次数占所有调用次数的百分比。支持正反排序。
- 健康度：当前外部服务在统计周期内的健康度。支持正反排序。

单击外部服务名称进入对应外部服务的分析页面，单击**性能**和**异常**页签可分别查看当前外部服务的性能数据和异常数据。

3.5.12.1 性能

- URI：可选择指定的 URI 来查看外部服务统计数据。默认统计的是应用所调用的所有外部服务的性能数据。
- 平均响应时间和吞吐率：外部服务平均响应时间是指在**粒度时间**内，该外部服务多次被调用的平均响应时间，响应时间为从应用发送外部服务调用请求开始到收到响应结果结束的耗时。吞吐率（qps）是指应用在粒度时间内每秒调用当前外部服务的次数。
- 调用者耗时百分比：统计周期内，当前外部服务在每个调用者中的耗时百分比，调用者为当前应用的事务、服务接口或后台任务。公式为：调用者各次调用该外部服务的总响应时间/各调用者的外部服务总耗时*100%。
- 追踪列表：展示当前外部服务的调用者每一次调用的详细情况。单击外部服务追踪列表的条目，可以查看该次外部服务的调用方法。



3.5.12.2 异常

- 错误率曲线图：当前应用或实例调用该外部服务发生异常（包括错误和未影响事务的异常）的次数占所有调用次数的百分比趋势图。
- 异常类型及数量：图表显示统计周期内数量排名前 5 的异常（包括错误和未影响事务的异常）发生次数。
- 错误列表：展示当前应用或实例发生的异常（包括错误和未影响事务的异常）。详细介绍请参见[错误列表](#)。

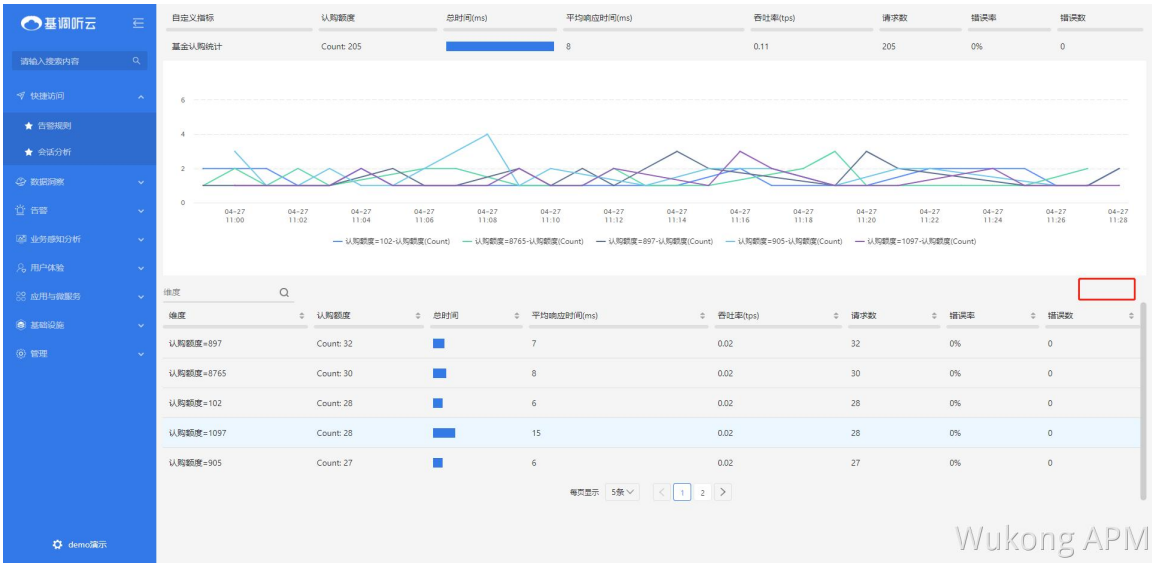
3.5.13 自定义指标

在应用概览页面 A 区域的右上角，单击自定义指标按钮，可进入应用的自定义指标分析页面。在该页面可查看每个指标中各个数据项的访问次数、指标的总响应时间、平均响应时间、请求次数、吞吐量、错误率和错误数。

说明：要分析自定义指标，请先在应用配置页面中对自定义指标进行配置。

- 单击页面右上角的导出自定义指标按钮，可将自定义指标列表导出为 Excel 到本地。
- 单击页面右上角的自定义表头按钮，勾选想要展示的列名，并可以调整列宽。部分字段是必选项，必须展示。单击恢复默认可恢复到系统默认展示的列和宽度。
- 单击下方维度分析列表右上角的导出按钮，可将维度分析列表数据导出为 Excel 到本地。
- 单击一个指标，系统会根据指标中的数据项的值进行排列组合，并展示每种组合的访问次数变化趋势。同时还支持以列表的形式分别统计每种组合的访问次数、指标的总响应时间、平均响应时间、请求次数、吞吐量、错误率和错误数。

- 单击一个维度，可跳转到请求追踪页面，查看该种参数组合条件下，涉及的所有请求的追踪列表。



3.5.14 线程剖析

当应用出现系统卡顿或者响应慢时，您可以以非常低的系统开销对应用实例进行线程剖析，采集线程状态，定位性能消耗最大的线程和方法，进而突破性能瓶颈。

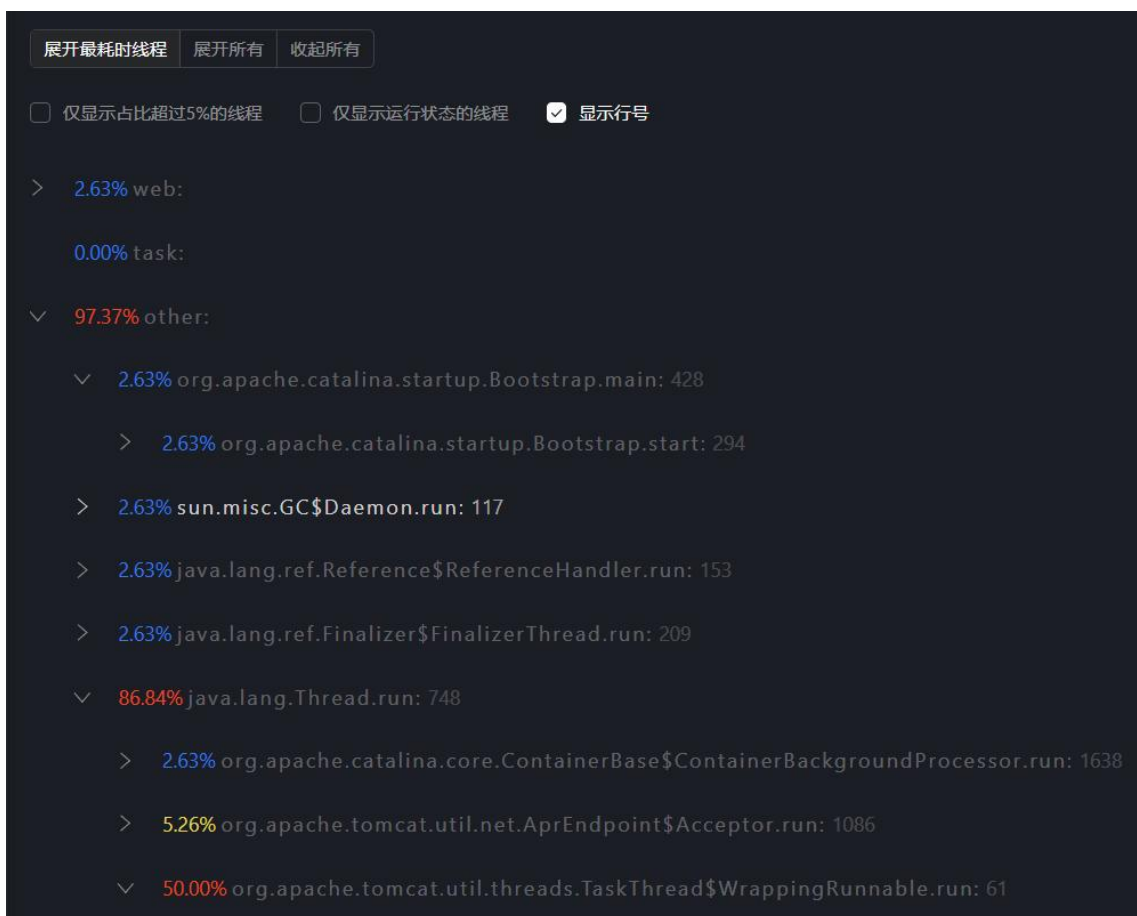


线程剖析的配置项说明如下：

- 选择实例：只可以选择一个实例，暂无实例时不能进行线程剖析。
- 采样持续时间：1~10 分钟的可选范围，默认选择 2 分钟。
- 采样时间间隔：50~100 毫秒的可选范围，默认选 60 毫秒。

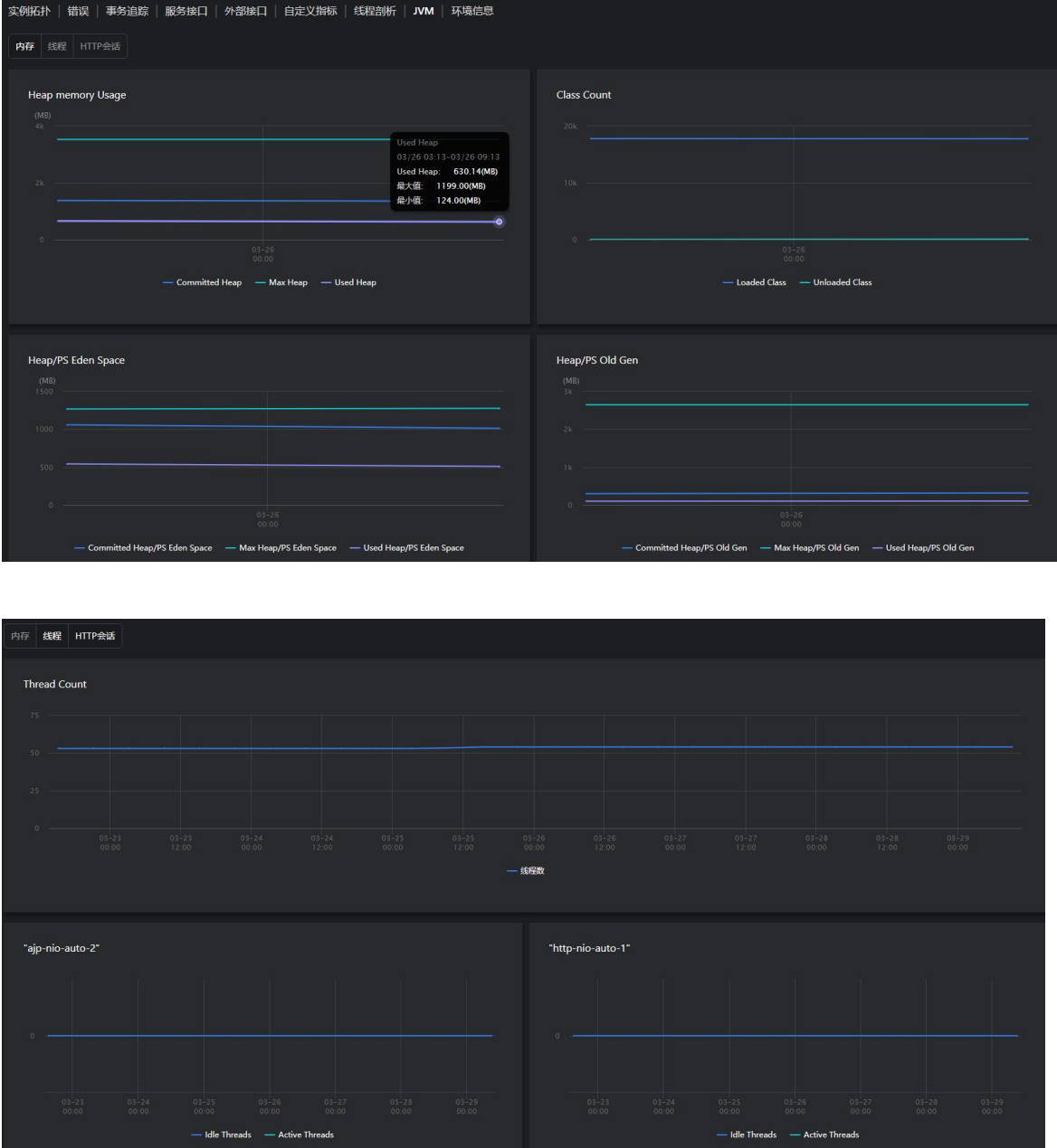
剖析完成后，可在右侧的线程剖析列表中查看剖析结果。线程剖析列表展示采集成功的剖析数据，包括：开始剖析时间、持续时间、采集人。用户可删除所选的剖析记录。单击一个剖析条目，可以查看所有线程的代码调用堆栈详细信息。默认展开最耗时线程的堆栈，即该应用的全部线程中耗时占比最高的线程。

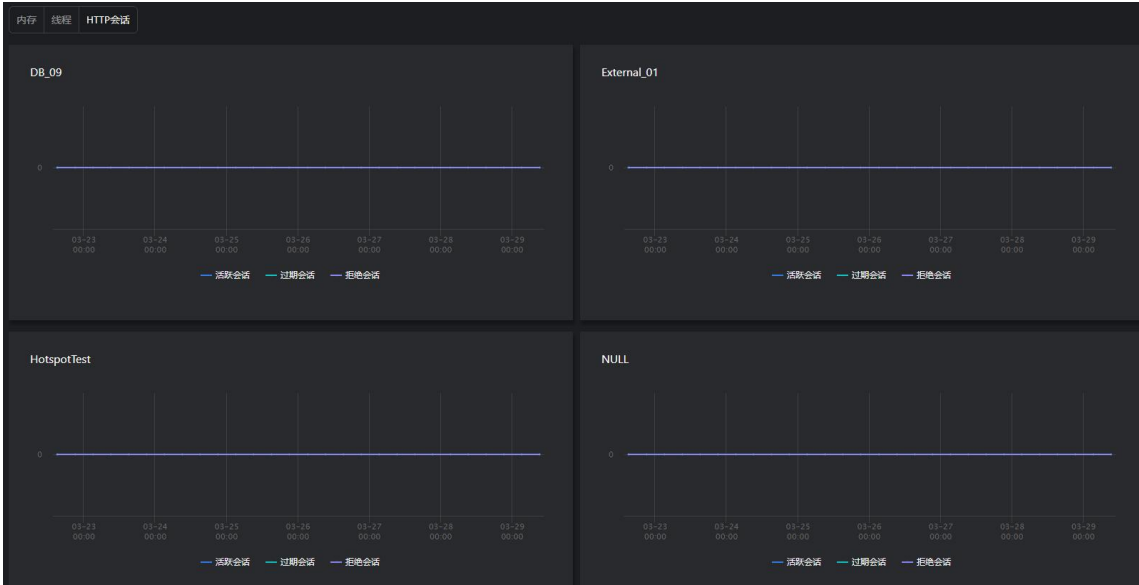
 **说明：**一旦单击了**开始剖析**按钮，单击取消后将不会真正停止，探针仍会采集堆栈信息。



3.5.15 JVM

当选择了具体的实例后，**JVM** 页签会显示在上方。该页面针对 Java 应用提供 JVM 运行环境的各类统计信息和图表，包括：内存曲线、线程曲线、HTTP 会话曲线。用户可通过单击上方的各个页签进行查看。





3.5.16 环境信息

当选择了具体的实例后，**环境信息**页签会显示在上方。该页面显示应用实例的 IP 地址、探针版本、最后更新时间（探针最后一次初始化启动的时间）和环境信息。

- 运行环境：展示当前应用实例所在主机的内存、线程、操作系统、JVM 虚拟机等信息。
- 系统环境变量：展示当前应用实例所在主机的环境变量设置情况。
- JVM 系统属性：展示当前应用实例所运行在的 JVM 的系统属性。
- JAR 包信息：展示对当前应用实例所安装探针中的 Jar 包及版本。当探针无数据时不显示 Jar 包信息。
- Instrument Detail：展示该实例的自定义嵌码详情。
- Container：展示探针所部署在的容器的信息。

3.6 请求

3.6.1请求列表

在左侧导航栏中依次单击**应用与微服务>请求**，可进入请求列表页面。该页面展示所有监控到的请求，包括安装了基调听云请求与微服务探针的请求、接入的第三方探针所监控的请求。列表项包括请求名称、请求类型、所属应用、响应时间中位数、吞吐率、请求次数、错误率和错误次数。

请求

过去30分钟

请求类型:Web请求

154个请求

请求名称	请求类型	应用	响应时间中位数	吞吐量	错误率	慢次数	异常次数	操作
SpringController/sqljdbc/vuln	Web请求	aspm-demo	2.12 min	< 0.01 /s	0	2	0	...
SpringController/testLinkedBlockingQueue	Web请求	OA	39.11 s	0.17 /s	0	313	0	...
UR(source-map-api/release/web/list	Web请求	wjn-mock-agent-1	6.39 s	< 0.01 /s	100.00 %	3	9	...
SpringController/testMysqlSleep	Web请求	OA	5.02 s	0.32 /s	100.00 %	297	576	...
SpringController/testHello-列表	Web请求	OA	3.16 s	0.16 /s	0	281	0	...
SpringController/product/submit	Web请求	wjn-mock-agent-1	3.00 s	< 0.01 /s	0	1	0	...
UR(mail-portal/order/list	Web请求	mail-gateway-deployme...	2.73 s	0.02 /s	0	34	0	...
SpringController/order/list	Web请求	mail-portal-deployment...	2.70 s	0.02 /s	0	34	0	...
SpringController/shop/list	Web请求	wjn-mock-agent-1	2.15 s	< 0.01 /s	0	4	0	...
UR(mail-portal/order/detail/*	Web请求	mail-gateway-deployme...	1.71 s	0.02 /s	0	2	0	...
SpringController/order/detail/{orderId}	Web请求	mail-portal-deployment...	1.71 s	0.02 /s	0	2	0	...
UR(mail-portal/order/paySuccess	Web请求	mail-gateway-deployme...	921.02 ms	0.02 /s	0	0	0	...
SpringController/order/paySuccess (POST)	Web请求	mail-portal-deployment...	920.44 ms	0.02 /s	0	0	0	...
UR(mail-portal/order/generateConfirmOrder	Web请求	mail-gateway-deployme...	870.90 ms	0.13 /s	0	0	0	...
SpringController/order/generateConfirmOrder (POST)	Web请求	mail-portal-deployment...	869.98 ms	0.13 /s	0	0	1	...

共 154 条数据

1 2 3 4 5 ... 11 >

15条/页

- 在请求列表中，您可以进行以下操作：
- 单击请求名称，可以进入目标请求的概览页面。
 - 单击页面上方的过滤框，可根据应用名称、业务系统、请求类型、请求名称、标签和关注状态筛选请求。每类条件只能选择一个，支持设置多类条件。
 - 单击列表右上角的自定义表头图标，可以勾选想要展示的列名，部分字段必选。
 - 单击右侧操作列的“...”，可以对目标请求进行多维观测和分析，可查看响应时间性能分解、错误分析、响应时间分布、分布式链路追踪和异常分析。

3.6.2请求概览

在请求列表页面单击请求名称，可以进入目标请求的概览页面。概览页面包括标题栏、指标、关系、分布式链路追踪、事件、日志。

3.6.2.1 标题栏

标题栏显示当前请求的请求名称、数据项、属性信息和标签。

- 点击数据项右侧抽屉显示数据项的详情。

User-Agent 数据项的详情

搜索 1 / 1

导出

名称	响应时间中位数	吞吐量	请求次数	错误率	错误次数
python-requests/2.32.3	155.23 ms	0.01 /s	23	0	0

- 点击**属性信息和标签**，右侧抽屉显示请求所属的应用、业务系统、标签等信息，且支持新增标签。

属性和标签

属性

请求名称

SpringController/vue-element-admin/component/transaction/all/v1

请求原名

SpringController/vue-element-admin/component/transaction/all/v1

应用

HawkApplication-wukong

业务系统

wukong-wb

标签

环境：测试环境

添加标签

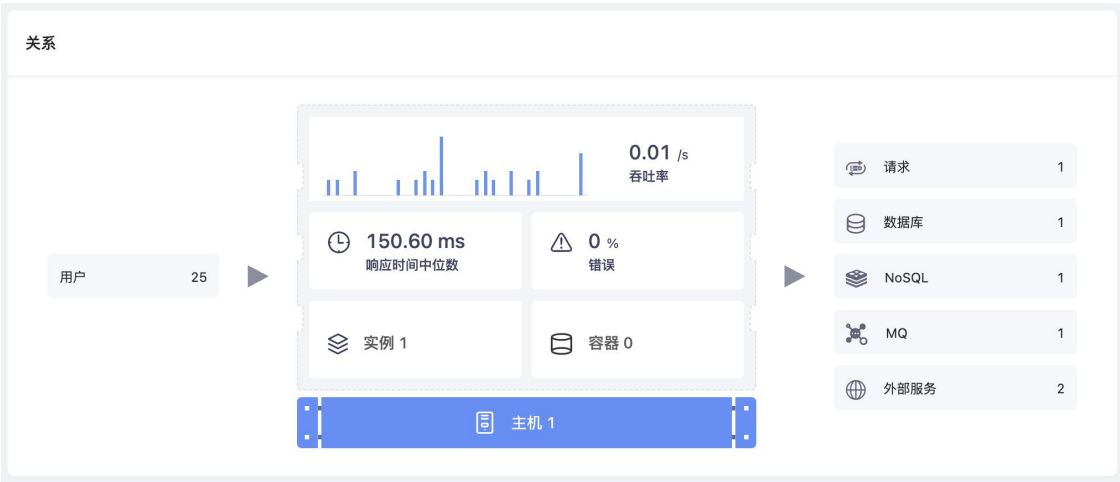
3.6.2.2 指标

指标区域展示当前请求的响应时间、错误率、吞吐率的趋势图及环比，且支持切换指标显示Apdex、异常次数的指标趋势图。

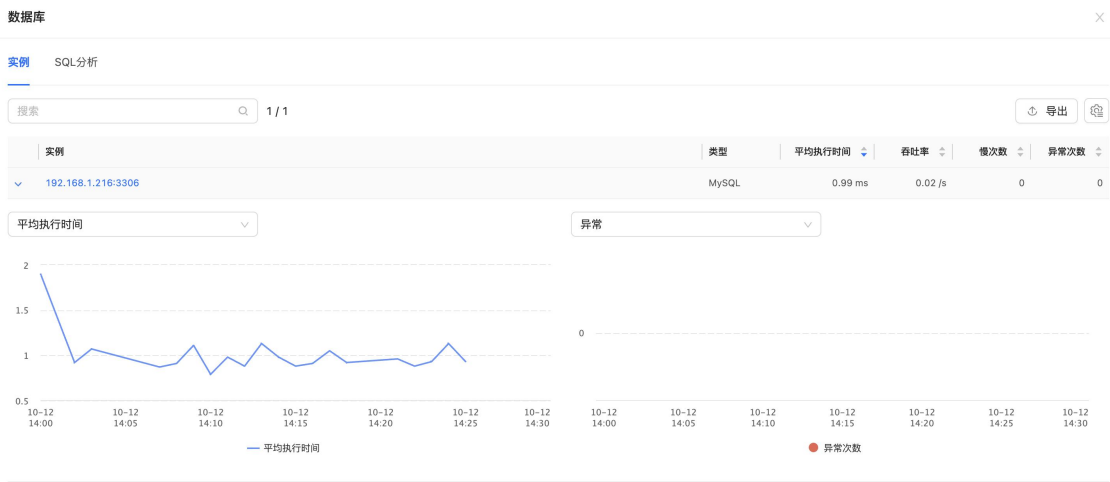


3.6.2.3 关系

关系区域显示当前请求的上下游调用关系，包括调用当前请求的上游请求、当前请求的实例、容器、主机及下游的请求、数据库、NoSQL、MQ、外部服务。



- 调用者区域展示调用当前请求的请求及个数。
 - 单击调用者请求个数，右侧会显示调用者请求列表。
- 当前请求区域展示当前请求的吞吐量、吞吐量柱状图、响应时间中位数、错误率、实例数量、容器数量及主机数量。
 - 单击实例数、容器数及主机数，右侧显示对应的详情信息。
- 下游请求区域展示当前请求所调用的下游请求或服务组件信息，包括请求、外部服务、数据库、NoSQL 和 MQ。单击后，右侧显示对应的列表信息。



3.6.2.4 分布式链路区域

分布式链路追踪区域显示当前请求的追踪次数堆叠图和追踪列表。



3.6.2.5 事件区域

- 事件：展示所选统计周期内对象统计中环比分析的结果事件。
 - 柱状图：蓝色柱状图代表对象数增长数，绿色代表对象数持续增长数。
 - 列表：展示事件的发生时间、事件类型和事件描述信息。事件描述的信息包括：事件名称、请求名称和实例名称。单击一个事件，可查看对象统计环比分析详情。
 - 中间展示的对象数增长、探针熔断等数据是对象统计中环比分析的结果事件数量。



3.6.2.6 日志区域

显示当前请求所属请求的应用日志，点击右上角的日志跳转到日志模块查看详情。



3.6.3响应时间性能分解

在请求详情页面单击详情部分的响应时间性能分解进入该页面。

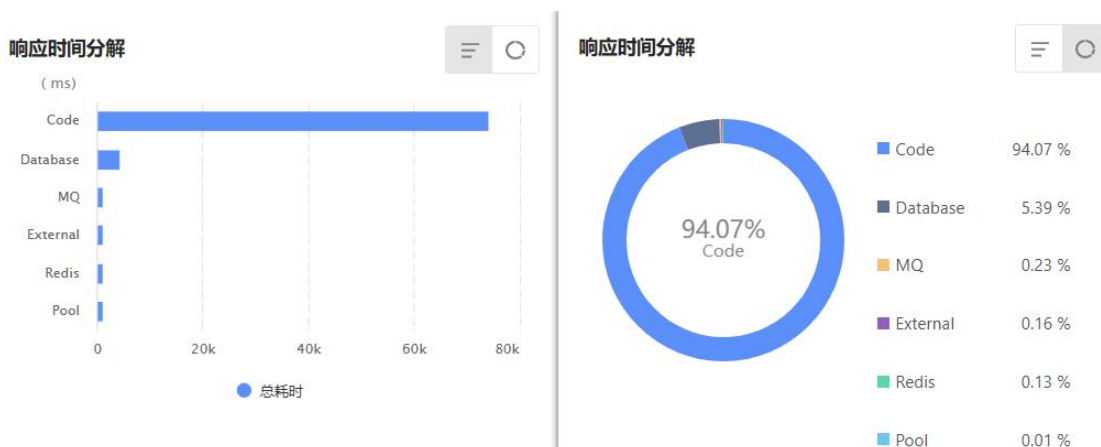
- 修改分析时间范围：该页面中，所有图表的分析时间范围是请求详情页面服务视角下选择的分析范围，可在页面顶部分析时间范围处修改时间范围。

说明：如果时间跨度超过 1 天，开始时间修改为结束时间减 1 天。
- 过滤：单击页面顶部的过滤框，弹出过滤条件，选择想要查询的条件，立即可看到过滤后的统计数据。过滤条件包括：实例名称、请求名称和请求类型。

3.6.3.1 响应时间分解

以 Bar 图（条形图）和环形图的方式展示请求响应时间的构成部分分别所占用的时间。图形默认展示 Bar 图（条形图）。

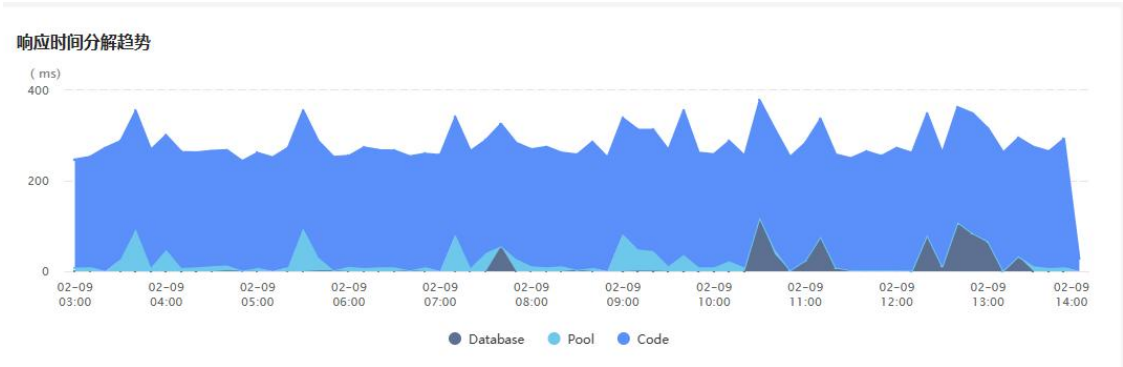
- Database：当前请求依赖的数据库组件。
- Code execution：当前请求代码执行。
- Redis：当前请求的缓存数据库组。
- External：当前请求依赖外部组件。
- MQ：当前请求的消息中间件。
- Memcached：当前请求的分布式高速缓存组件。
- Pool：当前请求连接池组件。



3.6.3.2 响应时间分解趋势

以堆叠图的形式展示请求响应时间构成的变化趋势。响应时间包含 Database(数据库)、Code execution(代码执行)、Redis、External(外部组件)、MQ、Memcached（分布式高速缓存组件）和连接池的获取连接耗时 7 种请求组件。

将鼠标悬浮在堆叠图曲线上，可查看粒度时间内当前请求组件的独占时间、调用次数、组件执行时间占比。



3.6.3.3 Top 响应时间分解

展示当前请求的各个重要方法（即代码段）的耗时趋势图。图表展示前 5 个最耗时的代码段以及其他（耗时小于 2%的代码段会汇总到“其他”类别中）。每个点的响应时间数值为粒度时间内代码段的平均执行时间。

将鼠标悬浮在堆叠图曲线上，可查看粒度时间内当前代码段的响应时间、总耗时以及总次数。



3.6.3.4 响应时间分解表格

展示当前请求涉及的所有代码段性能分类、代码段、所属请求和服务接口、总耗时、耗时百分比、调用次数和平均执行时间。代码段按总耗时降序排序。

响应时间分解表格

所有分类

代码段过滤

显示 13/13

性能分类	代码段	总耗时	总耗时占比	调用次数	平均执行时间
Java	com.mq.QueueReceiver/run	50.87 s	64.36 %	50	1.02 s
Java	oracle.*	20.03 s	25.34 %	100	200.30 ms
Oracle	10.128.1.29:1521	4.26 s	5.39 %	50	85.24 ms
Java	com.Exceptions.SQLExceptions/OracleErrorTest1	2.16 s	2.74 %	50	43.24 ms
Java	com.Exceptions.MySimpleJob/execute	1.03 s	1.31 %	50	20.64 ms
Java	com.Exceptions.SQLExceptions/OracleTest	240.00 ms	0.3 %	50	4.80 ms
ActiveMQ	192.168.2.213:61616	182.00 ms	0.23 %	250	0.73 ms
thrift	/Error/TestDemo/	130.00 ms	0.16 %	50	2.60 ms
Redis	192.168.2.213:6379/0	100.00 ms	0.13 %	100	1 ms
Java	org.springframework.*	12.00 ms	0.02 %	50	0.24 ms

< 1 2 >

10条/页

您可以对表格进行以下操作：

- 表格默认展示所有分类的代码块，单击所有分类下拉菜单，可以根据已有请求组件进行分类选择。
- 在搜索框中输入代码段的名称，然后单击搜索图标，可查看指定代码段的信息。支持模糊搜索，不区分大小写。
- 单击 Database 代码段可进入 **SQL 分析** 页面，具体描述可参见 [SQL 分析](#)。
- 单击 NoSQL 代码段可查看详情。
- 表格默认一页显示 10 条数据，单击右下角的下拉菜单可以选择 20 条/页、30 条/页、40 条/页、50 条/页。



- 单击列表右上角的自定义表头图标，可以勾选想要展示的列名。默认展示全部。
- 单击列表右上角的**导出**按钮，可将全部代码段列表导出为 CSV 格式。

3.6.4 响应时间分布

要进入指定请求的响应时间分布页面，有以下两种方式：

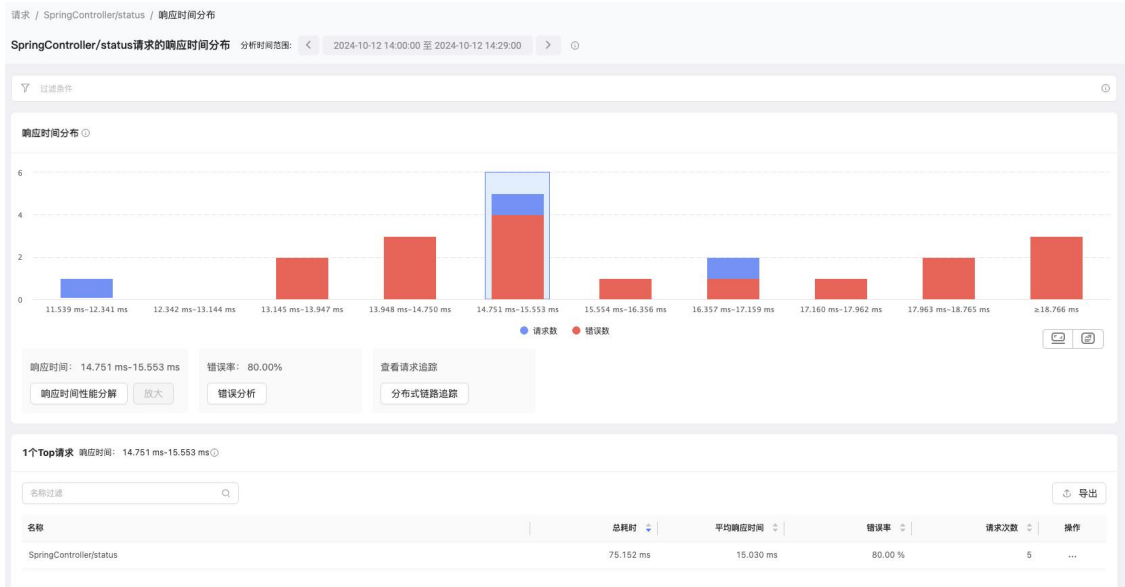
- 单击请求列表**操作**列的“…”，在菜单中选择**响应时间分布**。
- 在**请求概览**页面单击**指标**右上角部分“…”的**响应时间分布**按钮。

在该页面中，您可以修改分析时间范围和过滤数据。

- **修改分析时间范围**：该页面中，所有图表的分析时间范围是**请求详情**页面服务视角下选择的分析范围，可在页面顶部**分析时间范围**处修改时间范围。
- **过滤**：单击页面顶部的过滤框，弹出过滤条件，选择想要查询的条件，立即可看到过滤后的统计数据。过滤条件包括实例名称和响应时间。当按响应时间过滤时，左侧填写最小响应时间，右侧填写最大响应时间，响应时间支持输入小数，最多为三位小数，单击**确定**即可。

3.6.5 响应时间分布

响应时间分布图是将响应时间最小值和最大值的区间范围分成 10 个区间作为横坐标，响应时间的请求次数和错误次数作为纵坐标。默认选中请求次数最多的响应时间区间，如果请求次数相同，则选中响应时间最大的区间。



- 分布图底部展示已选择的响应时间区间和错误率。
- 单击**响应时间性能分解**按钮，可查看所选区间的响应时间性能分解详情。
- 单击**放大**，可以将当前响应时间范围添加到上方的过滤条件，并将所选时间范围继续分成 10 个区间展示在图表中。

说明：当所选时间范围的请求次数 ≤ 1 ，或响应时间区间 $< 1\text{ms}$ 时，**放大**按钮会置灰。

- 单击**错误分析**按钮，可进入错误的分析页面。
- 单击**分布式链路追踪**，可进入请求追踪页面，查看所选响应时间区间内的请求记录。

3.6.5.1 Top 请求

Top 请求列表展示所选响应时间区间内前 50 的请求信息，包括名称、总耗时、响应时间中位数、错误率和请求次数。默认按总耗时倒序展示。

您可以对列表进行以下操作：

- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定 Top 请求的信息。支持模糊搜索，不区分大小写。
- 单击列表右上角的**导出**按钮，可将全部 Top 请求列表导出为 CSV 格式。导出的数据会受选中的响应时间范围影响。
- 在**操作**列中单击  可将该 Top 请求名称作为过滤条件添加到顶部的过滤框中，同时可立即看到过滤后的统计数据。

3.6.6错误/异常分析

3.6.6.1 错误分析

在请求分析页面中单击**错误分析**进入**错误分析**页面，您可以查看当前请求发生的错误请求，并定位错误的影响范围和错误的根因。

您可以通过选择过滤条件筛选数据，过滤条件包括：实例和错误名称。

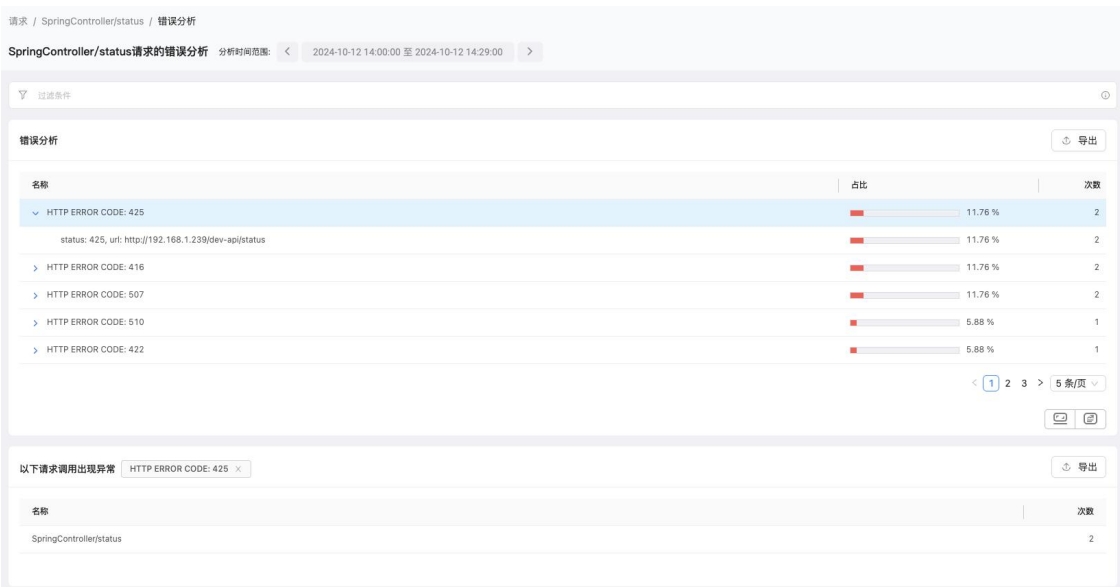
3.6.6.1.1 错误列表

错误分析列表展示当前请求出现的错误信息。列表默认选中错误次数最多的数据，如果没有选择任何数据时，页面仅显示错误分析列表和影响分析，不显示根因分析和追踪列表。

- 名称：请求错误名称。
- 占比：错误次数/请求的错误总数。
- 次数：请求错误次数，列表默认按照错误次数倒排序。

您可以对列表进行以下操作：

- 单击列表右上角的**导出**按钮，可以将整个错误分析列表导出为 CSV 格式。
- 单击一个错误，下方的影响分析、根因分析和追踪列表也相应的展示该错误的分析结果。



3.6.6.1.2 影响分析

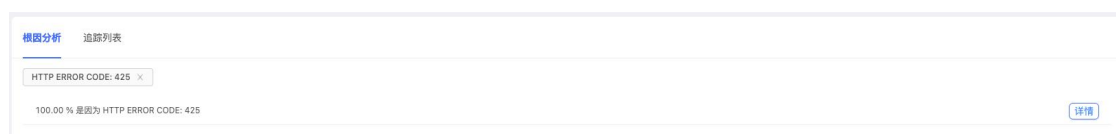
影响分析列表默认显示错误次数最多的错误的请求信息。当单击错误分析列表中任意一条错误信息，请求列表会对应展示所选的这条错误的请求信息。

- 单击标题右侧的**错误名称**后的叉号，可以删除所选错误信息。删除后会显示所有错误的请求信息。
- 单击列表右上角的**导出**按钮，可以将整个请求列表导出为 CSV 格式。
- 单击一个请求，下方的追踪列表展示相应的追踪结果。

3.6.6.1.3 根因分析

根因分析展示某个错误/异常的根因。单击 **StackTrace 分析** 进入新页面查看 root cause 的 Messages 和 StackTrace。

说明：当前错误存在异常堆栈时才显示根因分析，如果没有则默认置灰。



3.6.6.1.3.1 Exception Message 分析

展示当前错误的根因、根因占比和次数。列表默认按照次数倒序排列。

3.6.6.1.3.2 Exception StackTrace 分析

展示上方对应错误的 StackTrace（堆栈追踪）信息。

- +：合并单子集且包名相同的方法。
- 单击  将多个同级节点展开。
- 列表默认展示到第一次分支的地方，子方法默认收起。

3.6.6.1.4 追踪列表

追踪列表展示发生错误的请求追踪信息。

- 发生时间：请求追踪的发生时间，默认按发生时间倒序显示。
- Referer：请求来源地址信息。
- 单击请求名称进入请求追踪详情，具体描述可参考[请求追踪](#)。
- 单击**详情**会弹出错误根因详情，包括错误根因的 Messages 和 StackTrace。

说明：错误存在堆栈时才展示 StackTrace 信息，否则只显示 Messages。

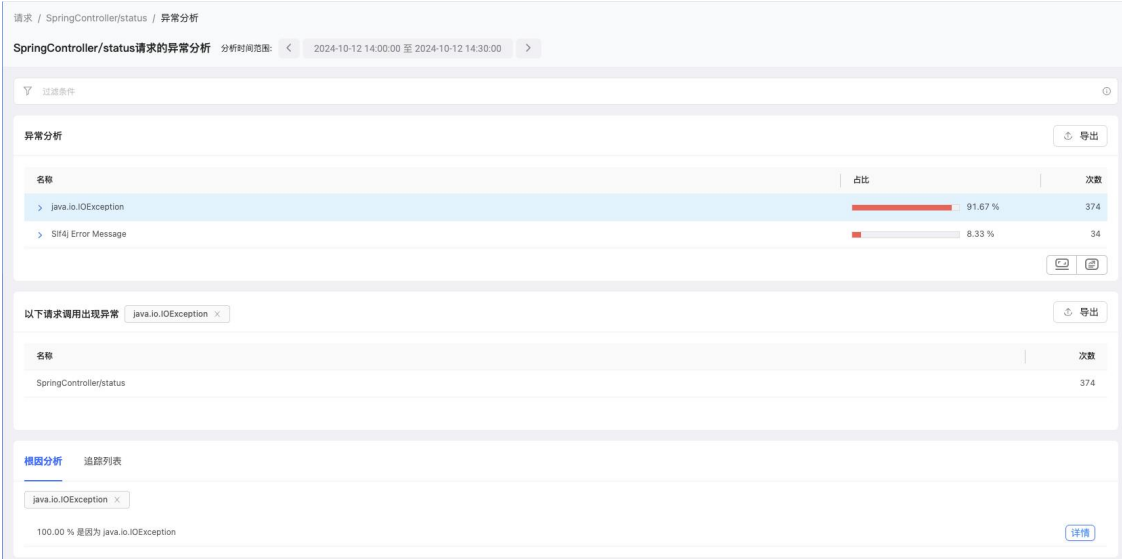
- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定请求信息。支持模糊搜索，不区分大小写。

根因分析 追踪列表				
名称过滤 显示 2 / 2 HTTP ERROR CODE: 425 X				
发生时间	请求名称	实例	Referer	详情
2024-10-12 14:22:29	SpringController/status	wb-node-2-8091	--	Exception details
2024-10-12 14:10:19	SpringController/status	wb-node-2-8091	--	Exception details

3.6.6.2 异常分析

在请求分析页面中单击**异常分析**进入异常分析页面，您可以查看当前请求发生的异常，并定位异常的影响范围和异常的根因。

您可以通过选择过滤条件筛选数据，过滤条件包括：实例、请求类型、请求名称和异常名称。



3.6.6.2.1 异常列表

异常列表展示当前请求出现的异常信息。列表默认选中异常次数最多的数据，如果没有选择任何数据时，页面仅显示异常列表和影响分析，不显示根因分析和追踪列表。

- 名称：请求异常名称。
- 占比：异常次数/请求的异常总数。

说明：异常总数为 Web 请求和后台任务的异常总数。如果请求类型选择了 Web 请求，那么异常总数为 Web 请求的异常总数，反之是后台任务的异常总数。

- 次数：请求异常次数，列表默认按照异常次数倒排序。

您可以对列表进行以下操作：

- 单击列表右上角的**导出**按钮，可以将整个异常分析列表导出为 CSV 格式。

- 单击一个异常，下方的影响分析、根因分析和追踪列表也相应的展示该异常的分析结果。

3.6.6.2.2 影响分析

请求列表默认显示异常次数最多的请求信息。当单击异常分析列表中任意一条异常信息，请求列表会对应展示所选的这条异常的请求信息。

- 单击标题右侧的**异常名称**后的叉号，可以删除所选异常信息。删除后会显示所有异常的请求信息。
- 单击列表右上角的**导出**按钮，可以将整个请求列表导出为 CSV 格式。
- 单击一个请求，下方的追踪列表展示相应的追踪结果。

3.6.6.2.3 根因分析

根因分析展示某个错误/异常的根因。单击**详情**进入新页面查看 root cause 的 Messages 和 StackTrace。

说明：当前异常存在异常堆栈时才展示根因分析，如果没有则默认置灰。

3.6.6.2.3.1 Exception Message 分析

展示当前异常的根因、根因占比和次数。列表默认按照次数倒序排列。

3.6.6.2.3.2 Exception StackTrace 分析

展示上方对应异常的 StackTrace（堆栈追踪）信息。

- +：合并单子集且包名相同的方法。
- 单击  将多个同级节点展开。
- 列表默认展示到第一次分支的地方，子方法默认收起。

3.6.6.2.4 追踪列表

追踪列表展示发生异常的请求追踪信息。

- 发生时间：请求追踪的发生时间，默认按发生时间倒序展示。
- Referer：请求来源地址信息。
- 单击请求名称进入请求追踪详情，具体描述可参考[请求追踪](#)。

- 单击**详情**会弹出异常根因详情，包括异常根因的 Messages 和 StackTrace。
- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定请求信息。支持模糊搜索，不区分大小写。

3.6.7线程剖析

当请求的响应时间过长时，您可以选择对某个应用实例进行请求级别的线程剖析，采集线程状态，定位该请求中性能消耗最大的线程和方法，进而突破性能瓶颈。

对请求进行线程剖析的步骤如下：

- 进入请求详情页面后，单击**线程剖析**页签。
- 在页面右上角单击**新建剖析**按钮，弹出**新建线程剖析**对话框。

新建线程剖析

选择实例

不允许选择CPU利用率>80%、内存使用率>80%或者探针处于熔断状态的实例。推荐单选，多选亦会为每个实例单独创建剖析，剖析结果独立查看

实例名称	CPU%	Memory(MB)	Memory(%)	Requests	响应时间	探针熔断
☒ localhost:localhost:8095	0.6	483.22	27.79	7	1004	否

CPU、Memory 为该实例 所有事务 最近5分钟的平均值，Requests、响应时间为该实例 当前事务 最近5分钟的平均值。

持续时间：2 分钟

采样间隔：10 毫秒

10
 请输入 1 ~ 100,000 的整数

1
 2
 3
 4
 5
 6
 7
 8
 9
 10

事务最大请求次数：100

100
 请输入 1 ~ 1000 的整数

剖析时间范围内，该事务的请求数超过设置的值时（默认：100），将不再进行剖析。持续时间和事务最大请求次数两个条件满足其一，即结束剖析。

开始剖析

取消

- 勾选要进行线程剖析的应用实例、持续时长、采样间隔和请求最大请求次数。

应用实例：一个请求可能会在多个应用实例中执行，因此涉及应用实例的选择。

请求最大请求次数：剖析时间范围内，请求的请求数超过设置的值时（默认值为 100），将不再进行剖析。持续时间和请求最大请求次数两个条件满足其一，即结束剖析。

说明：应用实例的 CPU 使用率和内存使用率大于 80%时，不允许执行线程剖析。

- 单击**开始剖析**按钮，剖析任务将显示在剖析列表中。

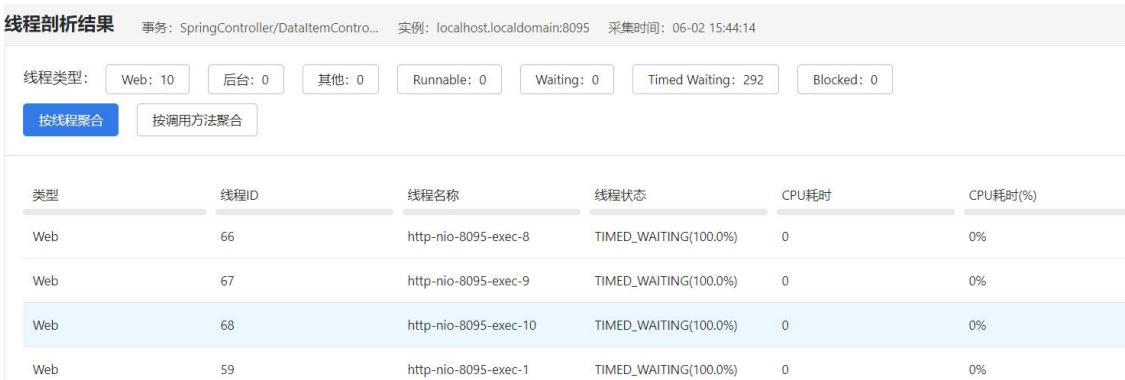
新的剖析任务会等待任务下发，大约 1 分钟后开始执行。支持自定义表头和批量删除剖析任务。



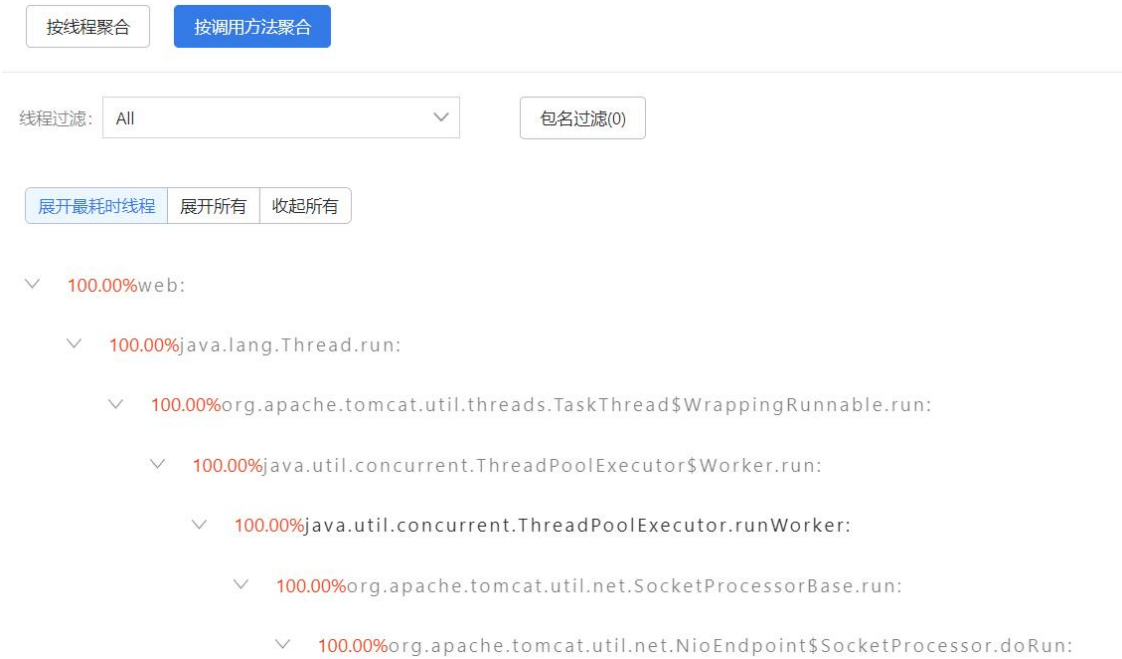
5. 当状态列显示为“已完成”时，可单击该条任务查看剖析结果。

上方显示各种类型、各种执行状态的线程的数量。

支持按线程聚合和按调用方法聚合 2 种聚合方式，单击按钮进行切换。



当选择按调用方法聚合时，可查看调用该请求的所有线程的代码调用堆栈详细信息。默认展开最耗时线程的堆栈。支持根据线程过滤调用堆栈。单击**包名过滤**按钮，选中的包名将不显示在堆栈中。单击**收起所有页签**，仅展示 Web、后台和其他 3 类线程的耗时百分比。



3.6.8 自定义指标

在请求概览页面的右上角，单击**自定义指标**按钮，可进入应用的自定义指标分析页面。在该页面可查看每个指标中各个数据项的访问次数、指标的总响应时间、平均响应时间、请求次数、吞吐率、错误率和错误数。

说明：要分析自定义指标，请先在**应用配置**页面中对自定义指标进行配置。

- 单击页面右上角的**导出自定义指标**按钮，可将自定义指标列表导出为 Excel 到本地。
- 单击页面右上角的**自定义表头**按钮，勾选想要展示的列名，并可以调整列宽。部分字段是必选项，必须展示。单击**恢复默认**可恢复到系统默认展示的列和宽度。
- 单击下方维度分析列表右上角的**导出**按钮，可将维度分析列表数据导出为 Excel 到本地。
- 单击一个指标，系统会根据指标中的数据项的值进行排列组合，并展示每种组合的访问次数变化趋势。同时还支持以列表的形式分别统计每种组合的访问次数、指标的总响应时间、平均响应时间、请求次数、吞吐率、错误率和错误数。
- 单击一个维度，可跳转到请求追踪页面，查看该种参数组合条件下，涉及的所有请求的追踪列表。

3.7 事务

事务是业务系统中完成某一业务操作的一组用户请求。例如电商业务系统中的提交订单，OA 系统中的审批签署等等。单个的用户请求是事务的一个实例。事务属于业务系统，可以穿透和跨越整个系统中的业务系统以及业务系统中的应用，以第一个收到请求的应用为事务的入口应用，穿透的其他的应用是该事务的下游应用，当事务穿透和跨越应用时，在下游应用上的访问不再产生新的事务。入口应用所在的业务系统为该事务所属的业务系统，当事务穿透和跨越业务系统时，在下游的业务系统不再生成新的事务，而是依然属于入口业务系统。

3.7.1 事务列表

单击左侧导航栏中的**事务**，进入事务列表页面。该列表默认展示当前业务系统下所有事务的基本信息，默认按总耗时进行排序。各个列表项说明如下：

- **健康度：**事务的健康状态。当显示为“严重”或者“警戒”时，可单击健康度进入**健康度分析**页面，查看该事务的健康详情，具体请参见[健康度分析](#)。
- **别名：**事务的别名。设置别名可方便用户识别事务。未设置时，别名默认展示为事务名称。

- 名称：事务名称。事务名称由探针自动识别或用户自定义的事务命名规则来命名。**事务名称**的下方会展示涉及的数据项，如果不涉及则不显示。单击事务名称可进入该事务的概览页面。
- 总耗时：统计周期内该事物被请求多次的耗时之和。
- 耗时百分比：统计周期内该事务的总耗时占应用中所有事务总耗时的百分比。
- 入口应用：当前事务所属应用，即产生当前事务的入口应用。单击该应用可进入该应用概览页面。
- 其他指标说明，请参见《应用与微服务指标说明》。

说明：每个应用仅展示总耗时 Top 2000 的请求名称，包括事务和服务接口，超出后将会被聚合成 **URI/OTHER/***。您可通过**事务命名**功能将指定的 URI 重新命名，从 **URI/OTHER/***拆解为独立的请求。

您可以对事务进行以下操作：

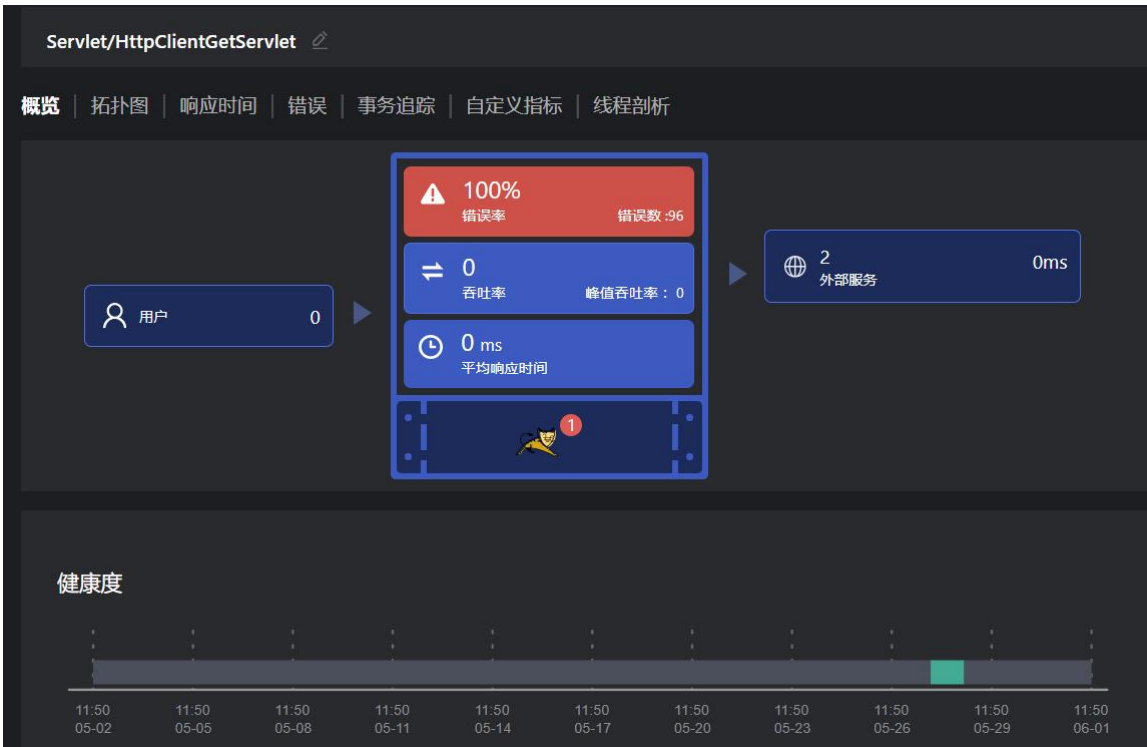
- 事务列表提供按业务系统、所属应用和事务名称（支持别名）进行搜索的功能，同时可以按各指标字段进行排序。
- 单击右上角的**自定义表头**，勾选想要展示的列名，并可以调整列宽。部分字段是必选项，必须展示。单击**恢复默认**可恢复到系统默认展示的列和宽度。
- 单击事务列表中的事务名称进入指定事务的详情页面，可查看事务概览、事务拓扑图、响应时间、错误、事务追踪、自定义指标和线程剖析。
- 单击**关注**列的图标，可将当前事务加入关注事务列表，在上方**我的关注**页签中查看。再次单击可取消关注。

业务系统	default	应用	全部	请输入事务名称	Q	查看全部	🔍	🔄	🔍	🔍
🔍	🔍	🔍	🔍	🔍	🔍	🔍	🔍	🔍	🔍	🔍
健康度	事务名称	名称	平均响应时间...	总耗时(ms)	耗时百分比...	请求数	吞吐量...	错误率(%)	错误数	入口应用
未知	SpringController/v1/cu/config/listener (POST) 123	SpringController/v1/cu/config/listener (POST)	9138	1151338	62.86	126	0	0	0	com.alibaba.nacos/nacos
未知	SpringController/shop/list	SpringController/shop/list	2009	122578	6.69	61	0	0	0	sign-mock-agent-1
未知	URI/server-api/graph/query/overview	URI/server-api/graph/query/overview	2235	101468	5.54	45	0	0	0	apm-api-gw-new
未知	URI/server-api/action/trace/detail/queryAgentVersionInfo	URI/server-api/action/trace/detail/queryAgentVersionInfo	60235	60235	3.29	1	0	0	0	apm-api-gw-new
未知	URI/proxy/*	URI/proxy/*	22	58065	3.17	2672	0	100	2672	localhost80
未知	URI/nginx.php	URI/nginx.php	3	55424	3.03	18385	0	0	0	localhost80
未知	URI/server-api/graph/query/diagram	URI/server-api/graph/query/diagram	2256	45125	2.46	20	0	0	0	apm-api-gw-new
未知	URI/server-api/graph/suggestions	URI/server-api/graph/suggestions	3346	42199	2.3	13	0	0	0	apm-api-gw-new

3.7.2 概览

单击事务列表中的事务名称进入指定事务的详情页面。系统默认展示事务的概览页签。概览页签展示当前事务在统计周期内的整体性能统计数据 and 图表。

单击事务名称后的编辑图标，可为事务设置别名，设置完成后，当前位置以及事务列表中将会展示事务的别名。该别名仅用于展示，目前不支持根据别名来搜索事务。如需恢复原事务名称，删除别名后单击**确定**即可。



概览标签页中，各个指标的说明如下：

- 用户：访问了该事务的用户数量。单击后，下方显示详细的用户列表。

用户 访问了该事务的用户信息。

请输入用户...

用户	平均响应时间(ms)	请求次数	错误次数	错误率(%)
cookie1	368	12	0	0

- 平均响应时间：该事务在统计周期内的平均响应时间。单击后，可跳转到**响应时间**页签查看详情。
- 错误率：该事务本身及其调用的服务接口在统计周期内的发生错误和不影响事务的异常的事务次数占事务总执行次数的百分比。单击后，可跳转到**错误**页签查看错误详情。
- 错误数：该事务本身及其调用的服务接口在统计周期内的发生错误和不影响事务的异常的数量。
- 吞吐量：该事务在统计周期内每秒平均被访问的次数。
- ：为该事务访问过程提供资源支撑的实例信息，亦包括服务接口的实例，图标**的**右上角显示实例的数量。单击实例名称，跳转到实例分析页面。

实例 为该事务访问过程提供资源支撑的实例信息，亦包括服务接口的实例，点击实例名称，跳转到实例分析页面。

实例名称	应用	平均响应时间(ms)	请求数	错误数/错误率	CPU(%)	内存(MB)
DESKTOP-A40OUDR:8104	liuqq_3.3.0_cloud	2,326	2	0 0%	0.29	830.6
DESKTOP-A40OUDR:8090	liuqq_3.3.0_p	429	2	0 0%	0.13	819.71
DESKTOP-A40OUDR:0	liuqq_3.3.0_cloud	368	12	0 0%	0.01	844.8
DESKTOP-A40OUDR:0	liuqq_3.3.0_p	41	11	0 0%	0.01	849.42

- 外部服务：当前事务调用的外部服务的次数和平均调用时间。单击某一行跳转到外部服务分析页面。支持自定义表头。

外部服务 分析当前事务调用的外部服务，点击某一行跳转到外部服务分析。

请输入 URL...

URL	平均响应时间(ms)	调用次数	错误率(%)	错误次数
DESKTOP-A40OUDR:8090	7	14	0.00	0

自定义表头

- 服务组件：如果事务调用了数据库、NoSQL 或者 MQ，也会相应的显示调用该组件的次数和组件的平均响应时间。单击某一行跳转到分析页面。支持自定义表头。

Redis 分析当前事务调用的Redis，点击某一行跳转到NoSQL分析详情。

请输入实例...

类型	实例	平均响应时间(n)	调用次数	占比(%)	频率	错误率(%)	错误次数	调用者(服务接口)
Redis	apm-redis-001.syh.tingyun.com:...	0	5520	100.00	3.52	0.00	0	

自定义表头

- 健康度：以堆叠条形图的形式展示当前统计周期内该事务的健康度变化情况，每个时间段的颜色以当时时间段内该事务的健康度颜色填充。当健康状态为严重或警戒时，支持单击条形图下钻到[健康度分析](#)页面查看详情。具体请参见[健康度分析](#)。
- Apdex 趋势图：当前事务的 Apdex 指数的变化趋势。右上角显示用于计算 Apdex 指数的 Apdex T 和统计时间周期内的平均 Apdex 指数值。
- 基本指标趋势图：展示当前事务在统计周期内的响应时间、请求和错误率三个重要指标的历史趋势图。
 - 响应时间：展示平均响应时间和 4 个自定义分位值响应时间的历史曲线，鼠标悬停在曲线上时显示当前时间点的对应响应时间指标数值并在其他两个趋势图中联动显示其他指标。右上角展示统计时间段内该事务的平均响应时间。
 - 请求：展示吞吐率的历史趋势图和请求数的条形图，并在其他两个趋势图中联动显示其他指标。右上角分别展示统计时间段内该事务的请求总次数和平均每秒请求数。

- 错误率：展示当前事务错误率的趋势图和错误数的条形图，并在其他两个趋势图中联动显示其他指标。右上角的 3 个数值分别代表统计时间段内该事务发生的错误总次数、每分钟的错误数和平均错误率，不含异常的统计。

3.7.3 事务拓扑图

- 拓扑图

系统以拓扑图的形式展示当前事务在统计周期内的调用关系，拓扑图中展示由当前事务的请求所调用的当前业务系统中的所有应用和服务组件，以及其间接调用的当前业务系统之外的其他业务系统。应用、服务组件和业务系统之间以连线的形式展示当前事务所经过的调用关系。拓扑图上，当前事务的入口应用以“入口”的字样标注出来。

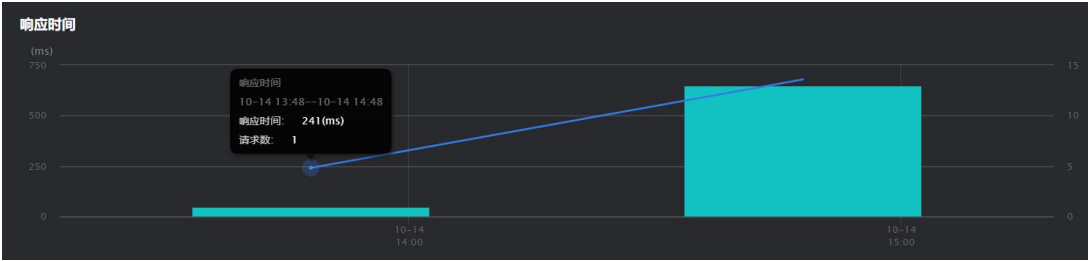
将鼠标悬停在应用、服务组件或业务系统的图标上，可查看平均响应时间、平均吞吐率等具体信息。单击图标名称可进入对应对象的概览页面。拓扑图连线上展示在当前事务调用过程中应用之间、应用与服务组件之间、应用与其他业务系统之间调用的平均响应时间、平均吞吐率和平均错误率，将鼠标悬浮在这些数字可查看平均响应时间缩略图等。单击连线会以弹出对话框窗口的形式展示应用与组件、应用与应用、应用与业务系统之间的详细调用情况，例如上游的事务或服务接口、数据库操作、相关的异常和追踪信息。

当组件数量超过 50 个时，系统会根据响应时间从高到低排序，仅展示前 50 个组件。

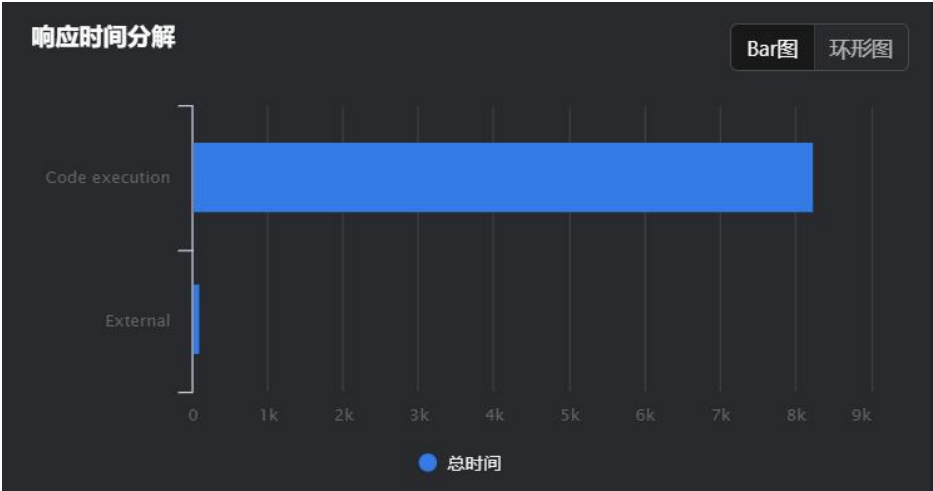
- Top 10 错误：显示当前事务中出现率最高的前 10 种错误的趋势图。
- 健康度：以堆叠条形图的形式展示当前统计周期内该事务的健康度变化情况，每个时间段的颜色以当时时间段内该事务的健康度颜色填充。当健康状态为严重或警戒时，支持单击条形图下钻到[健康度分析](#)页面查看详情。具体请参见[健康度分析](#)。
- 基本指标趋势图：展示当前事务在统计周期内的响应时间、请求和错误率三个重要指标的历史趋势图。
 - 响应时间：展示平均响应时间和 4 个自定义分位值响应时间的历史曲线，鼠标悬停在曲线上时显示当前时间点的对应响应时间指标数值并在其他两个趋势图中联动显示其他指标。
 - 请求：展示吞吐率的历史趋势图和请求数的条形图，并在其他两个趋势图中联动显示其他指标。
 - 错误率：展示当前事务错误率的趋势图和错误数的条形图，并在其他两个趋势图中联动显示其他指标。右上角的 3 个数值分别代表统计时间段内该事务发生的错误总次数、每分钟的错误数和平均错误率，不含异常的统计。

3.7.4 响应时间

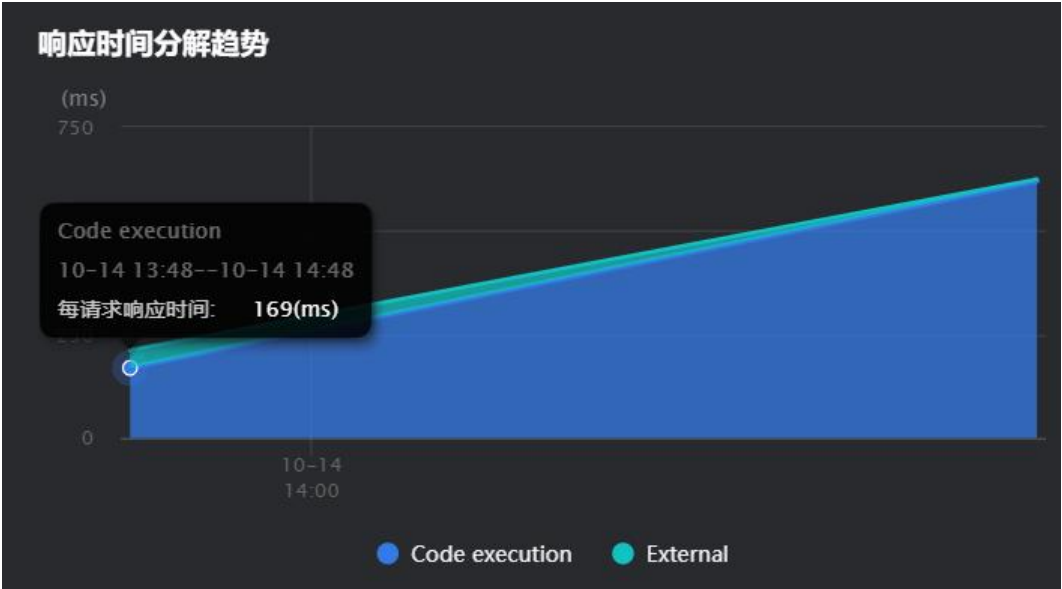
- 响应时间：显示以粒度时间为统计段的事务请求响应时间的变化趋势图和请求次数条形图。左侧的纵坐标为响应时间，右侧的纵坐标为请求次数。



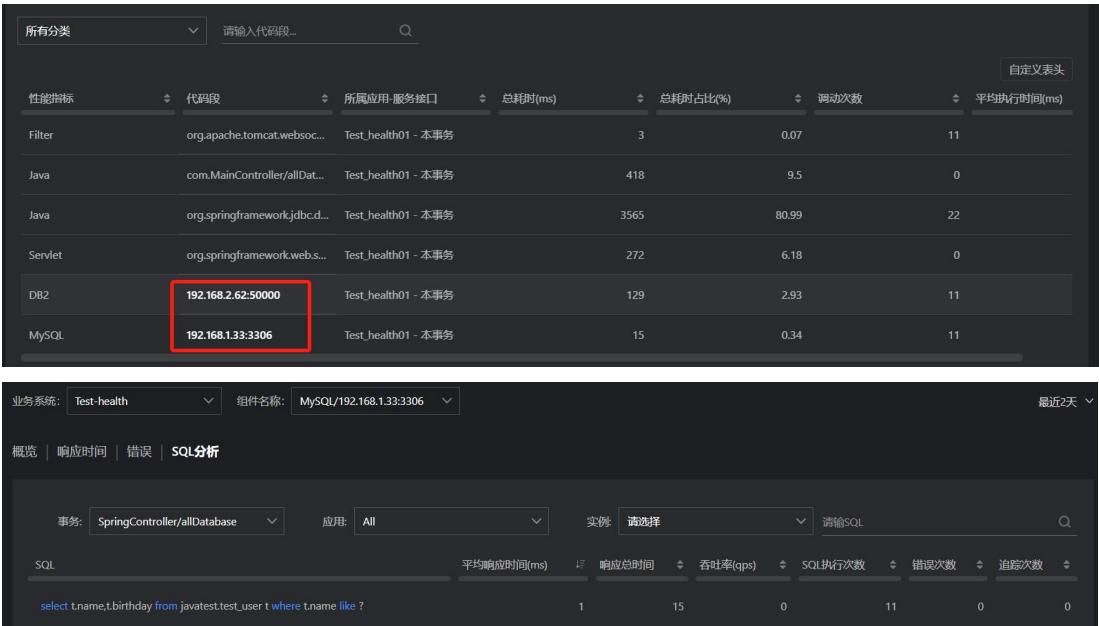
- 响应时间分解：以 Bar 图（条形图）和环形图的方式显示事务响应时间的构成部分分别所占用的时间。构成部分包含 Code execution（代码执行）、database（数据库访问）、NoSQL 访问、MQ 访问、External（即外部服务）和连接池的获取连接耗时 6 部分。



- 响应时间分解趋势：以堆叠图的形式展示每一次事务响应时间的构成部分，包含 Code execution（代码执行）、数据库访问、NoSQL 访问、MQ 访问、External（即外部服务）和连接池的获取连接耗时 6 部分。



- 响应时间分解表格：展示当前事务的各个重要方法（即代码段）的耗时趋势图。图表展示前 5 个最耗时的代码段以及其他（耗时小于 2%的代码段会汇总到“其他”类别中）。每个点的响应时间数值为粒度时间内代码段的平均执行时间。
- 代码段列表：展示当前事务（对应着多个请求）涉及的所有代码段性能分类、代码段、所属应用和服务接口、总耗时、耗时百分比、调用次数和平均执行时间。代码段按总耗时降序排序。支持自定义表头。
 - 代码段部分以更简洁、易懂的形式进行展现，例如调用数据库代码部分会展现为：域名/数据库名称。如果发现数据库代码耗时较长，可单击数据库代码段链接，跳转到该数据库组件的 **SQL 分析** 页面，查看该事务调用当前数据库的所有 SQL 语句详情。默认按照平均响应时间排序。

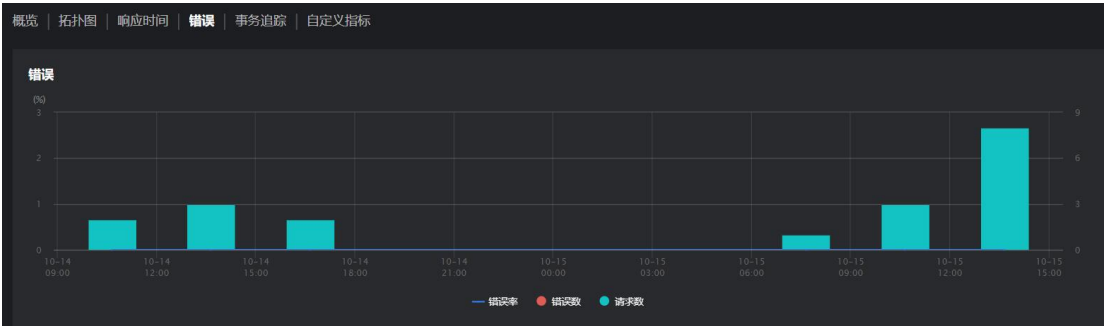


在 **SQL 分析** 页面中单击一条 SQL 语句，可查看该 SQL 语句的详细统计信息，包括概要信息、完整的 SQL 语句、统计周期内的平均响应时间&吞吐率图表、错误率图表、SQL 调用者的分析、错误分析、慢 SQL 追踪列表和发生错误和异常的 SQL 追踪列表。追踪列表部分展示包含该 SQL 语句的事务追踪的列表，单击列表项可跳转到对应的事务追踪详情页面，并定位到当前 SQL 语句的执行代码段位置。

- 性能指标是按照应用的代码类型对代码段大致归类。其中“External”类型是外部服务调用，“Hotspot”类型是热点方法，“Custom”类型是自定义嵌码的方法。
- 所属应用-服务接口：当前代码段所属的应用和所对应的下游应用的服务接口。
- 总耗时：各代码段在统计周期内的总耗时。
- 总耗时占比=当前代码段的总耗时/各代码段在统计周期内的总耗时。
- 调用次数：当前代码段在统计周期内被调用的次数。
- 平均响应时间：当前代码段在统计周期内的平均执行时间。

3.7.5 错误

错误页签中，下图显示错误率和错误数的趋势图、请求数的条形图。



下方的错误列表展示该事务发生的所有错误和异常。支持根据异常类型、异常名称、用户名称和参数进行过滤查询。

所有异常和错误	请输入异常名称...	请输入用户名称...	参数	+	搜索
持续时间(分钟)	异常类型	异常名称	影响用户数	发生次数	占比(%)
27	事务错误	500 error	1	9	100
					7

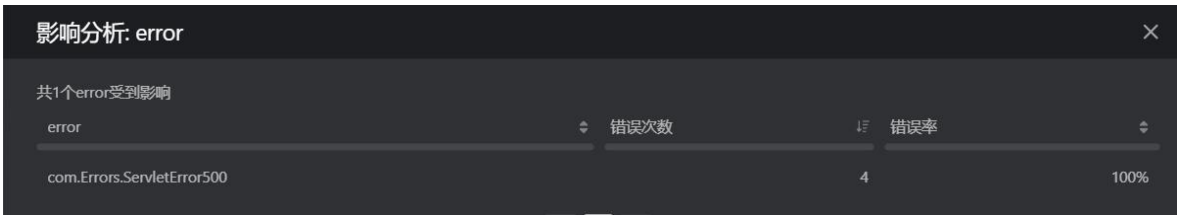
单击列表中的影响用户数，弹出用户信息窗口，显示具体受影响的用户。



单击列表中的追踪次数，可跳转到**事务追踪**页面，默认显示包含该错误的事务追踪。



列表中异常名称下会显示数据项，单击数据项弹出**影响分析**窗口，显示数据项各种取值情况下发生的事务错误数和错误率。错误列表可自定义表头。



在事务详情页面中，还可以通过上部的页签查看当前事务的追踪信息和事务涉及的自定义指标，详细介绍请参见**事务追踪**和**自定义指标**。

3.7.6 线程剖析

当事务的响应时间过长时，您可以选择对某个应用实例进行事务级别的线程剖析，采集线程状态，定位该事务中性能消耗最大的线程和方法，进而突破性能瓶颈。

对事务进行线程剖析的步骤如下：

- 1. 进入事务详情页面后，单击**线程剖析**页签。
- 2. 在页面右上角单击**新建剖析**按钮，弹出**新建线程剖析**对话框。

新建线程剖析

×

选择实例

不允许选择CPU利用率>80%、内存使用率>80%或者探针处于熔断状态的实例。推荐单选，多选亦会为每个实例单独创建剖析，剖析结果独立查看

☒	实例名称	CPU%	Memory(MB)	Memory(%)	Requests	响应时间	探针熔断
☒	localhost.localdomain:8095	0.6	483.22	27.79	7	1004	否

CPU、Memory 为该实例 所有事务 最近5分钟的平均值，Requests、响应时间为该实例 当前事务 最近5分钟的平均值。

持续时间：2 分钟

采样间隔：10 毫秒

100

请输入 1 ~ 1000 的整数

10

请输入 1 ~ 100,000 的整数

事务最大请求次数：100

100

请输入 1 ~ 1000 的整数

剖析时间范围内，该事务的请求数超过设置的值时（默认：100），将不再进行剖析。持续时间和事务最大请求次数两个条件满足其一，即结束剖析。

开始剖析

取消

3. 勾选要进行线程剖析的应用实例、持续时长、采样间隔和事务最大请求次数。
- 应用实例：一个事务可能会在多个应用实例中执行，因此涉及应用实例的选择。
- 事务最大请求次数：剖析时间范围内，事务的请求数超过设置的值时（默认值为 100），将不再进行剖析。持续时间和事务最大请求次数两个条件满足其一，即结束剖析。
- 说明：**应用实例的 CPU 使用率和内存使用率大于 80%时，不允许执行线程剖析。
4. 单击**开始剖析**按钮，剖析任务将显示在剖析列表中。
- 新的剖析任务会等待任务下发，大约 1 分钟后开始执行。支持自定义表头和批量删除剖析任务。

SpringController/DataItemController/paramShortReturnByteMapping

关注

概览

拓扑图

响应时间

错误

事务追踪

自定义指标

线程剖析

请输入实例

Q

新建剖析

删除

自定义表头

☐	时间	状态	持续时间	来源	实例	请求数	线程总数	Runnable	Blocked	Waiting	T
☐	06-02 15:44:14	已完成	2分	手动触发	localhost.localdomain:8095	32	10	0	0	0	

5. 当状态列显示为“已完成”时，可单击该条任务查看剖析结果。
- 上方显示各种类型、各种执行状态的线程的数量。
- 支持按线程聚合和按调用方法聚合 2 种聚合方式，单击按钮进行切换。

线程剖析结果

事务: SpringController/DataItemContro...

实例: localhost.localdomain:8095

采集时间: 06-02 15:44:14

线程类型:

Web: 10

后台: 0

其他: 0

Runnable: 0

Waiting: 0

Timed Waiting: 292

Blocked: 0

按线程聚合

按调用方法聚合

类型	线程ID	线程名称	线程状态	CPU耗时	CPU耗时(%)
Web	66	http-nio-8095-exec-8	TIMED_WAITING(100.0%)	0	0%
Web	67	http-nio-8095-exec-9	TIMED_WAITING(100.0%)	0	0%
Web	68	http-nio-8095-exec-10	TIMED_WAITING(100.0%)	0	0%
Web	59	http-nio-8095-exec-1	TIMED_WAITING(100.0%)	0	0%

当选择按调用方法聚合时，可查看调用该事务的所有线程的代码调用堆栈详细信息。默认展开最耗时线程的堆栈。支持根据线程过滤调用堆栈。单击**包名过滤**按钮，选中的包名将不显示在堆栈中。单击**收起所有**页签，仅展示 Web、后台和其他 3 类线程的耗时百分比。

按线程聚合

按调用方法聚合

线程过滤: All

包名过滤(0)

展开最耗时线程

展开所有

收起所有

100.00%web:

100.00%java.lang.Thread.run:

100.00%org.apache.tomcat.util.threads.TaskThread\$WrappingRunnable.run:

100.00%java.util.concurrent.ThreadPoolExecutor\$Worker.run:

100.00%java.util.concurrent.ThreadPoolExecutor.runWorker:

100.00%org.apache.tomcat.util.net.SocketProcessorBase.run:

100.00%org.apache.tomcat.util.net.NioEndpoint\$SocketProcessor.doRun:

3.8 服务接口

服务接口是业务系统中非事务入口应用（下游应用）中的一组服务请求，通常是事务请求在某个应用上的一段请求，例如认证接口或数据查询接口，单个的服务请求是服务接口的一个实例。每个服务接口都属于具体的一个应用。

在左侧导航栏中单击 **APM>服务接口**，进入服务接口列表界面。服务接口列表展示当前业务系统下统计周期内的所有活跃服务接口信息，默认按平均响应时间进行排序。列表中包含以下的项目：

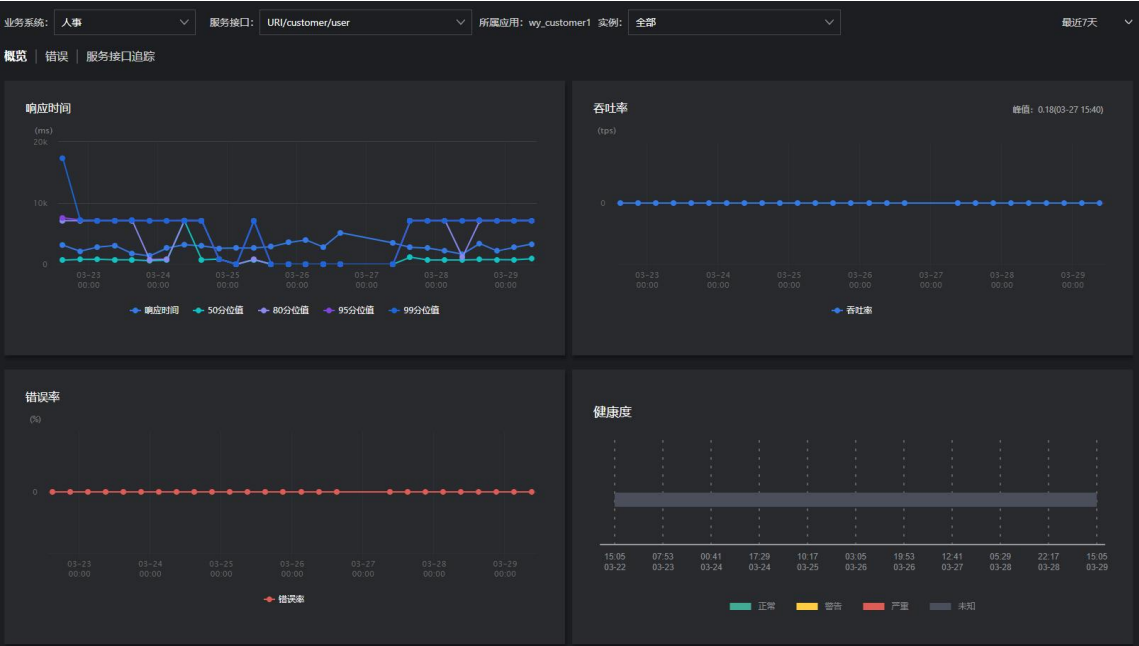
- 名称：服务接口名称，类似于事务名称，由探针自动命名或根据用户设置的自定义规则命名。支持正反排序。

101

- 应用：当前服务接口所属的应用名称，单击可跳转到对应应用的概览页面。支持正反排序。
- 响应时间：当前服务接口在统计周期内的响应时间历史趋势缩略图。
- 其他指标说明，请参见《应用与微服务_指标说明》。

名称	应用	响应时间	平均响应时间(ms)	吞吐量(tps)	健康度
Servlet/testServlet	Error_Zhang		3167	0	未知
Servlet/testservlet	Test01		3138	0	未知
SpringController/sql/oracleExcept...	Error_Zhang		2665	0	未知
SpringController/oraclejdbcTempl...	Error_Zhang		2487	0	未知
SpringController/oraclejdbcTempl...	Welcome to Tomcat		2137	0	未知

单击服务接口名称跳转到对应服务接口的概览界面，服务接口概览界面展示当前服务接口的平均响应时间、吞吐量、错误率、健康度、性能分解的趋势图和性能分解表格。具体的介绍跟事务概览页面一致，请参见 3.4 事务。



错误页签展示当前服务接口的错误率曲线、Top 10 错误统计和异常追踪信息。

服务接口追踪页签展示发生在当前服务接口上的追踪信息，与事务追踪类似，提供根据不同的字段进行查询和检索的功能，展示详情时只展示事务追踪信息中发生在当前服务接口部分的数据。

业务系统: AutoTaskDemo

服务接口: Servlet/default

所属应用: Auto_Application 实例: 全部

最近1个月

概览

错误

服务接口追踪

请输入用户标识...

请输入业务标识...

请输入追踪ID...

请输入请求ID...

输入最小响应时间...

输入最大响应时间...

参数

搜索

☐ 慢追踪	时间	追踪ID	请求ID	服务接口名称	响应时间(ms)	用户标识	业务标识	实例	☐ 异常
	04-02 11:30:29	be7a8c8596fe32c	65543ec3dd0aacc04	Servlet/testServlet	99			k8s-node1:9081	无
	04-02 11:30:26	ae5e2986e244939	8d37c7a890ebca83	Servlet/testServlet	269			k8s-node1:9081	无
	04-02 03:40:04	b5ce9e0887ca74ce	ad239c5ccb65121	Servlet/testServlet	283			k8s-node1:9081	无
	04-02 03:40:01	c80324d7443aedc6	679e1ae86e87b872	Servlet/testServlet	195			k8s-node1:9081	无
	04-02 01:50:04	147f3305570b9116	d3ad4cc09ef1ae24	Servlet/testServlet	67			k8s-node1:9081	无

3.9 后台任务

后台任务展示当前业务系统的应用中不是由事务访问直接或间接调用的后台定时执行的各类任务的统计信息。后台任务通常用来分析定期执行的批处理任务是否按预期的执行计划运行以及运行过程中是否存在性能问题。

单击左侧导航栏中的**后台任务**进入后台任务列表页面，该列表默认展示当前业务系统下所有的后台任务及其相关指标，默认按照平均执行时长从高到低进行排序。列表中显示以下字段：

业务系统: AutoTaskDemo

应用: 全部

请输入任务名称

最近1个月

名称	平均执行时长(ms)	吞吐量(tps)	错误率(%)	应用	实例
Initializer/ServletContextListener/org.apache...		10	0	Auto_Applica...	k8s-node1:9081
Job/DEFAULT/simpleJobDetail		1	0	Auto_Applica...	k8s-node4:9081
Job/DEFAULT/simpleJobDetail		1	0	Auto_Applica...	k8s-node1:9081
Job/DEFAULT/cronJobDetail		0	0	Auto_Applica...	k8s-node1:9081
Job/DEFAULT/cronJobDetail		0	0	Auto_Applica...	k8s-node4:9081

每页显示条数 20条

<
 1
 >

- 名称：后台任务名称。后台任务名称由探针自动识别或用户自定义的事务命名规则来命名。单击后台任务名称可进入该后台任务的概览页面。
- 应用：当前后台任务所属的应用。单击该应用可进入该应用概览页面。
- 实例：当前后台任务所运行在的应用实例。如果应用实例的名称发生了变化，后台任务列表中会新增一条数据来记录原后台任务与新应用实例的对应关系。
- 其他指标说明，请参见《应用与微服务_指标说明》。

后台任务列表提供按业务系统、所属应用和后台任务名称进行搜索的功能，同时可以按各指标字段进行排序。

3.9.1 概览

单击后台任务列表中的后台任务名称进入指定后台任务的概览页面。后台任务概览页面展示以下信息：

- 后台任务拓扑图

系统以拓扑图的形式展示当前后台任务在统计周期内的调用关系，拓扑图中展示由当前后台任务的请求所调用的当前业务系统中的所有应用和服务组件，以及其间接调用的当前业务系统之外的其他业务系统。应用、服务组件和业务系统之间以连线的形式展示当前后台任务所经过的调用关系。拓扑图上，当前后台任务的所属应用以“入口”的字样标注出来。

- 将鼠标悬停在应用、服务组件或业务系统的图标上，可查看平均响应时间、平均吞吐量、错误率等具体信息。
- 单击图标名称可进入对应对象的概览页面。
- 拓扑图连线上展示在当前后台任务调用过程中应用之间、应用与服务组件之间、应用与其他业务系统之间调用的平均响应时间、平均吞吐率和平均错误率，将鼠标悬浮在这些数字可查看响应时间、吞吐量、错误率的缩略图。
- 单击连线会以弹出对话框窗口的形式展示应用与组件、应用与应用、应用与业务系统之间的详细调用情况，例如上游的事务或服务接口、数据库操作、相关的异常和追踪信息。

- 基本指标趋势图：展示当前后台任务在统计周期内的响应时间、请求和错误率三个重要指标的历史趋势图。

- 响应时间：展示平均响应时间和 4 个自定义分位值响应时间的历史曲线，鼠标悬停在曲线上时显示当前时间点的对应响应时间指标数值并在其他两个趋势图中联动显示其他指标。右上角展示统计时间段内该后台任务的平均响应时间。
- 请求：展示吞吐率的历史趋势图和请求数的条形图，并在其他两个趋势图中联动显示其他指标。右上角分别展示统计时间段内该后台任务的请求总次数和平均每秒请求数。
- 错误率：展示当前后台任务错误率的趋势图和错误数的条形图，并在其他两个趋势图中联动显示其他指标。右上角的 3 个数值分别代表统计时间段内该后台任务发生的错误总次数、每分钟的错误数和平均错误率，不含异常的统计。

3.9.2 错误

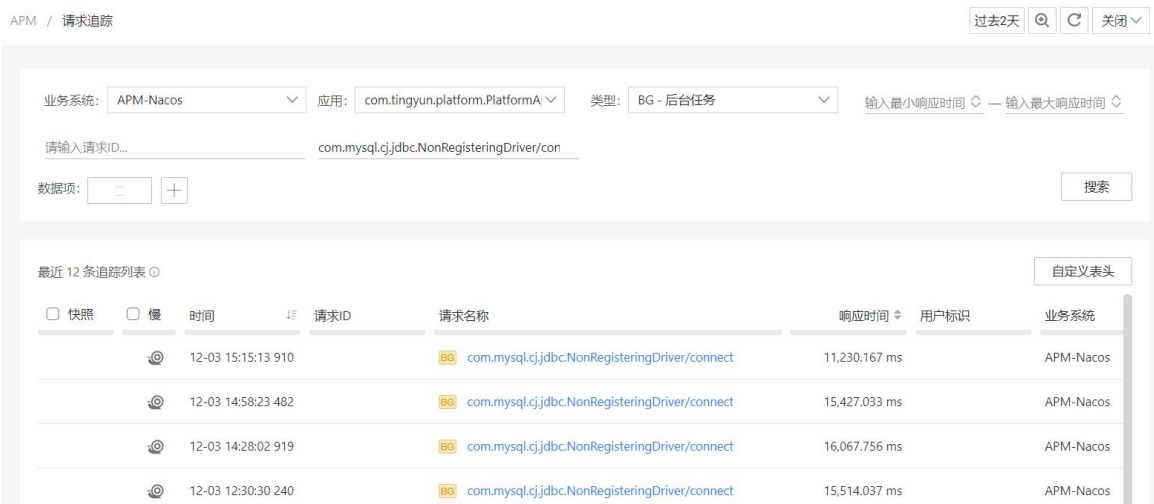
在后台任务概览页面中，通过单击上方的**错误**页签可以查看当前后台任务所发生的错误统计情况。

- 错误率曲线图：错误率是指当前后台任务在粒度时间内发生错误和不影响事务的异常的次数占总执行次数的百分比。
- TOP10 错误：显示在统计周期内当前后台任务出现次数最多的前 10 种错误的趋势图。

- 错误列表：显示在统计周期内当前后台任务出现的各种错误。单击错误条目可以查看当前错误每次发生的详细记录。默认按时间就近的顺序进行排序。

3.9.3 后台任务追踪

在后台任务概览页面中，通过单击上方的[后台任务追踪](#)页签，可以查看当前后台任务所发生的追踪记录。详细介绍请参见[请求追踪](#)。



3.10 Database 服务组件

服务组件是业务系统中的应用所使用的没有被应用探针直接嵌码监控的软件服务，但是其访问流量可以被相关应用中的应用探针监控到，例如数据库、队列服务或缓存服务等等。应用与微服务可监测的服务组件包括三类，分别是“Database 服务组件”、“NoSQL 服务组件”和“MQ 服务组件”。其中“Database 服务组件”展示当前业务系统中所访问的所有 SQL 数据库组件的性能相关数据。



数据库组件页面中，用户可以查看当前业务系统中在统计周期内应用所访问的所有 SQL 数据库组件的列表。数据库组件列表默认按照平均执行时间从高到低进行排序，支持按不同指标项进行排序。

您可以进行以下操作：

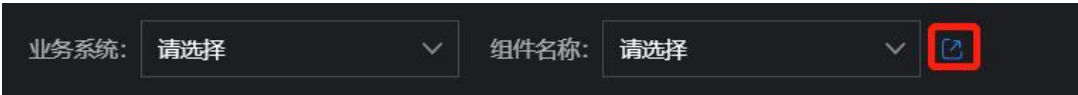
- 单击右上角的**实例**或 **Schema**，根据需要按实例或者数据库的 Schema 对列表中的数据库进行聚合，IP 地址和端口号相同，或者数据库的 Schema 相同，会被聚合成一条出现在列表中。
- 单击右上角的**自定义表头**，勾选想要展示的列名，并可以调整列宽。部分字段是必选项，必须展示。单击**恢复默认**可恢复到系统默认展示的列和宽度。

3.10.1 概览

单击数据库实例名称进入该实例的概览界面。您也可以通过上方的**组件名称**下拉菜单切换到业务系统中的其他数据库实例。数据库组件的概览界面包括以下内容：



- 选择业务系统和组件：在下拉菜单中可随意切换业务系统和 Database 服务组件，此处的组件名称显示：**组件类型/组件 IP 地址:端口号**。单击后面的图标，进入基调听云 Infra 产品，可查看该组件自身的性能情况。组件的钻取需要配置跳转链接，请参见[全局配置](#)中的组件钻取。



- 拓扑图：拓扑图展示当前业务系统中当前数据库实例被应用访问的逻辑调用关系。拓扑图上以图标形式显示当前数据库实例和直接访问该数据库的应用，应用与数据库之间的连线上展示应用对该数据库实例的平均查询时间和平均吞吐率。拓扑图中组件的颜色默认为蓝色。
- 概要信息：展示拓扑图中各个应用访问当前数据库实例的平均响应时间、响应总时间、执行次数、错误率、错误次数、调用该数据库实例的事务的个数、慢次数、当前数据库实例从连接池中获取连接的平均等待时间和最大等待时间、连接池已用连接数量和最大连接数、所在主机 IP 地址和端口号。
- 健康度：以堆叠条形图的形式展示，依据匹配的健康规则计算出的当前数据库实例的健康度变化趋势。

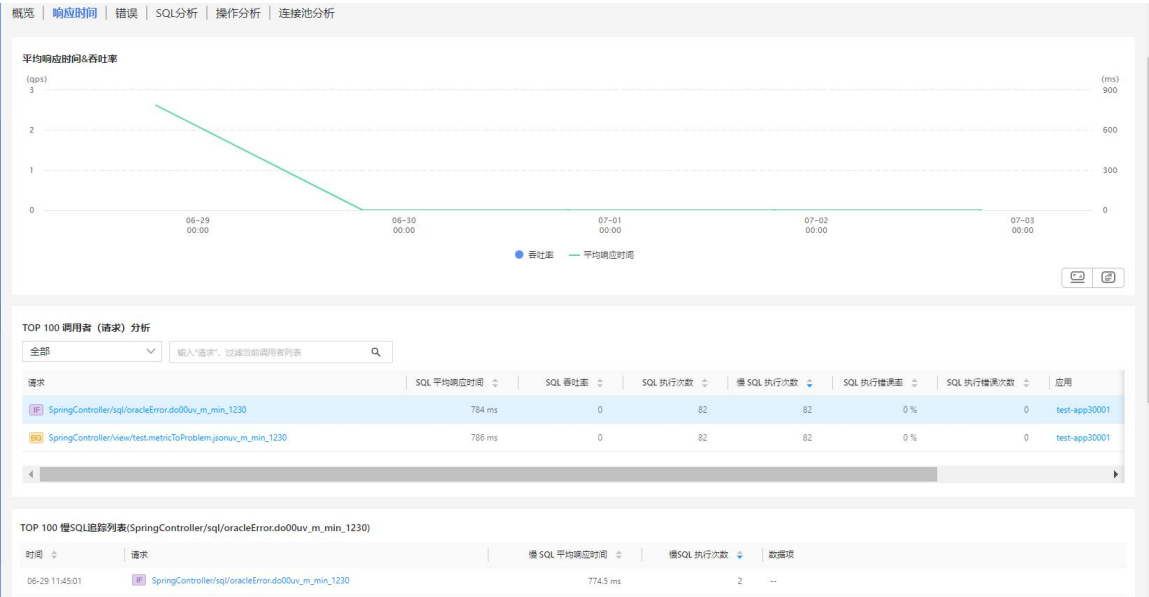
- 基本指标趋势图：以曲线图的形式展示统计周期内当前业务系统访问该数据库实例的平均执行时间、吞吐率和数据库异常数量随时间变化的历史趋势。

3.10.2 响应时间

平均响应时间&吞吐率图表展示当前数据库实例的平均响应时间趋势图和吞吐率条形图。左侧纵坐标为吞吐率，右侧纵坐标为平均响应时间。

TOP 100 调用者（请求）分析列表展示调用该数据库实例次数最多的前 100 个请求的性能详情，包括 SQL 平均响应时间、SQL 吞吐率、SQL 执行次数、慢 SQL 执行次数、SQL 执行错误率、SQL 执行错误次数以及请求所属应用。用户可通过下拉菜单根据请求类型（事务、服务接口和后台任务）进行过滤，输入请求名称进行查询。

TOP 100 慢 SQL 追踪列表展示调用当前数据库实例、SQL 平均响应时间排名前 100 的请求追踪，包括追踪发生时间、请求名称、慢 SQL 平均响应时间、慢 SQL 执行次数和请求中的数据项。在上方单击一个调用者，TOP 100 慢 SQL 追踪列表则展示该调用者调用当前数据库实例、SQL 平均响应时间排名前 100 的请求追踪。



3.10.3 错误

错误页签中显示的是该数据库实例被调用时抛出的异常的统计情况。

错误率图表展示数据库实例的请求错误率、错误数和调用总次数的变化趋势。



错误分析展示该数据库实例被调用时发生的所有异常信息，包括异常信息、发生次数和数据项。

异常	发生次数	数据项
PDOException	2	0

在上方单击一个异常条目，**SQLs** 会展示发生该异常的 SQL 语句以及异常发生次数，**Root cause** 展示该异常的根因。最下方的 **TOP 100 追踪列表** 展示发生该 SQL 错误的错误次数最高的前 100 个请求追踪。单击指定的 SQL，则可查看发生该异常的 SQL 语句的请求追踪。

SQL	次数
select 1	4
SELECT 1	2
SELECT m.id AS "id", m.agreement_id AS "agreementId", m.name AS "name", m.support_type AS "supportType", m.tag_filter_enable AS "tagFilterEnable", m.invalid_time AS "invalidTime" FROM OM_ALARM_MONITOR_POLICY m WHERE m.status = 1 AND m.support_type in (1,3)	1
SELECT license_code FROM PL_U_LICENSE_CODE ORDER BY id DESC LIMIT 1	1
select count(*) from roles where 1=1	1
select count(*) from users where 1=1	1

Root cause
 错误详情
 100% 因为 Communications link failure The last packet successfully received from the server was " milliseconds ago. The last packet sent successfully to the server was " milliseconds ago.

时间	请求	错误次数	数据项
07-04 05:27:27	SpringSchedule/com.alibaba.nacos.console.security.nacos.users.NacosUserDetailsServiceImpl/reload	1	...
07-05 16:48:12	SpringSchedule/com.alibaba.nacos.console.security.nacos.roles.NacosRoleServiceImpl/reload	1	...

3.10.4 SQL 分析

在数据库组件的概览界面上方选择 **SQL 分析** 页签，进入当前数据库实例的 SQL 分析页面。

页面展示当前统计周期内在当前数据库实例上执行的所有 SQL 语句列表及其性能信息，列表默认按平均响应时间从高到底排序。SQL 语句列表可以按发起 SQL 请求的事务、应用名称、实例名称和 SQL 语句内容进行查询。

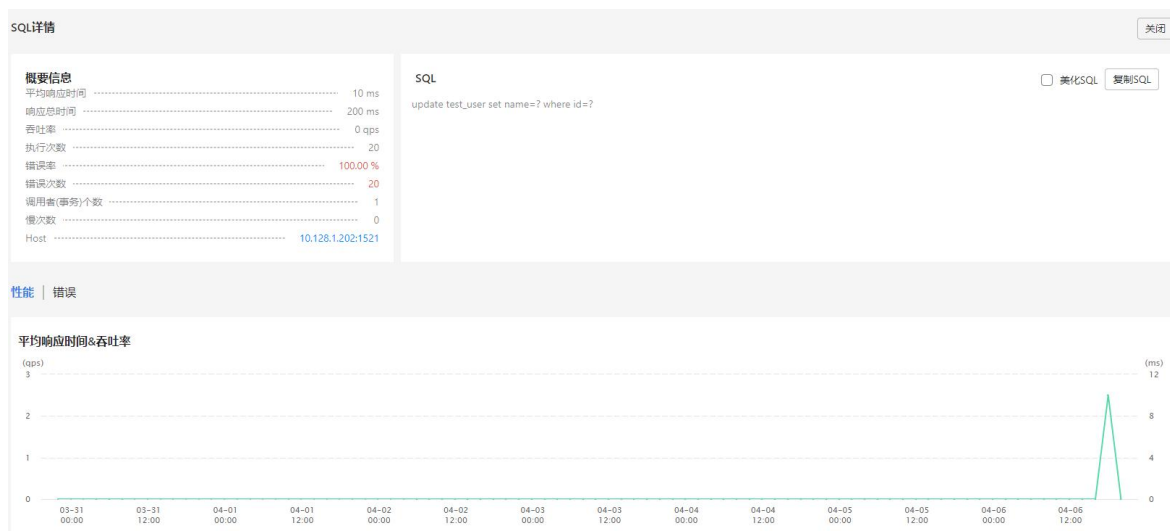
3.10.4.1 SQL 语句列表

SQL 语句列表指标项包括：

- SQL 语句：完整的标准化 SQL 语句，此处展示的 SQL 语句为原始 SQL 语句进行标准化（包括去掉参数值，去掉多余的空格和回车等等）之后聚合而成的 SQL 语句。该处的 SQL 语句中的参数信息会被强制混淆为“？”，不受 SQL 混淆配置影响。
- 平均响应时间：统计周期内，当前数据库实例处理该 SQL 访问的平均执行时间，可正反排序。默认按平均响应时间倒序排序（只查询前 10000 条数据）。
- 响应总时间：统计周期内，当前数据库实例处理多次 SQL 语句的总执行时间。
- 吞吐率：统计周期内，对当前数据库实例进行该 SQL 操作的每秒平均调用次数，可正反排序。
- SQL 执行次数：统计周期内，对当前数据库实例进行该 SQL 操作的总调用次数。
- 错误次数：统计周期内，处理该 SQL 请求时数据库实例发生异常的次数，可正反排序。
- 慢次数：统计周期内，慢 SQL 语句的请求次数，可正反排序。

3.10.4.2 SQL 语句详情

在 SQL 语句列表中单击一条 SQL 语句，可进入该 SQL 语句的详细统计图表页面。详细统计信息中包括概要信息、SQL 语句、性能分析和错误分析。



- 概要信息：展示各个应用访问当前 SQL 语句的平均响应时间、响应总时间、执行次数、错误率、错误次数、调用该 SQL 语句的事务的个数、慢次数、所在主机 IP 地址和端口。
- 完整 SQL 语句：勾选**美化 SQL** 复选框，可将 SQL 语句用彩色字体分行展示。单击右上角的**复制 SQL** 按钮，可复制完整 SQL 语句到剪贴板。

SQL

美化SQL

```
select
  name,
  birthday,
  sleep(?)
from
(
  select
    name,
    birthday
  from
```

- 性能分析
 - 平均响应时间&吞吐率曲线：显示所选的 SQL 语句在统计周期内的平均执行时间和平均吞吐率趋势曲线图。
 - Top 100 调用者（请求）分析：展示调用该 SQL 语句的、SQL 平均响应时间排名前 100 的请求性能详情，包括 SQL 平均响应时间、SQL 吞吐率、SQL 执行次数、慢 SQL 执行次数、SQL 执行错误率、SQL 执行错误次数以及请求所属应用。用户可通过下拉菜单根据请求类型（事务、服务接口和后台任务）进行过滤，输入请求名称进行查询。
 - Top 100 慢 SQL 追踪列表：展示调用该 SQL 语句、SQL 平均响应时间排名前 100 的请求追踪，包括追踪发生时间、请求名称、慢 SQL 平均响应时间、慢 SQL 执行次数和请求中的数据项。在上方单击一个调用者，**TOP 100 慢 SQL 追踪列表**则展示该调用者调用当前 SQL 语句、SQL 平均响应时间排名前 100 的请求追踪。
- 错误分析
 - 错误率曲线：展示所选的 SQL 语句在统计周期内出现异常的执行次数占总执行次数的百分比趋势图、错误数和请求总次数的条形图。
 - 错误分析：展示该 SQL 语句被调用时发生的所有异常信息，包括异常信息、发生次数和数据项。

错误分析		
异常	发生次数	数据项
PDOException	2	0

- 在上方单击一个异常条目，**SQLs** 会展示发生该异常的 SQL 语句以及异常发生次数，**Root cause** 展示该异常的根本原因。最下方的 **TOP 100 追踪列表**展示发生该 SQL 异常的错误次数最高的前 100 个请求追踪。单击指定的 SQL，则可查看发生该异常的 SQL 语句的请求追踪。

SQLs

SQL	次数
select 1	4
SELECT 1	2
SELECT m.id AS "id", m.agreement_id AS "agreementId", m.name AS "name", m.support_type AS "supportType", m.tag_filter_enable AS "tagFilterEnable", m.invalid_time AS "invalidTime" FROM ALARM3_MONITOR_POLICY m WHERE m.status = 1 AND m.support_type in (1,3)	1
SELECT license_code FROM FL_U_LICENSE_CODE ORDER BY id DESC LIMIT 1	1
select count(*) from roles where 1=1	1
select count(*) from users where 1=1	1

1 / 10 条/页

Root cause

错误信息

100% 因为 Communications link failure The last packet successfully received from the server was * milliseconds ago. The last packet sent successfully to the server was * milliseconds ago.

TOP 100 追踪列表(com.mysql.cj.jdbc.exceptions.CommunicationsException)

时间	请求	错误次数	数据项
07-04 09:27:27	SpringSchedule/com.alibaba.nacos.console.security.nacos.users.NacosUserDetailsServiceImpl/reload	1	...
07-05 16:48:12	SpringSchedule/com.alibaba.nacos.console.security.nacos.roles.NacosRoleServiceImpl/reload	1	...

3.10.5 操作分析

在数据库组件的概览页面上方选择操作分析页签，进入当前数据库实例的操作分析页面。

页面上以列表的形式列出当前统计周期内在当前数据库实例上执行的所有操作类型及其性能统计数据，按表名和操作类型进行聚合，列表默认按平均响应时间从高到底排序。用户可以根据操作对操作列表进行查询。

概览 | 响应时间 | 错误 | SQL分析 | 操作分析 | 连接池分析

请输入操作

操作	响应总时间(ms)	平均响应时间(ms)	吞吐率(qps)	执行次数	错误次数	错误率(%)	慢次数
nb_m_option/SELECT	314792	301	0	1046	0	0	353
Table/TRUNCATE	61	61	0	1	0	0	0
nb_m_probe_ad/SELECT	4564	16	0	278	0	0	1
nb_m_user_task/SELECT	10049	15	0	690	0	0	1
nb_m_user_task/INSERT	72	14	0	5	0	0	0

每页显示 5条
 1 2 3 4 5 ... 21

操作列表指标项包括：

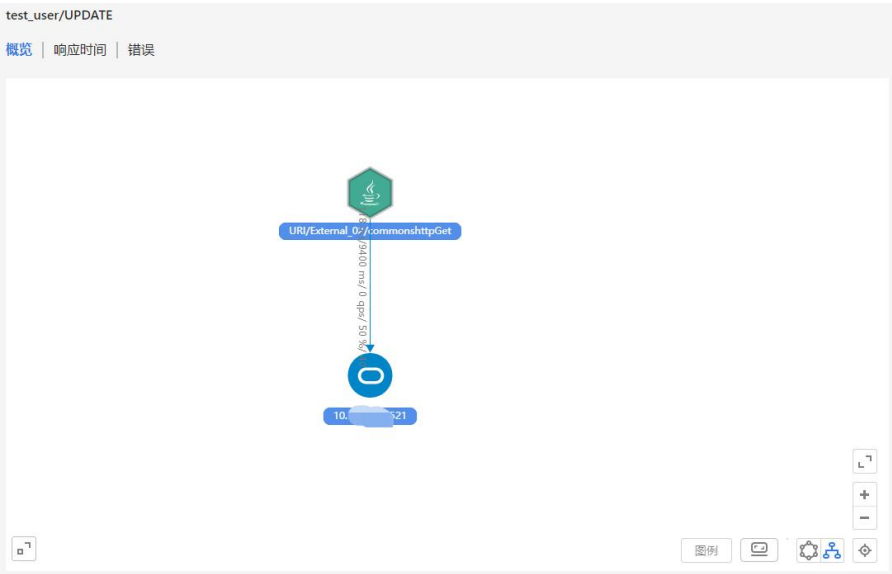
- 操作：由表名和操作类型构成。例如 test_user/select。
- 响应总时间：统计周期内，当前数据库实例处理多次操作的总执行时间，可正反排序。
- 平均响应时间：统计周期内，当前数据库实例处理该类操作的平均执行时间，可正反排序。默认按平均响应时间倒序排序（只查询前 10000 条数据）。
- 吞吐率：统计周期内，对当前数据库实例进行该类操作的每秒平均调用次数，可正反排序。
- 执行次数：统计周期内，对当前数据库实例进行该类操作的总调用次数，可正反排序。

- 错误次数：统计周期内，处理该类操作时数据库实例发生异常的次数，可正反排序。
- 错误率：统计周期内，数据库实例发生异常的次数占执行次数的比例，可正反排序。
- 慢次数：统计周期内慢操作的次数，可正反排序。

在列表中单击一个操作，进入该操作的详细统计信息页面，可查看操作概览、响应时间和错误。

3.10.5.1 概览

左侧展示调用该操作的应用与当前数据库实例的拓扑图。



右侧展示各个应用访问当前操作的平均响应时间、响应总时间、吞吐率、执行次数、错误率、错误次数、调用该操作的事务的个数、慢次数、当前操作从连接池中获取连接的平均等待时间和最大等待时间、连接池的已使用连接数和最大连接数、主机和端口。

概要信息		
平均响应时间	-----	301 ms
响应总时间	-----	314792 ms
吞吐率	-----	0 qps
执行次数	-----	1046
错误率	-----	0 %
错误次数	-----	0
调用者(事务)个数	-----	60
慢次数	-----	353
Connection Time 峰值	-----	0 ms 155 ms
连接池 (Connected Max Connections)	-----	7 896
主机和端口	-----	10.138.1.100:3306

页面下方的基本指标趋势图提供当前操作的平均响应时间、吞吐率和错误数三个重要指标的历史趋势图。

- 平均响应时间：展示当前操作的平均响应时间的历史曲线。
- 吞吐率：展示当前数据库实例处理该操作的吞吐率的历史趋势图。
- 错误数：展示当前数据库实例处理该操作时发生错误的次数的趋势图。

3.10.5.2 响应时间

- 平均响应时间&吞吐率曲线：展示所选的 SQL 操作在统计周期内的平均响应时间和平均吞吐率趋势曲线图。
- TOP 100 调用者（请求）分析：展示所有调用该 SQL 操作的请求的性能指标，包括 SQL 平均响应时间、SQL 吞吐率、SQL 执行次数、慢 SQL 执行次数、SQL 执行错误率、SQL 执行错误次数以及请求所属应用。用户可通过下拉菜单根据请求类型（事务、服务接口和后台任务）进行过滤，输入请求名称进行查询。
- TOP 100 慢 SQL 追踪列表：展示调用该 SQL 操作、SQL 平均响应时间排名前 100 的请求追踪，包括追踪发生时间、请求名称、慢 SQL 平均响应时间、慢 SQL 执行次数和请求中的数据项。在上方单击一个调用者，**TOP 100 慢 SQL 追踪列表**则展示该调用者调用当前 SQL 操作、SQL 平均响应时间排名前 100 的请求追踪。

3.10.5.3 错误

- 错误率曲线：展示所选的 SQL 操作在统计周期内出现异常的执行次数占总执行次数的百分比趋势图、错误数和请求总次数的条形图。
- 错误分析：展示 SQL 操作被调用时发生的所有异常信息，包括异常信息、发生次数和数据项。
- 在上方单击一个异常条目，**SQLs** 会展示发生该异常的 SQL 语句以及异常发生次数，**Root cause** 展示该异常的根因。最下方的 **TOP 100 追踪列表**展示发生该 SQL 异常的错误次数最高的前 100 个请求追踪。单击指定的 SQL，则可查看发生该异常的 SQL 语句的请求追踪。

3.10.6 连接池分析

连接池分析页面展示从当前数据库实例的连接池获取连接的等待时间的变化趋势，以及 4 个获取连接的等待时间分位值的变化趋势。4 个分位可在**配置**中自定义。您可通过上方的 Schema 下拉菜单进行库的切换。

3.11 NoSQL 服务组件

NoSQL 服务组件功能模块展示当前业务系统中使用到的所有 NoSQL 数据库的信息，目前支持 Redis、MongoDB 和 Memcached 三种类型的 NoSQL 数据库监控。

在左侧导航栏中依次选择**服务组件>NoSQL 组件**，进入 NoSQL 分析页面。NoSQL 列表展示所有的 NoSQL 组件。

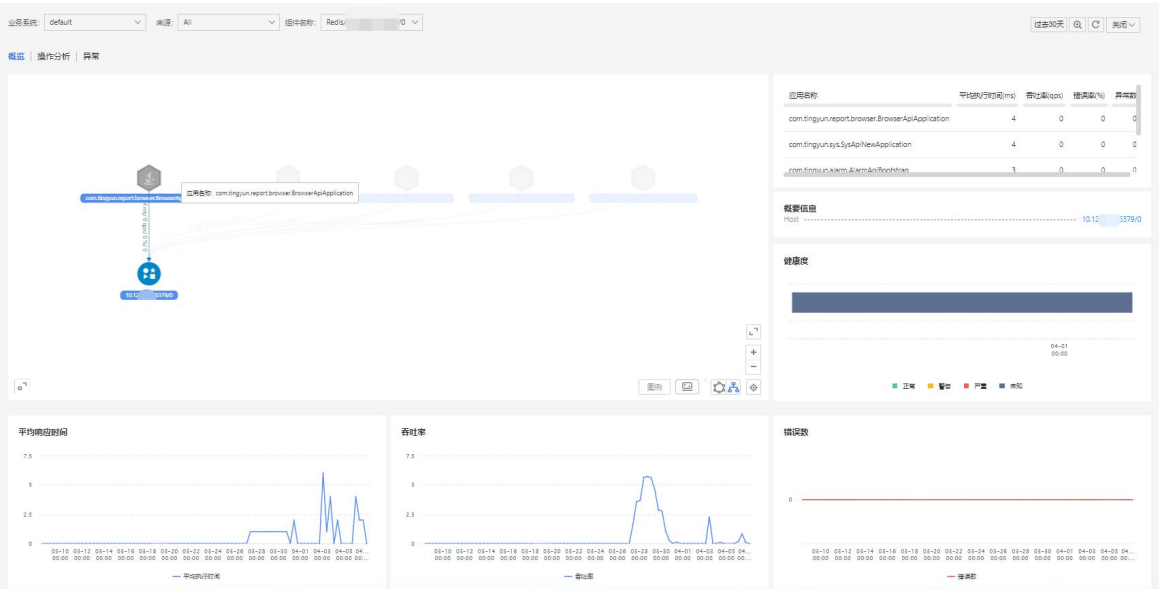
业务系统: default 来源: All 过去30天 关闭

类型	实例	平均执行时间(ms)	吞吐量(qps)	健康度	慢次数	异常次数
Redis	192.168.3.54:6379	5	0.01	未知	0	0
Memcached	192.168.1.8:4334	3	0.01	未知	0	0
Redis	10.128.2.41:6379/9	2	0.01	未知	2	0
Redis	192.168.1.218:6379/0	2	0	未知	0	0
Redis	10.128.2.41:6379/0	1	0.46	未知	0	0
Redis	10.128.2.41:6379/1	1	0.61	未知	3	0

每页显示 20条 1

打印

单击列表上的 NoSQL 服务组件实例可进入该组件的详细页面，与 [Database 服务组件](#) 的概览类似，提供组件拓扑图、健康度、关键指标趋势、NoSQL 操作分析和组件相关异常分析等详细分析。



3.12 MQ 服务组件

MQ 服务组件功能模块展示当前业务系统中使用到的所有消息队列的信息，方便用户分析消息队列上可能存在的性能瓶颈以及生产和消费消息队列的相关应用的性能问题。

在左侧导航栏中依次选择**服务组件>MQ 组件**，进入 MQ 分析页面。页面上以列表形式展示所有 MQ 组件。

业务系统: error-TX

过去7天

🔍

🔄

关闭

类型	实例	消息生产吞吐量(mps)	消息生产次数	健康度	消息消费吞吐量(mps)	消息消费次数	消息消费时间(ms)	🔍
RabbitMQ	192.168.1.174	0	0	未知	0	20	220	
RabbitMQ	10.16.1.4	0	20	未知	0	0	0	
RabbitMQ	10.16.1.74	0	20	未知	0	0	0	

单击 MQ 列表上的消息队列实例名称可跳转到对应消息队列的详情分析页面，包括概览中的 MQ 拓扑图、健康度、关键指标趋势图和按队列、消息生产者、消费者分别统计的队列性能指标统计报表。

业务系统:

error-TX

来源:

All

组件名称:

RabbitMQ/192.168.1.174

过去7天

🔍

🔄

关闭

概览

MQ分析

Queue/Topic	消息生产吞吐率(mps)	消息生产次数	消息消费吞吐率(mps)	消息消费次数	消息生产流量(KB/s)	消息消费流量(KB/s)	消费执行时间(ms)
Topic-81	0	0	0	20	0	0.2	220

3.13 错误分析

错误分析模块对当前业务系统、应用或者实例中出现的错误和异常进行汇总分析。

- 错误：影响用户访问的请求的异常称之为错误。当一个请求发生多个错误时，只为该请求保留优先级最高的一个错误，其他的错误不进行统计。错误按优先级从高到低排序，包括：
 - Business Error（业务错误）
 - Uncaught Exception
 - HTTP Error Code
 - Redirect Error Page
 - Logged Exception
 - Logged Error Message（开启设置事务状态为错误功能后）
- 异常：代码执行过程中产生的异常，不影响用户访问，统计范围如下：
 - 外部服务异常
 - 数据库异常

- NoSQL 异常
- MQ 异常
- 代码异常

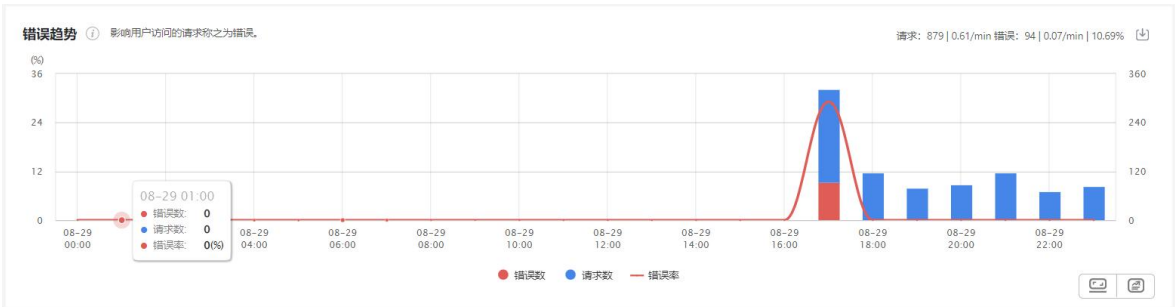
3.13.1 错误分析

3.13.1.1 错误统计

错误页签统计事务和服务接口的错误。

3.13.1.1.1 错误趋势图

错误趋势图分别展示请求数、错误数和错误率的变化趋势，可在页面上方的下拉菜单中选择统计对象。其中蓝色和红色柱图的总高度代表请求数。图的右上角展示统计周期内的总请求数量、平均每分钟的请求数、总错误数、平均每分钟的错误数、错误率。



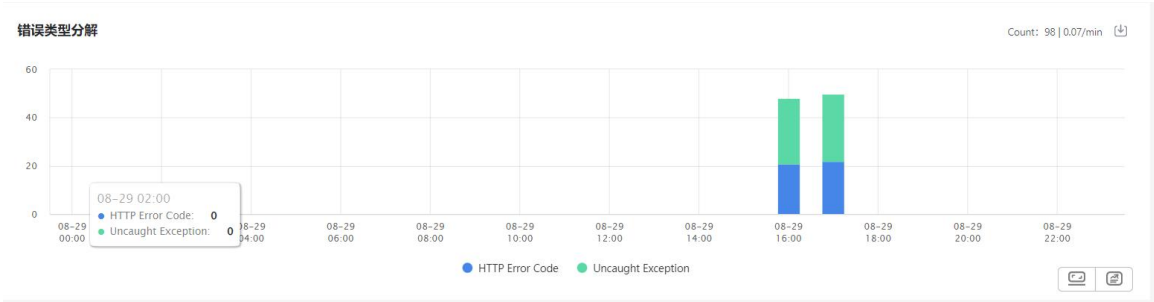
- 请求数：统计周期内业务系统、应用或实例中所有事务和服务接口粒度时间内的访问量。
- 错误数：统计周期内业务系统、应用或实例中所有事务和服务接口粒度时间内的发生错误的次数。
- 错误率：统计周期内发生错误的事务次数和服务接口次数占请求数的百分比。

3.13.1.1.2 过滤设置

在错误趋势图下面的过滤框中，单击空白处，系统会自动弹出条件下拉菜单供您选，支持根据调用者（请求）、错误类型、错误名称和数据项来过滤错误。选择完成后的过滤条件会显示在过滤条件栏，并可立即看到过滤后的统计数据。过滤条件的影响范围包括下方的错误类型分解图、错误列表和影响的请求列表。

3.13.1.1.3 错误类型分解图

分解图以堆叠条形图的方式展示错误的类型构成。图的右上角展示统计周期内的总错误数、平均每分钟的错误数。



3.13.1.1.4列表

错误列表中展示统计对象的所有错误信息。

错误列表影响的请求

错误/异常采集配置

Filter by 错误名称 显示7/7

导出 导出任务列表 自定义表头

开始出现时间	最后发生时间	持续时长	错误类型	错误名称	影响用...	次数	发生频率	
08-29 16:59:00	08-29 17:15:00	16分钟	HTTP Error Code	HTTP ERROR CODE: 500 name (10) account (10)	0	28	0.02/min	
08-29 16:59:00	08-29 17:15:00	16分钟	Uncaught Exception	java.io.IOException name (4) account (4)	0	21	0.01/min	
08-29 16:59:00	08-29 17:15:00	16分钟	Uncaught Exception	org.springframework.web.util.NestedServletException name (7) account (7)	0	19	0.01/min	

您可以对列表项进行以下操作：

- 在搜索框中输入错误的名称，然后单击搜索图标，可查看指定错误的信息。支持模糊搜索，不区分大小写。
- 单击影响用户数，可查看该错误影响到的用户数和用户信息。
- 单击错误的名称，会跳转到错误的详情页面，具体描述请参见[错误详情](#)。
- 单击错误名称后的数据项，可查看该错误影响的业务情况，例如影响的订单金额。
- 单击操作列的漏斗符号，可将该错误的名称作为过滤条件添加到错误趋势图下面的过滤框中，同时可立即看到过滤后的统计数据。
- 支持将错误列表导出为 CSV 格式。单击列表右上角的导出按钮，勾选要导出的字段，系统会创建导出任务，展示在导出任务列表中。当状态列显示“已完成”时，可单击操作列的下载链接进行下载。您可以随时单击追踪列表右上角的导出任务列表按钮，查看导出进度、下载列表或删除导出任务。

说明：

- 当过滤设置中有过滤条件时，导出的列表为过滤后的结果。
- 当错误趋势图下面的过滤框中存在数据项过滤条件时，导出选项对话框的导出字段部分会出现数据项。

- 单击列表右上角的**自定义表头**按钮，可自定义要显示的列与列宽。
- 单击列表右上角的**错误采集配置**，跳转到**全局配置**中的**错误及异常**页签，可对要采集的错误进行调整。

在列表区域上方单击**影响的请求**页签，可查看统计对象发生的错误影响到的所有事务和服务接口。

您可以对列表项进行以下操作：

- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定请求的信息。支持模糊搜索，不区分大小写。
- 单击**操作**列的漏斗符号，可将该请求的名称作为过滤条件添加到错误趋势图下面的过滤框中，同时可立即看到过滤后的统计数据。
- 支持将展示的请求列表结果下载到本地。

3.13.1.2 异常统计

异常页签统计事务、服务接口和后台任务的异常。

3.13.1.2.1 过滤设置

在过滤框中，单击空白处，系统会自动弹出条件下拉菜单供您选，支持根据调用者（请求）、异常类型、异常名称和数据项来过滤异常。选择完成后的过滤条件会显示在过滤条件栏，并可立即看到过滤后的统计数据。过滤条件的影响范围包括下方的异常类型分解图、异常列表和影响的请求列表。

3.13.1.2.2 异常类型分解图

分解图以堆叠条形图的方式展示异常的类型构成。图的右上角展示统计周期内的总异常数、平均每分钟的异常数。

3.13.1.2.3 列表

异常列表中展示统计对象的所有异常信息。

异常列表影响的请求

错误/异常采集配置

Filter by 异常名称 Q 显示13/13

导出导出任务列表自定义表头

开始出现时间	最后发生时间	持续时长	异常类型	异常名称	影响用...	次数	发生频率	
08-29 16:59:00	08-29 19:11:00	132分钟	Database Exception	java.sql.SQLException name (9) account (9)	0	134	0.09/min	
08-29 16:59:00	08-29 17:15:00	16分钟	External Exception	Status Code 500 name (9) account (9)	0	26	0.02/min	
08-29 17:04:00	08-29 19:11:00	127分钟	Code Exception	Log4j Error Message	0	24	0.02/min	

您可以对列表项进行以下操作：

- 在搜索框中输入异常的名称，然后单击搜索图标，可查看指定异常的信息。支持模糊搜索，不区分大小写。
- 单击影响用户数，可查看该异常影响到的用户数和用户信息。
- 单击异常的名称，会跳转到异常的详情页面，具体描述可参考[错误详情](#)。
- 单击异常名称后的数据项，可查看该异常影响的业务情况，例如影响的订单金额。
- 单击**操作**列的漏斗符号，可将该异常的名称作为过滤条件添加到过滤框中，同时可立即看到过滤后的统计数据。
- 支持将异常列表导出为 CSV 格式。单击列表右上角的**导出**按钮，勾选要导出的字段后，系统会创建导出任务，展示在导出任务列表中。当状态列显示“已完成”时，可单击**操作**列的下载链接进行下载。您可以随时单击追踪列表右上角的**导出任务列表**按钮，查看导出进度、下载列表或删除导出任务。

 说明：

- 当过滤设置中有过滤条件时，导出的列表为过滤后的结果。
 - 当过滤框中存在数据项过滤条件时，**导出选项**对话框的导出字段部分会出现数据项。
- 单击列表右上角的**自定义表头**按钮，可自定义要显示的列与列宽。
 - 单击列表右上角的**错误采集配置**，跳转到**全局配置**中的**错误及异常**页签，可对要采集的异常进行调整。

在列表区域上方单击**影响的请求**页签，可查看统计对象发生的异常影响到的所有事务、服务接口和后台任务。

您可以对列表项进行以下操作：

- 在搜索框中输入请求的名称，然后单击搜索图标，可查看指定请求的信息。支持模糊搜索，不区分大小写。
- 单击**操作**列的漏斗符号，可将该请求的名称作为过滤条件添加到过滤框中，同时可立即看到过滤后的统计数据。
- 支持将展示的请求列表结果下载到本地。

3.13.1.3 公共操作

单击错误分析模块中每个图右上方的  图标，即可将该图中展示的所有数据下载到本地，格式为.xls。

3.13.2 错误详情

3.13.2.1 错误影响分析

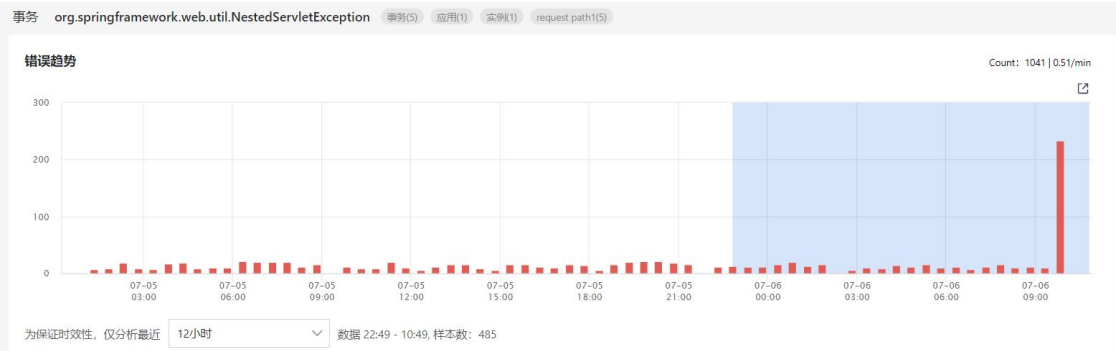
错误名称后展示该错误影响的请求、应用、应用实例和数据项，单击可展示影响详情。



3.13.2.2 错误趋势图

错误趋势图展示该种错误发生次数的变化趋势，可在页面上方的下拉菜单中选择统计对象。图的右上角展示统计周期内该种错误的总错误数、平均每分钟的错误数。

问题发生时，往往查看靠近问题发生时间点前附近时间的数据，才能相对高效的、有针对性的定位导致错误的问题。您可以在错误趋势图的下方下拉菜单中，选择当前时间点之前时间范围的数据进行分析，分析结果体现在下方的 Exception Messages、Stacktrace、Root cause、异常追踪、Referring pages、Client IPs 和 Caller Applications 中。



3.13.2.3 Exception Messages

该列表展示探针采集到的该种错误的所有错误 Message，包括日志框架打印错误信息的（前提是开启了采集日志异常功能以及勾选了日志级别高于'ERROR'的'message'）和代码抛出的错误信息。单击某一行后，下方的 Exception Messages、Stacktrace、Root cause、异常追踪、

Referring pages、Client IPs 和 Caller Applications 列表都会显示跟该条 Message 相关的数据。

Exception Messages

Filter by Message

Q

显示6/6

所有该异常的Message汇总，点击某一行，过滤下方信息。

Message	占比	次数
Request processing failed; nested exception is java.sql.SQLException: ORA-00904: "X": invalid**	72.99%	354
Request processing failed; nested exception is com.mongodb.MongoWriteException: E11000 duplicate key error collection: mydb.test index: _id_d...	25.15%	122
Request processing failed; nested exception is org.springframework.jdbc.CannotGetJdbcConnectionException: Could not get JDBC Connection: nes...	0.62%	3
Request processing failed; nested exception is java.sql.SQLRecoverableException: IO Error: * * one from a read call	0.62%	3
Request processing failed; nested exception is com.mongodb.MongoTimeoutException: Timed out after 30000 ms while waiting to connect. Client ...	0.41%	2

每页显示

5条

<

1

2

>

3.13.2.4 Stacktrace

在上方 Exception Messages 中指定异常信息后，Stacktrace 会显示所有该异常的错误堆栈，堆栈是经过聚合的结果。

您可以进行如下操作：

- 在搜索框中输入关键字，然后单击搜索图标，可看到蓝色字体高亮显示符合条件的堆栈。支持模糊搜索，不区分大小写。
- 展开或收起所有堆栈。
- 查看当前层级中该方法的调用次数所占各个方法总调用次数的百分比，以及该方法的调用次数。

Filter by Stacktrace

Q

展开所有

收起所有

Stacktrace

Stacktrace analyze，展示所有该异常的错误堆栈。

Stacktrace	占比	次数
com.Errors.ServletThrowError.doGet(ServletThrowError.java:36)	100%	10
javax.servlet.http.HttpServlet.service(HttpServlet.java:634)	100%	10
javax.servlet.http.HttpServlet.service(HttpServlet.java:741)	100%	10
org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:231)	100%	10
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:166)	100%	10
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:52)	100%	10
org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:193)	100%	10
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:166)	100%	10
org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:199)	100%	10
org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:96)	100%	10
org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:543)	90%	9
org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:528)	10%	1

3.13.2.5 Root cause

在上方 Exception Messages 中指定异常信息后，Root cause 会显示所有该异常的原因，各个原因是经过聚合的结果。

- 单击每个 root cause，下方会显示相应的堆栈。
- 占比：当前问题发生的次数占所有问题发生次数的百分比。

- 次数：当前问题发生的次数。

Filter by Root cause(cause by)

显示10/10

Exception chain analyze, 所有该异常的Cause by Exception及其Messages汇总, 点击某一行, 过滤下方信息.

Root cause(cause by)	占比	次数
java.sql.SQLException: ORA-00904: "X": invalid**\n	47.15%	446
java.lang.ArithmeticException: / by zero	13.11%	124
Request processing failed; nested exception is com.mongodb.MongoWriteException: E11000 duplicate key error collection: mydb.test index: _id_d...	12.9%	122
javax.jms.JMSException: I AM A EXCEPTION	6.55%	62
java.lang.NullPointerException	6.55%	62

每页显示 5条

<

1

2

>

3.13.2.6 异常追踪

在上方 Exception Messages 中指定异常信息后，该列表展示所有发生了该异常的事务。其中 **Referer** 列显示发生了异常的事务所对应的请求的来源。

您可以进行如下操作：

- 单击事务名称可追踪事务详情。
- 单击操作列的图标可查看错误 message 和错误堆栈。仅支持复制错误堆栈的内容到剪切板。

Filter by 事务名称

显示32/32

所有发生了该异常的事务追踪。点击事务名称, 跳转至事务追踪详情, 点击操作, 查看异常详情.

发生时间	事务名称	应用	实例	Referer	操作
08-13 18:12...	SpringController/mqException/mqPr...	Error_Zhang	localhost.localdomai...		⋮
08-13 18:12...	SpringController/mqException/mqPr...	Error_Zhang	localhost.localdomai...		⋮
08-13 18:11...	SpringController/mqException/mqPr...	Error_Zhang	localhost.localdomai...		⋮
08-13 18:11...	SpringController/mqException/mqPr...	Error_Zhang	localhost.localdomai...		⋮
08-13 18:11...	SpringController/mqException/mqPr...	Error_Zhang	localhost.localdomai...		⋮

3.13.2.7 Referring pages

在上方 Exception Messages 中指定异常信息后，该列表展示所有发生了该异常的事务所对应的请求的来源。

3.13.2.8 Client IPs

在上方 Exception Messages 中指定异常信息后，该列表展示所有发生了该异常的请求的客户端 IP 地址。

3.13.2.9 Caller Applications

在上方 Exception Messages 中指定异常信息后，该列表展示所有调用发生过该异常的请求的上一级应用。

3.14 诊断工具

应用可能会发生内存溢出而导致崩溃，利用诊断工具可以主动发现并分析问题。要分析内存问题，系统提供了两种诊断工具。一是您可以远程对进程打内存 Dump 文件，并可转存储到指定服务器。二是通过对内存进行对象统计和环比分析，判断对象数量是否存在持续增长的趋势，进而判断内存泄露的可能性。

注意：开启该功能后，对应用的 CPU 使用率和响应时间影响较大，并且会产生较大的尖刺。

在左侧导航栏中依次单击应用与微服务>诊断工具，进入诊断工具页面，该页面展示当前统计周期内触发的诊断记录，包括对象统计和内存 Dump 两类。

在页面左侧，您可以通过**诊断结论**、**诊断类型**和**状态**三个维度对诊断记录进行过滤。

- **诊断结论：**包括对象数持续增长和对象数疑似增长。
 - **对象数持续增长：**当环比分析评估增量大于 2000 且增量比大于等于 0.5 时，系统认为对象数持续增长。
 - **对象数疑似增长：**当环比分析评估增量大于 2000 且 0.1<增量比<0.5 时，认为对象数疑似增长。

 **说明：**评估增量和增量比的具体介绍，请参见[环比分析](#)。

- **诊断类型：**包括对象统计和内存 Dump。
- **状态：**包括进行中、分析中、已完成和失败。

应用与微服务 / 诊断工具

过去90天 🔍 ↺ 关闭 ▾

诊断工具

过滤条件

诊断结论

☐ 对象数持续增长 ①

1644

☐ 对象数疑似增长 ①

479

诊断类型

☐ 对象统计

7654

☐ 内存Dump

172

状态

☐ 进行中

2

☐ 分析中

0

☐ 已完成

6480

☐ 失败

1344

7826 个诊断记录

创建对象统计

创建内存Dump

状态	时间	应用名称	实例	诊断类型	诊断结论
进行中	2023-04-06 17:33:13	instance1	192.168.1.53:8083	对象统计	--
已完成	2023-04-06 17:08:13	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
已完成	2023-04-06 16:43:13	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
已完成	2023-04-06 16:14:30	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
已完成	2023-04-06 15:49:33	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
进行中	2023-04-06 15:49:30	test-366-memory	tingyun-Vostro-3690:8094	对象统计	--
已完成	2023-04-06 15:23:13	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
已完成	2023-04-06 14:58:13	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
已完成	2023-04-06 14:33:12	instance1	192.168.1.53:8083	对象统计	对象数疑似增长
已完成	2023-04-06 14:08:12	instance1	192.168.1.53:8083	对象统计	对象数疑似增长

< 1 2 3 4 5 ... 783 >

10 条/页 ▾

3.14.1 对象统计

对象统计功能是统计内存中每个类的对象数量，可以在两次对象统计间观察对象数的变化或新增加的类等。对象统计分为手动对象统计和自动对象统计两种模式，自动对象统计由平台根据应用 OldGC 情况，自动分配实例进行对象统计。其中 FullGC 或 OldGC 后采集的数据可以用来做环比分析。

对象统计是通过 Java 提供的 JMX 接口获取数据。如果无法通过 JMX 接口获取，则通过 **jmap** 命令获取。通过手动触发的对象统计可以选择是否 FullGC 后的采集，FullGC 后采集的数据会更加准确，也可以用来做环比分析使用。自动采集的方式都是在触发 OldGC 后采集且有次数限制，数据可用来做环比分析。

如果您想要对指定的应用实例进行对象统计分析，请先启用对象统计功能，然后创建对象统计。

3.14.1.1 启用对象统计

在左侧导航栏中单击**配置**，在页面上方选择**系统设置**或**应用设置**，然后单击**诊断工具**页签。启用内存分析（即对象统计）功能后，应用与微服务会自动开始监测发生 Old GC 的实例，并对实例进行对象统计和环比分析。

- 5 分钟内同应用发生 Old GC 的实例，按照 GC 时长倒排序，设置取前多少个实例（默认值为前 2 个，数据边界[1,5]），每次对象快照根据对象数排序，取前多少个类（默认值为前 2000 个，数据边界[前 1000,前 5000]）。
- 设置会话时长（默认值 30 分钟，可选择时间为[30、60、120]），在会话期间内，当发生 Old GC 后触发一次对象快照采集，设置每多少分钟（可选择时间为[15、30]）仅采集一次。

 **说明：**会话结束后会重新筛选实例，已经开启过会话的实例优先。

- 勾选复选框，可启用对 Parallel Old GC 的对象快照采集（在该 GC 下的对象快照采集会产生较长的 STW，请谨慎启用）。



对象统计

提示：对象统计会触发 JVM STW，导致应用停顿，请谨慎调节参数，避免频繁触发。



启用自动对象统计功能 ⓘ

5分钟内同应用发生Old GC的实例，按照GC时长倒排序，取前 个实例，开启对象统计，每次对象快照根据对象数倒排，取 类。

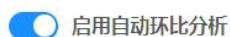
会话时长 分钟，会话期间内，当发生Old GC后触发一次对象快照采集，每 分钟仅采集一次

会话结束后重新筛选实例，已经开启过会话的实例优先。

☐ 启用对Parallel Old GC的对象快照采集（在该GC下的对象快照采集会产生较长的STW，请谨慎启用）。

如需进行对象的环比分析，您还需要启用自动环比分析功能，设置经过多少小时以上（默认值 3 小时）的因 Old GC 触发的对象快照采集，且每隔 1/6 时间都有采集点覆盖，则进行自动环比分析。若某个类的对象数持续上涨，则疑似内存泄漏。

环比分析



经过 小时以上的因Old GC触发的对象快照采集，且每隔1/6时间都有采集点覆盖，则进行自动环比分析，若某个类的对象数持续上涨，则疑似内存泄漏。

3.14.1.2 创建对象统计

1. 在左侧导航栏中依次单击**应用与微服务>诊断工具**，进入**诊断工具**页面。
2. 单击**诊断记录列表**右上角的**创建对象统计**按钮，弹出对话框。

创建对象统计

×

* 应用:

请选择

▼

* 实例:

请选择

▼

会话时长:

30

▼

分钟

采集间隔:

10

▼

分钟

☐ 先触发Full GC,然后再采集对象快照

- 勾选此选项后, 探针在采集对象快照时, 会先触发一次 Full GC,然后再采集对象快照。
- 勾选此选项后, 采集的对象统计数据更准确, 可以进行环比分析。
- Full GC可能会导致应用较长时间STW。

取消

开始

- ### 3. 配置要诊断的应用、实例、会话时长、采集间隔信息。
- 会话时长：设置要统计对象的会话持续时间，默认为 30 分钟。创建对象统计完成后即开始计时。
 - 采集间隔：设置采集对象快照的时间间隔，默认为 10 分钟采集一次对象快照。
 - 勾选**先触发 Full GC 然后再采集对象快照**复选框后，探针在采集对象快照时，会先触发一次 Full GC，然后再采集对象快照。这样采集的对象统计数据更准确，且数据可以用来做环比分析。

 **说明：** Full GC 可能会导致应用较长时间 STW。

4. 单击**开始**，完成配置。

创建成功后列表会显示一条状态为**进行中的**诊断记录。

3.14.1.3 对象统计诊断

单击诊断类型为**对象统计**的诊断记录，进入对象统计详情页面。左侧是当前应用实例所有的快照记录，采集时间即快照的名称，按日期和时间从近到远排序，支持通过关键字模糊搜索。选择一个快照后，右侧展示该快照的对象统计列表，可查看每个类中的对象数、对象占比、字节数和字节占比信息。默认按对象数从高到低排序，支持按对象数、对象占比、字节数和字节占比正反排序。

 **说明：**支持对象统计的 JDK 版本为：Oracle JDK 6~18、OpenJDK 6~18，支持对象统计的 GC 为：Parallel Old 、Serial Old、CMS、G1。

应用与微服务 / 诊断工具 / 对象统计 / 20230406170813

 192.168.1.53:8083 实例的对象统计



3.14.1.3.1 列表说明

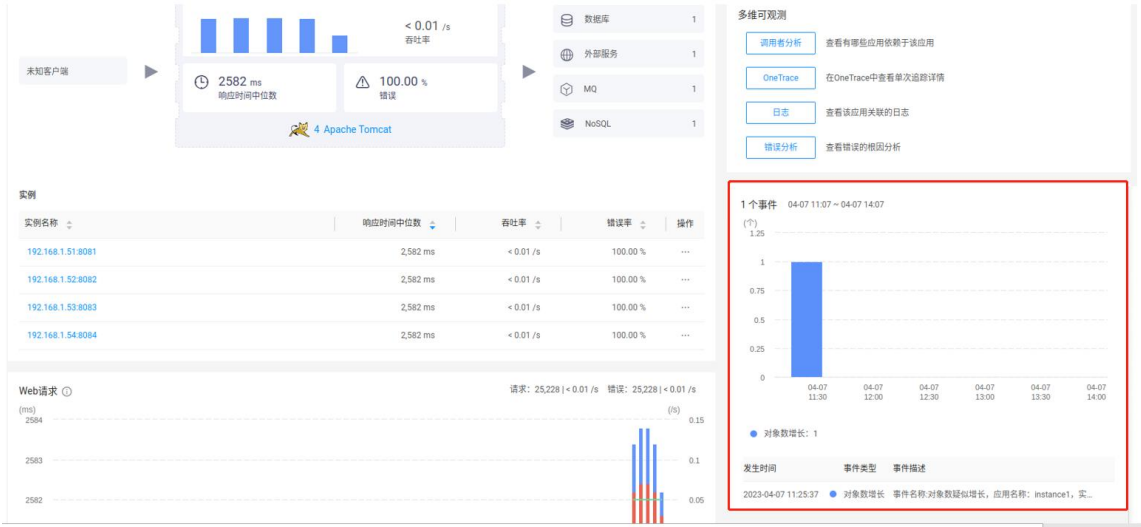
- 列表上方展示当前快照中的类、字节、对象统计数据。分子代表当前显示的类、字节、对象数，分母代表当前对象统计快照中所有的类、字节、对象数。字节是指类的所有对象实例占用内存空间的大小。
- 单击**统计来源**下拉菜单，可选择不同的统计来源：当前快照、同前一次快照增量对比和同前一次快照的新增类。
- 在搜索框中输入类名，然后单击搜索图标，可查看指定类的统计信息。支持模糊搜索，不区分大小写。
- 单击**环比分析**按钮或黄色提示条中的**查看详情**，进入环比分析页面，请参见**环比分析**。

 **说明：**当前对象统计如果不是在 Full GC 或 Old GC 后采集，则不能进行环比分析。

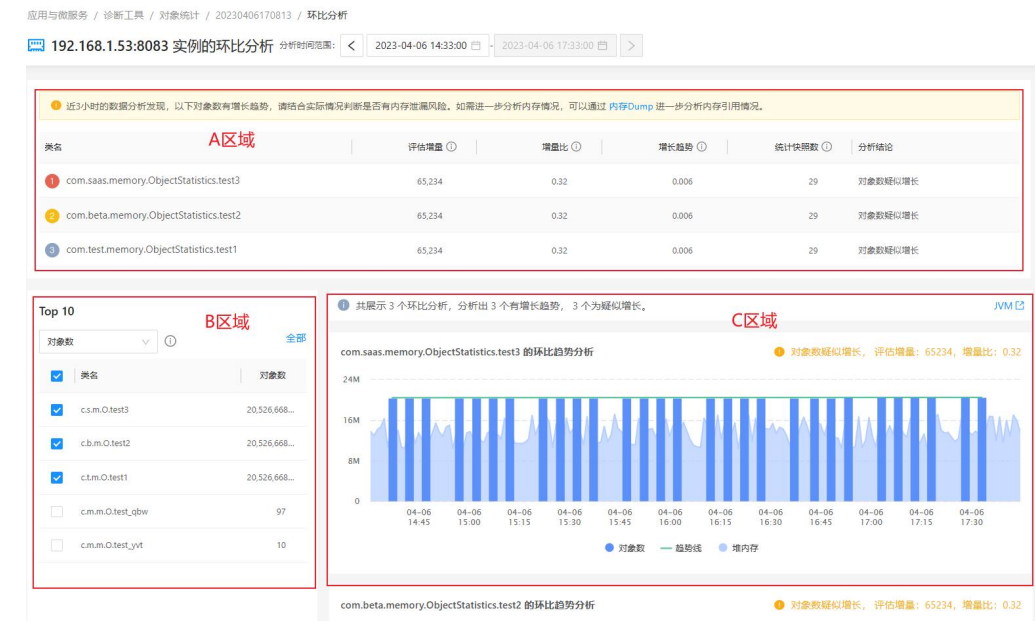
- 勾选**不显示黑名单类**后，列表不显示加入黑名单的类，黑名单请提前在**应用与微服务>配置>应用设置>常规选项**的页面中配置。基础类包括 java.*、javax.*和 sun.*，勾选**不显示基础类**，列表则不会显示。

3.14.1.3.2环比分析

环比分析功能会进一步对实例对象进行分析计算，找到存在内存泄露风险的相关应用类并给出提示。当某个类的对象数持续上涨时，则疑似内存泄露。当发生诊断结论后，系统会以事件的形式向应用发送对象数增长事件。



环比分析页面包括 A、B、C 三个区域。



A 区域展示经过数据分析发现的按评估增量排序 Top 3 的类信息。如果系统分析发现存在风险，页面上方会给出提示，如下图所示。如果需要内存 Dump，可以直接点击操作（在业务空闲时操作）。

近3小时的数据分析发现，以下对象数有增长趋势，请结合实际情况判断是否有内存泄漏风险。如需进一步分析内存情况，可以通过 [内存Dump](#) 进一步分析内存引用情况。

- **评估增量：**通过对这个类的近 N 小时（具体时间是配置中环比分析的配置时长，默认 3 小时）对象数进行趋势计算，形成一条趋势线，使用趋势线的终点值减去起点值，生成评估增量。通常此值出现明显的正值，则需进一步排查是否有泄漏情况。
- **增量比：**是用评估增量比趋势线的终点值，生成增量比。
- **增长趋势：**通过对这个类的近 N 小时（具体时间是配置中环比分析的配置时长）对象数进行趋势计算，形成一条趋势线，这条趋势线的斜率为增长趋势。
- **统计快照数：**当前环比分析中包含该类的对象的快照个数。

B 区域展示当前环比分析的对象数异常的 Top 10 类。系统根据此次环比分析的最后一次对象快照数据并根据对象数（或字节数）进行排名。

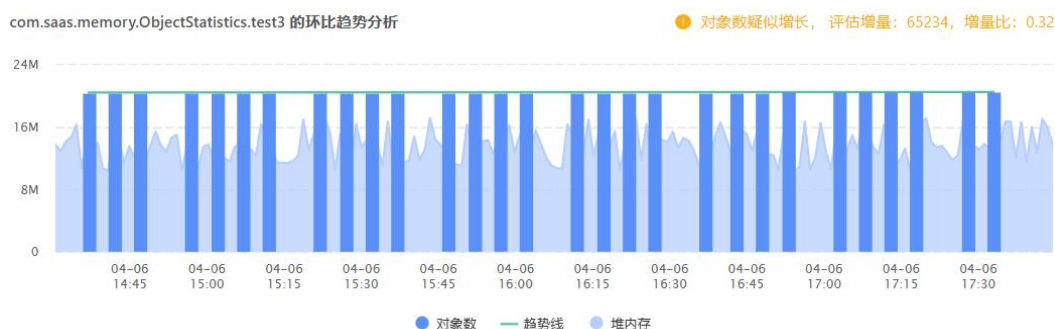
- 列表类名默认按照**对象数**排序，单击下拉菜单选择**字节数**可按照字节数来排序。
 - 对象数：指某个类的对象实例的数量。
 - 字节数：指某个类的所有对象实例占用内存空间的大小。

 **注意：**这里的大小仅代表这个对象内各个字段大小乘以对象数量，不包含引用对象的大小。

- 勾选类名后（类名展示缩写，取路径的首字母），右侧图表会对应显示对象数的变化趋势图。
- 单击右上角的**全部**，可完整的查看 Top 10 类的对象数和字节数。

C 区域对应展示 B 区域类的对象数变化趋势图。

- 趋势图中绿色的线为评估计算的趋势线，柱状图为每次快照里该类的对象数或字节数，面积为堆内存使用量。



- 趋势图右上方显示评估增量和增量比。当选择按对象数排序时, 则会增加显示分析结论: 对象疑似增长或对象持续增长。

● 对象数疑似增长, 评估增量: 65234, 增量比: 0.32

- 当需要查看该应用内存情况时, 可以单击 **JVM** 进入 JVM 页面, 时间范围会自动调整为环比分析的时间范围。
- 如果当前对象快照数据过少, 则无法计算趋势。

3.14.2 内存 Dump 诊断

通过该功能您可以方便快捷地进行内存 Dump, 例如在容器环境下, 使用内存 Dump 功能可以快捷的打出 Dump, 并将内存 Dump 转发至存储服务器上。

如果您想要对指定的应用实例进行内存 Dump 分析, 请先启用内存 Dump 功能, 然后创建内存 Dump。

3.14.2.1 启用内存 Dump

当系统崩溃时, 您可以通过开启内存 Dump 功能远程对进程打内存 Dump 文件, 并可将内存的 Dump 文件保存在其他服务器上, 方便相关人员进行排错分析。

在左侧导航栏中单击**全局配置**, 然后单击**内存 Dump** 页签, 打开**启用内存 Dump 功能**开关即可。对于内存转储配置您可以进行以下操作:

- 单击右侧**新建连接**按钮, 在输入框中填写新建转储的服务器信息, 包括连接名称、连接类型、服务器地址、服务器端口 (通常 SFTP 端口是 SSH 的端口)、用户名、密码以及存储路径。

说明: 存储路径是指服务器上的已存在路径。

新建连接

* 连接名称:

连接类型:

SFTP

* 服务器地址:

* 服务器端口:

* 用户名:

* 密码:

* 存储路径:

启用压缩:

隐私声明:

我们致力于保护您的隐私。我们对密码进行了加密存储，且保证仅用于 Dump 文件的传输使用。

关闭

保存

- 单击开启**启用压缩**，可压缩传输文件。
注意：内存 Dump 完成后压缩文件会增加一定的 CPU 使用率。
- 单击**操作列**的**修改**或**删除**，可对创建成功的服务器进行修改或删除。
- 开启**转储后自动清理文件**功能后，转储成功后自动删除本地的 Dump 文件，之后不能再做转储操作。

3.14.2.2 创建内存 Dump

1. 在左侧导航栏中依次单击**应用与微服务>诊断工具**，进入**诊断工具**页面。
2. 单击诊断记录列表右上角的**创建内存 Dump** 按钮，弹出对话框。



3. 配置要诊断的应用、实例、转储服务器信息。

转储至：设置内存 Dump 文件要保存到的服务器，默认为**不转储 Dump 文件**。转储服务器需要在**全局配置**下的**内存 Dump** 页签中提前配置，具体说明请参见[启用内存 Dump](#)。

4. 单击**开始**，完成配置。

创建成功后列表会显示一条状态为**进行中的**诊断记录。

3.14.2.3 内存 Dump 诊断

单击诊断类型为**内存 Dump** 的诊断记录，进入内 Dump 详情页面。

说明：支持内存 Dump 的 JDK 版本为：Oracle JDK 6~18、OpenJDK 6~18、Jrockit: 1.6、IBM J9 8~18。

- **Dump 信息**展示时间、应用、实例、Pid、容器 ID、实例状态、诊断状态、耗时、Dump 文件、文件大小和文件状态。
 - 实例状态：实例的运行状态，只有运行中的实例才可以进行删除和转储操作。
 - Dump 文件：Dump 文件的存储目录，未设置转储服务器可以在此路径下获取 Dump 文件。
 - 文件状态：单击删除失败的文件状态后的叹号图标，可查看原因。
- 单击右下角的**删除原文件**按钮，可删除内存 Dump 原文件。单击**转储**按钮，可进行转储操作。

Dump信息

时间: 2023-03-31 15:54:42	状态: 已完成
应用: test-dump	耗时: 44.62 s
实例: DESKTOP-20UM1148082	Dump文件: F:\tingyun3.0\tingyun3.6.6-105\tingyun\logs\dumps\java-1680249301409-20212-dump.h...
Pid: 20212	文件大小: 3.33 GB
容器ID: --	文件状态: 正常

实例状态: 离线

删除原文件

转储

- **转储记录**展示转储的目标服务器端口、目标路径、转储的开始时间和结束时间以及状态。
- 当转储状态为**进行中**时，支持取消转储。当转储失败时，单击叹号图标可查看失败原因。

转储记录

目标服务器	目标路径	开始时间	结束时间	状态
192.168.5.163:22	/home/dump/java-1680249301409-20212-dump.hprof.zip	2023-03-31 16:01:19	2023-03-31 16:04:14	转储成功
192.168.5.163:22	/home/dump/java-1680249301409-20212-dump.hprof.zip	2023-03-31 15:54:42	2023-03-31 15:58:44	转储成功

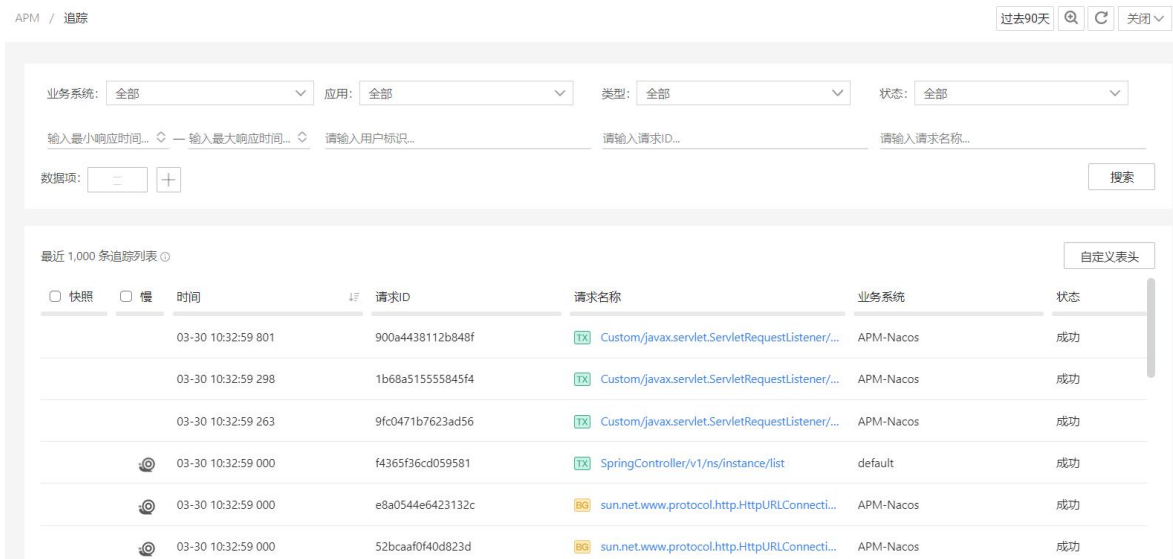
3.15 请求追踪

请求追踪用来记录一次事务/服务接口/后台任务请求过程中所经过和访问的所有业务系统、应用和相关服务组件的详细信息，包括业务数据、性能数据、代码堆栈、服务组件的详细操作以及错误和异常信息等等。当请求的响应时间大于设置的事务追踪阈值，那么该次请求的过程就会被系统详细记录。

用户可以从业务系统概览、事务概览、应用概览等页面进入请求追踪界面，也可以直接从菜单中选择**请求追踪**进入请求追踪列表页面。

3.15.1 请求追踪列表

请求追踪列表提供当前统计周期内所有追踪到的慢事务、慢服务接口和慢后台任务。



用户可根据查询条件进行过滤，查询条件包括业务系统、应用、请求类型、请求状态、用户标识、请求 ID、请求名称、响应时间范围和数据项。对于数据项，查询参数名称和参数值符合条件的请求追踪，可添加多个参数查询条件，格式为 Key=Value。支持输入数据项进行搜索。

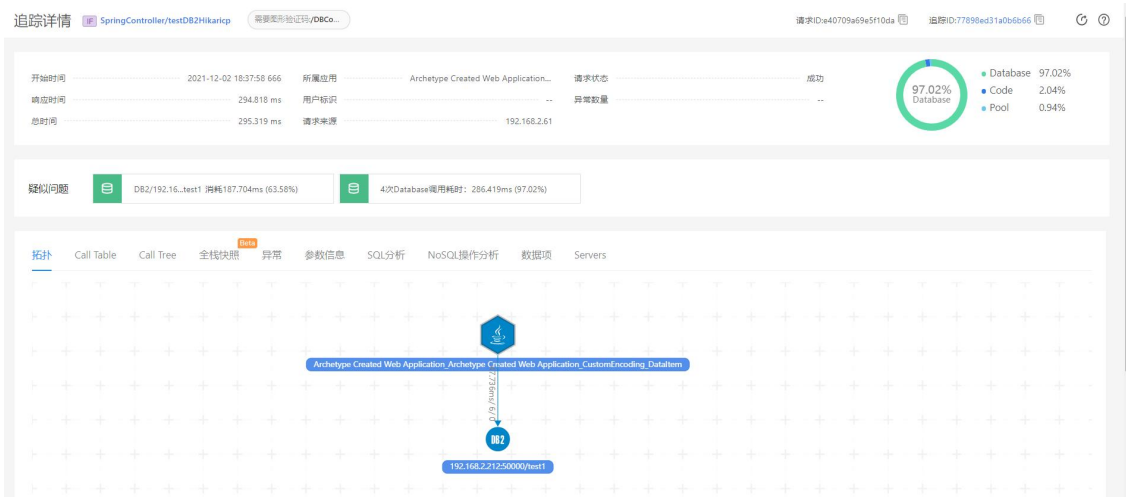
请求追踪列表项包括以下项目：

- 慢请求：标注当前追踪是否是慢请求追踪。当显示慢请求标志时，单击图标可跳转到该慢请求追踪的详情页面，查看拓扑、调用栈、异常和参数信息。当没有该图标时，代表只是一次普通的请求记录。勾选表头前的复选框，可过滤出所有慢请求条目。
- 快照：标注当前追踪是否含有全栈快照信息。当显示快照标志时，单击图标可跳转到该慢请求追踪的全栈快照页面，查看线程的调用栈信息。勾选表头前的复选框，可过滤出所有含有快照信息的追踪条目。
- 时间：追踪产生的时间点，精确到毫秒。
- 请求 ID：当前请求的 ID，单击可跳转到该请求追踪的详情页面。
- 请求名称：当前请求追踪所属的请求的名称，请求名称由探针自动识别或根据用户自定义的规则进行命名。名称前的图标标识请求的类型，表示事务，表示后台任务，表示服务接口。
- 响应时间：从该请求进入入口应用到该应用输出响应结果的时间。精确到微秒。
- 用户标识：该请求追踪中包含的用户标识（如果有）。用户标识是用来识别用户的标识，例如：用户名、用户 ID、用户手机号等，通过该用户标识，可以判断该次追踪的请求是由哪个用户访问的。配置用户标识的获取方式，请参见[用户溯源](#)。
- 业务系统：该请求所属的业务系统。
- 入口应用：该请求所属的入口应用，单击可跳转到该应用概览页面。

- 入口实例：该请求所属的入口应用实例，单击可跳转到该实例概览页面。

3.15.2 请求追踪详情

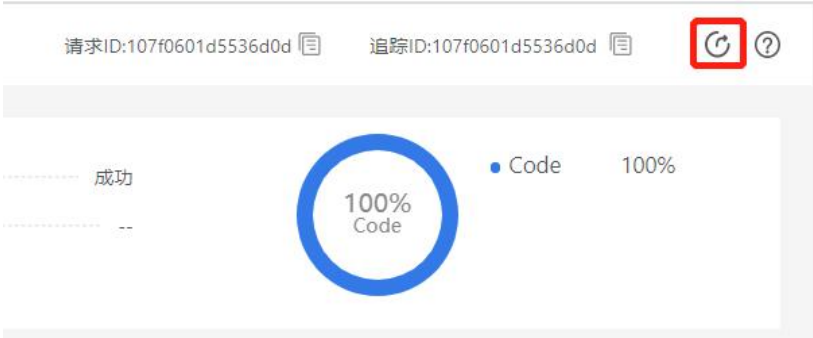
单击请求 ID 可以进入该请求追踪的详情页面，该页面展示此次请求访问中完整的全栈追踪信息，包括以下的功能和项目：



3.15.2.1 追踪概览

追踪概览展示本次追踪的概览信息。包括：

- 所属应用：此次追踪的入口应用名称。
- 请求名称：该追踪的请求名称。
- 请求 ID：此次请求的 ID。单击  图标可复制该 ID。
- 追踪 ID：此次追踪的唯一 ID。单击  图标可复制该 ID。
- 异常数量：此次追踪过程中发生的异常次数，包括异常和错误。
- 开始时间：此次请求追踪的起始时间点。精确到微秒。
- 响应时间：此次请求执行的总时间。精确到微秒。
- 执行时间：代码的执行时间，即从请求进入 Web 容器开始到接口执行结束的时间。如果有多个线程，是线程开始运行的时间到最后一个线程结束的时间，而不是所有异步执行的总和。
- 用户标识：此次请求执行过程中采集到的用户标识信息。
- 单击页面右上角的分享图标，可复制当前追踪详情的 URL 链接。



- 单击  图标，可查看追踪详情页面的帮助说明。

3.15.2.2 性能分解图

性能分解图以饼图的形式展示本次追踪的性能分解，包含代码执行、数据库访问、NoSQL 访问、MQ 访问、外部调用（即外部服务）和连接池的获取连接耗时 6 部分的性能占比。外部服务是指应用通过 HTTP 等协议请求调用的外部应用的统称，是以当前应用使用的外部服务的维度来展示相关性能数据。

3.15.2.3 疑似问题

疑似问题自动分析出该请求执行过程中最慢的代码段、最耗时的组件调用和最严重的错误和异常信息。

3.15.2.4 拓扑

拓扑图展示本次请求追踪的逻辑拓扑调用关系。拓扑展示本次追踪中所经过和访问的所有应用（包括入口业务系统的应用和其他相关的业务系统中的应用）、服务组件和外部服务及其调用关系和调用的过程。事务追踪中经过和访问的应用、服务组件和外部服务以图标形式展示，调用关系和调用过程的数据以连线及连线上的文字标识来展示。

3.15.2.5 Call Table

Call Table 以列表的形式展示本次请求的代码调用过程。

- 方法：展示方法的调用关系。在同一层级的调用中，如果一个方法被调用了多次，该方法会被合并展示，方法前展示调用次数，用星号（*）隔开，如下图所示。

拓扑 Call Table Call Tree 异常 参数信息 SQL分析 NoSQL操作分析 数据项 Servers							
方法	属性	开始时间	偏移量	耗时	耗时占比	独占耗时	类名
24 * getKey		11:40:41.044	44 ms	0 ms		0 ms	Tags
and		11:40:41.044	44 ms	0 ms		0 ms	ImmutableTag
and		11:40:41.044	44 ms	0 ms		0 ms	Tags
getOrCreateMeter		11:40:41.044	44 ms	0 ms		0 ms	MeterRegistry
3 * getKey		11:40:41.044	44 ms	0 ms		0 ms	ImmutableTag
lambda\$initialize\$2		11:40:41.051	51 ms	0 ms		0 ms	MicrometerMetricsPubl...

- 属性：展示具体请求的地址、HTTP 状态码、方法的参数值、数据库操作、数据库地址等方法执行的详细信息。
- 开始时间：展示代码段开始执行的时间，最后三位为毫秒。

- 偏移量：展示该代码段相对事务起始时间点的时间偏移量。
- 耗时：该代码段的独占耗时、网络耗时和异常耗时的总时间。
- 耗时占比：以条形图的形式展示代码段耗时的各组成部分，包括独占耗时、网络耗时和异常耗时。不同的颜色代表不同的耗时部分，具体请参见表格右上角的图例。
- 独占时间：该代码段自身的执行时间。
- 类名：该代码段所属的类。
- 应用：该代码段所属的应用。
- 详情：单击  图标，查看方法栈详情。

方法栈详情

详情

Stack trace

异常

调用者信息

调用者：localhost:80

自身信息

开始时间：

2021-08-18 18:35:59 000

响应时间：

5 ms

总时间：

5 ms

状态：

成功

应用：

localhost:80

实例：

VM_1_26_centos:0

基础信息

URL：

http://10.128.1.26/db-pdo-mysql-exec.php

HTTP method：

GET

Response status：

200

Client IP：

10.128.5.254

Thread name：

--

Request headers

Referer：

--

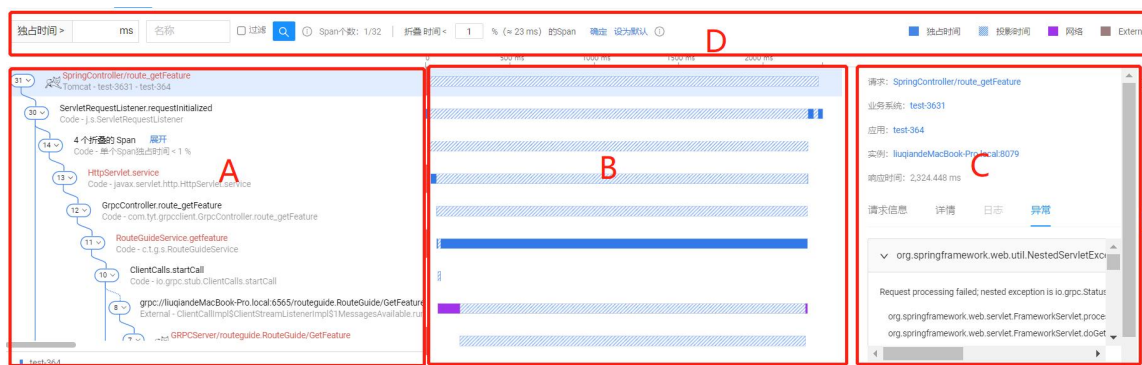
user-agent：

--

3.15.2.6 Call Tree

Call Tree 页面包括 A、B、C、D 四个区域。

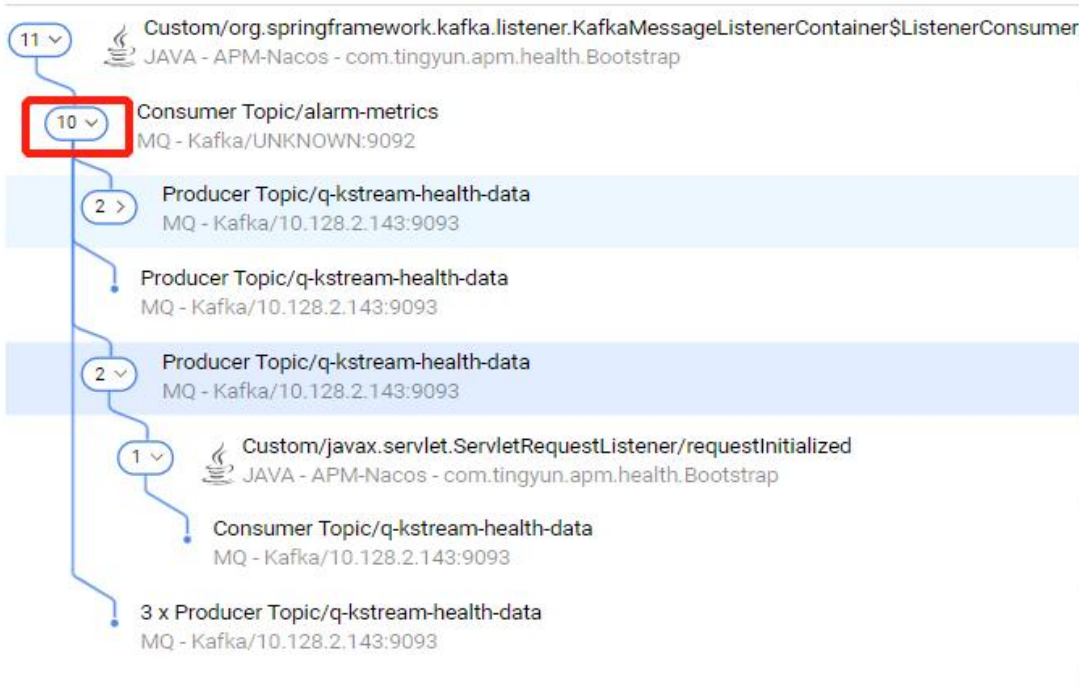
- 区域 A：调用栈。
- 区域 B：调用时序图。
- 区域 C：Span 详情。
- 区域 D：搜索。



3.15.2.6.1调用栈

区域 A 的调用树展示本次追踪的代码调用过程。

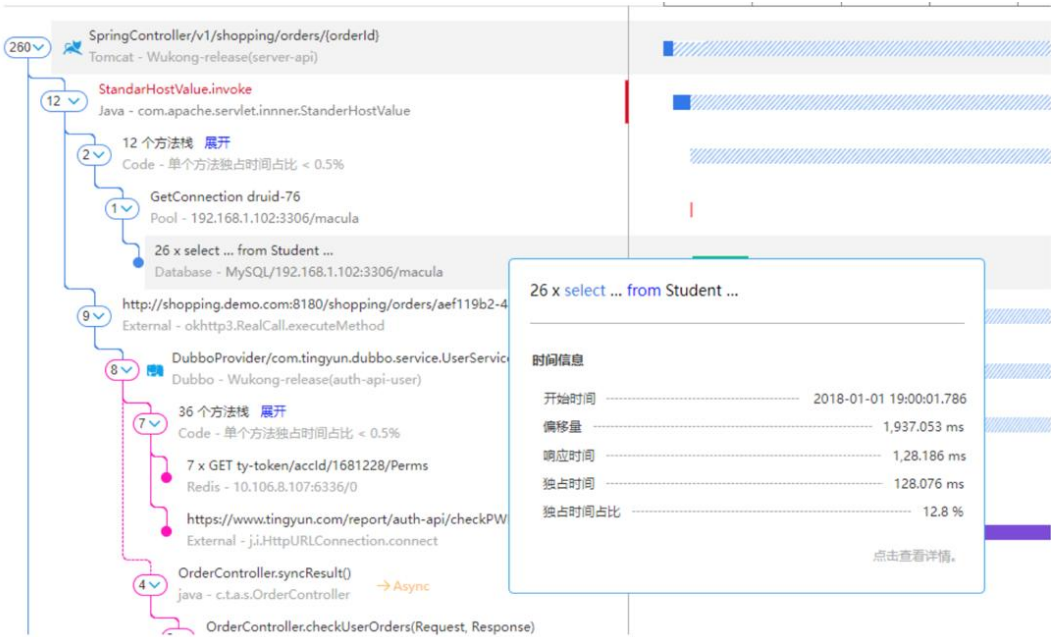
- 调用栈以连线的方式显示方法间的调用关系，不同的应用以不同连线颜色标识，异步调用以虚线表示。
- 各个应用的图例显示在调用栈的底部。
- 入口为虚拟节点（即第一个 Span），显示请求信息，第一行显示请求名称，第二行显示容器、业务系统、应用，容器图标显示在请求信息前。未采集到容器类型的显示语言图标。
- 除入口的虚拟节点外，还会展示以下 Span 信息：
 - Code：第一行显示类名.方法名（类名为最后一段），第二行显示类型（metric_scope）-包名.类名。
 - Database：第一行显示 SQL 语句关键字，第二行显示组件类型-组件子类型/实例。
 - NoSQL：第一行显示操作、key，第二行显示组件子类型-实例。
 - MQ：第一行显示 Producer/Consumer、topic，第二行显示组件类型-组件子类型/实例。
 - External：第一行显示 URL，第二行显示组件类型-类名.方法名。
 - Pool：第一行显示 GetConnection、连接池名称，第二行显示组件类型-实例。
 - 异步：异步的调用关系用虚线标识，第一行显示类名.方法名，第二行显示语言类型-包名.类名，右侧显示异步图标  Async。
- Span 前的数字表示当前方法所调用的所有方法的次数。最后一个被调用的方法前不显示次数。



- 当连续调用多个同级的相同方法、组件（类型、实例、操作相同）时，Span 会合并展示。“N×Span 名称”表示方法被调用的次数。开始时间、偏移量为合并的第一个方法的开始时间，响应时间、独占时间为所有方法的独占时间之和。

以下情况不合并：

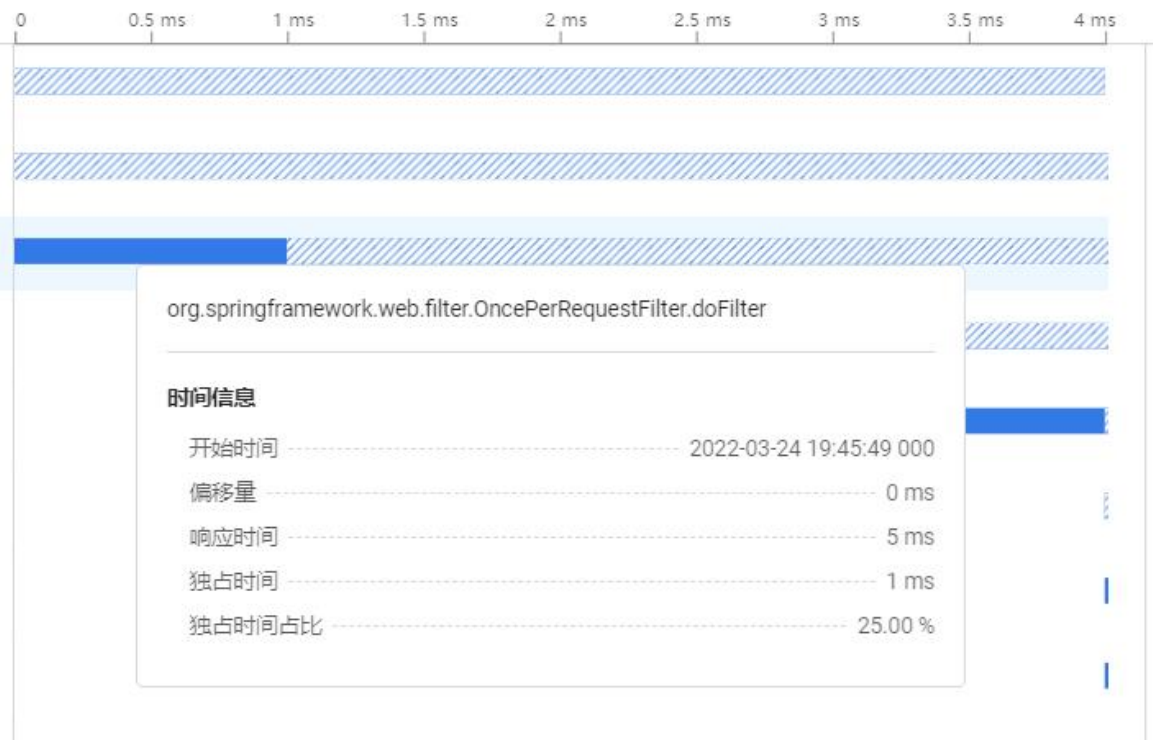
- 发生异常不合并
- 跨应用节点不合并
- 异步节点不合并



- 发生异常时，方法为红色。单击发生异常的 Span，右侧详情默认显示异常页签。

3.15.2.6.2调用时序图

区域 B 以条形图的形式展示了代码的调用时序。单击或者将鼠标悬浮在条形图上，可以查看此次追踪的请求执行过程中执行的 Span 名称（类名.方法名）、开始时间、偏移量（该 Span 相对请求起始时间点的时间偏移量）、响应时间、独占时间（该 Span 自身的执行时间）和独占时间占比等等。



3.15.2.6.3Span 详情

单击一个 Span，区域 C 会展示该 Span 的详情、Stacktrace 和异常信息（如果有才会显示）。单击右侧的  图标，可隐藏区域 C，再次单击可恢复显示。单击类名后红框中的图标，可查看反编译的源代码，并可下载源代码。查看源代码需要开启[获取源代码](#)功能。

URL: http://10.128.2.134:8848/nacos/v1/ns/instance/list

类名: sun.net.www.protocol.http.HttpURLConnection </>

方法: getInputStream

执行: 1 次

[详情](#) [Stacktrace](#) [异常](#)

时间信息

开始时间:	2022-03-29 17:16:59 000
偏移量:	0 ms
结束时间:	2022-03-29 17:16:59 003
响应时间:	3 ms

外部调用详情

http://10.128.2.134:8848/nacos/v1/n

源代码 在线反编译自: javax.servlet.http.HttpServlet

文件获取日期: 2021-05-10 18:40:04 [重新获取](#) 明亮模式 [下载Class文件](#)

```
1 import java.io.IOException;
2 import java.lang.reflect.Method;
3 import java.text.MessageFormat;
4 import java.util.Enumeration;
5 import java.util.ResourceBundle;
6 import javax.servlet.DispatcherType;
7 import javax.servlet.GenericServlet;
8 import javax.servlet.ServletException;
9 import javax.servlet.ServletOutputStream;
10 import javax.servlet.ServletRequest;
11 import javax.servlet.ServletResponse;
12 import javax.servlet.http.HttpServletRequest;
13 import javax.servlet.http.HttpServletResponse;
14 import javax.servlet.http.NoBodyResponse;
15
16 public abstract class HttpServlet extends GenericServlet {
17     private static final long serialVersionUID = 1L;
18
19     private static final String METHOD_DELETE = "DELETE";
20
21     private static final String METHOD_HEAD = "HEAD";
22
23     private static final String METHOD_GET = "GET";
24
25     private static final String METHOD_OPTIONS = "OPTIONS";
26
27     private static final String METHOD_POST = "POST";
```

当单击的 Span 为连接池调用时，在详情中可查看所对应的连接池的详细指标。

- Pool name: 连接池名称，为连接池的唯一标识。通常由连接池框架名（C3P0、Druid 等）+ 随机码组成。
- Max Active: 应用启动时，注册连接池初始化配置的最大活跃连接数。
- Init Active: 应用启动时，注册连接池初始化配置的活跃连接数，也即最小连接数。
- Max Idle: 应用启动时，注册连接池初始化配置的最大空闲连接数。有些框架不需要设置，此时 Max Idle 等于 Max Active。
- Min Idle: 应用启动时，注册连接池初始化配置的最小空闲连接数。
- Current Wait Count: 当前等待获取连接的线程数。

3.15.2.6.4 搜索

在区域 D 中，可输入独占时间或方法名称来过滤 Span。直接单击查询按钮  后，查询结果会高亮显示；如果勾选过滤复选框，再单击查询按钮，那么下方仅展示符合条件的 Span。

连续多个独占时间小于 1% 的方法默认折叠显示，但不包括 External、Database、MQP、MQC、NoSQL 和错误的 Span。该时间可设置，修改完成后单击**确定**即可生效，单击**设为默认**可恢复默认值。折叠后，第一行显示折叠方法的个数，第二行显示单个方法独占时间占比<1%（或设置值）。

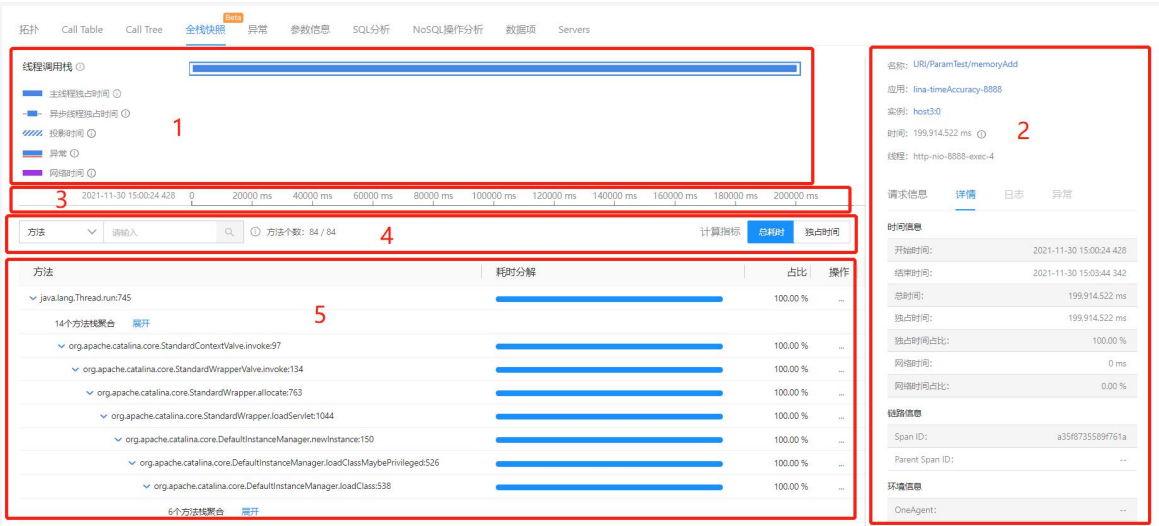
右侧展示条形图的图例。

- 独占时间：Span 自身的调用耗时。
- 投影时间：调用其他服务的耗时。
- Pool：从连接池获取连接的时间。
- 网络：网络耗时。
- External：外部服务调用耗时。
- Database：数据库调用耗时。
- NoSQL：NoSQL 调用耗时。
- MQ：MQ 调用耗时。

 独占时间  投影时间  Pool  网络  External  Database

3.15.2.7 全栈快照

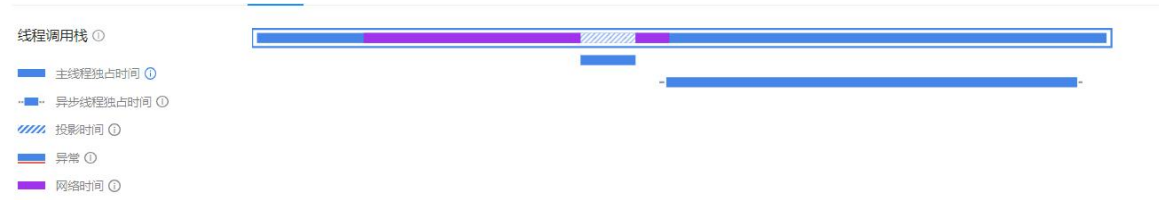
全栈快照页面展示当前事务/服务接口/后台任务的线程调用信息。该页面分为 5 个功能区，包括线程调用时序图、线程信息、图例及时间线、搜索区、数据视图。



- 1：调用时序图。
- 2：线程信息。
- 3：图例及时间线。
- 4：搜索区。
- 5：数据视图。

3.15.2.7.1调用时序图

线程调用时序图展示本次事务完整调用链的所有服务里所有线程的调用时序。图的说明如下：

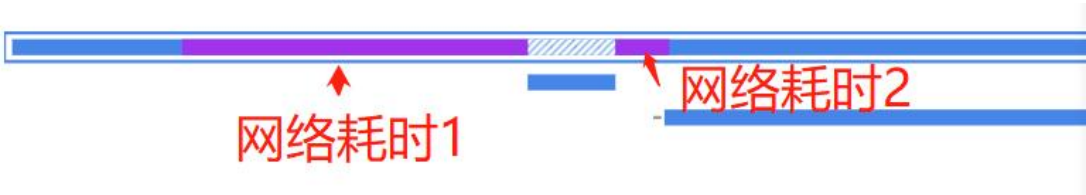


- 每一行表示一个服务中的线程，可能是同步线程，也可能是异步线程。
- 实线区块表示自身调用耗时，虚线区块表示投影时间，即调用其他服务的耗时。表示异步线程独占时间。如果区块中带有红线，表示发生了异常。表示网络时间。
- 单击一个线程，下方可展示该线程的快照数据。如果线程下没有数据，可能是因为线程的执行时间小于快照采集的时间间隔。

- 将鼠标悬浮在一个线程上，可在悬浮框中查看线程的时间信息和概要信息。



- 网络耗时 1=下级方法的开始时间-上级方法的开始时间，网络耗时 2=上级方法的结束时间-下级方法的结束时间。



3.15.2.7.2线程信息

线程信息区展示当前选中的线程的概要信息、请求信息、详情和异常。异步线程仅显示详情和异常。

- 概要信息：展示请求名称、应用、应用实例、独占时间和线程名，异步线程有 → Async 字样。单击请求名称，可跳转到事务详情页面；单击应用名称，可跳转到应用详情页面；单击实例名称，可跳转到实例详情页面。
- 请求信息：展示 URL、HTTP method、Response Status、Client IP、threadId、Request headers、Response headers、Request Parameters、Posts。
- 详情：展示时间信息、链路信息和环境信息。Span ID 为 action_id，Parent Span ID 为调用者的 action_id。
- 异常：展示当前线程的错误和异常。默认展开第一个异常。主线程的异常显示主线程和异步线程的所有异常。异步线程的异常只显示当前线程的异常。

3.15.2.7.3图例及时间线

图例的深色线区块表示自身调用耗时，即独占时间，浅色线区块表示投影时间，即调用其他服务的耗时，所有区块的并集为总时间，红色线区块表示线程发生异常。



3.15.2.7.4搜索

线程下的方法支持通过方法名称和占比耗时来搜索。在下拉菜单中进行切换即可。当选择**方法**时，支持模糊搜索。当选择**占比**时，大于指定比例的方法会出现在搜索结果中。

方法个数展示搜索结果中方法的个数/方法总个数。

方法耗时支持按方法总耗时和独占时间进行展示，单击右侧页签进行切换即可。

占比 >

10

%

Q

① 方法个数: 2 / 64

计算指标

总耗时

独占时间

3.15.2.7.5数据视图

- **方法列**展示合并后的方法调用栈，格式为：包名.类名.方法名:行号。当连续多个（3个及以上）方法调用的独占时间占比<0.5%时，会被合并展示。以下情况除外：
 - 根节点不合并
 - 最后一个方法不合并
 - 当子级有多条时，该方法不合并
- **占比**：方法独占时间/线程总耗时，保留 2 位小数。
- 单击**操作列**的 图标，选择**自定义嵌码**，可将该方法快速加入监控。
- 单击**操作列**的 图标，选择**查看源代码**，可查看应用的源代码。只有在全局配置中开启获取源代码功能后，才能展示该选项。

方法	耗时分解	占比	操作
java.lang.Thread.run:745 23个方法栈聚合 展开		100.00 %	...
com.ningasynchttpclient.AsyncHttpClientHandlerServlet.doPost:19 7个方法栈聚合 展开		100.00 %	...
com.ningasynchttpclient.AsyncHttpClientHandlerServlet.doGet:40 7个方法栈聚合 展开		100.00 %	...
java.lang.Throwable.printStackTrace:-2 7个方法栈聚合 展开		100.00 %	...
com.ningasynchttpclient.AsyncHttpClientHandlerServlet.doGet:43 7个方法栈聚合 展开		100.00 %	...
java.lang.Object.wait:-2 6个方法栈聚合 展开		100.00 %	...
com.ningasynchttpclient.AsyncHttpClientHandlerServlet.doGet:77 6个方法栈聚合 展开		100.00 %	...
sun.misc.Unsafe.park:-2 6个方法栈聚合 展开		100.00 %	...

3.15.2.8 异常

异常页面集中展示当前请求追踪过程中捕获的所有异常信息。所有的异常信息按异常出现的时间顺序排列，列表中包括：

- 偏移量：异常出现的时间点相对请求起始时间点的偏移量。
- 异常类型：异常类型包括事务错误（会导致事务失败）、代码异常、数据库异常、外部服务异常等多种类型。
- 异常名称：异常的名称，例如 Java 异常类名、404 等状态码异常和数据库异常。
- 所属应用：发生该异常的应用名称。
- 所属实例：发生该异常的应用实例名称。

单击异常列表中的条目，可在下方的内容区域查看对应异常的详细信息，包括异常或错误信息、错误类型和错误堆栈。

拓扑

调用栈

异常

参数信息

智能分析

偏移量	异常类型	异常名称	所属应用	所属实例
14ms	错误	org.apache.http.conn.HttpHostConnectException	Tingyun_portal	agent.syh.tingyun.com:8080
13ms	外部接口异常	org.apache.http.conn.HttpHostConnectException	Tingyun_portal	agent.syh.tingyun.com:8080

错误信息

Connect to localhost:10120 [localhost/127.0.0.1] failed: Connection refused

错误类型

org.apache.http.conn.HttpHostConnectException

堆栈

org.apache.http.impl.conn.DefaultHttpClientConnectionOperator.connect(DefaultHttpClientConnectionOperator.java:158)
 org.apache.http.impl.conn.PoolingHttpClientConnectionManager.connect(PoolingHttpClientConnectionManager.java:353)
 org.apache.http.impl.execchain.MainClientExec.establishRoute(MainClientExec.java:380)
 org.apache.http.impl.execchain.MainClientExec.execute(MainClientExec.java:236)
 org.apache.http.impl.execchain.ProtocolExec.execute(ProtocolExec.java:184)
 org.apache.http.impl.execchain.RetryExec.execute(RetryExec.java:88)
 org.apache.http.impl.execchain.RedirectExec.execute(RedirectExec.java:110)
 org.apache.http.client.internal.HttpClient.execute(HttpClient.java:184)

3.15.2.9 参数信息

参数信息页面展示此次请求追踪中采集到的 HTTP 请求和响应头中的参数，以及从方法参数或返回值中采集的用户自定义参数。

3.15.2.10 Code 统计

Code 统计页面展示该请求中耗时占比排名前 100 的方法信息。默认按照耗时占比从高到低排序。支持根据方法名称的首字母正反排序，也支持根据耗时占比、调用次数、总独占时间、平均独占时间和最大独占时间进行正反排序。

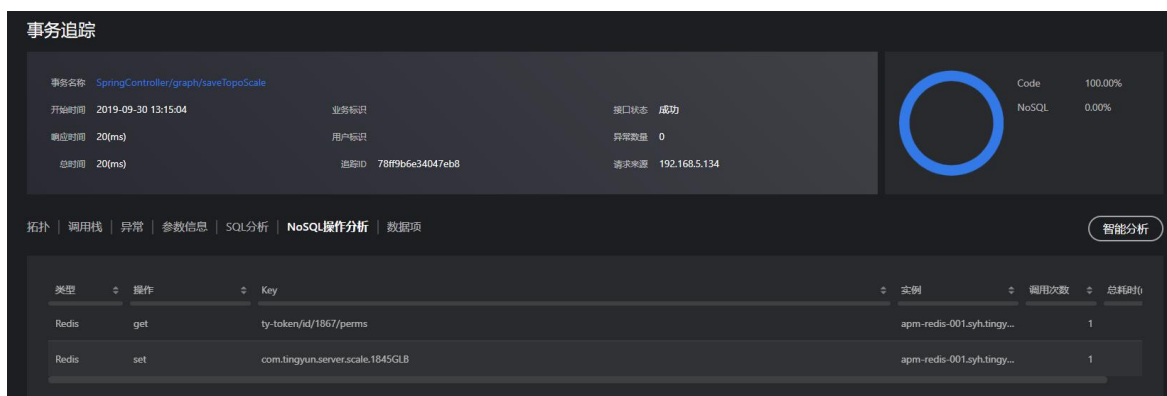
Top 100						
方法	耗时占比	调用次数	总独占时间	平均独占时间	最大独占时间	最小独占时间
com.example.demos.controller.HttpClientController.HttpClientController	72.20%	1	2.961 ms	2.961 ms	2.961 ms	2.961 ms
javax.servlet.http.HttpServlet.service	21.17%	3	0.868 ms	0.289 ms	0.868 ms	< 0.01 ms
javax.servlet.ServletRequestListener.requestInitialized	3.95%	3	0.162 ms	0.054 ms	0.162 ms	< 0.01 ms
org.springframework.web.filter.OncePerRequestFilter.doFilter	2.49%	9	0.102 ms	0.011 ms	0.055 ms	< 0.01 ms
org.apache.tomcat.websocket.server.WsFilter.doFilter	0.19%	3	0.008 ms	0.003 ms	0.008 ms	< 0.01 ms
org.apache.http.protocol.HttpRequestExecutor.execute	< 0.01%	2	< 0.01 ms	< 0.01 ms	< 0.01 ms	< 0.01 ms
com.example.demos.controller.OradeController.OradeErrorTest	< 0.01%	1	< 0.01 ms	< 0.01 ms	< 0.01 ms	< 0.01 ms
oracle.jdbc.driver.OracleStatement.execute	< 0.01%	1	< 0.01 ms	< 0.01 ms	< 0.01 ms	< 0.01 ms
oracle.jdbc.driver.OracleDriver.connect	< 0.01%	1	< 0.01 ms	< 0.01 ms	< 0.01 ms	< 0.01 ms
com.example.demos.controller.HttpClientController.HttpClientController	< 0.01%	1	< 0.01 ms	< 0.01 ms	< 0.01 ms	< 0.01 ms

3.15.2.11 SQL 统计

SQL 分析页面展示该请求的所有 SQL 语句列表，包括详细的 SQL 语句、调用次数、总耗时、平均执行时间和调用 SQL 语句的代码所在的行数。支持排序。

3.15.2.12 NoSQL 统计

NoSQL 分析页面展示该请求的所有 NoSQL 操作列表，包括 NoSQL 类型、NoSQL 操作、Key、NoSQL 实例、调用次数和总耗时。支持排序。



3.15.2.13 Servers

Servers 页面提供该请求追踪发生前后 30 分钟之内该事务入口应用的各个实例的 JVM CPU、内存、线程和会话等指标的变化情况。

3.15.2.14 日志

日志页面展示该请求追踪发生前后的日志信息。

- 支持根据应用定位到指定应用上打印的日志，默认展示全部应用的日志。
- 支持根据追踪 ID 定位指定事务、服务接口或后台任务的完整日志，默认根据当前追踪 ID 过滤出当前请求的完整日志。
- 支持选择前后 15 分钟、前后 30 分钟、前后 1 小时、前后 2 小时和前后 8 小时的日志进行查看。
- 支持通过输入日志关键字搜索日志。

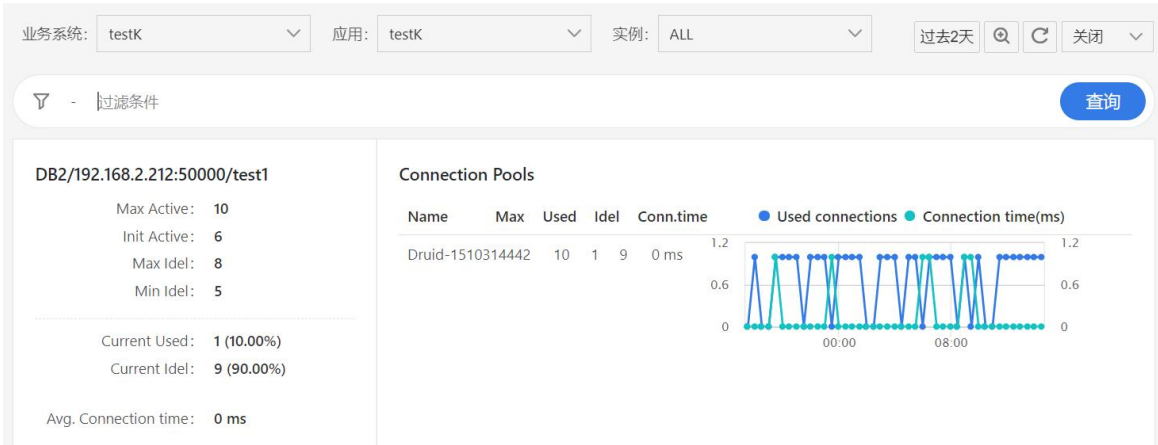
3.16 连接池

在实际应用访问过程中，经常会出现因为数据库（包含 NoSQL）连接池资源紧张，导致事务响应时间较长。在这种场景下想要定位应用性能问题的根因，不但需要问题排查人员能熟练使用 APM，还需要很强的研发能力。监控数据库连接池可以有效的解决上述问题，提高使用者独立排查问题的效率。目前支持采集 Database、NoSQL（Jedis）连接池信息，可分别查看业务系统、应用、实例、事务不同级别的 Connection Pool 信息。

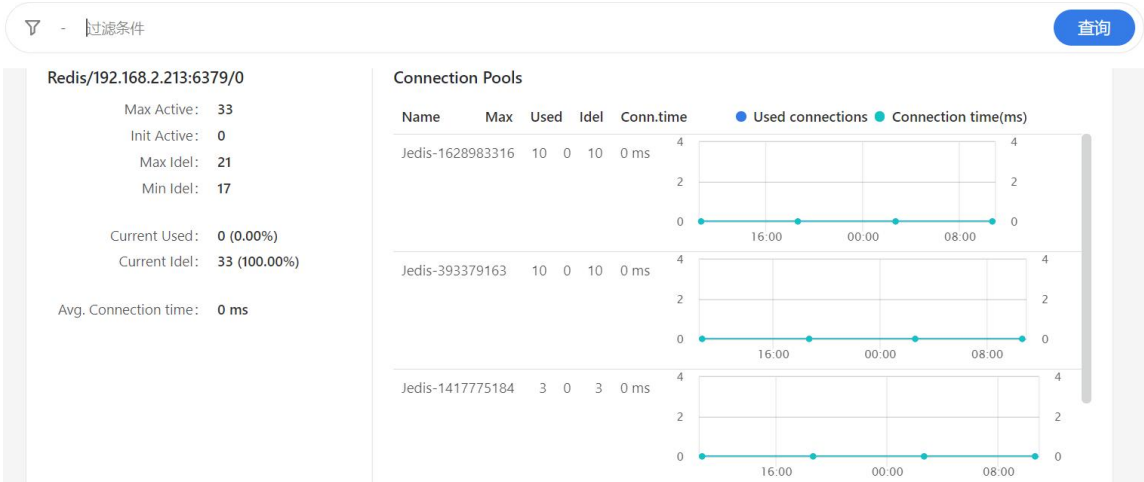
在左侧导航栏中选择**连接池**进入连接池列表页面。在页面上方指定业务系统、应用或实例后，该页面以列表形式展示所访问的所有连接池指标，包括以下内容：

- 数据库实例：由类型、主机+端口、Schema 组成。其中类型包括 MySQL、Oracle、DB2、SQL Server、PostgreSQL、Redis。
- Max Active：应用启动时，注册连接池初始化配置的最大活跃连接数。
- Init Active：应用启动时，注册连接池初始化配置的活跃连接数，即最小连接数。
- Max Idle：应用启动时，注册连接池初始化配置的最大空闲连接数。有些框架不需要设置，此时 Max Idle 等于 Max Active。
- Min Idle：应用启动时，注册连接池初始化配置的最小空闲连接数。
- Current Used：连接池已被使用的连接数。
- Current Idle：目前连接池空闲的连接数。
- Avg. Connection time：指定时间段内，从连接池中获取连接的平均等待时间。
- Connection Pools：展示选定时间段内每个连接池的统计数据，以及连接获取耗时和被使用的连接数的历史曲线。
 - Name：连接池名称，为连接池的唯一标识。通常由连接池框架名（C3P0、Druid、Hikaricp、WebLogic、Jedis）+随机码组成。随机码由探针随机生成。
 - Max：该连接池的最大活跃连接数。

- Used：最近一分钟该连接池中已使用的连接数。
- Idle：最近一分钟该连接池中空闲的连接数。
- Conn.time：最近一分钟从该连接池获取连接的平均等待时间。



一个数据库拥有多个连接池的情况下，右侧会排列展示多个连接池的统计数据。



支持根据连接池名称和组件过滤连接池列表。

3.17 动态基线和健康度

应用与微服务中引入了动态基线和健康度的概念和功能，用来更精确地对被监控的对象进行性能和可用性等健康状态的评估。被监控对象的健康度计算依据健康规则的定义，当监控规则中的规则条件配置了动态阈值时，需要使用动态基线来进行对比。

3.17.1 动态基线

为了更精确计算健康度和警报阈值，提供了对动态基线的配置和管理功能，用户可选择对不同的指标项设置不同的动态基线配置，每个动态基线配置提供一套对动态基线算法的配置参数。当在警报或健康规则中选定基线之后，系统将根据选择的动态基线配置参数自动进行动态基线的计算，计算指标将包括平均值、四个分位值（根据系统的设定，只针对事务、服务接口的响应时间有效）和标准差在内的基线指标数据。

3.17.1.1 新建动态基线

在左侧导航栏中依次选择**配置>平台配置>基线配置**，进入**基线配置**页面。在页面的右上角，单击**新建基线配置**按钮进入新建动态基线配置的界面。



新建基线配置对话框，包含以下配置项：

- 基线名称：** 请输入基线名称
- 趋势：** ☒ 无趋势 ☐ 每日小时趋势 ☐ 按周每日小时趋势 ☐ 按月每日小时趋势
- 时间窗口：** ☒ 动态时间窗口 ☐ 固定时间窗口
- 动态时间窗口配置：** * 最近 天 (最小1天最大15天)
- 固定时间窗口配置：** 请选择固定时间窗口
- 操作按钮：** 确定 取消

- 名称：当前基线配置的名称。
- 趋势：当前基线配置里的趋势选项，包括以下四种趋势选项：
 - 无趋势：计算整个基线时间窗口内所有数据的平均值、分位值和标准差来作为基线，无趋势粒度的数据，即一个基线时间窗口内仅包含一套平均值、分位值和标准差。
 - 每日小时趋势：以小时粒度分组汇总统计过去一段时间窗口内数据作为基线，数据变化粒度为小时，即每小时汇总统计一套基线数据样本点（包括平均值、分位值和标准差），分组方式为小时（0-23 点）。
 - 按周每日小时趋势：以星期几和小时粒度分组汇总统计过去一段时间窗口内数据作为基线，数据变化粒度为小时，即每周几的每个小时汇总统计一套基线数据样本，分组方式为小时（0 到 23 点）加星期几（星期一到星期日）。
 - 按月每日小时趋势：以月份内日期和小时粒度分组汇总统计的过去一段时间窗口内数据作为基线，数据变化粒度为小时，分组方式为小时（0 到 23 点）加月份内日期（1 日-31 日）。
- 时间窗口：用来计算动态基线的数据的时间段选取方式和范围，包括以下两种时间窗口类型：

- 动态时间窗口：使用滚动的时间窗口，即从当前日期往前选择指定的时间长度的数据来进行基线计算。选择该类型时，用户需要设置时间窗口长度，单位为天。

时间窗口根据不同的趋势选项，对其可设置的最小、最大值以及缺省值定义如下表：

趋势选项	动态窗口缺省值（天）	最小值（天）	最大值（天）
无趋势	15	1	31
每日小时趋势	30	1	31
按周每日小时趋势	30	7	91
按月每日小时趋势	30	30	365

- 固定时间窗口：使用固定的一段时间窗口内的数据来进行基线计算。选择该类型时，提供设置时间窗口开始和截止时间的选项。时间窗口的起始时间点精确到分钟。固定时间窗口用来将用户指定的有参考价值（例如：系统没有故障地平稳运行）的一段时间内的性能数据作为基线。

时间窗口以天为单位进行设置，设置固定时间窗口时，可精确到小时，但跨度必须是 24 小时的整数倍。

单击**确定**按钮后，新创建的动态基线将会显示在动态基线列表中。

3.17.1.2 管理动态基线

动态基线配置表展示当前系统中已经设置的所有动态基线配置，一组动态基线配置包括一系列参数选项来决定动态基线的计算方式。列表中配置项说明如下：

- 缺省：当前基线是否是缺省基线，系统中只能有一个缺省基线。
- 操作：可对基线配置列表中的基线配置进行修改、删除和设置缺省三种操作。动态基线配置修改界面与新建界面完全一致，动态基线配置修改后，系统将根据该设置重新计算使用了该动态基线配置的所有基线数据。系统不允许删除正在被使用中的和系统缺省的基线配置。

基线配置					+ 新建基线配置	
名称	趋势	时间窗口	缺省	操作		
最近30天每日趋势基线	每日小时趋势	最近30天	是	修改 删除		
15日平均值基线	无趋势	最近15天	否	修改 删除 设置为缺省		
最近三个月每周趋势基线	按周每日小时趋势	最近90天	否	修改 删除 设置为缺省		
最近1年每月趋势基线	按月每日小时趋势	最近365天	否	修改 删除 设置为缺省		

3.17.2 健康度

健康度是应用与微服务中用来评估各类被监控对象健康状态的重要指标，目前支持评估业务系统、服务组、应用、实例、事务、服务接口、数据库、NoSQL、MQ、外部服务、错误和异常。系统通过不同的颜色来表示不同状态的健康度，说明如下。

- 绿色：表示健康（正常）。
- 黄色：表示警戒。
- 红色：表示严重。
- 灰色：表示未知状态，当无法评估健康度时会显示未知状态，例如：没有匹配的健康规则、被监控对象没有足够符合要求的数据、配置变更但新的健康规则未触发等等。

健康度的计算由健康规则来决定，自定义健康规则请参见[新建健康规则](#)。

3.17.2.1 新建健康规则

当新建一个业务系统后，对该业务系统，系统提供 10 条缺省的健康规则来对业务系统、应用、事务和实例等被监控对象进行健康度评估。单击[新建规则](#)按钮可创建用户自定义的健康规则。

新建一条健康规则，要经过以下四个步骤：

1. 设置评估对象。
2. 设置严重条件。
3. 设置警戒条件。
4. 设置时间计划。

3.17.2.1.1 设置评估对象

需要分别设置以下的规则项：

1. 规则名称：用户自定义的健康规则名称，可使用任意字符，包括中文字符，最长 256 个字符长度。
2. 是否启用：该健康规则是否启用的开关，当启用是规则生效，禁用时系统将不再对该健康规则进行评估。
3. 评估对象：指定该健康规则评估的被监控对象，评估对象类型详见下面表格。该对象作为当前健康规则的目标评估对象，即最终当前规则计算出的健康度展示在此对象上。

评估对象	说明
业务系统	以整体业务系统的所有事务的相关的指标组合（例如：响应时间、吞吐率、慢事务次数）等来评估的健康度。
事务	以事务的相关指标组合来评估的健康度。
服务接口	以服务接口的相关指标组合来评估的健康度。
应用	以应用实例的硬件、JVM 或 CLR 等相关指标组合（例如：CPU、内存等）以及应用自身所有事务的相关指标组合来评估的健康度。
实例	以应用实例的硬件、JVM 或 CLR 等相关指标组合（例如：CPU、内存等）以及应用实例自身所有事务的相关指

	标组合等来评估的健康度。
服务组件（数据库，NoSQL，MQ，外部服务）	以应用内服务组件的相关指标组合（例如：SQL 查询响应时间、吞吐率等）来评估的健康度。
错误或异常	以错误、异常、返回码以及用户自定义的错误信息组合来评估健康度。

4. 实例筛选条件：只有当评估对象是**实例**时显示该选项。您可以按实例类型进行筛选，用来对特定语言实例的特定指标（例如：Java 语言的 JVM Heap 指标）进行健康度评估。

新建规则

返回列表

1

2

3

4

评估对象

严重条件

警戒条件

时间计划

规则名称: 请输入规则名称...

规则名称,可以使用中文,最大256字符

启用:

评估对象:

请选择

评估对象筛选:

请选择

下一步

5. 评估对象筛选：根据不同的评估对象类型确定当前健康规则的适用对象范围，对不同的评估对象类型提供不同的筛选方式和筛选条件。用户可将评估对象类型中的所有对象纳入健康规则的评估范围，也可以通过指定筛选条件，只将符合条件的对象纳入健康规则评估范围。

3.17.2.1.2设置健康条件

健康规则条件组按健康度的“警戒”和“严重”两个状态分为警戒条件和严重条件两个级别。系统在计算健康度时优先检测“严重”级别的条件，满足条件时健康度即为“严重”状态，否则就检测“警戒”级别的条件，满足条件时健康度即为“警戒”，否则健康度为“正常”。如果两级健康条件配置项目相同，可以单击**从警戒条件中拷贝所有规则**或者**从严重条件中拷贝所有规则**选择从将一个基本的条件配置复制到另一个级别中再进行修改，以提高配置效率。健康规则条件包括以下的项目：

- 条件逻辑关系：每个级别的健康规则条件中都可以包含多个检测条件，该选项用来决定这多个检测条件之间的关系，当选择 or 关系时，条件中任何一个检测条件满足即为满足该条件；当选择 and 关系时，条件中所有的检测条件满足才判断为满足条件。

- 检测条件：每个基本的健康规则条件中都可包含多个检测条件，用户可通过**新增条件**按钮增加新的检测条件。每个检测条件包含以下项目：
- 指标：选择用来作为检测条件的性能指标项，例如：平均响应时间、吞吐率等等。根据不同类型的被评估对象，可选择的指标项不同。
 - 指标取值方式：选择的被评估对象和指标项不同，则可选的指标取值方式就不同。取值方式包括：最大值、最小值、平均值、累计值、计数、去重、当前值、实例数、分位值第一个参数、分位值第二个参数、分位值第三个参数、分位值第四个参数。
 - 比较方式：设置当前检测条件下指定的指标值与条件阈值的对比方式，包括静态阈值对比和动态基线对比两种类型，每一种比较类型的比较方式、阈值项和阈值单位的设置范围如下表所示：

类型	比较方式	阈值值	阈值单位
动态基线	偏离基线超过 xx 范围	见动态基线取值表定义	- 基线标准差的 N 倍 - N%基线值
	大于基线		
	小于基线		
静态阈值	大于特定值	指定静态阈值	同指标值单位
	小于特定值		

- 无数据判断为满足：当检测条件中的指标没有数据时判定该条件为满足。

- 分组评估：当选择的健康规则评估对象包含多个实例时（包括：事务、应用、错误、服务接口这 4 种评估对象），提供按实例分组进行健康条件评估的选项。该选项决定了指标数据的统计按何种方式分组统计后再与健康评估条件进行比较，以及多少分组数据满足评估条件后进行健康度指标的更新。包括以下的项目：

检测分组	说明
无分组	业务系统、事务或应用中所有实例的平均相关

	指标满足检测条件时才进行健康度的更新
任意实例	业务系统、事务或应用中的任何一个实例的相关指标满足检测条件时即进行健康度的更新。 缺省选项值为“任意实例”
N%的实例	业务系统、事务或应用中百分之 N 的实例的相关指标满足检测条件时即进行健康度的更新
N 个实例	业务系统、事务或应用中 N 个实例的相关指标满足检测条件时即进行健康度的更新

3.17.2.1.3时间计划

- 规则生效时段：设置该健康规则的生效时间段。用户可选择让健康规则只在自定义的时间段内生效，在生效时间段之外，该健康规则将不做评估计算。支持对当前选择的生效时段重新编辑和删除。如果设置为始终生效，该健康规则将会一直生效。新建的生效时段支持简单模式和高级模式。
 - 简单模式：自定义周一到周日的起始和结束时间段。
 - 高级模式：使用 Quartz Scheduler Cron 表达式来设置时段的开始和结束时间。

Quartz Scheduler Cron表达式说明

支持6个字段的设置，以空格分割，分别表示：

- 秒 0-59，该字段在此场景下无效，始终为*
- 分 0-59，该字段在此场景下无效，始终为*
- 小时 0-23，支持数字，逗号分割列表，范围和通配符“*”

例：“10”表示生效时间段是10:00-11:00，

- “10, 11, 18”表示生效时间段是10:00-12:00和18:00-19:00
- “9-18,20-22”表示生效时间段为9:00-19:00和20:00-23:00
- 当开始时间大于结束时间，例：“22-5”表示当天晚上22:00到次日凌晨6:00

- 日期 1-31，支持数字，逗号分割列表，范围和通配符“*，？”
- 月 1-12，支持数字，英文缩写，逗号分割列表，范围和通配符“*”
- 星期 0-7，0和7都表示周日，支持数字，英文缩写，逗号分割列表，范围和通配符“*，？”

- 时间窗口：指定使用最近多长时间内的数据来计算和评估健康度。最小值为 2 分钟，最大值 6 小时。

3.17.2.2 管理健康规则

从左侧导航栏中选择**健康规则**进入健康规则的列表页面。页面中以列表形式展示当前系统中已有的健康规则。用户对健康规则可进行修改、删除和检查等操作。

业务系统名称: zdu

最近14天

健康规则列表

请输入搜索内容

Q

+ 新建规则

启动状态	评估对象	规则名称	操作
✓	业务系统	业务系统错误率缺省健康规则	修改 删除 检查
✓	业务系统	业务系统性能缺省健康规则	修改 删除 检查
✓	应用	应用错误率缺省健康规则	修改 删除 检查
✓	应用	应用性能缺省健康规则	修改 删除 检查
✓	事务	事务错误率缺省健康规则	修改 删除 检查
✓	事务	事务性能缺省健康规则	修改 删除 检查
✓	实例	CPU 使用率缺省健康规则	修改 删除 检查

健康规则修改界面与新建健康规则的界面相同。

对健康规则进行删除时，已经触发的“警戒”和“严重”状态的被评估对象会产生对应的“解除”事件，由当前健康规则评估的状态将变成“未知”。

检查操作用来检查指定的健康规则的运行状态，单击**检查**，在**检查**页面以列表的形式显示当前健康规则所评估的所有的被评估对象的健康状态。列表展示以下项目：

- 应用：被评估对象所属的应用，如果被评估对象是总体业务系统本身则不显示该字段数据。
- 评估对象：被当前健康规则所评估的对象，此处根据健康规则中被评估对象筛选条件的设定显示每一个符合筛选条件的对象名称，并标注对象类型。单击评估对象名称在新的窗口跳转到对应的对象的概览页面。
- 健康度：当前被评估对象的健康度状态不一定是当前规则所触发的，单击健康度链接，在弹出的窗口中可查看当前评估对象在统计周期内的健康事件。事件列表中的事件记录只按评估对象筛选，与健康规则无关，会列出所有可能起作用的健康规则触发的所有事件。列表中展示以下的项目：
 - 状态：展示评估对象的健康度。
 - 时间：事件发生的时间，精确到秒。
 - 事件类型：健康度事件的类型，请参见[健康度定义](#)。
 - 评估对象：产生该健康度事件的对象，例如特定的业务系统、应用、事务或实例等等，单击跳转到指定的对象的概览界面。
 - 概述：描述此次事件的内容。单击该内容，可以查看此次事件在持续时间内的状态变化以及原因。单击[查看](#)，可跳转到该评估对象的概览页面。

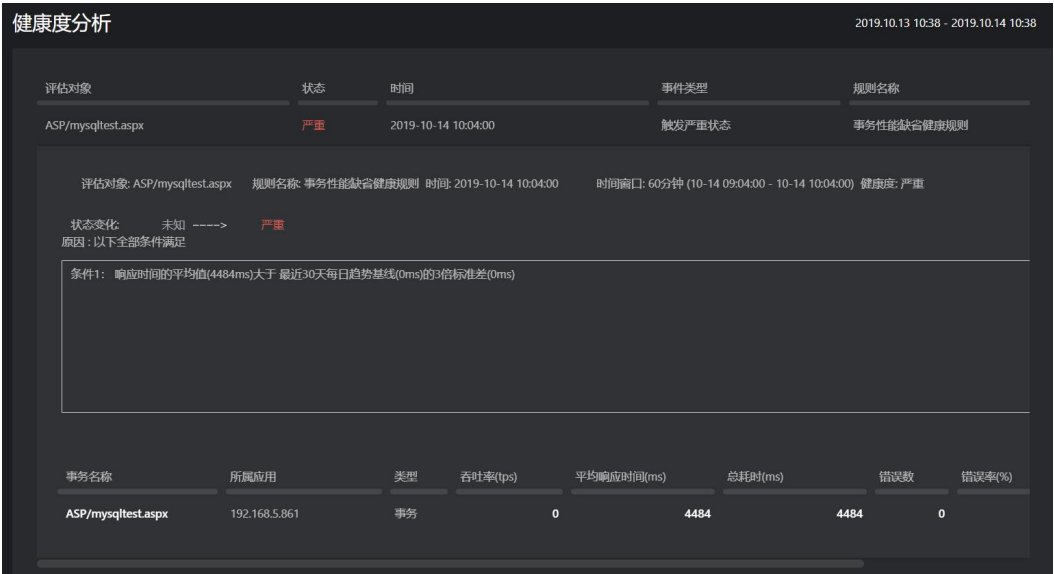


- 实例健康度：以多个带颜色的图标显示当前被评估对象在每种健康状态下的实例数量。
- 被评估对象的健康度数据也可以在各监控对象的概览页面中看到。

3.17.2.3 健康度分析

在**健康度分析**页面中，您可以看到以下内容：

- 当前事务触发的健康规则。
- 用来计算和评估健康度所采用的时间窗口。
- 健康状态的变化情况。
- 触发健康状态发生变化的原因，即健康规则中的设置的条件。
- 最下方显示事务列表。支持自定义表头。



3.18 部署管理

3.18.1 TingyunAgents 管理

在左侧导航栏中单击**管理>部署状态**，进入 TingyunAgents 管理页面，用户可查看已监控的主机，包括 Linux 主机和 Windows 主机。

说明：TingyunAgent 暂不支持采集 Windows 操作系统的性能数据。

只展示最近一周的Host列表

过滤条件

查询

自定义表头

批量升级 批量启用 批量禁用 新增 删除 导出

☐	Host Name	Agent类型	部署环境	IP	CPU核数	总内存	版本	升级状态	禁用/启用	操作
☐	 k8s-demo	Linux	k8s-demo	10.10.10.1	2	8.07 GB	2.1.0.0	--	☒	编辑
☐	+  mall-machine	Linux	tx_gz	10.10.10.2	--	--	2.0.0.0	升级成功	☒	编辑
☐	+  middleware-1	Linux	tx_gz	10.10.10.3	--	--	2.0.0.0	升级成功	☒	编辑
☐	 oracle-3.0	Linux	tx_gz	10.10.10.4	--	--	2.0.0.0	升级成功	☒	编辑
☐	+  middleware-2	Linux	tx_gz	10.10.10.5	--	--	2.0.0.0	升级成功	☒	编辑
☐	 netop-c1-jenkins-001.bgz.tingyun.com	Linux	tx_gz	10.10.10.6	--	--	2.0.0.0	升级成功	☒	编辑
☐	+  middleware-3	Linux	tx_gz	10.10.10.7	--	--	2.0.0.0	升级成功	☒	编辑
☐	+  manager	Linux	tx_gz	10.10.10.8	--	--	2.0.0.0	--	☒	编辑

3.18.1.1 列表项说明

各个列表项的说明如下：

- 圆点颜色：绿色表示 TingyunAgent 状态正常，红色表示异常，蓝色表示未监控，灰色表示无数据。
- Host Name：主机名称。
- Agent 类型：分为 Linux 和 Windows 两种。
- 部署环境：TingyunAgent 所在主机所属的数据中心、VPC、机房等。
- IP：TingyunAgent 所在主机的 IP 地址。
- 端口：TingyunAgent 监听的端口号。
- CPU 核数：TingyunAgent 所在主机的 CPU 核数。如果 TingyunAgent 是部署在虚拟机或者 Docker 中，则是指虚拟机或者 Docker 本身的 CPU 核数。
- 总内存：TingyunAgent 所在主机的内存。如果 TingyunAgent 是部署在虚拟机或者 Docker 中，则是指虚拟机或者 Docker 本身的内存。
- 版本：TingyunAgent 版本。
- 升级状态：TingyunAgent 的在线升级状态，包括升级中、升级成功、升级失败等。
- 安装时间：TingyunAgent 安装时间。
- 心跳时间：TingyunAgent 最后的数据更新时间，即最后心跳时间。

3.18.1.2 操作说明

用户可在该页面中进行以下操作：

- **查看进程信息：**单击主机名称前的  图标，可查看该主机上所有的进程信息，如进程名称、进程 PID 和进程进行通信所使用的端口号等，具体介绍请参见[进程信息](#)。单击 PID，可跳转至[进程](#)页面查看进程详情；单击 App Name，可跳转至应用分析页面；单击 Instance Name，可跳转至应用实例的分析页面。

☐	Host Name	Agent类型	部署环境	IP	CPU核数 ...	总内存	版本	升级状态	禁用/启用
☐	+ oracle-3.0	Linux	test_1_174	10.128.2.35	2	8.20 GB	2.2.0.0	升级成功	☒
☐	+ VM-2-55-centos	Linux	test_1_174	10.128.2.55	24	101.20 GB	2.2.0.0	--	☒
☐	+ mq1	Linux	test_1_174	192.168.1.174	1	3.98 GB	2.2.0.0	--	☒
 Tomcat Welcome to Tomcat_axis2demo_axis2demo PID: 25098 App Name: Welcome to Tomcat_axis2demo_axis2demo Port: 8080 Instance Name: mq1:8080									
☒	SpringBoot com.tingyun.agent.collector.AgentCollector								☒
	 Process need restart PID: 5315								☒
☒	PHP httpd								☒
	 Process need restart PID: 19585								☒
☒	Java org.elasticsearch.bootstrap.Elasticsearch								☒
	 Process need restart PID: 720								☒

- **禁用 TingyunAgent：**在禁用/启用列关闭开关，主机下不再显示进程信息，TingyunAgent 探针对重启或新起的进程不再嵌码，即不再采集该主机上主机资源、所有基础组件进程和应用进程的数据，已嵌码的应用正常上传数据。
- **编辑主机：**单击操作列的编辑，可修改主机名称，该名称只作为别名展示在基调听云悟空平台中，实际的主机名称不会发生变动。如果在部署 TingyunAgent 时没有开启全栈数据采集功能，可编辑主机开启。开启[全栈数据采集](#)开关后，TingyunAgent 可采集基础设施、服务端的性能数据，如果关闭开关，仅采集基础设施的性能数据（需重启应用才可生效）。
- **搜索主机：**单击页面顶端的过滤框，弹出过滤条件。选择过滤条件，输入内容，然后单击[查询](#)按钮查询主机，也可直接按回车键查询。支持通过单个关键字、多个关键字、单个列字段、单个列字段多个值、多个列字段、单个标签、多个标签进行过滤，具体可单击[查询](#)按钮前的  图标查看搜索帮助。用户可设置多个过滤条件，支持通过 Agent 类型、Host Name、部署环境、IP 地址和版本进行搜索。
- **自定义表头：**单击 TingyunAgent 列表左上角的自定义表头按钮，可勾选想要展示的列名。部分字段是必选项，必须展示。
- **只展示最近一周的 Host 列表：**默认 TingyunAgent 列表中展示的是所有已安装 TingyunAgent 的主机信息，无数据的也会展示在其中。勾选页面右上角的[只展示最近一周的 Host 列表](#)复选框，列表中仅会展示最近一周有数据的主机，系统会以 TingyunAgent 最后一次心跳时间进行筛选。
- **批量启用：**勾选多个主机名称前的复选框，TingyunAgent 列表右上角的[批量启用](#)按钮变为可用状态，用户可单击按钮批量启用 TingyunAgent。
- **批量禁用：**勾选多个主机名称前的复选框，TingyunAgent 列表右上角的[批量禁用](#)按钮变为可用状态，用户可单击按钮批量禁用 TingyunAgent。禁用的主机下不再显示进程信息，

TingyunAgent 探针对重启或新起的进程不再嵌码，即不再采集该主机上主机资源、所有基础组件进程和应用进程的数据，已嵌码的应用正常上传数据。

- **批量升级：**勾选一个或多个主机名称前的复选框，TingyunAgent 列表右上角的**批量升级**按钮变为可用状态，用户可单击按钮批量升级 TingyunAgent。在弹出的对话框中选择要升级到的 TingyunAgent 版本，然后单击**确定**按钮，即可开始升级。**升级状态**列显示各个主机上 TingyunAgent 的升级状态，单击状态，可查看升级日志。

升级建议：在同一个部署环境下，先灰度升级一台主机，观察无问题后，再进行批量升级，建议每批次批量升级 5~20 台主机，稳定后再升级下一批次。



- **导出 TingyunAgent 列表数据：**单击 TingyunAgent 列表右上角的**导出**按钮，可导出所有 TingyunAgent 数据。如果设置了过滤条件，导出的不是搜索后的结果，仍然是所有 TingyunAgent 的数据。
- **新增 TingyunAgent：**单击 TingyunAgent 列表右上角的**新增**按钮，可进入 TingyunAgent 部署页面，用户可以新部署一个 TingyunAgent，详情请参见《**TingyunAgent 部署&管理手册**》中**自动部署**小节。
- **删除主机探针：**单击 TingyunAgent 列表右上角的**删除**按钮，可删除 TingyunAgent。

3.18.1.3 进程信息

- **进程名：**进程名由启动方式和进程的类型决定。Docker 启动时，会含有 using 信息，using 后是镜像名。

说明：进程信息中不展示 Go、Python 和 Node.js 应用。

Java：启动方式分为三种，非 Docker 启动、Docker 启动、kubernetes 启动 Docker。进程类型目前支持五种，Tomcat、WebLogic、SpringBoot、Jar、Java。以 Tomcat 为例，如下图所示：

➤ 非 Docker 启动

 Tomcat CustomEncoding_DataItem,DBConnectionPool-demo 🔴

PID: 23462 | Agent Version: 3.4.5 | App Name: CustomEncoding_DataItem,DBConnectionPool-demo | Port: 8081 |

Instance Name: k8s-master:8081

➤ Docker 启动

 Tomcat CustomEncoding_DataItem,DBConnectionPool-demo using "autotest"  🔴

PID: 6439 | DockerId: 76d9d9c0 | Agent Version: 3.4.5 | App Name: docker_autoapp | Port: 8080 | Instance Name: 76d9d9c0e2d5:8080

➤ Docker+kubernetes 启动

 Tomcat tomcat using "sha256:cd6cc8bc02aefec1bbc27fa298453b10f5ecb4eac5a42835717d72b1a351a15"   🔴

PID: 14179 | DockerId: b6bfedf2 | k8s pod UID: 5de24fe4-85f3-11ea-8d54-005056a521ee | Agent Version: 3.4.5 |

App Name: External_01,reportConfig | Port: 8080 | Instance Name: sun1.8-192.168.2.207-8070-tomcat:8080

.NET Core：启动方式分为三种，非 Docker 启动、Docker 启动、kubernetes 启动 Docker。由于.NET Core 不需要容器启动，所以进程类型只有一种。每种启动方式示例如下：

➤ 非 Docker 启动

 Dotnet core_redis 🔴

PID: 3439 | Agent Version: 3.0.0 | App Name: core_redis | Port: 5050 | Instance Name: k8s-node2:5050

➤ Docker 启动

 Dotnet core_redis using "autocore"  🔴

PID: 4799 | DockerId: d341a7a0 | Agent Version: 3.0.0.0 | App Name: core_redis | Port: 5050 | Instance Name: b79273e66701:5050

➤ kubernetes 启动 Docker

 Dotnet autocore using "sha256:8ef2fe7fdb1c144058eabfc1c88a586ba587eaf4870988158e1772545d60a260"   🔴

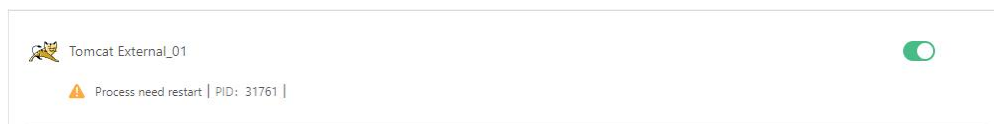
PID: 4627 | DockerId: 8321ae44 | k8s pod UID: 12dc5247-8606-11ea-8d54-005056a521ee | Agent Version: 3.0.0.0 | App Name: core_redis |

Port: 5050 | Instance Name: netcore-192.168.2.207-5050-autocore:5050

- **PID**：进程 ID。
- **DockerId**：若是 Docker 容器中起的进程，则取 DockerId 前八位。
- **kubernetes pod UID**：kubernetes 的 podId。
- **Agent Version**：探针版本。

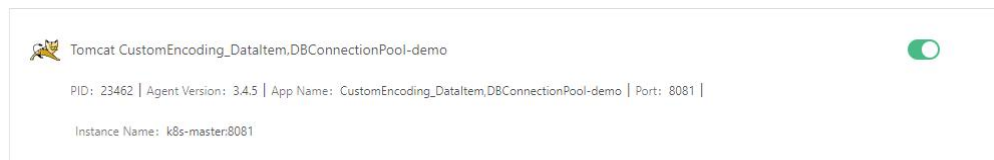
- **App Name:** 应用名，应用命名规则详情参见《TingyunAgent 部署&管理手册》中应用命名规则小节。
- **Port:** 进程监听端口。
- **Instance Name:** 实例名。分为以下四种情况（当 port 取不到时，默认为 0）：
 - **普通进程:** hostName:port。
 - **Weblogic 进程:** ServerName:port。由于.NET Core 是无容器部署，不会存在 Weblogic 进程，因此不会出现此种实例名称。
 - **Docker 容器进程:** ContainerId 前 12 位:port。
 - **kubernetes 启动 Docker 容器进程:** PodName:port。
- **开关:** 进程嵌码禁用/启用开关。当是禁用状态时，不对进程嵌码，需重启进程才能生效。
- **进程状态**

➢ 待重启状态



Process need restart: TingyunAgent 安装前启动的进程，在安装 TingyunAgent 后，需要重启进程才能嵌入探针。

➢ 正常嵌码状态



➢ 禁用待重启状态



Process need to be restarted to remove application agent: 禁用进程后，需要在服务器上重启进程才能生效。

➢ 禁用状态



Process is disabled: 不嵌码状态。

3.18.2 Collector 管理

Agent Collector 简称 Collector，负责汇总性能数据，一套基调听云悟空平台系统可以部署多个 Agent Collector。

在左侧导航栏中单击**管理>部署状态**，然后再上方选择 **Collectors** 页签，进入 Collector 管理页面。

3.18.2.1 Collector 列表

Collector 列表部分展示 7 天内有数据的 Collector 信息。列表左上角显示后的数字代表：7 天内有数据的 Collector 数量/所有。

OneAgents管理

Collectors管理

下载中心

过去30分钟

🔍

🔄

关闭

🔍 过滤条件

🔍 查询

自定义表头

显示: 21/21

批量升级

批量启用

批量禁用

新增

导出

☐	IP	部署环境	版本	协议	负载	CPU	内存	带宽	状态	升级状态	操作
☐	+ 192.168.2.241	javatest	3.6.5.0	3.2.0	0	0 %	25.79 MB(0.45 %)	4.83 KB	空闲	升级成功	禁用
☐	+ 10.128.2.38	default_38	3.6.5.0	3.2.0	0	0 %	140.59 MB(2.45 %)	5.40 KB	空闲	升级成功	启用
☐	+ 192.168.1.173	default_173	3.6.5.0	3.2.0	0.01	1 %	38.53 MB(0.61 %)	3.14 KB	空闲	升级成功	启用
☐	+ 192.168.2.207	zhaodc	3.6.5.0	--	--	--	--(--)	--	--	升级成功	
☐	+ 10.128.2.230	default_230	3.6.5.0	3.2.0	0	0 %	53.24 MB(0.85 %)	1.46 KB	空闲	升级成功	禁用

列表项说明如下：

- IP：Collector 的 IP 地址。
- 部署环境：数据中心、VPC、机房等名称。
- 版本：Collector 的版本号。
- 协议：Collector 所采用的协议版本号。
- 负载：该 Collector 所在主机的 CPU 繁忙程度。当负载的值在[0,1]之间时，表明 CPU 不需等待即可处理指令。
- CPU：Collector 当前的 CPU 使用率。
- 内存：Collector 当前的内存使用量和使用率。
- 吞吐率：选定时间段内，平均每秒钟 Collector 处理的数据收集汇总请求数量，单位 Rps。
- 带宽：选定时间段内，Collector 处理的数据收集汇总请求的平均带宽，单位为 Byte/s。
- 状态：当 CPU 使用率大于等于 70%或内存使用率大于等于 80%时，即被视为资源紧张状态。当管理的探针数量为 0 时，被视为空闲状态。其他情况为正常状态。Collector 如果 10 分钟没有心跳，将处于离线状态。

- 升级状态：Collector 的在线升级状态，包括升级中、升级成功、升级失败等。
- 启动时间：Collector 最近一次的启动时间。
- 心跳检测时间：最近一次检测到 Collector 心跳的时间。

用户可以对 Collector 进行以下操作。

- **搜索 Collector:** 搜索单击页面顶端的过滤框，弹出过滤条件。选择过滤条件，输入内容，然后单击**查询**按钮查询 Collector，也可直接按回车键查询。支持通过单个关键字、多个关键字、单个列字段、单个列字段多个值、多个列字段、单个标签、多个标签进行过滤，具体可单击**查询**按钮前的  图标查看搜索帮助。用户可设置多个过滤条件，支持通过部署环境、IP 地址和版本进行搜索。
- **查看 Collector 各组成部分的运行指标趋势图:** 单击主机名称前的  图标，可看到该 Collector 中各组成部分的图标，包括 APM Collector、Infra Collector 和 12 种组件探针图标。

TingyunAgents管理

Collectors管理

下载中心

过去30分钟

🔍

🔄

关闭

🔍 过滤条件

🔍 查询

自定义表头

显示: 10/10

批量升级

批量启用

批量禁用

新增

删除

导出

☐	IP/Name	部署环境	版本	升级状态	禁用/启用
☐	+ tingyun-collector-0.tingyun-collector.tingyun-beta2	tingyun-beta2	3.6.5.0	--	☒
☐	- 10.128.2.38	beta2_38_1014	3.6.5.0	升级成功	☒
					

单击这些图标，可查看 CPU、内存、延迟时间和运行时长（纵轴表示秒数）的变化趋势图。

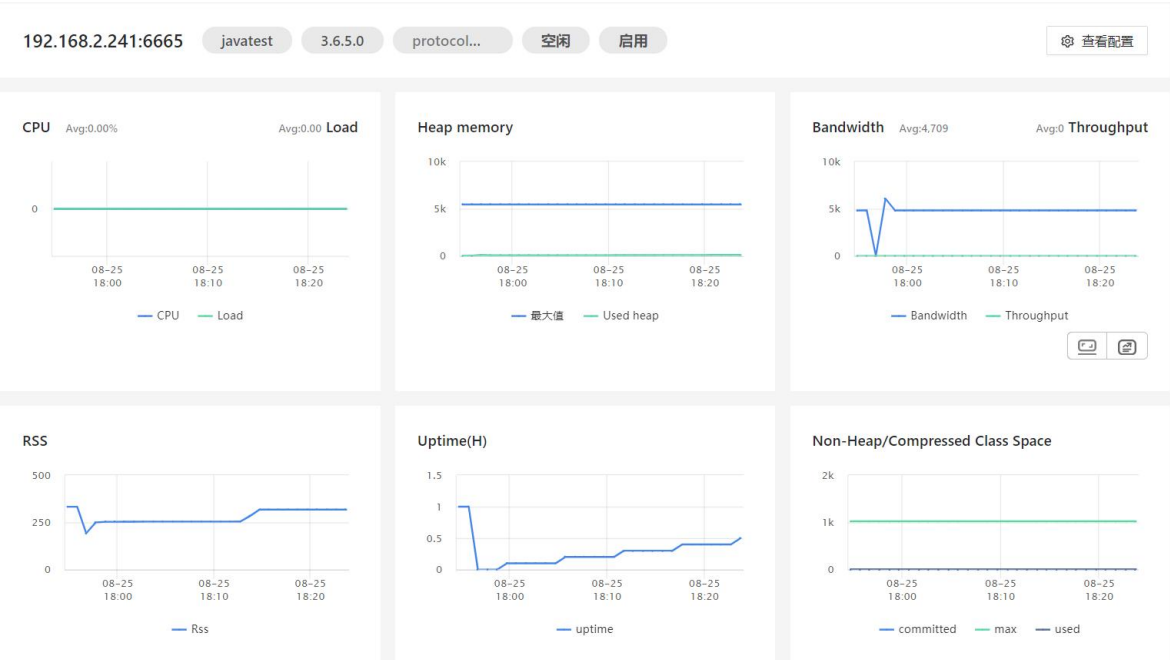


- **自定义表头：**单击列表左上角的自定义表头按钮，可勾选想要展示的列名。部分字段是必选项，必须展示。
 - **禁用或启用 Collector：**关闭禁用/启用列的开关可禁用 Collector，禁用后仍保留 Collector 心跳，以便重新启用。已经连接的 Agent 会重新负载其他的 Collector。
 - **批量启用：**勾选多个 Collector 前的复选框，Collector 列表右上角的**批量启用**按钮变为可用状态，用户可单击按钮批量启用 Collector。启用后，Collector 会加入到 IDC（部署环境）中，共同负载 IDC 下的 Agent 连接。
 - **批量禁用：**勾选多个 Collector 前的复选框，Collector 列表右上角的**批量禁用**按钮变为可用状态，用户可单击按钮批量禁用 Collector。禁用后仍保留 Collector 心跳，以便重新启用。已经连接的 Agent 会重新负载其他的 Collector。
 - **批量升级：**勾选多个 Collector 前的复选框，Collector 列表右上角的**批量升级**按钮变为可用状态，用户可单击按钮批量升级 Collector。在弹出的对话框中选择要升级到的 Collector 版本，然后单击**确定**按钮，即可开始升级。**升级状态**列显示 Collector 的升级状态，单击状态，可查看升级日志。
- 升级建议：先灰度升级一台主机，确认没有问题后再逐一升级，不建议批量升级。
- **导出 Collector 列表数据：**单击 Collector 列表右上角的**导出**按钮，可导出所有 Collector 数据。如果设置了过滤条件，导出的不是搜索后的结果，仍然是所有 Collector 的数据。

- **新增 Collector:** 单击 Collector 列表右上角的**新增**按钮，可进入 Collector 部署页面，用户可以新部署一个 Collector，详情请参见《基调听云_应用与微服务_Agent Collector 部署手册》。
- **删除 Collector:** 列表中勾选要删除的 Collector，单击 Collector 列表右上角的**删除**按钮，除主机探针外，该 Collector 下的所有组件探针实例全部都会被删除。

3.18.2.2 Collector 主机详情

任意单击列表中的一项后，列表下方会展示该 Collector 所在主机的 CPU、各种内存、带宽、管理的探针数量等的变化趋势图。



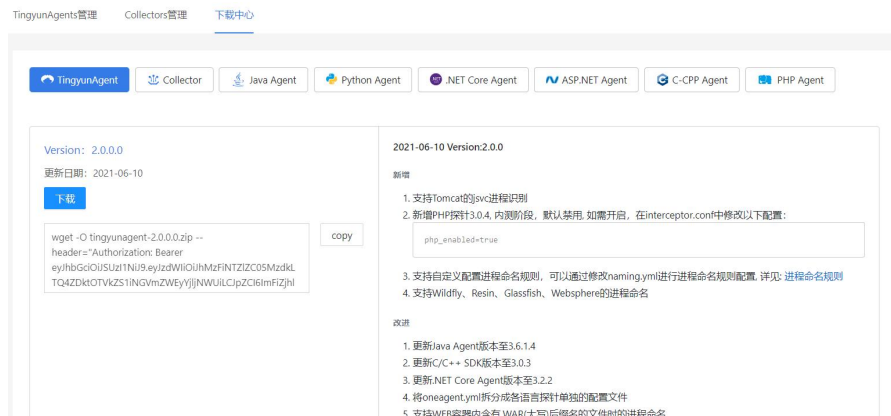
单击右上角的**查看配置**按钮，可查看属性配置、进程和环境变量信息。每个配置项的具体含义可在 collector.properties 中查看。

3.18.3 下载中心

3.18.3.1 TingyunAgent 下载

在 TingyunAgent 页面用于下载各个 TingyunAgent 版本，每个版本包含以下信息：

- Version: TingyunAgent 版本。
- 更新日期: TingyunAgent 发版日期。
- 功能改动说明: 包含 TingyunAgent 当前版本新增、改进及修复的功能。



3.18.3.2 Agent Collector 下载

在 Agent Collector 页面用于下载各个 Agent Collector 版本，在下载 Agent Collector 之前，需要选择 With JRE(Oracle JRE 1.8.125)或 No JRE，因为 Collector 要求 JRE1.8+，当选择 No JRE 时，请确认目标服务器已安装好要求的 JRE。Agent Collector 包含以下版本信息：

- Version：Agent Collector 版本。
- 更新日期：Agent Collector 发版日期。
- 下载按钮：下载 Agent Collector 安装包。
- copy 按钮：复制 Agent Collector 下载命令。
- 功能改动说明：包含 Agent Collector 当前版本新增、改进及修复的功能。

3.18.3.3 Agent 下载

各语言 Agent 页面用于下载各个版本的 Agent 或者 SDK，每个版本包含以下信息：

- Version：Agent 版本。
- 更新日期：Agent 发版日期。
- 下载按钮：下载 Agent 安装包。
- copy 按钮：复制 Agent 下载命令。
- 功能改动说明：包含 Agent 当前版本新增、改进及修复的功能。

3.18.4 第三方探针接入

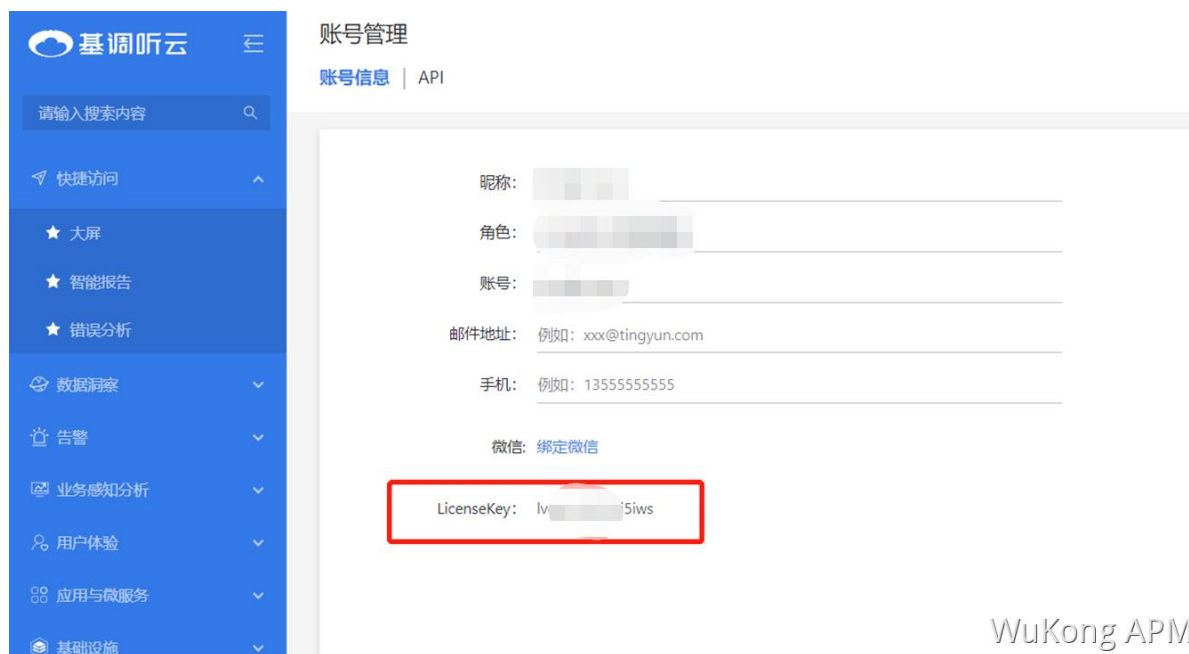
3.18.4.1 通过 OpenTelemetry 接入 Java 应用数据

3.18.4.1.1 OpenTelemetry 简介

OpenTelemetry 是 CNCF 的一个可观测性项目，旨在提供可观测性领域的标准化方案，解决观测数据的数据模型、采集、处理、导出等的标准化问题，提供与三方供应商无关的服务。它是一组标准和工具的集合，旨在管理观测类数据，如 Traces、Metrics、Logs 等，目前已经是业内的标准。

3.18.4.1.2 前提条件

- 下载 OpenTelemetry Agent
- 下载 OpenTelemetry Collector
- 目前基调听云应用与微服务只支持 OpenTelemetry Java 探针的数据接入。
- 在平台获取 License 及接入点信息，具体操作如下：
- 获取 License：将鼠标悬浮在平台左下角的账号名称位置，在悬浮菜单中选择账户管理，在账号信息页签中查看 LicenseKey。



获取接入点信息：接入点为应用与微服务 Data Center 开放的一个端口。您可以在 SkyWalking 探针接入页面中获取接入点信息。

SkyWalking
 OpenTelemetry

1

下载最新的Agent，前往[官网](#)下载。

2

打开config/agent.config，配置Agent。

2.1 配置接入点

collector.backend_service=\${SW_AGENT_COLLECTOR_BACKEND_SERVICES:http://10.128.2.95:11800}

copy

2.2 修改license

agent.instance_properties[tingyun.license]=9OgQXob4VGGFFYcm

copy

2.3 配置应用名称

agent.service_name=<ServiceName>

copy

3

根据应用的运行环境，选择相应的方法来指定SkyWalking Agent的路径。

Linux Tomcat 7 / Tomcat 8

在tomcat/bin/catalina.sh第一行添加以下内容：

CATALINA_OPTS="\$CATALINA_OPTS -javaagent:<skywalking-agent-path>"; export CATALINA_OPTS

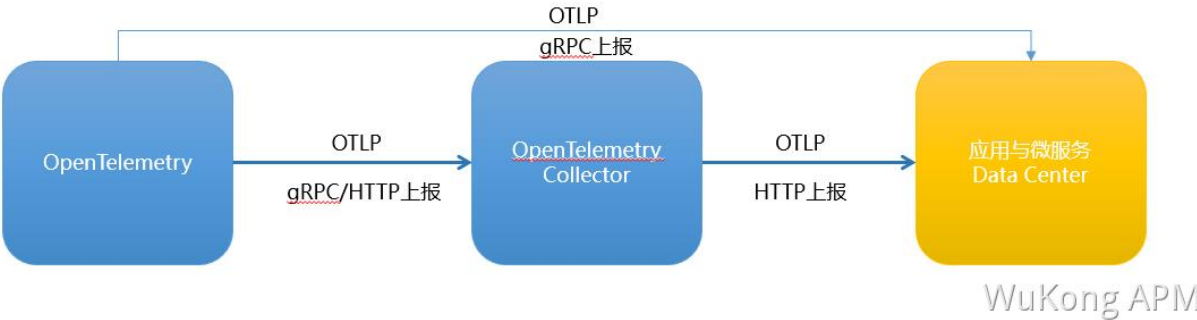
copy

Windows Tomcat 7 / Tomcat 8

在tomcat/bin/catalina.bat第一行添加以下内容：

3.18.4.1.3 数据上报说明

您可以通过两种方式集成 OpenTelemetry Trace 和 Metric 数据，分别是直接上报和 OpenTelemetry Collector 转发。向基调听云应用与微服务 Data Center 上报数据时目前仅支持 HTTP 协议。



直接上报：

如果您的应用使用了 OpenTelemetry Agent，可以通过 OpenTelemetry gRPC 协议直接向 DC 发送数据，您只需要配置接入点信息以及 license 信息。您可以通过以下两种方式直接上报数据。

- 方式一：通过修改 Java 启动的 VM 参数上报数据。

```

java -javaagent:<opentelemetry-agent-path>

```

```
-Dotel.service.name=<appName>
-Dotel.exporter.otlp.protocol=http/protobuf
-Dotel.exporter.otlp.endpoint=<endpoint>
-Dotel.resource.attributes=tingyun.license=<license>
-jar myapp.jar
```

- 方式二：通过新增环境变量上报数据。

```
#新增环境变量
OTEL_RESOURCE_ATTRIBUTES=tingyun.license=<license>
OTEL_SERVICE_NAME=<appName>
OTEL_EXPORTER_OTLP_ENDPOINT=<endpoint>
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf

#启动应用
java -javaagent:<opentelemetry-agent-path> \
-jar myapp.jar
```

OpenTelemetry Collector 转发：

如果您的应用使用了 OpenTelemetry Agent 和 OpenTelemetry Collector，OpenTelemetry Agent 将接入点配置为本地部署的 OpenTelemetry Collector 地址后（无需修改其他配置），需在 OpenTelemetry Collector 配置 OTLP Exporter 向应用与微服务的 DC 上报数据。

1. 修改 collector 配置文件 otel-config.yaml，并按照下发配置修改 exporters 部分。

```
exporters:
  otlphttp:
    endpoint: <endpoint>
service:
  pipelines:
    traces:
      exporters: [otlphttp]
    metrics:
      exporters: [otlphttp]
```

2. 启动 OpenTelemetry Collector。

3.18.4.1.4 组件支持列表

OpenTelemetry Java 探针支持的组件列表，请参见 <https://github.com/open-telemetry/opentelemetry-java-instrumentation/blob/main/docs/supported-libraries.md> 网页。

3.18.4.1.5 平台暂不支持功能

基调听云应用与微服务对接入的 OpenTelemetry 探针数据暂不支持以下功能：

- 无心跳告警
- 应用分析
 - 定义指标
 - 线程剖析
 - JVM 信息下 HTTP 信息展示
 - 环境信息下除环境信息以外的信息
- 事务分析
 - 自定义指标
 - 线程剖析
- 连接池
- 健康规则
- 应用配置（包括所有配置：自定义嵌码、采样率、配置数据项等）
- 追踪详情：全栈快照、数据项

3.18.4.2 通过 SkyWalking 接入 Java 应用数据

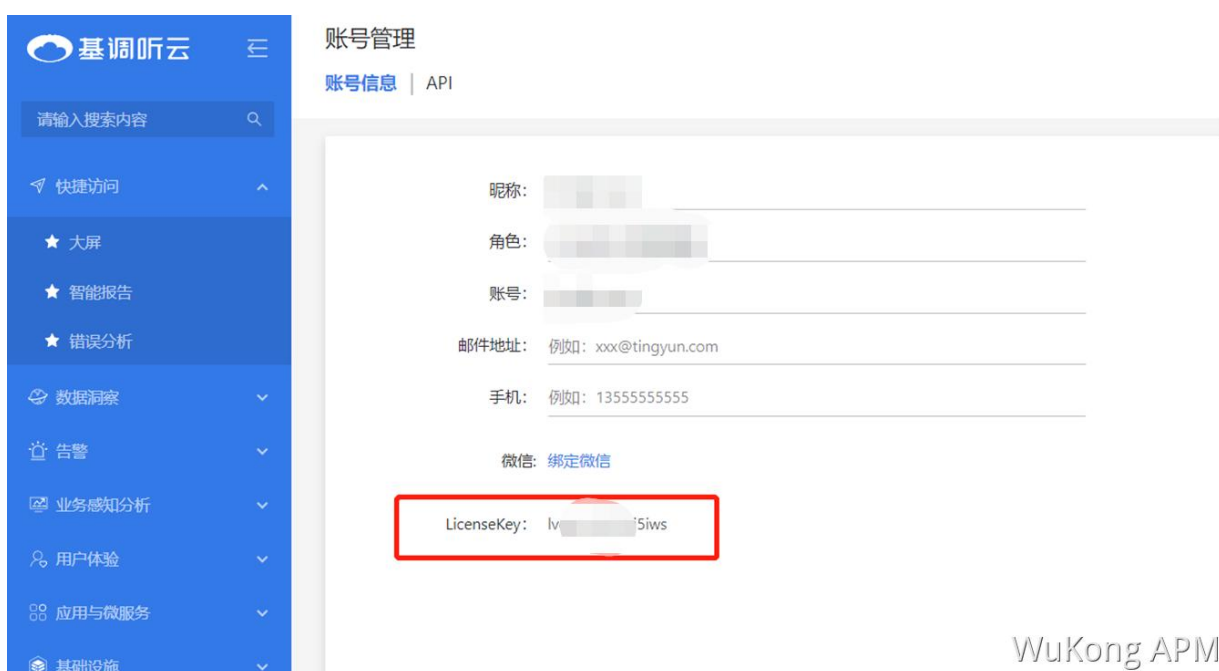
3.18.4.2.1 SkyWalking 简介

SkyWalking 是一款广受欢迎的 APM（Application Performance Monitoring，应用性能监控）产品，主要针对微服务、Cloud Native 和容器化（Docker、Kubernetes、Mesos）架构的应用。SkyWalking 的核心是一个分布式追踪系统。

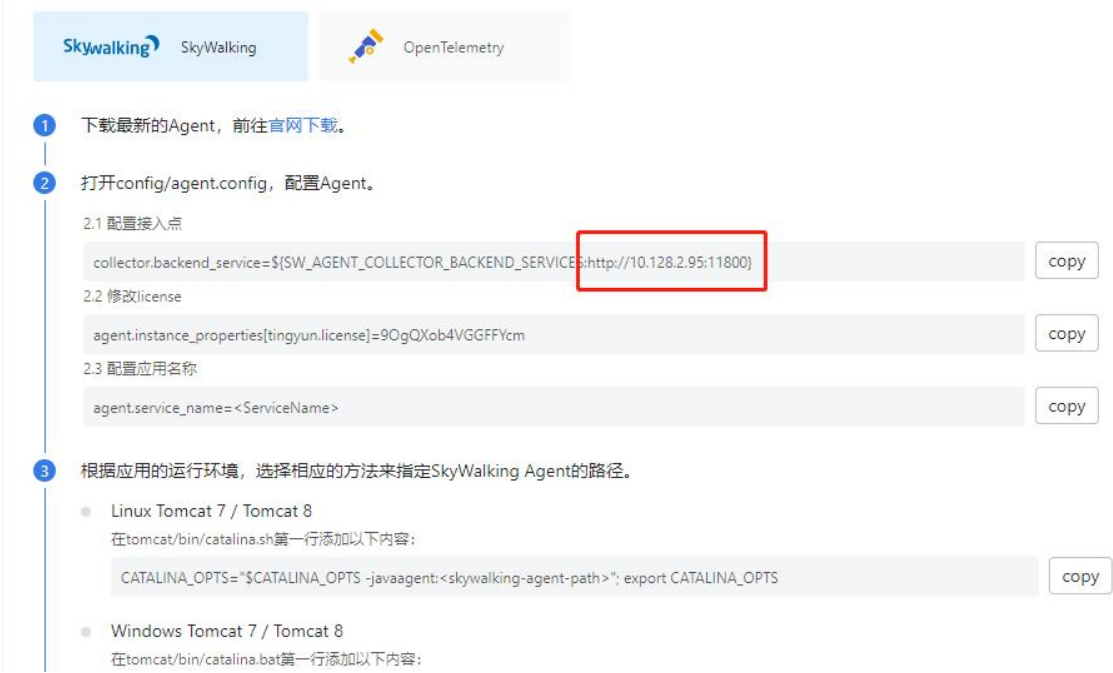
要通过 SkyWalking 将 Java 应用数据上报至基调听云应用与微服务控制台，首先需要完成埋点工作。SkyWalking 既支持自动埋点（Dubbo、gRPC、JDBC、OkHttp、Spring、Tomcat、Struts、Jedis 等），也支持手动埋点（OpenTracing）。本文介绍自动埋点方法。

3.18.4.2.2 前提条件

- 打开 SkyWalking 探针下载页面，下载 SkyWalking8.8.0 及以上版本探针，并将解压后的 Agent 文件夹放至 Java 进程有访问权限的目录。
- 插件均放置在/plugins 目录中。在启动阶段将新的插件放进该目录，即可令插件生效。将插件从该目录删除，即可令其失效。另外，日志文件默认输出到/logs 目录中。
- 目前基调听云应用与微服务只支持 SkyWalking Java 探针的数据接入。
- 在平台获取 License 及接入点信息，具体操作如下：
- 获取 License：将鼠标悬浮在平台左下角的账号名称位置，在悬浮菜单中选择账户管理，在账号信息页签中查看 LicenseKey。

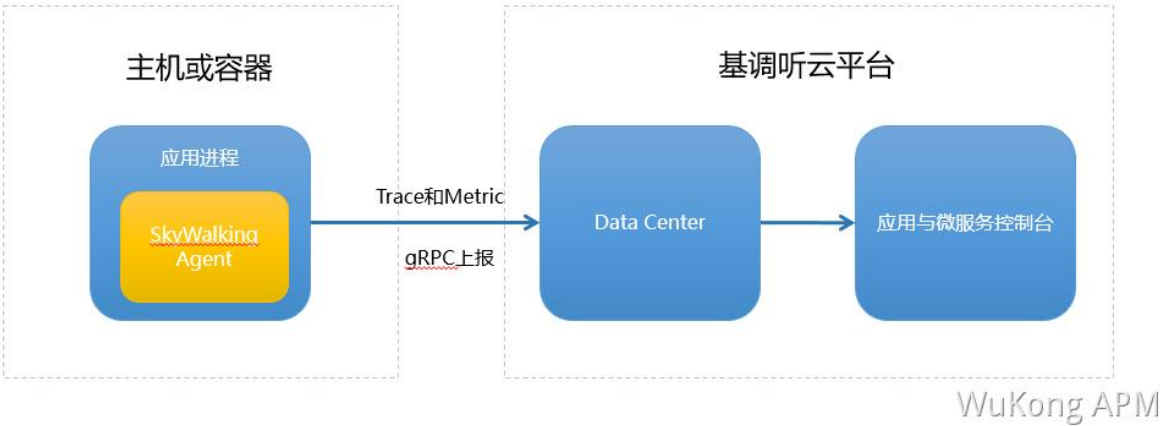


获取接入点信息：接入点为应用与微服务 Data Center 开放的一个端口。您可以在 SkyWalking 探针接入页面中获取接入点信息。



3.18.4.2.3 数据上报说明

您可以通过直接上报的方式集成 SkyWalking Trace 和 Metric 数据，目前只支持私有化部署，不支持 SaaS 环境。



1. 打开 config/agent.config，配置接入点和 license。

配置接入点：

- 方式一：将接入点默认值 127.0.0.1:11800 替换为前提条件中获取到的接入点信息。建议单击 SkyWalking 探针接入页面中的整行接入点配置后的 Copy 按钮，然后替换掉 collector.backend_service 配置项。

```
collector.backend_service=${SW_AGENT_COLLECTOR_BACKEND_SERVICES:127.0.0.1:11800}
```

- 方式二：配置环境变量。将接入点默认值 127.0.0.1:11800 替换为前提条件中获取到的接入点信息。

```
skywalking.collector.backend_service=127.0.0.1:11800  
#或者  
SW_AGENT_COLLECTOR_BACKEND_SERVICES=127.0.0.1:11800
```

配置 license：

- 方式一：请将<license>替换为在前提条件中获取到的 LicenseKey 信息。

```
#方式一：适用于 8.8.0 及以前的 Agent 版本  
agent.instance_properties[tingyun.license]=<license>  
#方式二：适用于 8.9.0 及以后的 Agent 版本  
agent.instance_properties_json={"tingyun.license": "<license>"}  
#方式三：  
agent.authentication=${SW_AGENT_AUTHENTICATION:<license>}
```

- 方式二：配置环境变量。请将<license>替换为在前提条件中获取到的 LicenseKey 信息。

```
#第一种方式：适用于 8.8.0 及以前的 Agent 版本  
skywalking.agent.instance_properties[tingyun.license]=<license>  
#第二种方式：适用于 8.9.0 及以后的 Agent 版本  
skywalking.agent.instance_properties_json={"tingyun.license": "<license>"}  
#第三种方式：适用于 8.9.0 及以后的 Agent 版本  
SW_INSTANCE_PROPERTIES_JSON={"tingyun.license": "<license>"}  
#第四种方式：  
skywalking.agent.authentication=<license>  
#第五种方式  
SW_AGENT_AUTHENTICATION=<license>
```

2. 采用以下方法之一配置应用名称（Service Name）。

- 打开 config/agent.config，配置应用名称。请将<ServiceName>替换为您的应用名称。

```
agent.service_name=<ServiceName>
```

或者配置环境变量：

```
skywalking.agent.service_name=<ServiceName>  
#或者  
SW_AGENT_NAME=<ServiceName>
```

- 在应用程序的启动命令行中添加-Dskywalking.agent.service_name 参数。

```
java -javaagent:<skywalking-agent-path> -  
Dskywalking.agent.service_name=<ServiceName> -jar yourApp.jar
```

3. 根据应用的运行环境，选择相应的方法来指定 SkyWalking Agent 的路径。

请将以下示例代码中的<skywalking-agent-path>替换为 Agent 文件夹中的 skywalking-agent.jar 的绝对路径。

- Linux Tomcat 7/Tomcat 8

在 tomcat/bin/catalina.sh 第一行添加以下内容：

```
CATALINA_OPTS="$CATALINA_OPTS -javaagent:<skywalking-agent-path>"; export  
CATALINA_OPTS
```

- Windows Tomcat 7/Tomcat 8

在 tomcat/bin/catalina.bat 第一行添加以下内容：

```
set "CATALINA_OPTS=-javaagent:<skywalking-agent-path>"
```

- JAR File 或 Spring Boot

在应用程序的启动命令行中添加 javaagent 参数。

注意：javaagent 参数一定要在 jar 参数之前。

```
java -javaagent:<skywalking-agent-path> -jar yourApp.jar
```

- Jetty

在{JETTY_HOME}/start.ini 配置文件中添加以下内容：

```
--exec    # 去掉前面的井号取消注释。  
-javaagent:<skywalking-agent-path>
```

3.18.4.2.4 组件支持列表

SkyWalking Java 探针支持的组件列表，请参见
<https://skywalking.apache.org/docs/skywalking-java/latest/en/setup/service-agent/java-agent/supported-list> 文档

3.18.4.2.5 平台暂不支持的功能

基调听云应用与微服务对接入的 探针 SkyWalking 数据暂不支持以下功能：

- 无心跳告警
- 应用分析
 - 定义指标
 - 线程剖析
 - JVM 信息下 HTTP 信息展示
 - 环境信息下除环境信息以外的信息
- 事务分析
 - 自定义指标
 - 线程剖析
- 连接池
- 健康规则
- 应用配置（包括所有配置：自定义嵌码、采样率、配置数据项等）
- 追踪详情：全栈快照、数据项

3.19 配置

应用与微服务中提供 4 种级别的选项配置，配置级别（即配置的顺序）从高到低分别是全局配置、业务系统配置、应用配置和实例配置。部分配置项会同时出现在多种级别的配置页，配置生效情况说明如下：

- 如果在低级别的配置中选择使用上级配置时，当前级别的被监控对象继承并保持与上一级完全一样的设置值。
- 如果低级别的配置中勾选了单独配置功能，且打开开关设置了与上级配置不一样的设置值，则当前低级别的设置值会生效，即配置生效的优先级为：全局配置<业务系统配置<应用配置<实例配置。

- 如果低级别的配置中勾选了单独配置功能，但是没有打开开关，则代表当前监控对象不启用该功能。

3.19.1 全局配置

全局配置的配置项允许用户设置对整个系统统一生效，包括日志溯源开关、开启追踪、错误及异常采集设置和探针熔断等选项。

3.19.1.1 Apdex T

Apdex 定义了应用响应时间的最优门槛 T （即 Apdex T ），另外根据应用响应时间结合 T 定义了三种不同的性能表现：

- Satisfied（满意）：应用响应时间低于或等于 T （ T 由性能评估人员根据预期性能要求确定），则认为用户对应用的性能表现满意。比如 T 为 1.5s，则一个耗时 1s 的响应结果则可以认为是满意的。
- Tolerating（可容忍）：应用响应时间大于 T ，但同时小于或等于 $4T$ 。假设应用设定的 T 值为 1s，则 $4 * 1 = 4$ 秒，即为应用响应时间的容忍上限。
- Frustrated（不能忍受）：应用响应时间大于 $4T$ 。另外，当一个请求的响应状态码为 400 及以上（401 除外）或发生最外层异常时，无论响应时间长短，均归类为不能忍受。

应用与微服务取应用的平均响应时间作为计算指标，默认定义 Apdex T 为 500 毫秒。全局配置中的 Apdex T 配置适用于所有业务系统中的应用。

Apdex 标准从用户的角度出发，对真实的响应时间进行采样，采集一定时间之后，将应用响应时间的表现，转化为用户对于应用性能的可量化的满意度评价，我们称之为 Apdex 指数，范围为 0~1。0 代表没有满意的用户，1 代表所有用户都满意。

计算公式为：Apdex 指数 = (满意数 + 可容忍数 / 2) / 成功次数

应用与微服务定义 Apdex 指数范围对应的颜色展示如下：

- 0.94~1: 
- 0.85~0.94: 
- 0.7~0.85: 
- 0.7 以下: 

3.19.1.2 日志溯源

3.19.1.2.1 Java Agent 3.6.3.3 及以下版本

一个事务往往会跨越很多个应用，事务的日志都分散在各个应用下，查找单次请求的所有日志较为困难。应用与微服务的日志溯源功能可以很好的解决这个问题。开启日志溯源功能后，APM 探针可以自动在用户的日志内容中输出 NBS.APPID 和 NBS.REQUEST_GUID 属性，通

过 NBS.REQUEST_GUID 属性可以关联不同应用和实例上单次请求的日志。当用户通过应用与微服务追踪到慢事务时，可以根据该次慢追踪的追踪 ID 在应用日志中查询该慢事务的所有日志信息。

说明：

- NBS.REQUEST_GUID 即应用与微服务控制台页面中的追踪 ID。
- 日志溯源默认是关闭状态。
- 只有对 Java 应用支持该功能。
- 日志溯源功能支持主流的 Log4j 和 Logback 日志框架。
- 要使用日志溯源功能，链路的所有应用都需要部署应用与微服务探针。

配置日志溯源，请按照以下步骤进行操作：

1. 如果应用初次使用该功能，需要在探针配置文件 tingyun.properties 中开启以下设置。

日志追溯相关 Plugin，开启后可以在应用的日志中，打印 APM 相关数据，如应用 ID、追踪 ID 等。

```
class_transformer.tingyun-log4j-plugin-2.0.0.enabled=true
class_transformer.tingyun-log4j-plugin-2.3.enabled=true
class_transformer.tingyun-log4j-plugin-1.2.enabled=true
class_transformer.tingyun-logback-plugin-1.2.enabled=true
```

2. 登录应用与微服务控制台，在左侧导航栏中选择**全局配置**，在**常规选项**页签中开启日志溯源功能。



3. 配置被监控应用的日志配置文件。

➤ Log4j 配置

```
log4j.appender.order-file-appender.layout.ConversionPattern=[%d] [%-5p] [%t] [%c] [%R][%A]%m%n
```


➤ Log4j2 配置

```
<Console name="Console" target="SYSTEM_OUT">
  <PatternLayout pattern="%d{HH:mm:ss.SSS} [%log4j2] %-5level %logger{36} -
    %msg%n[%A][%R]">
  </Console>
```

➤ Logback 配置

```
<encoder>
  <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} -
    %msg%n[%A][%R]</pattern>
</encoder>
```

4. 查看输出的应用日志。

例如，Log4j 日志输出格式如下：

```
[2020-10-29T16:30:27.730-07:00][DEBUG][com.tingyun.Log-
0][com.tingyun.Test][NBS.REQUEST_GUID:167d328d-b82a-4b8c-8049-
7a3a13af158f][NBS_APPID:0017]
Test Log Message
Test Log Message
```

其中，[%R]负责输出请求的 NBS.REQUEST_GUID，配置后日志会增加 NBS.REQUEST_GUID 信息。在整个请求链路中，入口事务生成的 NBS.REQUEST_GUID 将作为整个请求链路的唯一标识，此值将在链路中不断传递直到链路的最后请求（如果遇到无法实现跨应用追踪的组件的情况除外），以实现全链路追踪。调用链所涉及的各个应用的日志都显示同一个 NBS.REQUEST_GUID。[%A]负责输出应用的 NBS.APPID，是应用与微服务为每一个监控的应用生成的唯一 id，配置后日志会增加 NBS.APPID 信息。如果日志溯源功能是关闭的，即使用户配置了[%R]，NBS.REQUEST_GUID 也不会被嵌入。

```
[NBS.APPID:883][NBS.REQUEST_GUID:5aa2d73e37139eb] 15:14:19.301 [http-nio-8080-exec-1] WARN c.tingyun.controller.MainController - ===== warn
[NBS.APPID:883][NBS.REQUEST_GUID:5aa2d73e37139eb] 15:14:19.301 [http-nio-8080-exec-1] ERROR c.tingyun.controller.MainController - ===== error
[NBS.APPID:883][NBS.REQUEST_GUID:5aa2d73e37139eb] 2018-12-05 15:14:19.301 [http-nio-8080-exec-1] [13232 83] DEBUG - UserInfo: Start record user info.
2018-12-05 15:14:19.301 [http-nio-8080-exec-1] [13232 83] DEBUG - UserInfo: Application configed get user info from: UserInfoRules[url='null', heade
2018-12-05 15:14:19.301 [http-nio-8080-exec-1] [13232 83] DEBUG - UserInfo: Agent get user info from uri or body parameters, result: UserInfo[origin
2018-12-05 15:14:19.301 [http-nio-8080-exec-1] [13232 83] DEBUG - Sending X-Tingyun-Id header: ;c=SlpT2Cl3AdhM4#lqet6-iv8gA,x=5aa2d73e37139eb,e=340
2018-12-05 15:14:19.309 [http-nio-8080-exec-2] [13232 84] DEBUG - backtrace: [java.lang.Thread.getStackTrace(Thread.java:1556), com.tingyun.agent.co
2018-12-05 15:14:19.309 [http-nio-8080-exec-2] [13232 84] DEBUG - Setting action name using servlet name: mvc-dispatcher
2018-12-05 15:14:19.309 [http-nio-8080-exec-2] [13232 84] DEBUG - Setting action name to "mvc-dispatcher"
2018-12-05 15:14:19.309 [http-nio-8080-exec-2] [13232 84] DEBUG - Client tingyun id is ;c=SlpT2Cl3AdhM4#lqet6-iv8gA,x=5aa2d73e37139eb,e=3400d4f7df
2018-12-05 15:14:19.310 [http-nio-8080-exec-2] [13232 84] DEBUG - ( exclusive:0.0000, duration:0.0000)Spring/HandlerInterceptor/preHandle Spring/org
2018-12-05 15:14:19.310 [http-nio-8080-exec-2] [13232 84] DEBUG - Setting action name to "/hello"
2018-12-05 15:14:19.310 [http-nio-8080-exec-2] [13232 84] DEBUG - Setting action name to "/hello"
15:14:19.310 [http-nio-8080-exec-2] INFO c.tingyun.controller.MainController - info===== info
[NBS.APPID:883][NBS.REQUEST_GUID:5aa2d73e37139eb] 15:14:19.310 [http-nio-8080-exec-2] WARN c.tingyun.controller.MainController - warn===== warn
[NBS.APPID:883][NBS.REQUEST_GUID:5aa2d73e37139eb] 15:14:19.310 [http-nio-8080-exec-2] ERROR c.tingyun.controller.MainController - error===== error
[NBS.APPID:883][NBS.REQUEST_GUID:5aa2d73e37139eb] 2018-12-05 15:14:19.310 [http-nio-8080-exec-2] [13232 84] DEBUG - ( exclusive:0.0003, duration:0.00
2018-12-05 15:14:19.310 [http-nio-8080-exec-2] [13232 84] DEBUG - GET: match resource, resource status code is 200
```

3.19.1.2.2 Java Agent 3.6.4 及以上版本

一个事务往往会跨越很多个应用，事务的日志都分散在各个应用下，查找单次请求的所有日志较为困难。应用与微服务的日志溯源功能可以很好的解决这个问题。开启日志溯源功能后，APM 探针可以自动在用户的日志内容中输出 `tingyun.app_id`、`tingyun.trace_id` 和 `tingyun.span_id` 属性，通过 `tingyun.trace_id` 属性可以关联不同应用和实例上单次请求的日志。当用户通过应用与微服务追踪到慢事务时，可以根据该次慢追踪的追踪 ID 在应用日志中查询该慢事务的所有日志信息。

说明：

- `tingyun.trace_id` 即应用与微服务控制台页面中的追踪 ID。
- 日志溯源默认是关闭状态。
- 只有对 Java 应用支持该功能。
- 日志溯源功能支持主流的 Log4j 和 Logback 日志框架。
- 要使用日志溯源功能，链路的所有应用都需要部署应用与微服务探针。

配置日志溯源，请按照以下步骤进行操作：

1. 登录应用与微服务控制台，在左侧导航栏中选择**全局配置**，在**常规选项**页签中开启日志溯源功能。



2. 默认情况下，探针采用**自动注入**方式在日志中增加 `tingyun.app_id`、`tingyun.trace_id` 和 `tingyun.span_id` 属性。该方式的优点是探针自动将数据写入应用日志，无需修改应用配置。缺点是您不能在日志配置文件中自定义上述属性输出的位置。

日志输出示例：

```
2022-03-24 16:43:52.302 [http-nio-8079-exec-1] INFO
[tingyun.app_id:2503,tingyun.trace_id:77469a69f8b9fdce,tingyun.span_id:77469a69f8b9fdce]
com.tyt.controller.LogbackController -Thread[http-nio-8079-exec-1,5,main]
```

- 如果您的环境中对日志格式做了一些定义，日志溯源可能影响日志解析，此时如果您仍想使用日志溯源功能，可根据业务自定义输出位置。您可采用**手动注入方式**（不推荐），该方式需要配置被监控应用的日志配置文件。例如，您希望将上述属性输出在日志的最前面，可做如下配置：

```
log4j.appender.order-file-appender.layout.ConversionPattern= [TINGYUN] [%d] [%-5p] [%t] [%c]%m%n
```

日志输出示例：

```
[tingyun.app_id:2503,tingyun.trace_id:77469a69f8b9fdce,tingyun.span_id:77469a69f8b9fdce] 2022-03-24 16:43:52.302 [http-nio-8079-exec-1] INFO com.tyt.controller.LogbackController -Thread[http-nio-8079-exec-1,5,main]
```

其中，[TINGYUN]负责输出请求的 `tingyun.app_id`、`tingyun.trace_id` 和 `tingyun.span_id`。在整个请求链路中，入口事务生成的 `tingyun.trace_id` 将作为整个请求链路的唯一标识，此值将在链路中不断传递直到链路的最后请求（如果遇到无法实现跨应用追踪的组件的情况除外），以实现全链路追踪。调用链所涉及的各个应用的日志都显示同一个 `tingyun.trace_id`。`tingyun.app_id` 是应用与微服务为每一个监控的应用生成的唯一 id。`tingyun.span_id` 是应用与微服务为请求链路中每一段请求生成的唯一 id。如果日志溯源功能是关闭的，即使用户配置了[TINGYUN]，上述 3 个属性也不会被嵌入。

```
17:14:25.400 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error0
17:14:25.417 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error1
17:14:25.417 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error2
17:14:25.417 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error3
17:14:25.417 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error4
17:14:25.417 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error5
17:14:25.417 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error6
17:14:25.418 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error7
17:14:25.418 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error8
17:14:25.418 [log4j2] ERROR com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== error9
17:14:25.418 [log4j2] DEBUG com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== debug
17:14:25.418 [log4j2] INFO com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] =====被调用者 test
17:14:25.418 [log4j2] WARN com.tingyun.controller.Log4j2Controller - [tingyun.app_id:2083,tingyun.trace_id:aaa8483cbbff457d,tingyun.span_id:aaa8483cbbff457d] ===== warn
```

3.19.1.3 采样

采样是针对链路追踪数据是否保留的一种策略。开启采样不会对指标数据造成影响。基调听云平台通过“调用链采样”和“请求采样”两种模式实现采样，两种模式只能开启其中一种。

3.19.1.3.1 调用链采样

默认情况下，应用与微服务探针提供全量的业务和性能数据采集。但是在应用访问量比较大时，为了让探针减少对 CPU、内存等资源的消耗，可以对采集的调用链数据按照一定的比例进行保留，这样能够帮助您以较低的性能开销记录最有价值的链路数据。调用链采样适用于以下场景：

- 压测或大促期间流量过大，为了避免调用链数据全量上传影响客户端性能，可以考虑按需调整采样率的比例。

- 日常情况下，全量调用链数据上传导致网络带宽成本高，可以考虑按需调整采样率的比
例。

调用链采样就是根据追踪 ID 记录一定比例的调用链数据。例如，固定比例为 10‰，则每 1000 条调用链数据记录 10 条。采样不会导致调用链数据本身不完整，要么保留整条链路数据，要么丢弃整条链路数据。开启调用链采样后，Agent Collector 在保障调用链完整的情况下，依据采样率保留事务追踪，性能汇总数据依然保持全量采集。默认调用链采样开关为关闭状态，开启后，默认采样率为 2‰，请根据实际需要调整采样率。

 **说明：**仅 V3.6.1.2 以上版本的 Agent Collector 支持调用链采样。

3.19.1.3.2 请求采样

开启请求采样后，基调听云平台将忽略调用链完整性诉求，依据作用在实例级上的采样配置，针对每次请求，保留其追踪数据。该模式下，链路完整性无法保证，请谨慎开启。

您可以根据固定比例或者固定个数两种模式来采集链路追踪数据。

- 默认情况下，每实例每分钟采集 5‰的链路追踪数据，可根据实际需要进行配置。
- 默认情况下，每实例每分钟固定采集 10000 个链路追踪数据，可根据实际需要进行配置。

3.19.1.4 采集调用成功率和响应率

当开启采集调用成功率和响应率时，应用探针会定期通过 JVM 接口上报外部服务和请求的成功率和响应率数据。并将获取的数据进行计算。

获取的数据如下：

- time：发生时刻
- url：外部服务/请求的 URL
- success：成功数
- exception：发生异常数
- timeout：超时数
- httpError：发生 HTTP 错误数
- error：发生错误数

调用成功率

调用成功率是当前应用调用其他请求的成功率。调用成功率=（调用成功数/调用总次数）*100%。

调用总次数=调用成功数+调用超时数+调用 HTTP 错误数+调用异常数。

- 调用超时数：调用请求发生超时的次数。

- 调用 HTTP 错误数：HTTP 调用错误数为状态码 ≥ 400 的调用次数，非 HTTP 调用不计数。
- 调用异常数：调用请求抛出的异常的次数，调用请先判断是否超时，再判断是否属于异常，最后判断是否调用错误。

 **说明：**对于每次外部服务调用或请求，仅计数一次。例如：超时时间设置为 30 秒时，当一个调用执行了 40 秒并返回 200 状态码时，应该仅对 timeout 计数，不应该对 success 计数。

调用响应率

调用响应率是当前应用响应其他接口调用的成功率。响应成功率=（响应成功数/请求次数）*100%。

请求次数=响应成功数+响应超时数+响应异常数。

- 请求次数：是当前应用收到其他接口的访问次数。
- 响应成功数：是当前应用响应其他接口调用的成果次数。
- 响应超时数：是当前应用响应其他接口的超时次数。
- 响应异常数：是当前应用响应其他接口的异常次数。

开启采集

当单击开启采集调用成功率和响应率时需要配置响应率超时时间和调用成功率超时时间。

启用采集调用成功
率和响应率: 

响应率超时时间: ms

配置应用接收到的请求超时时间，用于计算响应率。

调用成功率超时时
间: ms

配置应用调用其他请求超时的时间，用于计算调用成功率。

响应率超时时间：配置应用接收到的请求超时时间，用于计算响应率。开启后响应率默认超时时间为 2000ms。

调用成功率超时时间：配置应用调用其他请求超时时间，用于计算调用成功率。开启后默认调用成功率超时时间为 2000ms。

3.19.1.5 获取源代码

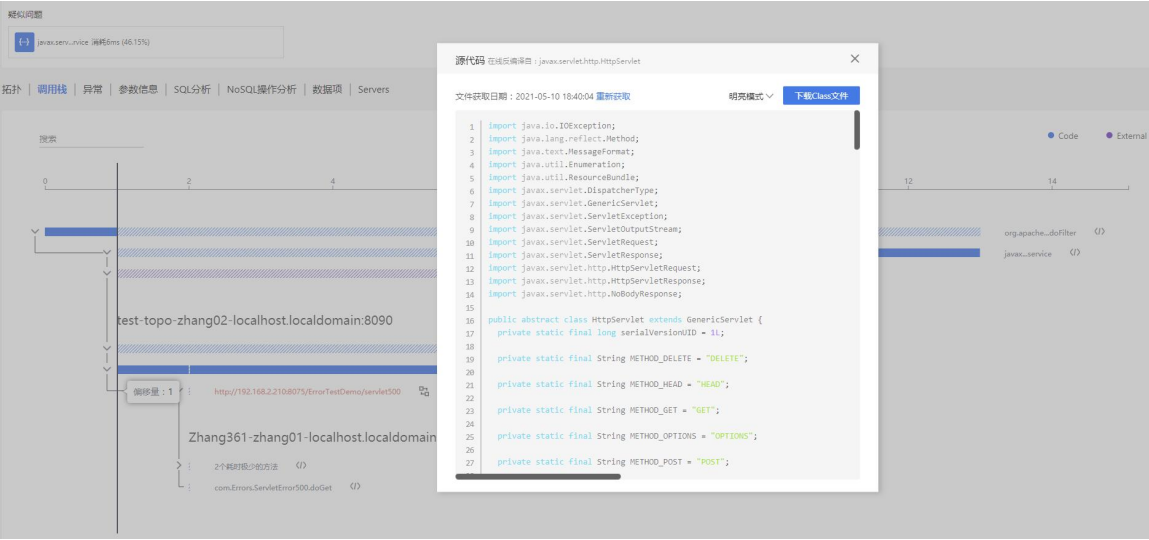
为了更快地帮助用户找到代码问题，应用与微服务可通过获取用户应用的源代码进行问题分析。此处的源代码是指从 JVM ClassLoader 中获取 Java Classs 文件，经反编译而转存的代码。基调听云不会主动获取任何 Classes 文件，只有当用户手动触发获取源文件时，才从 JVM

中动态获取。转存的 Class 文件，只供用户在线反编译使用，不会做其他用处。基调听云不会向任何第三方分发，暂存在磁盘上 7 天后会自动清理。

要获取源代码，请按以下步骤进行操作：

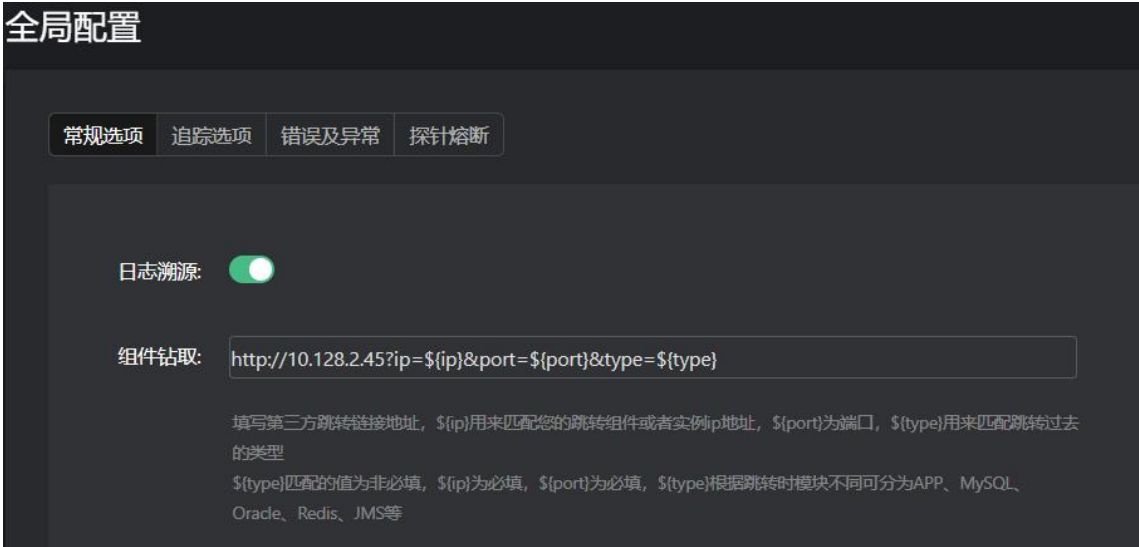
1. 实施人员配置 Nacos 组件的 apm-api 和 apm-config，将 applicationService.showSourceCode 的值改为 1。默认值为 0，报表上不显示获取源代码功能项。
2. 在报表上开启获取源代码功能。
3. 重启应用。

用户可在事务/服务接口/后台任务追踪页面的调用栈中，查看某个类的源代码。



3.19.1.6 组件钻取

填写第三方跳转链接地址，\${ip}用来匹配您的跳转组件或者实例 ip 地址，\${port}为端口，
 \${type}用来匹配跳转过去的类型。\${type}匹配的值为非必填，\${ip}为必填，\${port}为必填，
 \${type}根据跳转时模块不同可分为 APP、MySQL、Oracle、Redis、JMS 等。

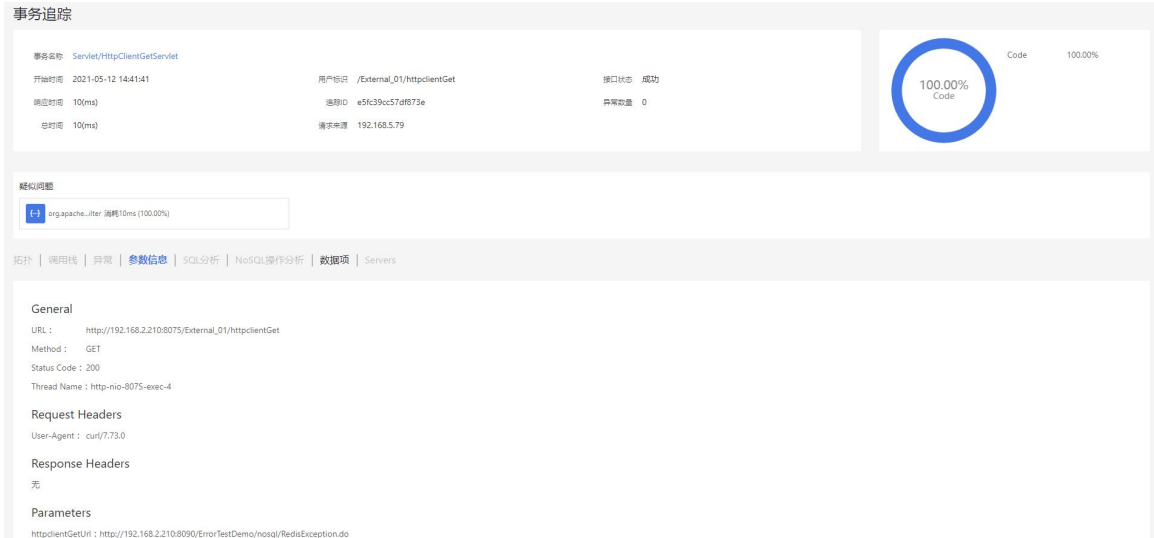


3.19.1.7 追踪选项

- 事务追踪阈值：当请求的响应时间大于阈值时，会被应用与微服务标记为慢事务。事务追踪条目前会显示 图标。

☐ 慢...	时间	追踪ID	事务名称	响应时间(ms)	用户标识	业务系统	状态	入口应用
	05-10 21:06:56	860bc27b1abb25...	Servlet/healthChec...	0		tingyun-server2.0	成功	1_23_report-sys
	05-10 21:06:50	c74ec57d63bf6ba6	Servlet/HttpClient...	1839	kongjie	Metrics	失败	jmeter-192.168.5.181
	05-10 21:06:50	5a5i244bda15	URI/Servlet/testser...	2127	kongjie	Metrics	成功	jmeter-192.168.5.181
	05-10 21:06:50	bfd8556ec5f2301	URI/Servlet/Comm...	2761	kongjie	Metrics	成功	jmeter-192.168.5.181

- 事务追踪明细：默认情况下，应用探针对业务系统中请求的所有事务保留各种跟踪记录数据，例如代码的调用堆栈和执行时间、HTTP 请求参数、查询慢的 SQL 语句等等。开启该选项后，默认配置值为 10ms，当事务响应时间大于等于 10ms 时，事务的追踪明细数据会被保留；响应时间小于 10ms 的事务请求调用栈详情会被丢弃，单次追踪拓扑、组件调用会受影响，请求参数、数据项不受影响，体现在非慢事务追踪详情页面中，只有**参数信息**和**数据项**会有数据。目前仅支持 Agent Collector V3.6.1 以上版本。该功能默认为关闭状态，当开启时，配置的值必须小于等于上方的慢事务阈值。



- 慢方法堆栈：对于事务涉及的方法、SQL 语句、NoSQL 操作、自定义方法，当执行时间大于阈值时才记录方法的堆栈、慢 SQL 详情和慢 NoSQL 操作详情。默认开启。
- 混淆 SQL：指对跟踪到的 SQL 语句中的数字和字符串值进行混淆操作，以问号“?”替换。默认开启。仅对慢 SQL 追踪和请求追踪生效。
- 采集请求参数：采集 HTTP 的请求头和请求参数。默认开启。
- 忽略的参数列表：如果不想采集个别敏感的参数，可以进行设置，多个参数以英文逗号进行分隔。

3.19.1.8 错误及异常

- 采集错误和异常：开启时，应用探针将采集各业务系统中所有应用的错误和异常信息。默认为开启状态。
- 捕捉异常堆栈：启用后，事务追踪中的异常信息将包含堆栈信息。默认为关闭状态。
- 选择语言类型：目前支持 Java、.NET 和.NET Core 三种语言。不同语言显示的配置项不同。
- 采集日志异常：勾选 Log4j 或 Logback 复选框后，可采集和统计日志组件输出的 error 及以上级别日志中的异常信息，例如应用输出了日志 log.error("message","IOException")，那么 IOException 将会被采集。.NET 和.NET Core 还支持 System 日志组件。当勾选日志组件的复选框后，就可以勾选日志级别高于‘error’的‘message’和设置事务状态为错误复选框了。
 - 日志级别高于‘error’的‘message’：默认情况下，对于 error 及以上级别的日志，如果日志中只有 message，而没有异常信息，是不会被统计为异常的。勾选该复选框后，上述日志也会被统计为异常。
 - 设置事务状态为错误：勾选后，error 及以上级别的日志所对应的事务会被视为错误，否则只被视为异常。Logback 只有 error 级别的日志。
- 根据 Logged Message 忽略：符合日志信息匹配规则的日志异常不会被视为异常或错误。单击添加按钮，选择信息匹配方式，输入异常信息，该处不支持正则表达式。规则可被编辑和删除。

- 根据异常类忽略：您可将不想计入错误和异常的异常类加入列表，后续统计错误时这些类将被忽略。单击**添加**按钮，输入包含包名的完整异常类名。可勾选 **Exception Message 过滤**复选框，通过 `exception.getMessage()` 打印出的异常信息忽略异常，如果匹配成功，则该 `Exception` 不会被认为是 `Exception`。选择信息匹配方式，输入异常信息，该处不支持正则表达式。规则可被编辑和删除。
- 自定义 HTTP 状态码：默认情况下，基调听云会将 Status Code 400 ~ 505 的事务状态设置为错误。但是业务上的错误可能并不符合这个规则，如：900 在金融场景下可能代表“余额不足”，需要被作为事务错误，而 401 被认为是正常的事务请求。通过自定义 HTTP 状态码，可分别设置错误状态码和正常状态码来标识事务是否正常，以符合上述 2 种场景的需要。“错误状态码”类型的自定义 HTTP Status code 的名称在错误分析中会作为错误类型显示，格式为：HTTP Error Code: \${名称}。状态码规则可被编辑、删除和批量删除。

说明：

- 该选项必须在**采集错误和异常**选项开启的情况下才生效。
- 当某个状态码同时被设置成了错误状态码和正常状态码时，基调听云将会根据配置项的优先级判定最终是错误状态码还是正常状态码，排列越靠上的优先级越高。
- 自定义业务错误：业务错误指业务上的一些非正常结果，通过对数据项的值进行规则定义，来识别业务错误。当满足业务错误的规则时，事务的状态会被设置为错误，错误的名称为自定义业务错误的描述。业务错误在错误模块中可以独立分析其趋势、堆栈、调用者、调用栈及请求上下文。单击**添加**按钮，输入业务错误描述，选择数据项（支持创建新的数据项），对值设置匹配规则后，单击**确认**。匹配规则最多可添加 3 条。自定义业务错误可被编辑、删除和批量删除。
- 自定义 Redirect Pages：当用户在操作应用的过程中，经常遇到操作失败后跳转到一个错误提示页面的情况。您可以对应用中发生错误后重定向到的错误页面进行定义，当匹配成功后，操作对应的这个事务的状态会被设置为错误。单击**添加**按钮，输入名称，设置匹配条件和重定向页面的 URL，单击**确认**。自定义 Redirect Pages 可被编辑、删除和批量删除。

3.19.1.9 探针熔断

Java Agent 在运行过程中会消耗应用进程的资源，包括 CPU 和内存，当应用进程资源紧张时，为了应用进程稳定运行，Java Agent 将触发熔断机制，关闭部分数据采集以减少对进程资源的消耗。

相关名词解释如下：

- Heap 内存：JVM 堆内存。
- Garbage Collection：JVM 垃圾回收机制。
- Garbage Collection CPU 时间占比：Garbage Collection 消耗的 CPU 时间/CPU 总时间。

触发探针熔断有两种情况：

1、当 Heap 内存使用率超过配置值时（默认值：70%）或者 Garbage Collection CPU 时间占比超过配置值时（默认值：10%），将对数据采集进行**采样**（默认值：50%）。

2、当 Heap 内存使用率超过配置值时（默认值：80%）或者 Garbage Collection CPU 时间占比超过配置值时（默认值：20%），将**禁用**数据采集，但仍保留探针心跳。

当应用进程的资源恢复后，恢复数据采集。

3.19.2 业务系统设置

业务系统配置作用于每个独立的业务系统。如果您希望业务系统的配置情况与全局配置完全一致，可以选择从全局配置中继承设置值。您也可以单独配置每个业务系统自己的独立配置。

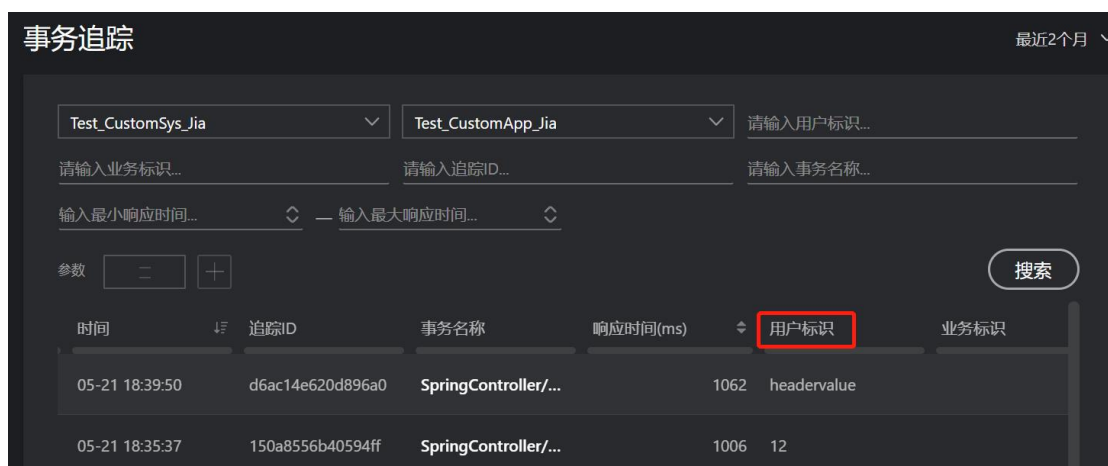
除分位数、用户溯源自定义嵌码、数据项和自定义指标外，其他的业务系统配置项和全局配置的配置项完全相同，具体介绍请参见[全局配置](#)。

3.19.2.1 常规选项

分位数：对于指定的业务系统，您可以单独设置要显示的分位值曲线，即勾选**单独配置**后进行设置。业务系统相关的分位值统计图表默认展示 50、80、95 和 99 分位值曲线。

3.19.2.2 用户溯源

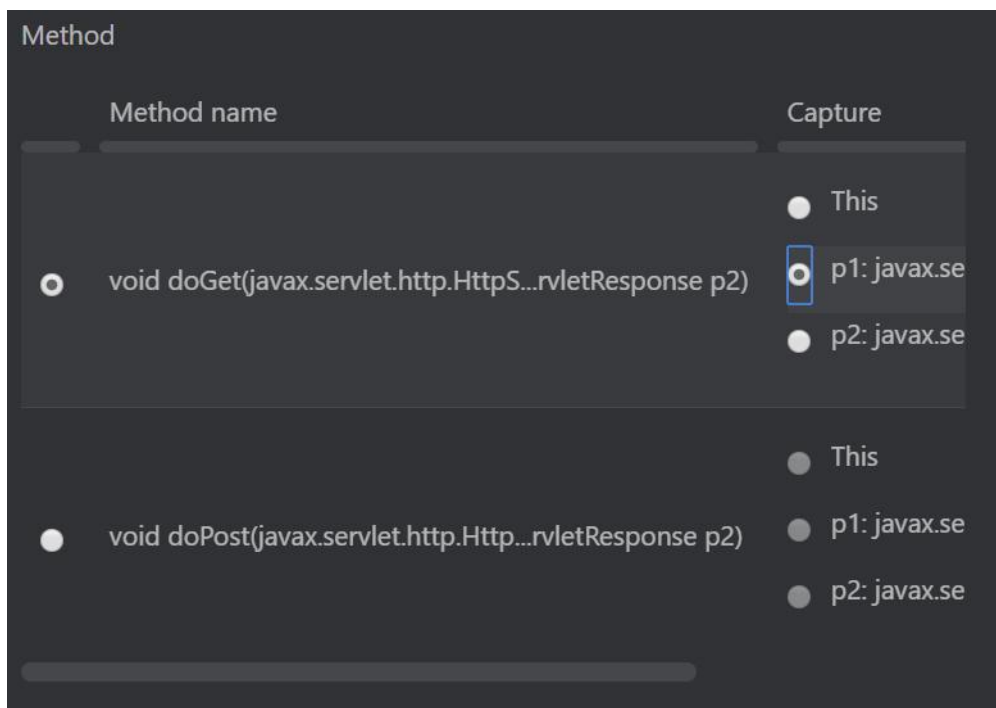
当需要追踪某一个事务是哪个用户进行的，请先设置用户信息获取方式。当发生慢事务追踪时，悟空平台会根据您设置的数据源按照优先级抓取用户标识，直到获取到为止。当您在**事务追踪**页面查看列表时，只需查看用户标识一列，就可以知道慢事务是由哪个用户触发的。



配置用户溯源，单击**添加数据源**，在下拉列表中选择获取方式，包括以下 8 种方式：

- Java method parameter(s): 从方法参数获取，该方式为默认方式。通过该方式获取参数会自动生成一条自定义嵌码，并自动执行，显示在**自定义嵌码**页签中。在 Class 搜索框中输入类名的关键字，找到类名后，在 Method 部分选择要获取用户标识参数所在的方法，然后选择要获取的参数。

- p1:xxxx 或 p2:xxxx 代表当前方法的参数，按方法入参的顺序排序。
- this: 当前 Class 的 this 对象。
- Getter Chain: 方法调用链，可以获取对象中一个具体的值，例如 `getPerson().getName()`。



- Web request query parameter: 从请求的 URL 参数获取。在文本框中输入代表用户信息的参数名称。
- HTTP request header: 从 HTTP 请求头中的参数获取。在文本框中输入代表用户信息的参数名称。
- HTTP post parameter: 从 HTTP 请求体中的参数获取。在文本框中输入代表用户信息的参数名称。
- HTTP response header: 从 HTTP 响应头中的参数获取。在文本框中输入代表用户信息的参数名称。
- Servlet session attribure: 从 Session 中保存用户标识的参数获取。在 Key 文本框中输入代表用户信息的属性名称。如果需要获取对象中一个具体的值，请填写 Getter Chain 方法调用链，例如 `getPerson().getName()`。
- Web request path: 从请求 URL 的 URI 部分获取。
- HTTP cookie: 从 Cookie 中保存用户标识的参数获取。在文本框中输入代表用户信息的参数名称。

某些情况下，获取到的参数信息并不是准确的用户信息，例如参数中的某个字段才是真正的用户信息，对于这种情况，您可以勾选**数据处理**，对参数进行二次精确，包括 `between`、`before`

和 after 三种方式。例如 Web 请求为
https://help.tingyun.com/document_detail/106690.html?spm=a2c4g-11174359.6.643.54e960f80sE7s8，其中 spm 这个参数中只有“-”前面的部分为用户信息，那么我们将 Substring 方式设置为 before，内容设置为“-”，悟空平台获取到的用户信息为 a2c4g。您还可以在下方的预览文本框中输入测试内容，单击**查看结果**进行确认。

☒ 数据处理

Substring

before

-

预览

https://help.tingyun.com/document_detail/106690.html?spm=a2c4g-

查看结果

https://help.tingyun.com/document_detail/106690.html?spm=a2c4g

保存配置
 取消

单击**保存配置**后，新添加的数据源将会显示在列表的最上方，即优先级最高。您可以单击**顺序**列中的箭头来调整数据源的优先级，排列越靠上，优先级越高。表示移到顶部，表示移到底部。表示上移一行，表示下移一行。悟空平台会按照优先级依次匹配数据源获取用户标识。当您不想使用某个数据源时，可在**状态**栏中将其关闭。

在搜索框中可根据 Key 的关键字搜索数据源。

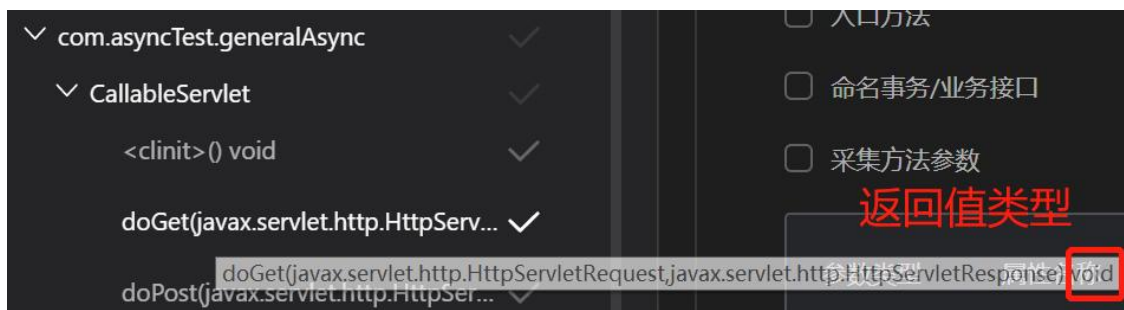
3.19.2.3 自定义嵌码

用户可以对各种非标准化应用组件的某个方法或者基调听云暂不支持的组件的某个方法进行监控。当方法被调用时，探针将对该方法进行性能数据（如调用次数、平均响应时间等）搜索，获取到的内容将在事务分解表格、事务追踪的调用栈中进行展示。

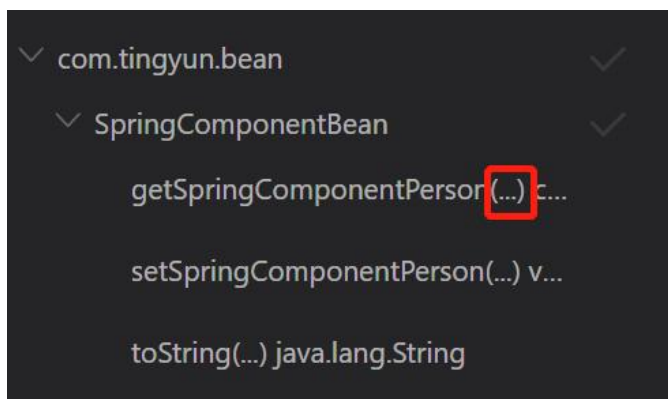
自定义嵌码状态栏：自定义嵌码状态开关开启后，您所配置的自定义嵌码可以被执行。关闭后，仍然可以配置自定义嵌码，但是不能执行嵌码。

未执行的嵌码列表：显示配置完成但还未执行嵌码的自定义嵌码配置。单击右上方的**添加自定义嵌码**按钮，您可以通过代码包添加和手工录入两种方式配置。

- 通过代码包添加：当您不能准确的输入具体的类名和方法名时，可以在应用的代码中按层级来定位方法，基础包不会显示在此处。在下拉菜单中选择应用，可以通过类名、方法名或者参数名来搜索，匹配到的内容将会显示为蓝色，搜索到的方法会自动被展开展示。您也可直接单击  可展开包和类，以便定位到方法。括号中是方法的参数类型，最后显示的是返回值类型。



勾选**忽略方法参数**后，方法中的参数会显示为省略号，如下图所示，重载的方法会合并显示为一个。



在左侧确定一个或多个方法后，右侧面板中会显示**嵌码选项**和**当调用方法时忽略事务**两个选项按钮。

嵌码选项

包含子类

入口方法

命名事务/业务接口

采集方法参数

参数类型	属性名称	Getter Chain	参数命名
java.util.function.Fun...	请输入...	请输入...	
org.reactivestreams.P...	请输入...	请输入...	

当调用方法时忽略事务

■ 嵌码选项：

➢ 包含子类：勾选后，当前方法所在的类的所有子类都会被监控到。

➢ 入口方法：有些方法由于没有入口方法，就算自定义嵌码了也不会作为一个事务显示，因此这类方法如果要作为一个事务来监控，必须要勾选该项。

➢ 命名事务/业务接口：勾选后，该方法的名称将作为事务名称或服务接口名称。事务或服务接口的命名方式为：Custom/类名/方法名。如果不勾选，事务名称是默认的一些方法名。

➢ 采集方法参数：勾选后，可以采集当前方法的参数。属性名称是给参数一个命名，默认会生成一个数据项，并展示在数据项页签列表中。只有配置了属性名称的参数才会被采集。当参数类型为对象时，如果您还想采集具体的一个值，请配置 Getter Chain。如果需要在命名事务后的名称中继续加入参数的值，请勾选参数命名复选框，该复选框只有在命名事务/业务接口和采集方法参数功能同时开启的情况下才可勾选。命名方式为：Custom/类名/方法名？key=参数值。

■ 当调用方法时忽略事务：

当该方法被调用时，该方法所涉及的事务不会被监控到，即事务列表或者事务追踪列表中不会再出现上述事务。

● 手工录入：

如果您对应应用代码非常熟悉，可以使用该方式。包含 5 种匹配规则（即 Match Type），说明如下：

■ Class method by signature：

请准确的输入类名（包括包名）和方法名。

➢ 忽略方法参数：不管该方法有几个参数，所有参数都会被采集。

191

- 指定方法参数：由于在 Java 代码中会出现方法重载的情况，因此需要指定参数类型来唯一确定一个方法。在重载的情况下，只采集与配置的参数个数一样的方法。例如拥有相同方法名称的方法有 3 个，分别有 1 个参数、2 个参数和 3 个参数，我们只配置了一个参数类型，则悟空平台只采集 1 个参数的方法。
- 嵌码选项：请参见通过代码包添加下的[嵌码选项](#)说明。
- 当调用方法时忽略事务：当该方法被调用时，该方法所涉及的事务不会被监控到，即事务列表或者事务追踪列表中不会再出现上述事务。
- Class method by return type：请准确的输入类名（包括包名）和返回值类型。
 - 包含子类：勾选后，只要是包含该返回值类型的方法所在的类的所有子类都会被监控到。
 - 入口方法：有些方法由于没有入口方法，就算自定义嵌码了也不会作为一个事务显示，因此这类方法如果要作为一个事务来监控，必须要勾选该项。
 - 命名事务/业务接口：勾选后，该方法的名称将作为事务名称或服务接口名称。事务或服务接口的命名方式为：Custom/类名/方法名。如果不勾选，事务名称是默认的一些方法名。
 - 采集方法返回值：勾选后，可以采集该类中包含该返回值类型的方法的返回值。属性名称是给返回值一个命名，默认会生成一个数据项，并展示在[数据项页签](#)列表中。当返回值类型为对象时，如果您还想采集具体的一个值，请配置 Getter Chain。
 - 当调用方法时忽略事务：当该类中包含该返回值类型的方法被调用时，所涉及的事务不会被监控到，即事务列表或者事务追踪列表中不会再出现上述事务。
- Interface method by signature：请准确的输入接口名和方法名。
 - 忽略方法参数：不管该方法有几个参数，所有参数都会被采集。
 - 指定方法参数：在重载的情况下，只采集与配置的参数个数一样的方法。例如拥有相同方法名称的方法有 3 个，分别有 1 个参数、2 个参数和 3 个参数，我们只配置了一个参数类型，则悟空平台只采集 1 个参数的方法。
 - 嵌码选项：请参见通过代码包添加下的[嵌码选项](#)说明。
 - 当调用方法时忽略事务：当该方法被调用时，该方法所涉及的事务不会被监控到，即事务列表或者事务追踪列表中不会再出现上述事务。
- Interface method by signature：请准确的输入接口名和返回值类型。
 - 入口方法：有些方法由于没有入口方法，就算自定义嵌码了也不会作为一个事务显示，因此这类方法如果要作为一个事务来监控，必须要勾选该项。
 - 命名事务/业务接口：勾选后，该方法的名称将作为事务名称或服务接口名称。事务或服务接口的命名方式为：Custom/类名/方法名。如果不勾选，事务名称是默认的一些方法名。

- 采集方法返回值：勾选后，可以采集包含该返回值类型的方法的返回值。属性名称是给返回值一个命名，默认会生成一个数据项，并展示在**数据项**页签列表中。当返回值类型为对象时，如果您还想采集具体的一个值，请配置 Getter Chain。
- 当调用方法时忽略事务：当包含该返回值类型的方法被调用时，所涉及的事务不会被监控到，即事务列表或者事务追踪列表中不会再出现上述事务。
- Method annotation for class：请准确输入注解名。
 - 入口方法：有些方法由于没有入口方法，就算自定义嵌码了也不会作为一个事务显示，因此这类方法如果要作为一个事务来监控，必须要勾选该项。
 - 命名事务/业务接口：勾选后，该方法的名称将作为事务名称或服务接口名称。事务或服务接口的命名方式为：Custom/类名/方法名。如果不勾选，事务名称是默认的一些方法名。
 - 当调用方法时忽略事务：当包含该注解名的方法被调用时，所涉及的事务不会被监控到，即事务列表或者事务追踪列表中不会再出现上述事务。

自定义嵌码创建完成后，会显示在未执行的嵌码列表中，默认为开启状态。如果您不想在单击**执行嵌码**按钮后对某一条配置进行嵌码，可以在 **Enable** 列关闭该条配置。在搜索框中输入类名、接口名、方法注解名、返回值类型或者方法名，都可以对列表进行搜索。

已执行的嵌码列表：单击最下方的**执行嵌码**按钮后，开启状态的自定义嵌码配置开始执行，显示在该列表中。包含四种状态：

- ◆ 嵌码成功：业务系统中所有应用的所有实例都执行嵌码成功。
- ◆ 部分成功：业务系统中部分应用的部分实例执行嵌码成功，部分失败。
- ◆ 嵌码失败：业务系统中所有应用的所有实例都执行嵌码失败。
- ◆ 等待执行：按照计划等待执行中。探针 5 分钟内有心跳才可执行自定义嵌码。

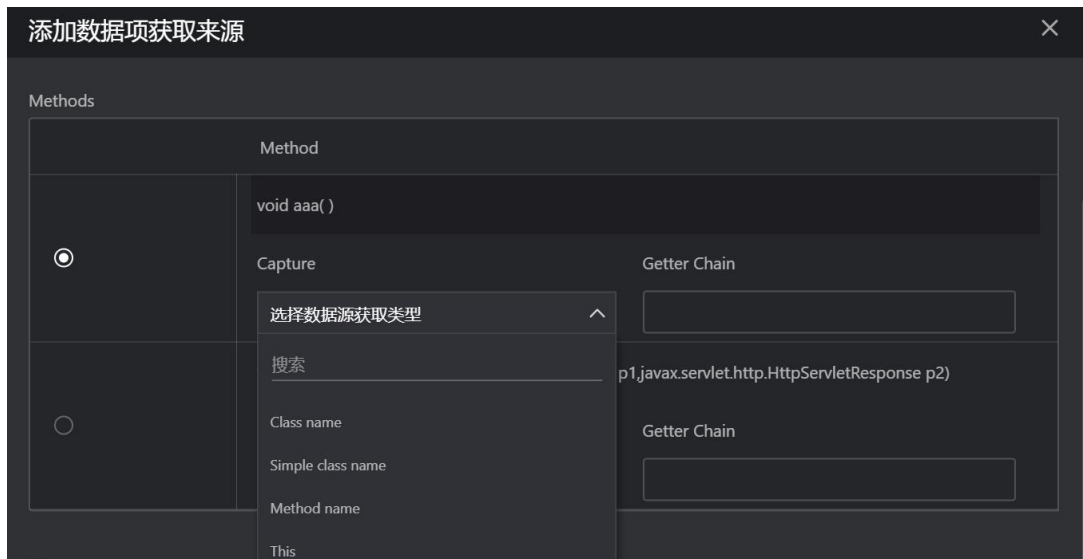
执行完成的自定义嵌码配置可以在 **Enable** 列关闭，目的是让探针停止采集该条配置的数据。在搜索框中输入类名、接口名、方法注解名、返回值类型或者方法名，都可以对列表进行搜索。

3.19.2.4 数据项

用户可以将业务相关的参数定义为一个数据项。数据项可以被自定义指标所使用。单击右上角的**添加**按钮，进行数据项的配置。

- 名称：输入数据项的名称。
- 数据类型：选择数据项的数据类型，包括 String、Integer 和 Double 三种类型。
- 当出现多值时，保留：对于一个请求，同一个方法可能会被调用多次，例如方法的递归调用，每次调用时入口参数的值很可能会不同，这时，您可以设置保留第一个（取第一次调用时参数的值）、最后一个（取最后一次调用时参数的值）或者取出现次数。
- 添加数据项来源：来源包括以下 8 种方式。

- Java method parameter(s): 从方法参数获取，该方式为默认方式。通过该方式获取参数会自动生成一条自定义嵌码，并自动执行，显示在**自定义嵌码**页签中。在 Class 下拉菜单中选择类名后（支持搜索），在 Methods 部分选择要作为数据源来源的参数所在的方法，然后选择要获取的数据源。



- ◆ Class name: 类名，包含包名。
 - ◆ Simple class name: 简单类名，不包含包名。
 - ◆ Method name: 方法名。
 - ◆ this: 当前 Class 的 this 对象。当需要获取对象中一个具体的值时可以填写 Getter Chain，例如 getPerson().getName()。
 - ◆ p1:xxxx 或 p2:xxxx 代表当前方法的参数，按方法入参的顺序排序。
- Web request query parameter: 从请求的 URL 参数获取。在文本框中输入代表该数据项的参数名称。
 - HTTP request header: 从 HTTP 请求头中的参数获取。在文本框中输入代表该数据项的参数名称。
 - HTTP post parameter: 从 HTTP 请求体中的参数获取。在文本框中输入代表该数据项的参数名称。
 - HTTP response header: 从 HTTP 响应头中的参数获取。在文本框中输入代表该数据项的参数名称。
 - Servlet session attribute: 从 Session 中获取参数。在文本框中输入代表该数据项的属性名称。如果需要获取对象中一个具体的值，请填写 Getter Chain 方法调用链，例如 getPerson().getName()。
 - Web request URL: 从请求的 URL 获取。
 - Web request path: 从请求 URL 的 URI 部分获取。

某些情况下，获取到的参数信息并不是准确的数据源信息，例如参数中的某个字段才是真正的用户信息，对于这种情况，您可以勾选**数据处理**，对参数进行二次精确，包括 between、before 和 after 三种方式。例如 Web 请求为 https://help.tingyun.com/document_detail/106690.html?spm=a2c4g-11174359.6.643.54e960f80sE7s8，其中 spm 这个参数中只有“-”前面的部分为用户信息，那么我们将 Substring 方式设置为 before，内容设置为“-”，悟空平台获取到的用户信息为 a2c4g。您还可以在下方的预览文本框中输入测试内容，单击**查看结果**进行确认。

☒ 数据处理

Substring

before

-

预览

https://help.tingyun.com/document_detail/106690.html?spm=a2c4g-11174359.6.643.54e960f80sE7s8

查看结果

https://help.tingyun.com/document_detail/106690.html?spm=a2c4g-11174359.6.643.54e960f80sE7s8

保存配置

取消

新创建的获取来源会显示在顶部第一条，即优先级最高。从上往下优先级依次降低。悟空平台会按照优先级依次匹配数据源获取数据项的值。当您不想使用某个获取来源时，可在**启用**栏中将其关闭。

3.19.2.5 自定义指标

当用户想从业务角度查看符合某个条件（例如订单金额大于 100 元）的事务发生次数时，可配置自定义指标来实现。

配置项	说明
-----	----

指标名称	输入自定义指标的名称。
指标 (SELECT)	配置要采集的目标数据项，例如价格和数量两个数据项。该项相当于 SQL 语句中 SELECT 的作用，为必填项。 单击新建按钮进行添加，选择数据项和聚合规则，聚合规则即计算方式，会根据数据项类型的不同而改变，例如 String 类型只包含 count，表示计数。勾选一个指标后，单击修改按钮进行修改。勾选一个或多个指标后，单击删除可直接将其删除。
过滤器(WHERE)	配置采集目标数据项数据的条件，当满足过滤器内所有条件时探针才进行采集。该项相当于 SQL 语句中 WHERE 的作用，为可选项。 单击新建按钮进行添加，选择数据项、比较条件和数值。勾选一个指标后，单击修改按钮进行修改。勾选一个或多个指标后，单击删除可直接将其删除。
维度(GROUP BY)	配置要采集的目标数据项的统计维度。该项相当于 SQL 语句中 GROUP BY 的分组作用，为可选项。 单击新建按钮进行添加，选择数据项即可。勾选一个或多个指标后，单击删除可直接将其删除。

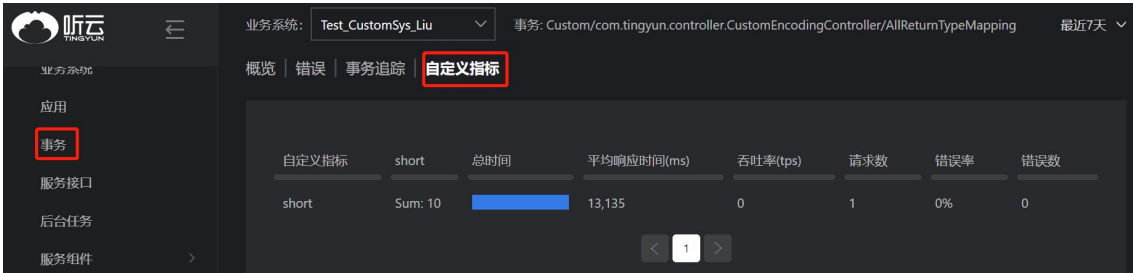
举例说明，如果您想知道订单金额大于 100 的订单有多少笔，即是要统计提交订单这个业务操作所涉及的事务中金额参数大于 100 的发生了多少次。如假设订单金额已经被设置成了数据项“金额”，则我们可以定义自定义指标为“订单金额大于 100”，指标添加“金额”，聚合规则为 count。过滤器添加“金额”，比较条件为大于，值为 100。维度不设置。

指标列表

+ 添加

名称	数据项(SELECT)	过滤器(WHERE)	维度(GROUP BY)	操作
test_custom_B	test_data_three, test_data_three	test_data_three, test_data_three	test_data_three	编辑 删除
test_custom_A	test_data_one, test_data_two	test_data_one, test_data_two	test_data_one, test_data_two	编辑 删除

自定义指标的监控统结果请到应用下的自定义指标页签或者事务下的自定义指标页签查看。



单击具体的指标条目可查看统计详情。



用户可以按维度查询该指标的维度展示详情。



3.19.2.6 性能诊断

默认情况下，探针监控到的方法有限，而对于一个请求的完整调用链来说，可能会存在某个方法耗时特别长，又无法进一步细粒度的分析。全栈快照通过对请求线程做多次快照剖析，得到更为详细的调用链信息，来更准确地分析性能瓶颈。探针在收到快照采集指令后，会对某一个请求线程开启快照采集诊断会话，采集当前线程的线程名、线程 ID 等信息。

采集全栈快照对性能有一定的消耗，因此用户可以按需控制是否开启快照采集。开启后，还可以控制单片快照采集的时间间隔。



全栈快照的采集模式有如下三种：

- 触发采集：有条件的触发全栈快照诊断会话。

触发条件为：1 分钟内慢请求占比超过指定阈值(默认值 10%，数据边界：[1, 100]%) 且慢请求次数超过 20 时，启动诊断会话。诊断会话的持续时间默认为 5 分钟（数据边界：[1, 10]），该诊断周期内，会为每个请求，每分钟尝试采集 10 个（数据边界：[1, 100]）快照样本，总的快照样本限制为 100 个（数据边界：[1, 1000]）。

为了避免因为持续的性能问题而导致过多的采集快照数据，可以配置诊断会话之间的等待时间，默认诊断会话间隔 10 分钟（数据边界：[5, 30]）。

- 固定周期采集：按照固定周期采集全栈快照。勾选复选框后才能生效。

触发条件为：每 1 分钟（数据边界：[1, 100]）采集 10 个全栈快照。当处于诊断会话（触发采集）时，固定周期的采集暂停。

- 强制采集：根据条件强制采集全栈快照。勾选复选框后才能生效。该模式默认关闭。

触发条件为：当请求执行时间超过 3 倍慢请求阈值时（数据边界：[慢请求阈值, ∞]），开启针对该请求的全栈快照，每分钟最多采集 100 个快照。

单击底部的**保存配置**按钮，配置开始生效。用户可在请求追踪详情的**全栈快照**页签中查看结果。

3.19.3 应用设置

应用设置是以应用为单位进行配置项的设置。部分配置项可以继承业务系统级别的配置，在此我们将不再讲解，具体描述请见[业务系统设置](#)。需要说明的是，应用不能继承业务系统配置的自定义指标。

3.19.3.1 常规选项

The screenshot shows a configuration interface for an application named 'Test03'. It includes several toggle switches and radio buttons for different monitoring features. The '应用别名' (Application Alias) is set to 'Test03'. The '自动命名事务' (Automatic Transaction Naming) is enabled. The '日志溯源' (Log Tracing) is set to '使用系统配置: 当前状态为打开' (Use system configuration: current status is open). The '启用Thrift监控' (Enable Thrift Monitoring) is disabled. The '启用MQ消费端监控' (Enable MQ Consumer Monitoring) is disabled. The 'ApdexT' is set to '使用系统配置: 500ms' (Use system configuration: 500ms).

应用别名: Test03
您监控的应用依然是“Test03”，别名仅作为更友好的显示，最长128个字符。

自动命名事务: ☒ 开启该选项后将使用应用组件作为事务的名称。关闭该选项将用Web请求的URI作为事务的名称。

日志溯源: ☐ 使用系统配置: 当前状态为打开
☒ 单独配置: ☐

启用Thrift监控: ☐ 监控非官方generator生成的Thrift程序可能会造成应用崩溃，如果有定制或者二次开发，务必在测试环境充分测试后，谨慎开启此功能。

启用MQ消费端监控: ☐ 监控MQ消费端可能造成处理能力下降，务必在测试环境充分测试后，谨慎开启此功能。

ApdexT: ☒ 使用系统配置: 500ms
☐ 单独配置: ms

- 应用别名

设置应用别名只是为了用户在界面中更方面的查看应用，便于识别。

- 自动命名事务

默认情况下，应用探针通过在 Web 请求的 [URI](#) 前加上 URI 前缀来命名事务，即事务名称结构为：URI/Web 请求的 URI 部分。当启用自动命名事务功能后，应用探针根据应用框架或组件来命名事务，以增强事务的可识别性，事务名称结构为：应用框架或组件名称/Web 请求的 URI 部分。

下面我们用一个例子来说明自动命名事务功能。假如用户从浏览器端发起一个 HTTP 请求，完整的 URL 为 <http://www.tingyun.com/sql/oracleException1.do?city=beijing>，如果处理该事务的组件为 Servlet，当未开启自动命名事务功能时，事务的名称为 URI/sql/oracleException1.do，如果开启，则事务的名称为 Servlet/sql/oracleException1.do。

- 日志溯源

请参见[全局配置](#)下的日志溯源介绍。

- 启用 Thrift 监控

开启该功能后，通过 Thrift 框架进行的 RPC 调用会被监控到。监控非官方 generator 生成的 Thrift 程序可能会造成应用崩溃，如果有定制或者二次开发，务必在测试环境充分测试后，谨慎开启此功能。

- MQ 消费端监控

监控 MQ 消费端可能造成应用处理消息能力下降，进而 MQ 消息队列中的数据会快速增长导致积压，因此请务必在测试环境充分测试后，谨慎开启此功能。

3.19.3.2 采样

请参见[全局配置](#)下的采样介绍。

3.19.3.3 探针熔断

请参见[全局配置](#)下的探针熔断介绍。

3.19.3.4 错误及异常

请参见[全局配置](#)下的采集错误和异常介绍。

3.19.3.5 事务

- 开启追踪

请参见[全局配置](#)下的追踪选项介绍。

- 事务命名

事务命名功能能够丰富事务的命名方式，包括根据请求方法来命名事务、根据请求 URI（URI 是指 URL 中除域名或 IP 地址、端口号、ContextPath 和参数之外的部分）的部分来命名事务，或者根据请求的各种参数来命名事务。

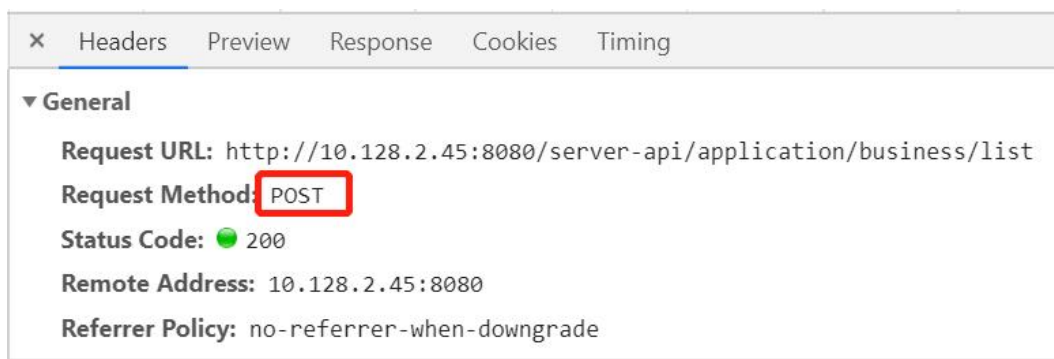
命名原理：探针会根据配置好的匹配规则聚合符合条件的事务，再根据命名规则重新给事务进行命名，以方便您识别事务。

匹配规则：

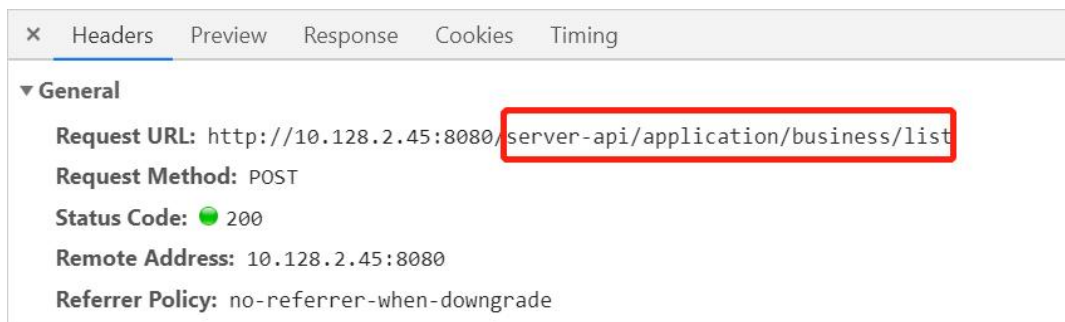
探针采集到一个请求后，会把请求和匹配规则进行匹配，符合同一匹配规则的请求就是同一类请求。

匹配规则中，应用探针对 URL 的匹配方式如下：

- 支持多种请求方式：GET、POST、PUT、DELETE、HEAD。

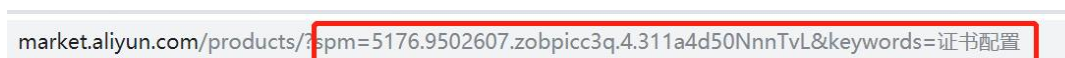


- 支持多样化的 URI 匹配方式：等于、开始于、结束于、包含、正则。

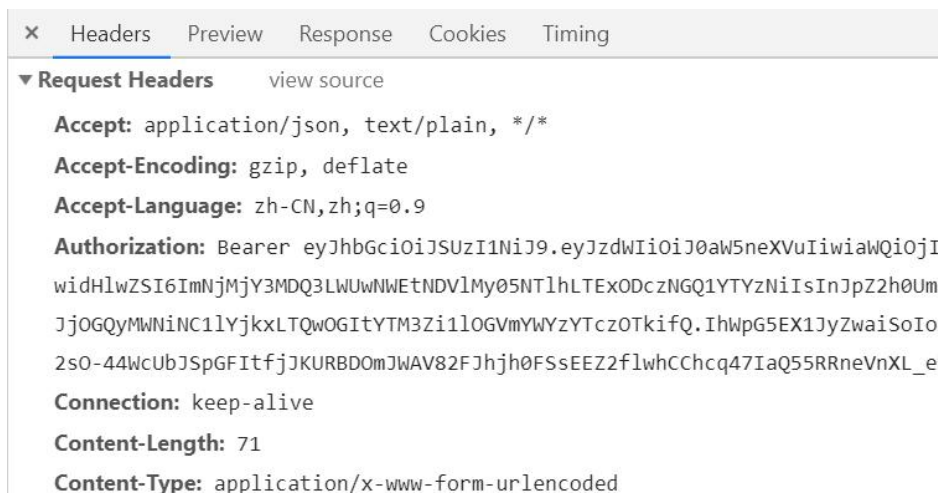


- 支持按照 URL 参数名、Header 参数名和 Body 参数名匹配 URL，该部分可根据用户的需要选择性配置。如果配置了多条参数规则，各规则必须都满足才匹配请求。

URL 参数（适用于 GET 请求时）：



请求头参数：



请求体参数（适用于 POST 请求时）：



命名规则：

命名规则使事务命名的方法更加丰富，既可以指定请求方式作为事务名称的一部分，也可以指定 URI 分段作为事务名称的一部分，还可以指定一些参数作为事务名称的一部分，从而增强事务的可识别性。

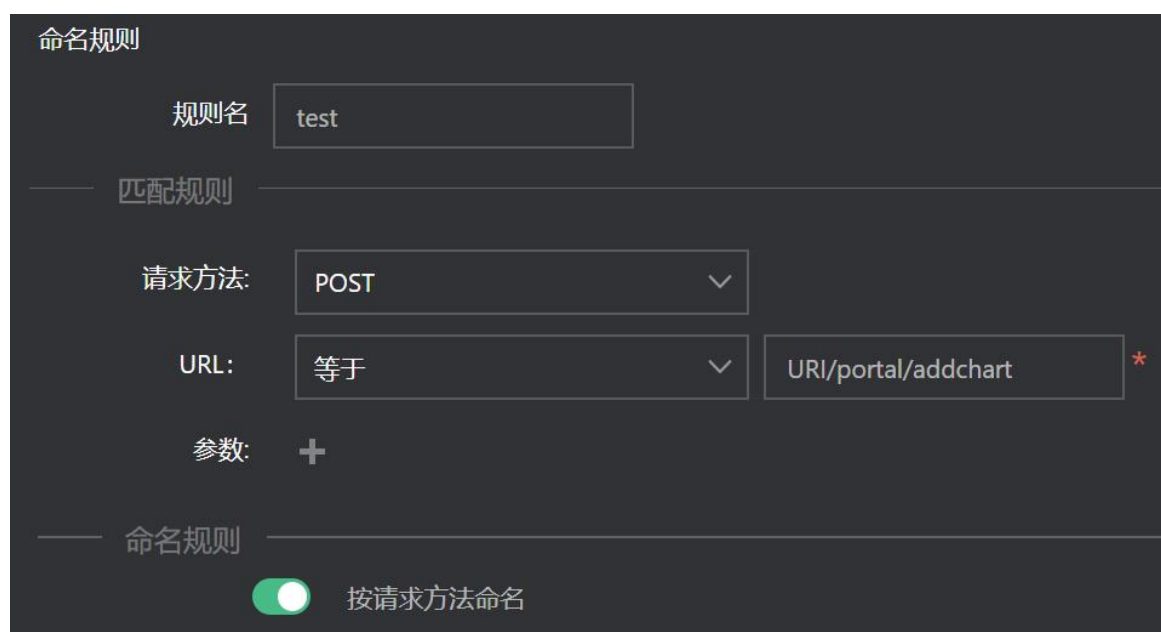
如果仅指定匹配规则而不指定命名规则，那么符合条件的 URL 请求生成的事务的名称为事务命名规则名称。如果指定了命名规则，那么事务名称的结构为：

- ◆ 按请求方法命名：匹配规则名称/(请求方法)。
- ◆ 按 URI 分段命名：匹配规则名称/URI 分段。
- ◆ 按 URL 参数名进行命名：匹配规则名称/?URL 参数 key=参数值。如果是多个参数，以&符号连接。
- ◆ 按 Header 参数名进行命名：匹配规则名称/?Header 参数 key=参数值。如果是多个参数，以&符号连接。
- ◆ 按 Body 参数名进行命名：匹配规则名称/?Body 参数 key=参数值。如果是多个参数，以&符号连接。
- ◆ 按 Cookie 参数名进行命名：匹配规则名称/?Cookie 参数 key=参数值。如果是多个参数，以&符号连接。

下面我们举例说明事务命名功能。

按请求方式命名举例：

对于 URI/portal/addchart 这个事务，我们想对 POST 请求方式和 GET 请求方式进行的请求分别命名，所以需要配置**按请求方式命名**，最终会生成 test/（POST）和 test/（GET）两个事务。



命名规则

规则名

匹配规则

请求方法:

URL:

参数: +

命名规则 ☒ 按请求方法命名

按 URL 命名举例：

假设有 URI/portal/addchart、URI/redirect/addchart 多个类似的事务，我们只将 POST 方式下且 URI 的第 3 段为 addchart 的请求定义为操作，在配置匹配（聚合）规则时我们可以做如下配置：

匹配规则

请求方法:

POST

▼

URL:

等于

▼

URI/portal/addchart

*

参数:

+

命名规则

☐

按请求方法命名

☒

按URL命名

☒

取URL前

段

☐

取URL后

段

☐

取URL第

3

段

最终生成的事务名称为 test/addchart。

按 URL 参数命名举例：

假设有请求/order，当 action=1 时代表提交订单，当 action=2 时代表支付，在配置规则时我们可以做如下配置（规则名为 test）：



图 45

通过上述聚合规则就会产生两个聚合的请求：

1. test?action=1
2. test?action=2

我们可以把第一类请求定义为提交订单的操作，把第二类请求定义为支付的操作。

按 Header 参数命名举例：

假设对于/order 这个事务，我们只对 POST 方式下的请求按照 Header 参数 Referer 进行命名，在配置规则时我们可以做如下配置（规则名为 test）：

请求方法: POST

URL: 等于 /order *

参数: +

Header参数名 Referer

等于 http://10.128.2.48:8080/serv 田

命名规则

☐ 按请求方法命名

☐ 按URL命名

☒ 取URL前 段

☐ 取URL后 段

☐ 取URL第 段

☐ 按url参数命名

☒ 按Header参数命名 Referer

当 Referer 为 http://10.128.2.48:8080/server/system 时，生成的事务名称为 test? Referer=http://10.128.2.48:8080/server/system。

按 Body 参数命名举例：

假设对于/order 这个事务，我们只对 POST 方式下的请求按照 Body 参数中 action 进行命名，在配置聚合规则时我们可以做如下配置（规则名为 test）：

请求方法:

POST

URL:

等于

/order

 *

参数: +

Body参数名

action

等于

2

命名规则

按请求方法命名

按URL命名

取URL前

段

取URL后

段

取URL第

段

按url参数命名

按Header参数命名

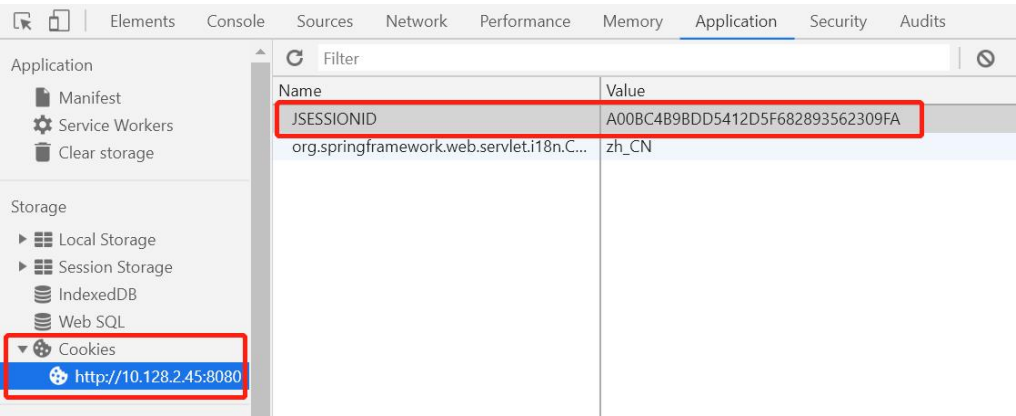
按Body参数命名

action

当 action 为 2 时，生成的事务名称为 test?action=2。

按 Cookie 参数命名举例：

假设有请求 https://10.128.2.45:8080/order，我们需要根据 Cookie 参数中的 JSESSIONID 参数进行命名：



在配置聚合规则时我们可以做如下配置（规则名为 test）：

匹配规则

请求方法: POST

URL: 等于

://10.10.128.2.45:8080/order*

参数: +

命名规则

按请求方法命名

按URL命名

取URL前

段

取URL后

段

取URL第

段

按url参数命名

按Header参数命名

按Body参数命名

按Cookie参数命名

JSESSIONID

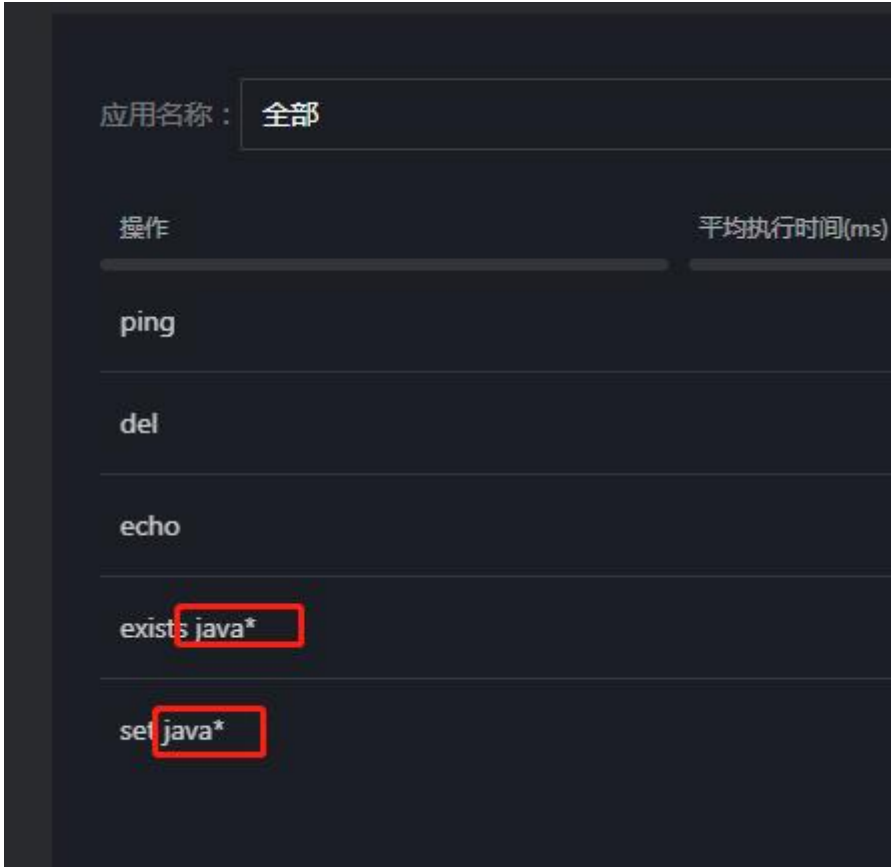
那么，当 JSESSIONID 为 1111 时，生成的事务名称为 test?JSESSIONID=1111。

● 事务追踪阈值

此处可针对已经发生过的具体事务单独设置追踪阈值。

● NoSQL 键值

通过在规则中设置以某字符串开头的键（Key），系统将会对涉及以该字符串开头的键的 NoSQL 操作名称进行聚合显示，格式为：操作+java*。例如，设置了键以“java”开头，那么操作名称会有如下显示：



● 事务过滤

事务过滤也可以称为事务/服务接口黑名单，即加入黑名单的事务/服务接口，系统不再收集其性能数据。

事务和服务接口的响应时间是影响应用评分的重要因素之一，大量的慢事务、慢服务接口会直接降低应用性能评分。但在一些特定场景下，例如长连接、批处理等已知的 QPS 高、响应时间长的慢事务，如果不希望它们的响应时间影响到应用评分，则可以通过配置过滤掉指定事务或服务接口，避免其对评分的影响。

添加需要过滤的事务或服务接口到右侧列表，单击提交即可，配置成功后不需要重启探针。勾选列表上方的事务复选框，列表中所有的事务会被选中；勾选列表上方的服务接口复选框，列表中所有的服务接口会被选中。

说明：事务过滤仅 Agent Collector 3.4.5 及以上版本支持，服务接口过滤仅 Agent Collector 3.5.3 及以上版本支持。列表显示最近 30 分钟有数据的事务及服务接口，不包含聚合后的事务/服务接口，即聚合后的事务/服务接口不支持过滤。

3.19.3.6 热点方法

● 热点方法学习

开启该选项后，探针会自主学习应用中比较耗时的方法，相应的方法会被嵌码，探针将采集性能数据，最终展示在事务和后台任务的性能分解表格以及追踪详情中，性能分类中会增加“Hotspot”类别。默认为关闭状态。

代码段	性能分类	总耗时(ms)	耗时占比(%)	调用次数	平均执行时间(ms)
javax.servlet.S_ialized	Java	12386	25.31	108	115
org.springframework...ofilter	Filter	9196	18.79	108	85
org.apache.tomc...ofilter	Filter	9175	18.75	108	85
org.springframework...service	Servlet	9170	18.74	108	85
com.tingyun.con...et/save	Java	5391	11.02	108	50
com.tingyun.utl...erties	HotSpot	1320	2.7	216	6

- 白名单

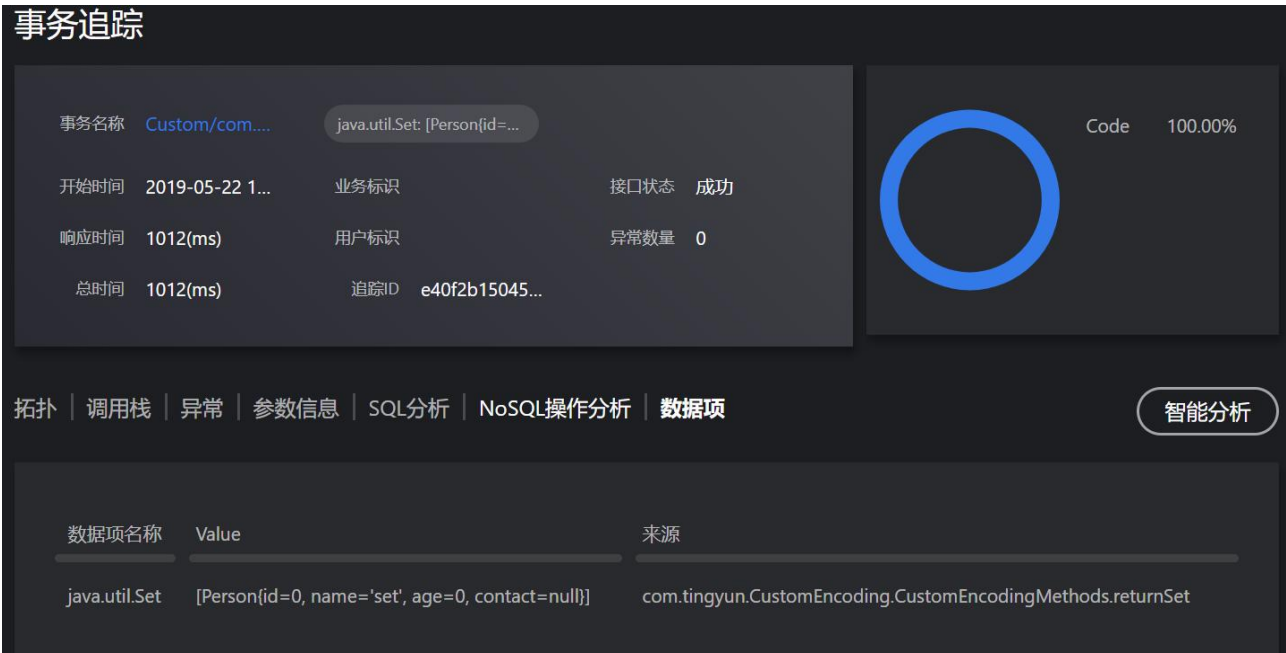
用户可以添加指定的几个方法到白名单中，探针只自主学习白名单中的方法。
- 黑名单

探针不会自主学习黑名单中的方法。默认会显示 Java 基础包。
- 学习结果

学习次数越多，说明该方法被调用的频率越高，耗时也越长。学习次数前 100 名的方法将显示在学习结果列表中。对方法单击**启用**后，相应的方法会被嵌码，探针将采集性能数据，最终展示在事务、业务接口和后台任务的性能分解表格以及追踪详情中。单击**禁用**后，探针将停止采集性能数据。单击**删除**后，相应的方法将从学习结果中删除。勾选**选择**前面的复选框可选择全部方法进行操作。

3.19.3.7 数据项

针对应用配置数据项时，可以关联具体的事务。当该事务发生慢追踪时，您可以在追踪详情中看到**数据项**页签。



3.19.4 实例设置

实例设置可以对每个应用的每个独立实例进行配置管理，提供热点方法监控和探针熔断两个配置项，可以从应用级别继承配置值，也可以对特定的实例设置自己的配置项值。



3.20 权限说明

管理员对超级管理员 admin 为其分配的其他管理员所创建的业务系统、应用等，具有“只读”权限。以管理员 A 为例，A 自己没有创建任何业务系统和应用，admin 赋予它所有 APM 的数据权限（即其他管理员所创建的业务系统和应用）和菜单权限后，具体影响说明如下：

- 业务系统列表和应用列表可以正常访问，但是不能编辑和删除。
- 系统配置、应用配置和实例配置菜单可以正常访问，但没有操作权限。
- 对于 APM 的其他页面，管理员 A 均有查看、导出文件、下载文件、执行等权限，操作权限与创建业务系统或者应用的管理员一致。

4 产品架构

APM 产品架构包括以下组成部分：



- Agent：应用探针，运行在用户的应用内，负责嵌码和原始数据采集。
- Agent Collector：原始数据采集器，负责从探针接收原始数据并进行关联汇总。
- DC：数据中心采集器，负责与 Agent Collector 进行数据交换，包括收集数据和下发指令。
- 数据处理和存储层：DC 收集的数据通过此层架构中的组件进行分析和持久化到数据库中。
- Report：系统报表，提供数据查询和配置界面。
- Alert：警报，负责根据配置发送警报通知。

5 安全和可靠性

5.1 安全

由于应用与微服务产品使用了应用内探针技术，通过用户的配置可以采集性能和业务等敏感数据，因此整个系统对安全需要有较高的要求。应用与微服务系统通过以下几方面来保证安全性。

- **采集安全**

在采集数据的探针上，通过配置选项，允许用户对采集到的敏感数据做混淆，例如：对用户名、密码、手机号等敏感信息。数据混淆在探针端完成，保证传输到 Agent Collector 上的数据不包含敏感数据，从而保证数据的安全。

同时探针提供审计模式，在审计模式运行时，探针将采集并上报的数据以日志的形式输出到本地的文件中，以便用户对上报数据进行安全审计，发现有敏感信息上报即可通过修改混淆设置进行屏蔽。

- **传输安全**

在数据传输上，探针与 Collector，Collector 与 DC 端采用加密的 HTTPS 传输协议，保证数据在传输网络过程中的安全。

- **存储安全**

应用与微服务采用高可用的存储架构对数据进行存储，对每种类型的数据存储都提供高度冗余和分布式的高可用存储方案，并提供健全的自动备份、恢复和容灾措施，最大限度的保证数据存储的安全性和可靠性。

- **账号安全**

应用与微服务提供完整的角色和权限管理系统、可对每个功能模块和数据项设置特定的用户或角色访问权限，不同角色和用户只能访问被允许访问的功能和数据，特别是敏感的业务分析和业务数据，可以限定到特定的访问用户和角色上。

同时平台使用安全的 HTTPS 加密访问和权限校验，保证用户账号与密码的安全。

5.2 可靠性

应用与微服务探针的可靠性保证机制如下：

- 当被监控的应用较繁忙或者已经没有足够的资源时，为了保证被监控系统的稳定运行，探针将关闭部分功能以减少对应用资源的消耗。当内存使用率或者 Garbage Collection CPU 时间占比超过采样阈值时，探针将对 Trace 数据进行采样；当内存使用率或者 Garbage Collection CPU 时间占比超过停止采集数据阈值时，探针将不再采集 Trace 数据。
- 当网络出现异常时，探针和 Agent Collector 之间的缓冲区能够保存 1024 条 trace 数据，最新采集的数据会不断的覆盖旧的数据，待网络恢复正常后，Agent Collector 会将缓冲区中的数据继续上传到基调听云系统。
- 当部署多个 Agent Collector 时（推荐），即使其中一个 Agent Collector 出现故障不能正常上传数据，其他的 Agent Collector 会在 5 分钟内接替其工作继续上传，确保数据不会丢失。